

Building Up RL: From Dynamics and Control to Learning

Pierre-Luc Bacon

Friday 7th August, 2026



1 Syllabus · Plan de cours

**IFT 6162 — Apprentissage par renforcement, commande optimale
(Reinforcement Learning, Optimal Control)**

Note

This page is bilingual. Use the tabs below to switch between English and French. Cette page est bilingue. Utilisez les onglets ci-dessous pour basculer entre l'anglais et le français.

2 Browser Labs

The browser lab is a small, install-free companion to the book. Its six notebooks focus on examples that fit comfortably in a WebAssembly Python kernel; the expensive JAX, MPC, and nonlinear solver demonstrations remain precomputed in the main text.

[Launch the browser labs](#)

Each notebook begins with a working baseline, asks you to predict an outcome before changing parameters, and ends with executable assertions:

1. finite-MDP policy evaluation;
2. value iteration and policy iteration;
3. Monte Carlo error and maximization bias;
4. Euler, trapezoidal, and higher-order ODE discretization;
5. finite-horizon LQR and the Riccati recursion;
6. a small SciPy trajectory-optimization problem.

The notebooks use only NumPy, SciPy, Matplotlib, and ipywidgets. Changes are stored in your browser. Download a notebook from JupyterLab if you want to keep or submit it.

To serve the lab locally after building it, run:

```
uv run jupyter lite serve - -lite -dir lab - -contents notebooks
```

3 Modeling

3.1 Dynamics and State-Space Models

Learning Goals

After reading this chapter, you should be able to:

- Distinguish between predictive models and decision-making models, and explain why the latter require more than correlation
- Write a dynamical system in state-space form and identify the state, control, disturbance, and output
- Formulate dynamics in both discrete and continuous time, and explain when each is appropriate
- Extend a deterministic model to include stochastic uncertainty using either the function-plus-noise view or the transition kernel view
- Recognize when partial observability matters and write down an observation model
- Distinguish between analytical models (where f is given) and simulation-based models (where only trajectory queries are possible)
- Identify when a system is best modeled as discrete-event, hybrid, or agent-based

Prerequisites: Basic linear algebra (matrices, vectors, matrix-vector multiplication) and ordinary differential equations (what $\dot{x} = f(x)$ means). The second half uses probability and expectation.

3.1.1 Models for Decision Making

“The sciences do not try to explain, they hardly even try to interpret, they mainly make models. By a model is meant a mathematical construct which, with the addition of certain verbal interpretations, describes observed phenomena. The justification of such a mathematical construct is solely and precisely that it is expected to work.”

— John von Neumann

The word *model* means different things depending on who you ask.

In machine learning, it typically refers to a parameterized function (often a neural network) fit to data. When we say “we trained a model,” we usually mean adjusting parameters so it makes good predictions. This is a narrow view.

In control, operations research, or structural economics, a model refers more broadly to a formal specification of a decision problem. It includes how a system evolves over time, what parts of the world we choose to represent, what decisions are available, what can be observed or measured, and how outcomes

are evaluated. It also encodes assumptions about time (discrete or continuous, finite or infinite horizon), uncertainty, and information structure.

To clarify terminology, we use the term **decision-making model** to refer to this broader object: one that includes system dynamics, state, control, observations, objectives, time structure, and information assumptions. In this sense, the model defines the structure of the decision problem. It is the formal scaffold on which we build optimization or learning procedures.

Depending on the setting, we may ask different things from a decision-making model. Sometimes we want a model that supports counterfactual reasoning or policy evaluation, and are willing to bake in more assumptions to get there. Other times, we just need a model that supports prediction or simulation, even if it remains agnostic about internal mechanisms.

This mirrors a distinction in econometrics between structural and reduced-form approaches. Structural models aim to capture the underlying process that generates behavior, enabling reasoning about what would happen under alternative policies or conditions. Reduced-form models, by contrast, focus on capturing statistical regularities (often to estimate causal effects) without necessarily modeling the mechanisms that generate them. Both are forms of modeling, just with different goals. The same applies in control and RL: some models are built to support simulation and optimization, while others serve more diagnostic or predictive roles, with fewer assumptions about how the system works internally.

This chapter focuses on the modeling side. What kinds of models do we need to support decision-making from data? What are their assumptions? What do they let us express or ignore? And how do they shape what learning and optimization can even mean?

3.1.2 Modeling, Realism, and Control

Realism is only one way to assess a model. When the purpose of modeling is to support control or decision making, accuracy in reproducing every detail of the system is not always necessary. What matters more is whether the model leads to decisions that perform well when applied in practice. A model may simplify the physics, ignore some variables, or group complex interactions into a disturbance term. As long as it retains the core feedback structure relevant to the control task, it can still be effective.

In some cases, high-fidelity models can be counterproductive. Their complexity makes them harder to understand, slower to simulate, and more difficult to tune. Worse, they may include uncertain parameters that do not affect the control decisions but still influence the outcome of optimization. The resulting decisions can become fragile or overfitted to details that are not stable across different operating conditions.

A useful model for control is one that focuses on the variables, dynamics, and constraints that shape the decisions to be made. It should capture the key trade-offs without trying to account for every effect. In traditional control design, this principle appears through model simplification: engineers reduce the system to

a manageable form, then use feedback to absorb remaining uncertainty. Reinforcement learning adopts a similar mindset, though often implicitly. It allows for model error and evaluates success based on the quality of the policy when deployed, rather than on the accuracy of the model itself.

Example: A simple model that supports better decisions Researchers at the U.S. National Renewable Energy Laboratory investigated how to reduce cooling costs in a typical home in Austin, Texas [Cole et al., 2014]. They had access to a detailed EnergyPlus simulation of the building, which included thousands of internal variables: layered wall models, HVAC cycling behavior, occupancy schedules, and detailed weather inputs.

Although this simulator could closely reproduce indoor temperatures, it was too slow and too complex to use as a planning tool. Instead, the researchers constructed a much simpler model using just two parameters: an effective thermal resistance R and an effective thermal capacitance C . Treating the building as a single thermal mass, the indoor temperature T evolves according to

$$C \frac{dT}{dt} = \frac{T_{\text{out}} - T}{R} - Q_{\text{cool}}, \quad (1)$$

where T_{out} is the outdoor temperature and Q_{cool} is the cooling power applied by the air conditioner. The first term on the right captures heat leaking in through the walls; the second captures heat removed by the cooling system. This is a first-order linear ODE, one of the simplest dynamics models possible.

This reduced model did not capture short-term temperature fluctuations and could be off by as much as two degrees on hot afternoons. Despite these inaccuracies, it proved useful for testing different cooling strategies. One such strategy involved cooling the house early in the morning when electricity prices were low, letting the temperature rise slowly during the expensive late-afternoon period, and resuming cooling only in the evening. When this strategy was simulated in the full EnergyPlus model, it reduced peak compressor power by approximately 70 percent and lowered total cooling cost by about 60 percent compared to a standard thermostat schedule.

The reason this worked is that the simple model captured the most important structural feature of the system: the thermal mass of the building (encoded in C) acts as a buffer that allows load shifting over time. The time constant $\tau = RC$ determines how quickly the indoor temperature responds to changes. That single number was enough to discover a control strategy that exploited this property. The many other effects present in the full simulation did not change the main conclusions and could be treated as part of the background variability.

This example shows that a model can be inaccurate in detail but still highly effective in guiding decisions. For control, what matters is not whether the model matches reality in every respect, but whether it helps identify actions that perform well under real-world conditions. We will return to this kind of thermal model, and its richer multi-zone extensions, once we have introduced the formal machinery.

3.1.3 Dynamics Models for Decision Making

The kind of model we need here is a **dynamics model**. It does not just describe correlations. It tells us how a system **evolves in time** and, for control purposes, how that evolution **responds to inputs** we choose.

A dynamics model earns its keep by answering counterfactuals of the form: *given an initial condition and an input schedule, what trajectory should I expect?* That ability to roll a trajectory forward under different candidate inputs is the backbone of planning, policy evaluation, and learning from interaction.

At this level, we can think of the model as a trajectory generator:

$$(\mathbf{x}_0, \{\mathbf{u}_t\}, \{\mathbf{d}_t\}) \longmapsto \{\mathbf{x}_t, \mathbf{y}_t\}_{t=0:T}, \quad (2)$$

where $\mathbf{u}_t$ are **controls** we set, $\mathbf{d}_t$ are **exogenous drivers** we do not control (weather, inflow, demand), $\mathbf{x}_t$ are internal **system variables**, and $\mathbf{y}_t$ are **observations**. The split between $\mathbf{u}$ and $\mathbf{d}$ is practical: it separates what we can act on from what we must accommodate.

Two design pressures shape such models:

1. **Responsiveness to inputs.** The model must expose the levers that matter for the decision problem, even if everything underneath is approximate.
2. **Memory management.** To simulate step by step, we need a compact summary of the past that is sufficient to predict the next step once an input arrives. That summary is what we will call the **state**.

This brings us to a standard representation. Rather than carry the full history, we look for a variable $\mathbf{x}_t$ that captures “what matters so far” for predicting what comes next under a given input. With that variable in hand, the model advances in small increments and can be composed with estimators and controllers.

With this motivation in place, we can now introduce the formalism.

3.1.4 The State-Space Perspective

Most dynamics models, whether derived from physics or learned from data, can be cast into **state-space form**. The state $\mathbf{x}$ is the compact memory that summarizes the past for prediction and control. Inputs $\mathbf{u}$ perturb that state, exogenous drivers $\mathbf{d}$ push it around, and outputs $\mathbf{y}$ are what we can measure. The equations look the same whether time is treated in discrete steps or as a continuous variable.

Discrete versus continuous time How we represent time is dictated by how we sense and actuate: digital controllers sample and apply inputs in steps; the underlying physics evolve continuously.

Time can be represented in two complementary ways, depending on how the system is sensed, actuated, or modelled.

In **discrete time**, we treat time as an integer counter, $t = 0, 1, 2, \dots$, advancing in fixed steps. This matches how digital systems operate: sensors are sampled periodically, decisions are made at regular intervals, and most logged data takes this form.

Continuous time treats time as a real variable, $t \in \mathbb{R}_{\geq 0}$. Many physical systems (mechanical, thermal, chemical) are most naturally expressed this way, using differential equations to describe how state changes.

The two views are interchangeable to some extent. A continuous-time model can be discretized through numerical integration, although this involves approximation. The degree of approximation depends on both the step size Δt and the integration algorithm used. Conversely, a discrete-time policy can be extended to continuous time by holding inputs constant over time intervals (a zeroth-order hold), or by interpolating between values.

In physical systems, this hybrid setup is almost always present. Control software sends discrete commands to hardware (say, the output of a PID controller) which are then processed by a DAC (digital-to-analog converter) and applied to the plant through analog signals. The hardware might hold a voltage constant, ramp it, or apply some analog shaping. On the sensing side, continuous signals are sampled via ADCs before reaching a digital controller. So in practice, even systems governed by continuous dynamics end up interfacing with the digital world through discrete-time approximations.

This raises a natural question: if everything eventually gets discretized anyway, why not just model everything in discrete time from the start?

In many cases, we do. But continuous-time models can still be useful, sometimes even necessary. They often make physical assumptions more explicit, connect more naturally to domain knowledge (e.g. differential equations in mechanics or thermodynamics), and expose invariances or conserved quantities that get obscured by time discretization. They also make it easier to model systems at different time scales, or to reason about how behaviors change as resolution increases. So while implementation happens in discrete time, thinking in continuous time can clarify the structure of the model.

It is helpful to see how both representations look in mathematical form. The state-space equations are nearly identical with different notations depending on how time is represented.

Discrete time

Having defined state as the summary we carry forward, a step of prediction applies the chosen input and advances the state.

$$\mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t), \quad \mathbf{y}_t = h_t(\mathbf{x}_t, \mathbf{u}_t).$$

Continuous time

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{y}(t) = h(\mathbf{x}(t), \mathbf{u}(t)).$$

The dot denotes a derivative with respect to real time; everything else (state, control, observation) remains the same.

When the functions f and h are linear we obtain

Linearity is not a belief about the world, it is a modeling choice that trades fidelity for transparency and speed.

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \quad \mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}.$$

The matrices A, B, C, D may vary with t . Readers with an ML background will recognise the parallel with recurrent neural networks: the state is the hidden vector, the control the input, and the output the read-out layer.

Classical control often moves to the frequency domain, using Laplace and Z-transforms to turn differential and difference equations into algebraic ones. That is invaluable for stability analysis of linear time-invariant systems, but the time-domain state-space view is more flexible for learning and simulation, so we will keep our primary focus there.

3.1.5 Examples of Deterministic Dynamics: HVAC Control

Consider a building in Montréal in the middle of February. Outside it is -20°C , and a thermostat tries to keep the interior comfortable. When the indoor temperature drops below your setpoint, the heating system kicks in. That system (a small building, a heater, the surrounding weather) can be modeled mathematically.

We start with a very simple approximation: treat the entire room as a single “thermal mass,” like a big air-filled box that heats up or cools down depending on how much heat flows in or out.

Let $\mathbf{x}(t)$ be the indoor air temperature at time t , and $\mathbf{u}(t)$ be the heating power supplied by the HVAC system. The outside air temperature, denoted $\mathbf{d}(t)$, affects the system too, acting as a known disturbance. Then the rate of change of indoor temperature is:

$$\dot{\mathbf{x}}(t) = -\frac{1}{RC}\mathbf{x}(t) + \frac{1}{RC}\mathbf{d}(t) + \frac{1}{C}\mathbf{u}(t). \quad (3)$$

Here:

- R is a thermal resistance: how well the walls insulate.
- C is a thermal capacitance: how much energy it takes to heat the air.

This is a **continuous-time linear system**, and we can write it in standard state-space form:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{E}\mathbf{d}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{x}(t), \quad (4)$$

with:

- $\mathbf{x}(t)$: indoor air temperature (the state)
- $\mathbf{u}(t)$: heater input (the control)
- $\mathbf{d}(t)$: outdoor temperature (disturbance)
- $\mathbf{y}(t)$: observed indoor temperature (output)
- $\mathbf{A} = -\frac{1}{RC}$
- $\mathbf{B} = \frac{1}{C}$

- $\mathbf{E} = \frac{1}{RC}$
- $\mathbf{C} = 1$

This model is simple, but too simplistic. It ignores the fact that the walls themselves store heat and release it slowly. This kind of delay is called **thermal inertia**: even if you turn the heater off, the walls might continue to warm the room for a while.

Before reading on, try to guess: if we want to capture the fact that walls store heat, what new variable should we add to the state? How would heat flow between the air and this new variable?

To capture this effect, we need to expand our state to include the wall temperature. We now model two coupled thermal masses: one for the air, and one for the wall. Heat can flow from the heater into the air, from the air into the wall, and from the wall out to the environment. This gives a more realistic description of how heat moves through a building envelope.

We write down an energy balance for each mass:

- For the air:

$$C_{\text{air}} \frac{dT_{\text{in}}}{dt} = \frac{T_{\text{wall}} - T_{\text{in}}}{R_{\text{ia}}} + u(t), \quad (5)$$

- For the wall:

$$C_{\text{wall}} \frac{dT_{\text{wall}}}{dt} = \frac{T_{\text{out}} - T_{\text{wall}}}{R_{\text{wo}}} - \frac{T_{\text{wall}} - T_{\text{in}}}{R_{\text{ia}}}. \quad (6)$$

Each term on the right-hand side corresponds to a flow of heat: the air gains heat from the wall and the heater, and the wall exchanges heat with both the air and the outside.

Now define the state vector:

$$\mathbf{x}(t) = \begin{bmatrix} T_{\text{in}}(t) \\ T_{\text{wall}}(t) \end{bmatrix}, \quad \mathbf{u}(t) = u(t), \quad \mathbf{d}(t) = T_{\text{out}}(t). \quad (7)$$

Dividing both equations by their respective capacitances and rearranging terms, we arrive at the coupled system:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{E}\mathbf{d}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{x}(t), \quad (8)$$

with:

$$\mathbf{A} = \begin{bmatrix} -\frac{1}{R_{\text{ia}}C_{\text{air}}} & \frac{1}{R_{\text{ia}}C_{\text{air}}} \\ \frac{1}{R_{\text{ia}}C_{\text{wall}}} & -\left(\frac{1}{R_{\text{ia}}} + \frac{1}{R_{\text{wo}}}\right) \frac{1}{C_{\text{wall}}} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \frac{1}{C_{\text{air}}} \\ 0 \end{bmatrix}, \quad \mathbf{E} = \begin{bmatrix} 0 \\ \frac{1}{R_{\text{wo}}C_{\text{wall}}} \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}. \quad (9)$$

Each entry in $\mathbf{A}$ has a physical interpretation:

- A_{11} : heat loss from the air to the wall
- A_{12} : heat gain by the air from the wall
- A_{21} : heat gain by the wall from the air
- A_{22} : net loss from the wall to both the air and the outside

The temperatures are now dynamically coupled: any change in one affects the other. The wall acts as a buffer that absorbs and releases heat over time.

This is still a linear system, just with a 2D state. But already it behaves differently. The walls absorb and release heat, smoothing out fluctuations and slowing down the response of the system.

As we add more rooms, walls, or building elements, the system grows. Each new temperature adds a new state. The equations still have the same structure, and their sparsity follows the building layout. Nodes represent temperatures; edges encode how heat flows between them.

Control Abstraction Levels This network of states is what we control. What we mean by “control input” $\mathbf{u}(t)$ depends on both what we want to achieve and what we can implement in practice.

The most direct interpretation is to let $\mathbf{u}(t)$ represent the actual heating power delivered to the system, measured in watts. This makes sense when modeling from physical principles or simulating a system with fine-grained actuation.

In many real buildings, however, thermostats do not issue power commands. They activate a relay, turning the heater on or off based on whether the measured temperature crosses a setpoint. Some systems allow for modulated control—such as varying fan speed or partially opening a valve—but those details are often hidden behind firmware or closed controllers.

A common implementation involves a **PID control loop** that compares the measured temperature to a setpoint and adjusts the control signal accordingly. While the actual logic might be simple, the resulting behavior appears smoothed or delayed from the perspective of the building.

Depending on the abstraction level, we might:

- Treat $\mathbf{u}(t)$ as continuous power input, if designing the full control logic.
- Use it as a setpoint input, assuming a lower-level controller handles the rest.
- Or reduce it to a binary signal—heater on or off—when working with logged behavior from a smart thermostat.

Each perspective shapes the kind of model we build and the kind of control problem we pose. If we are designing a controller from scratch, it may be worth modeling the full closed-loop dynamics. If the goal is to tune setpoints or learn policies from data, a coarser abstraction may be both sufficient and more robust.

Advantages of Physics-Based Models At this point, one might ask: why build this kind of physics-based model at all? If we can log indoor temperatures, thermostat actions, and weather data, we could instead learn a model directly from data. A neural ODE, for example, would let us define a parameterized function:

$$\dot{\mathbf{z}}(t) = f_{\boldsymbol{\theta}}(\mathbf{z}(t), \mathbf{u}(t), \mathbf{d}(t)), \quad \mathbf{y}(t) = g_{\boldsymbol{\theta}}(\mathbf{z}(t)), \quad (10)$$

with both $f_{\boldsymbol{\theta}}$ and $g_{\boldsymbol{\theta}}$ learned from data. The internal state $\mathbf{z}(t)$ is not tied to any physical quantity. It just needs to be expressive enough to explain the observations.

That flexibility can be useful, particularly when a large dataset is already available. But in building control and energy modeling, the constraints are usually different.

Often, the engineer or consultant on site is working under tight time and information budgets. A floor plan might be available, along with some basic specs on insulation or window types, and a few days of logged sensor data. The task might be to simulate load under different weather scenarios, tune a controller, or just help understand why a room is slow to heat up. The model has to be built quickly, adapted easily, and remain understandable to others working on the same system.

In that context, RC models are often the default choice: not because they are inherently better, but because they fit the workflow.

The parameters of an RC model correspond to quantities one can reason about: thermal resistance, capacitance, heat transfer between zones. Values can be cross-checked against architectural plans or adjusted manually when something does not line up. It is straightforward to tell which wall or zone is contributing to slow recovery times.

RC models can often be calibrated from short data traces, even when not all state variables are directly observable. The structure already imposes constraints: heat flows from hot to cold, dynamics are passive, responses are smooth. Those properties help narrow the space of valid parameter settings. A neural ODE, in contrast, typically needs more data to settle into stable and plausible dynamics, especially if no additional constraints are enforced during training.

Once built, an RC model is straightforward to modify. If a window is replaced, or a wall gets insulated, only a few numbers need updating. The model is easy to pass along to another engineer or embed in a larger simulation. A model like

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu} + \mathbf{Ed} \quad (11)$$

is linear and low-dimensional. Simulating it is cheap, even if you do it many times. That may not matter now, but it will matter later, when we want to optimize over trajectories or learn from them.

RC models are not always sufficient. They simplify or ignore many effects: solar gains, occupancy, nonlinearities, humidity, equipment switching behavior.

If those effects are significant, and you have enough data, a black-box model (neural ODE or otherwise) might achieve lower prediction error. In practice, it is common to combine the two: use the RC structure as a backbone, and learn a residual model to correct for unmodeled dynamics.

Questions to consider: What changes if the building has multiple zones with different setpoints? How would you model a heat pump that can both heat and cool? If the thermostat only records on/off events, can you still identify the thermal parameters?

Simulating the 1R1C Model Given a dynamics model, the next step is often simulation: computing a trajectory from an initial condition under a given input schedule. For a continuous-time ODE like the 1R1C model, this means numerical integration.

The simplest approach is the **forward Euler method**: approximate the derivative by a finite difference and step forward in time.

```
import numpy as np
import matplotlib.pyplot as plt

# 1R1C building model parameters
R = 2.0    # thermal resistance (°C/kW)
C = 10.0   # thermal capacitance (kWh/°C)
tau = R * C # time constant (hours)

# Simulation parameters
dt = 0.1   # time step (hours)
T = 24     # total simulation time (hours)
n_steps = int(T / dt)

# Initial condition and inputs
T_in = np.zeros(n_steps + 1)
T_in[0] = 18.0 # initial indoor temperature (°C)

# Outdoor temperature: sinusoidal with daily cycle
t = np.linspace(0, T, n_steps + 1)
T_out = -10 + 5 * np.sin(2 * np.pi * t / 24 - np.pi/2) # cold winter day

# Heating power: constant 2 kW
Q_heat = 2.0 * np.ones(n_steps)

# Forward Euler integration
for k in range(n_steps):
    dT_dt = (1/(R*C)) * (T_out[k] - T_in[k]) + (1/C) * Q_heat[k]
    T_in[k+1] = T_in[k] + dt * dT_dt

# Plot results
```

```

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 5), sharex=True)
ax1.plot(t, T_in, label='Indoor', linewidth=2)
ax1.plot(t, T_out, ' - -', label='Outdoor', linewidth=1.5)
ax1.set_ylabel('Temperature (°C)')
ax1.legend()
ax1.set_title(f'1R1C Building Model ( \tau = {tau:.0f} hours)')
ax1.grid(True, alpha=0.3)

ax2.step(t[:-1], Q_heat, where='post', linewidth=2)
ax2.set_xlabel('Time (hours)')
ax2.set_ylabel('Heating power (kW)')
ax2.set_ylim(0, 3)
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

The time constant $\tau = RC = 20$ hours means the building responds slowly to changes. Even with constant heating, it takes several hours for the indoor temperature to stabilize. This slow response is exactly what enables load-shifting strategies: pre-heat when electricity is cheap, and coast through expensive periods.

3.1.6 From Deterministic to Stochastic

The models considered so far were deterministic: given an initial state and input sequence, the system evolves in a fixed, predictable way. But real systems rarely behave so neatly. Sensors are noisy. Parameters drift. The world changes in ways we cannot fully model.

To account for this uncertainty, we move from deterministic dynamics to **stochastic models**. There are two equivalent but conceptually distinct ways to do this.

Function plus Noise The most direct extension adds a noise term to the dynamics:

$$\mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t), \quad \mathbf{w}_t \sim p_{\mathbf{w}}. \quad (12)$$

If the noise is additive and Gaussian, we recover the standard linear-Gaussian setup used in Kalman filtering:

$$\mathbf{x}_{t+1} = A\mathbf{x}_t + B\mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, Q). \quad (13)$$

We are not restricted to Gaussian or additive noise. For instance, if the noise distribution is non-Gaussian:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t, \quad \mathbf{w}_t \sim \text{Laplace, or Student-t}, \quad (14)$$

then $\mathbf{x}_{t+1}$ inherits those properties. This is known as a **convolution model**: the next-state distribution is a shifted version of the noise distribution, centered around the deterministic prediction. More formally, this is a special case of a **pushforward measure**: the randomness from $\mathbf{w}_t$ is “pushed forward” through the function f to yield a distribution over outcomes.

Or the noise might enter multiplicatively:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) + \Gamma(\mathbf{x}_t, \mathbf{u}_t)\mathbf{w}_t, \quad (15)$$

where Γ is a matrix that modulates the effect of the noise, potentially depending on state and control. If Γ is invertible, we can even write down an explicit density via a change-of-variables:

$$p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t) = p_{\mathbf{w}}(\Gamma^{-1}(\mathbf{x}_t, \mathbf{u}_t)[\mathbf{x}_{t+1} - f(\mathbf{x}_t, \mathbf{u}_t)]) \cdot |\det \Gamma^{-1}|. \quad (16)$$

This kind of structured noise is common in practice, for example, when disturbances are amplified at certain operating points.

The **function-plus-noise** view is natural when we have a physical or simulator-based model and want to account for uncertainty around it. It is **constructive**: we know how the system evolves and how the randomness enters. This means we can **track the source of variability along a trajectory**, which is particularly useful for techniques like **reparameterization** or **infinitesimal perturbation analysis (IPA)**. These methods rely on being able to differentiate through the noise injection mechanism, something that is much easier when the noise is explicit and structured.

Consider: if all we can do is sample next states from a simulator—without knowing the internal noise source—can we still write down a dynamics model? What form would it take?

Transition Kernel The second perspective skips over the internal noise and defines the system directly in terms of the probability distribution over next states:

$$p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t). \quad (17)$$

This **transition kernel** encodes all the uncertainty in the evolution of the system, without reference to any underlying noise source or functional form.

This view is strictly more general: it includes the function-plus-noise case as a special instance. If we do know the function f and the noise distribution $p_{\mathbf{w}}$ from the generative model, then the transition kernel is obtained by “pushing” the randomness through the function:

$$p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t) = \int \delta(\mathbf{x}_{t+1} - f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w})) p_{\mathbf{w}}(\mathbf{w}) d\mathbf{w}. \quad (18)$$

This may look abstract, but it is just marginalization: for each possible noise value $\mathbf{w}$, we compute the resulting next state, and then average over all possible $\mathbf{w}$, weighted by how likely each one is.

If the noise were discrete, this becomes a sum:

$$p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t) = \sum_{i=1}^k 1\{f(\mathbf{x}_t, \mathbf{u}_t, w_i) = \mathbf{x}_{t+1}\} \cdot p_i \quad (19)$$

This abstraction is especially useful when we do not know (or do not care about) the underlying function or noise distribution. All we need is the ability to sample transitions or estimate their likelihoods. This is the default formulation in reinforcement learning, econometrics, and other settings focused on behavior rather than mechanism.

To summarize: we have two views of stochastic dynamics. The **function-plus-noise** view is constructive; it tells us exactly how randomness enters and allows differentiation through the noise. The **transition kernel** view is more abstract but more general; it only requires that we can sample or evaluate likelihoods. Both describe the same object; the choice depends on what we know and what we need to compute.

Continuous-Time Analogue In continuous time, the stochastic dynamics of a system are often described using a **stochastic differential equation (SDE)**:

$$d\mathbf{X}_t = f(\mathbf{X}_t, \mathbf{U}_t) dt + \sigma(\mathbf{X}_t, \mathbf{U}_t) d\mathbf{W}_t, \quad (20)$$

where $\mathbf{W}_t$ is Brownian motion. The first term, called the **drift**, describes the average motion of the system. The second, scaled by σ , models how random fluctuations (diffusion) enter over time. Just like in discrete time, this is a **function + noise** model: the state evolves through a deterministic path perturbed by stochastic input.

This generative view again induces a probability distribution over future states. At any future time $t + \Delta t$, the system does not land at a single state but is described by a distribution that depends on the initial condition and the noise along the way.

Mathematically, this distribution evolves according to the **Fokker–Planck equation**, a partial differential equation that governs how probability density “flows” through time. It plays the same role here as the transition kernel did in discrete time: describing how likely the system is to be in any given state, without referring to the noise directly.

While the mathematical generalization is clean, working with continuous-time stochastic models can be more challenging. Simulating sample paths is often straightforward (eg. nowadays diffusion models in generative AI), but writing down or computing the exact transition distribution usually is not. For this reason, many practical methods still rely on discrete-time approximations, even when the underlying system is continuous.

Example: Managing a Québec Hydroelectric Reservoir On the James Bay plateau, 1 400 km north of Montréal, the Robert-Bourassa reservoir stores roughly 62 km³ of water, more than the volume of Lake Ontario

above its minimum operating level. Sixteen giant turbines sit 140 m below the surface, converting that stored head into 5.6 GW of electricity, about a fifth of Hydro-Québec’s total capacity. A steady share of that output feeds Québec’s aluminium smelters, which depend on stable, uninterrupted power.

Water managers face competing objectives:

- **Flood safety.** Sudden snowmelt or storms can overflow the basin, forcing emergency spillways to open. These events are spectacular, but carry real downstream risk and economic cost.
- **Energy reliability.** If the level falls too low, turbines sit idle and contracts go unmet. Voltage dips at the smelters are measured in lost millions.

A basic deterministic model for the mass balance of the reservoir is straightforward bookkeeping:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{r}_t - \mathbf{u}_t, \quad (21)$$

where $\mathbf{x}_t$ is the current reservoir level, $\mathbf{u}_t$ is the controlled outflow through turbines, and $\mathbf{r}_t$ is the natural inflow from rainfall and upstream runoff.

But inflow is variable, and its statistical structure matters. Two hydrological regimes dominate:

- In spring, melting snow over days can produce a long-tailed inflow distribution, often modeled as log-normal or Gamma.
- In summer, convective storms yield a skewed mixture: a point mass at zero (no rain), and a thin but heavy tail capturing sudden bursts.

This motivates a simple stochastic extension:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \begin{cases} 0 & \text{with prob. } p_0, \\ \text{LogNormal}(\mu, \sigma^2) & \text{with prob. } 1 - p_0. \end{cases} \quad (22)$$

Here the physics is fixed, and all uncertainty sits in the inflow term $\mathbf{w}_t$. Rather than fitting a full transition model from $(\mathbf{x}_t, \mathbf{u}_t)$ to $\mathbf{x}_{t+1}$, we can isolate the inflow by rearranging the mass balance:

$$\hat{\mathbf{w}}_t = \mathbf{x}_{t+1} - \mathbf{x}_t + \mathbf{u}_t. \quad (23)$$

This gives a direct estimate of the realized inflow at each timestep. From there, the problem becomes one of density estimation: fit a probabilistic model to the residuals $\hat{\mathbf{w}}_t$. In spring, this might be a log-normal distribution. In summer, a two-part mixture: a point mass at zero, and an exponential tail. These distributions can be estimated by maximum likelihood, or adjusted using additional features (covariates) such as upstream snowpack or forecasted temperature.

This setup has practical benefits. Fixing the physical part of the model (how levels respond to inflow and outflow) helps focus the statistical modeling effort. Rather than fitting a full system model, we only need to estimate the variability in inflows. This reduces the number of degrees of freedom and makes the estimation problem easier to interpret. It also avoids conflating uncertainty in inflow with uncertainty in the response of the system.

To get a sense of scale: the Robert-Bourassa reservoir has a usable storage range of roughly 20 km^3 . Weekly inflows during spring freshet might average 1.5 km^3 with a standard deviation of 0.4 km^3 . A manager planning turbine releases might simulate 1000 inflow scenarios over a 52-week horizon to estimate the probability of overflow or shortage under different release policies. Each scenario is just a draw from the fitted inflow distribution, pushed through the deterministic mass balance.

Compare this to a more generic approach, such as linear regression:

$$\mathbf{x}_{t+1} = a\mathbf{x}_t + b\mathbf{u}_t + \varepsilon_t. \quad (24)$$

This is straightforward to fit, but offers no guarantee that the result behaves sensibly. The model might violate conservation of mass, or compensate for inflow variation by adjusting coefficients a and b . This can lead to misleading conclusions, especially when extrapolating beyond the training data.

Questions to consider: How would you incorporate weather forecasts into the inflow model? What if the reservoir is part of a cascade, where releases from one dam become inflows to the next? How would you handle the fact that inflow distributions differ by season?

3.1.7 Partial Observability

So far, we have assumed that the full system state $\mathbf{x}_t$ is available. But in most real-world settings, only a partial or noisy observation is accessible. Sensors have limited coverage, measurements come with noise, and some variables are not observable at all.

To model this, we introduce an **observation equation** alongside the system dynamics:

$$\begin{aligned} \mathbf{x}_{t+1} &= f_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t), \quad \mathbf{w}_t \sim p_{\mathbf{w}}, \\ \mathbf{y}_t &= h_t(\mathbf{x}_t, \mathbf{v}_t), \quad \mathbf{v}_t \sim p_{\mathbf{v}}. \end{aligned} \quad (25)$$

The state $\mathbf{x}_t$ evolves under control inputs $\mathbf{u}_t$ and process noise $\mathbf{w}_t$, but we do not observe $\mathbf{x}_t$ directly. Instead, we observe $\mathbf{y}_t$, which depends on $\mathbf{x}_t$ through some possibly nonlinear, noisy function h_t . The noise $\mathbf{v}_t$ captures measurement uncertainty.

This setup defines a partially observed system. Even if the underlying dynamics are known, we still face uncertainty due to limited visibility into the true state. The controller or estimator must rely on the observations $\mathbf{y}_{0:t}$ to make sense of the hidden trajectory.

In the **deterministic** case, if the output map h_t is full-rank and invertible, we may be able to reconstruct the state directly from the output: no filtering required. But once noise is introduced, that invertibility becomes more subtle: even if h_t is bijective, the presence of $\mathbf{v}_t$ prevents us from recovering $\mathbf{x}_t$ exactly. In this case, we must shift from inversion to estimation, often via probabilistic inference.

In the **linear-Gaussian case**, the model simplifies to:

$$\begin{aligned}\mathbf{x}_{t+1} &= A\mathbf{x}_t + B\mathbf{u}_t + \mathbf{w}_t, & \mathbf{w}_t &\sim \mathcal{N}(0, Q), \\ \mathbf{y}_t &= C\mathbf{x}_t + D\mathbf{u}_t + \mathbf{v}_t, & \mathbf{v}_t &\sim \mathcal{N}(0, R).\end{aligned}\tag{26}$$

This is the classical state-space model used in signal processing and control. The model is fully specified by the system matrices and the covariances Q and R . The state is no longer known, but under these assumptions it can be estimated recursively using the **Kalman filter**, which maintains a Gaussian belief over $\mathbf{x}_t$.

Even when the model is nonlinear or non-Gaussian, the structure remains the same: a dynamic state evolves, and a separate observation process links it to the data we see. Many modern estimation techniques, including extended and unscented Kalman filters, particle filters, and learned neural estimators, build on this core structure.

Observation Kernel View Just as we moved from function-based dynamics to transition kernels, we can abstract away the noise source and define the **observation distribution** directly:

$$p(\mathbf{y}_t \mid \mathbf{x}_t).\tag{27}$$

This kernel summarizes what the sensors tell us about the hidden state. If we know the generative model—say, that $\mathbf{y}_t = h_t(\mathbf{x}_t) + \mathbf{v}_t$ with known $p_{\mathbf{v}}$ —then this kernel is induced by marginalizing out $\mathbf{v}_t$:

$$p(\mathbf{y}_t \mid \mathbf{x}_t) = \int \delta(\mathbf{y}_t - h_t(\mathbf{x}_t, \mathbf{v})) p_{\mathbf{v}}(\mathbf{v}) d\mathbf{v}.\tag{28}$$

But we do not have to start from the generative form. In practice, we might define or learn $p(\mathbf{y}_t \mid \mathbf{x}_t)$ directly, especially when dealing with black-box sensors, perception models, or abstract measurement processes.

Example – Stabilizing a Telescope’s Vision with Adaptive Optics On Earth, even the largest telescopes cannot see perfectly. As starlight travels through the atmosphere, tiny air pockets with different temperatures bend the light in slightly different directions. The result is a distorted image: instead of a sharp point, a star looks like a flickering blob. The distortion happens fast, on the order of milliseconds, and changes continuously as wind moves the turbulent layers overhead.

Adaptive optics (AO) systems cancel out these distortions in real time by measuring how the incoming wavefront of light is distorted and using a flexible

mirror to apply a counter-distortion that straightens it back out. The difficulty is that the wavefront cannot be observed directly. The sensor provides only noisy measurements of its **slopes** (the angles of tilt at various points), and the controller must act fast, before the atmosphere changes again.

To design a controller here, we need a model of how the distortions evolve. And that means building a decision-making model: one that includes uncertainty, partial observability, and fast feedback.

The main object to track is the distortion of the incoming wavefront. This phase field $\phi(\mathbf{r}, t)$ cannot be observed directly, but we can represent it approximately using a finite basis (e.g., Fourier or Zernike). The coefficients of this expansion form our internal state:

$$\mathbf{x}_t \in R^n \quad (\text{wavefront distortion at time } t). \quad (29)$$

A typical system might use $n = 100$ to 500 Zernike modes, sampled at 1–2 kHz. The state dimension is modest, but the control loop must complete in under a millisecond to keep up with the atmosphere.

The atmosphere evolves in time. A simple but effective model assumes the turbulence is “frozen” and just blown across the telescope by the wind. That gives us a discrete-time linear model:

$$\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{w}_t, \quad (30)$$

where $\mathbf{A}$ shifts the distortion pattern in space, and $\mathbf{w}_t$ is a small random change from evolving turbulence. This noise is not arbitrary: its statistics follow a power law derived from Kolmogorov’s turbulence model. In particular, higher spatial frequencies (small-scale wiggles) have less energy than low ones. This allows us to build a prior on how likely different distortions are.

The full wavefront is not directly observable. Instead, we use a wavefront sensor: a camera that captures how the light bends. What it actually measures are local slopes, the gradients of the wavefront, not the wavefront itself. So our observation model is:

$$\mathbf{y}_t = \mathbf{C}\mathbf{x}_t + \boldsymbol{\varepsilon}_t, \quad (31)$$

where $\mathbf{C}$ is a known matrix that maps wavefront distortion to measurable slope angles, and $\boldsymbol{\varepsilon}_t$ is measurement noise (e.g., due to photon limits).

The control objective is to flatten the wavefront using a deformable mirror. The mirror can apply a small counter-distortion $\mathbf{u}_t$ that subtracts from the atmospheric one:

$$\text{Residual state: } \mathbf{x}_t^{\text{res}} = \mathbf{x}_t - \mathbf{B}\mathbf{u}_t. \quad (32)$$

The goal is to choose $\mathbf{u}_t$ to minimize the residual distortion by making the light flat again.

Without a model, we would just react to the current noisy measurements. With a model, we can predict how the wavefront will evolve, filter out noise, and act preemptively. This is essential in AO, where decisions must be made

every millisecond. Kalman filters are often used to track the hidden state $\mathbf{x}_t$, combining model predictions with noisy measurements, and linear-quadratic regulators (LQR) or other optimal controllers use those estimates to choose the best correction.

Adaptive optics is a rare case where continuous-time modeling also plays a role. The true evolution of the turbulence is continuous, and we can model it using a stochastic differential equation (SDE):

$$d\mathbf{x}(t) = \mathbf{F}\mathbf{x}(t) dt + \mathbf{G} d\mathbf{W}(t), \quad (33)$$

where $\mathbf{W}(t)$ is Brownian motion and the matrix $\mathbf{G}$ encodes the Kolmogorov spectrum. Discretizing this equation gives us the $\mathbf{A}$ and $\mathbf{Q}$ matrices for the discrete-time model above.

Questions to consider: What happens if the wind speed changes suddenly—does the frozen-flow assumption break down? How might you adapt the model online as conditions change? If the guide star is faint and photon noise dominates, how does that affect the observation model?

3.1.8 Programs as Models

Consider again the HVAC example from earlier in this chapter. We wrote down a simple RC model: $\dot{T} = \frac{1}{RC}(T_{\text{out}} - T) + \frac{1}{C}Q$. Given the parameters R and C , we can compute the temperature at any future time by solving the differential equation. We have direct access to the dynamics function itself.

Now imagine instead that we are given only a software tool—say, EnergyPlus—that simulates the building. We can feed it an initial temperature and a schedule of heating commands, press “run,” and observe the resulting temperature trajectory. But we cannot inspect the internal equations. We cannot take a derivative of the output with respect to a parameter. We can only query the simulator as a black box.

This distinction matters for control. Many algorithms assume we can evaluate $f(\mathbf{x}, \mathbf{u})$ at arbitrary points, or even differentiate through it. When the model is a program rather than an equation, these operations are not always available, and we must adapt our methods accordingly.

Up to this point, we have described models using systems of equations—either differential or difference equations—that express how a system evolves over time. These **analytical models** define the transition structure explicitly. For instance, in discrete time, the evolution of the state is governed by a known function:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \quad (34)$$

Given access to f , we can construct trajectories, analyze system behavior, and design control policies. The important feature here is not that the model evolves one step at a time, but that we are given the **local dynamics function** f itself.

In contrast, **simulation-based models** do not expose f directly. Instead, they define a procedure—implemented in code—that takes an initial state and input sequence and returns the resulting trajectory:

$$\{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T\} = \mathcal{S}(\mathbf{x}_0, \{\mathbf{u}_t\}_{t=0}^{T-1}) \quad (35)$$

Here, $\mathcal{S}$ represents the full simulator. Internally, it may apply numerical integration, scheduling logic, branching rules, or other computations. But these details are encapsulated. From the outside, we can only query the simulator by running it.

This distinction is subtle but important. Both types of models can generate trajectories. What matters is the **interface**: analytical models provide direct access to f ; simulation models do not. They offer a trajectory-generation interface, but hide the internal structure that produces it.

Before reading on, think about which category the examples from earlier in this chapter fall into. Is the RC thermal model analytical or simulation-based? What about the reservoir model? The adaptive optics system?

Case Study: Robotics with MuJoCo MuJoCo simulates the dynamics of articulated rigid bodies under contact constraints. The equations it solves include:

$$\begin{aligned} M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) &= \boldsymbol{\tau} + J^\top \boldsymbol{\lambda} \\ \phi(\mathbf{q}) &= 0, \quad \boldsymbol{\lambda} \geq 0, \quad \boldsymbol{\lambda}^\top \boldsymbol{\phi} = 0 \end{aligned}$$

Here $\mathbf{q}$ are joint positions, M is the mass matrix, and $\boldsymbol{\lambda}$ are contact forces enforcing non-penetration. But these physical equations are part of a larger simulator that also includes collision detection, contact force models, sensor and actuator emulation, and visual rendering.

The full behavior of a robot interacting with its environment emerges only when the simulator is executed. While the underlying physics are well-understood, the complexity of contact dynamics, collision detection, and sensor modeling makes it impractical to expose the local dynamics function f directly.

Questions to consider: If you can only query MuJoCo as a black box, how would you estimate the gradient of a cost function with respect to the control inputs? What changes if you have access to the simulator’s source code?

Systems with Discrete Events Many simulation models arise when a system’s dynamics are driven not by time-continuous evolution, but by the occurrence of events. These **discrete-event systems** (DES) change state only at specific, often asynchronous points in time. Between events, the state remains fixed.

A discrete-event system can be described by:

- a set of discrete states $\mathcal{X}$,
- a set of events $\mathcal{E}$,
- a transition function $f : \mathcal{X} \times \mathcal{E} \rightarrow \mathcal{X}$,

- and a time-advance function $t_a : \mathcal{X} \rightarrow R_{\geq 0}$.

At each point, the system checks which events are enabled and advances to the next scheduled one.

Consider a software-defined networking (SDN) controller managing traffic routing in a data center. The system must make real-time decisions about packet forwarding paths based on network conditions and service requirements.

The discrete states $\mathcal{X}$ represent the current network configuration: active routing tables, link utilization levels, and quality-of-service priority queues at each switch.

The events $\mathcal{E}$ include new flow requests arriving (video streaming, database queries, file transfers), link failures or congestion threshold violations, flow completion notifications, load balancing triggers when servers exceed capacity, and network policy updates from administrators.

The transition function f captures how routing decisions change the network state. When a high-priority video conference flow arrives while a link is congested, the controller might transition to a new state where low-priority background traffic is rerouted through alternative paths.

The time-advance function t_a determines when the next routing decision occurs. Flow arrivals follow traffic patterns (bursty during business hours), while link failures are rare but unpredictable events.

Between events, packets follow the established routing rules; the same forwarding tables remain active across all switches. The control problem here is to adapt routing decisions to discrete network events, balancing throughput, latency, and reliability constraints.

Questions to consider: How would you define a “state” for this system in a way that is Markovian? What information would you need to predict the next event time?

Hybrid Systems Some systems evolve continuously most of the time but undergo discrete jumps in response to certain conditions. These **hybrid systems** are common in control applications.

The system consists of:

- a set of discrete modes $q \in \mathcal{Q}$,
- continuous dynamics in each mode: $\dot{\mathbf{x}} = f_q(\mathbf{x})$,
- guards that specify when transitions between modes occur,
- and reset maps that update the state during such transitions.

An HVAC system can be in one of several modes: **heating**, **cooling**, or **off**. The temperature evolves continuously according to physical laws, but when it crosses certain thresholds, the system switches modes:


```

if x < setpoint - delta:
    mode = "heating"
elif x > setpoint + delta:
    mode = "cooling"
else:
    mode = "off"

```

Within each mode, a different differential equation applies. This results in a piecewise-smooth trajectory with mode-dependent dynamics.

Case Study: Building Energy with EnergyPlus EnergyPlus provides a detailed example of hybrid systems in building energy simulation. At its core are physical equations describing heat flows:

$$C_i \frac{dT_i}{dt} = \sum_j h_{ij} A_{ij} (T_j - T_i) + Q_i \quad (36)$$

It also solves implicit equations representing HVAC component behavior:

$$0 = f(T, \dot{m}, P) \quad (37)$$

But the actual simulator includes hundreds of thousands of lines of code handling interpolated weather data, occupancy schedules, equipment performance curves, and control logic implemented as finite-state machines.

The result is a program that emulates how a building behaves over time, given environmental inputs and schedules. The hybrid nature emerges from the interaction between continuous thermal dynamics and discrete control decisions made by thermostats, occupancy sensors, and HVAC equipment.

Questions to consider: How does EnergyPlus relate to the simple RC models we derived earlier? Could you use an RC model as a surrogate for optimization, then validate in EnergyPlus?

Agent-Based Models Some simulation models do not describe systems via global state transitions, but instead simulate the behavior of many individual components or **agents**, each following local rules. These **agent-based models** (ABMs) are widely used in epidemiology, ecology, and social modeling.

Each agent maintains its own internal state and acts according to probabilistic or rule-based logic. The system's behavior arises from the interactions among agents.

Consider a neighborhood where each household is an agent making energy consumption decisions based on real-time electricity pricing and thermal comfort preferences. Each household agent has internal state (current temperature, HVAC settings, comfort preferences, price sensitivity), local decision rules (MPC algorithms that optimize the trade-off between energy cost and thermal comfort), and unique characteristics (different utility functions, thermal mass, occupancy patterns).

The simulator might execute something like:

```

for household in neighborhood:
    # Each household solves its own MPC optimization
    current_price = utility.get_current_price()
    comfort_weight = household.comfort_preference

    # Optimize over prediction horizon
    optimal_setpoint = household.mpc_controller.optimize(
        current_temp=household.temperature,
        price_forecast=utility.price_forecast,
        comfort_weight=comfort_weight
    )

    household.set_hvac_setpoint(optimal_setpoint)

    # Update shared grid load
    neighborhood.total_demand += household.power_consumption

```

The macro-level demand patterns—peak shifting, load leveling, rebound effects—emerge from individual household optimization decisions. No single equation describes the neighborhood’s energy consumption; it arises from the collective behavior of autonomous agents each solving their own control problems.

Case Study: Traffic Simulation with SUMO SUMO is an agent-based traffic simulator used in transportation research. Each vehicle is an agent with its own route, driving behavior, and decision-making logic. The Krauss car-following rule shows how individual vehicle agents behave:

```

def update_vehicle(v, v_leader, gap, dt):
    v_safe = v_leader + (gap - min_gap) / tau
    v_desired = min(v_max, v_safe)
    \epsilon = random.uniform(0, 1)
    v_new = max(0, v_desired - \epsilon * a_max * dt)
    x_new = x + v_new * dt
    return x_new, v_new

```

Beyond car-following, each vehicle agent also plans routes through the network based on travel time estimates, responds to traffic signals and road conditions, makes lane-changing decisions based on utility functions, and exhibits individual driving characteristics (aggressiveness, reaction time).

The emergent traffic patterns—congestion formation, traffic waves, bottlenecks—arise from the collective behavior of thousands of individual vehicle agents, each following local rules and making autonomous decisions.

Questions to consider: In an agent-based model, what plays the role of the “state”? Is it the collection of all agent states? How would you define a policy that controls traffic signals based on this high-dimensional state?

To summarize the taxonomy so far: we have moved from analytical models (where the dynamics function f is given explicitly) to simulation-based models (where we can only query trajectories). Within simulation-based models, we distinguished three patterns: discrete-event systems, where the state changes only at specific events; hybrid systems, where continuous dynamics interact with discrete mode switches; and agent-based models, where macro-level behavior emerges from local agent interactions.

This distinction will matter throughout the book. When we have access to the dynamics function f , we can use gradient-based trajectory optimization (Chapter 3) or derive closed-form solutions like the Riccati equation for LQR. When we only have simulator access, we must rely on sampling-based methods: Monte Carlo estimation, reinforcement learning, and derivative-free optimization. The model interface shapes the algorithms we can apply.

The following case study illustrates how these ideas combine in practice.

Case Study: Curbside Access at Montréal–Trudeau Airport Afternoon traffic at Montréal–Trudeau airport regularly backs up along the two-lane ramp leading to the departures curb. As passenger volumes rebound, the mix of private drop-offs, taxis, and shuttles converging in a confined space produces frequent delays. When curb dwell times rise, especially around wide-body departures, queues can spill back onto the access road and interfere with other flows on the airport campus.

To manage the situation, the airport operator relies on a dense sensor network. Cameras and license plate readers track vehicle trajectories across virtual gates, generating a real-time stream of entry points, curb interactions, and exit times. According to public statements, AI-based forecasting solutions have been deployed to anticipate congestion and suggest alternative routing options for passengers and drivers. While no technical details have been disclosed, this is a typical instance of a traffic prediction and control problem that lends itself to agent-based modeling.

In such a model, each vehicle is treated as an individual agent with internal state:

$$s_t = (\text{lane}, x_t, v_t, \text{intent}), \quad (38)$$

where x_t and v_t denote longitudinal position and speed, and lane and intent capture higher-level behavioral traits. The simulation proceeds in discrete time. At each step, agents update their acceleration based on local traffic density (e.g., via a car-following model like IDM), evaluate potential lane changes (e.g., using a utility or incentive rule), and advance position accordingly.

The layout of the ramp—its geometry, merges, and constraints—is fixed. What changes are the traffic patterns and driver behaviors. These can be estimated from historical trajectories and updated as new data arrives. In a real-time setting, a filtering step adjusts the simulation so that its predicted flows remain consistent with current observations.

While the behavior of each individual agent is governed by program logic and heuristics—such as car-following rules, desired speeds, or gap acceptance—some parameters are identified offline from historical data, while others are estimated online. This adaptation helps the model track observed conditions. But even with such adjustments, not all effects are easily captured.

Construction activity, weather disturbances, and irregular flight scheduling can introduce sudden shifts in flow that lie outside the scope of the structural model. To account for these, one can overlay a data-driven correction on top of the simulation. Suppose the simulator produces a queue length forecast q_{t+h}^{sim} over horizon h . A statistical model can be trained to predict the residual between this forecast and the observed outcome:

$$r_{t+h} = q_{t+h}^{\text{obs}} - q_{t+h}^{\text{sim}}, \quad (39)$$

as a function of exogenous features z_t , such as weather, incident flags, or scheduled arrivals. The final forecast then becomes:

$$\hat{q}_{t+h} = q_{t+h}^{\text{sim}} + \phi(z_t), \quad (40)$$

where ϕ is a learned mapping from external conditions to the expected correction. The result is a hybrid model: the simulation enforces physical structure and agent-level behavior, while the residual model compensates for aspects of the system that are harder to express analytically.

Questions to consider: What are the tradeoffs between using a purely data-driven model versus a simulation with residual correction? How would you decide how much structure to build into the simulation versus how much to learn from data?

3.1.9 Summary

This chapter introduced the modeling vocabulary we use throughout the book.

A **dynamics model** describes how a system evolves in response to inputs we control and disturbances we cannot. The **state-space representation** provides a common language: the state $\mathbf{x}$ is a compact summary of the past sufficient to predict the future given the current input. This representation works for both discrete-time systems (difference equations) and continuous-time systems (differential equations), and for both linear and nonlinear dynamics.

The transition from **deterministic to stochastic models** adds uncertainty. We saw two equivalent views: the **function-plus-noise** perspective, which is constructive and allows differentiation through the randomness, and the **transition kernel** perspective, which is more abstract but requires only the ability to sample or evaluate likelihoods. The choice depends on what we know about the system and what we need to compute.

Partial observability adds another layer: the state is hidden, and we only see noisy projections through an observation model. This structure—hidden

state, stochastic dynamics, noisy observations—is the foundation for state estimation (Kalman filtering, particle filtering) and for decision-making under uncertainty.

Beyond analytical models, we also considered **simulation-based models** where the dynamics function is not exposed directly. Instead, a program takes an initial state and input sequence and returns the resulting trajectory. This includes discrete-event systems (where state changes only at specific events), hybrid systems (where continuous dynamics interact with discrete mode switches), and agent-based models (where macro-level behavior emerges from local agent interactions). These simulation-based approaches are common in robotics, building energy, and traffic modeling.

Throughout, we emphasized that models for control need not be accurate in every detail. What matters is whether they capture the structure relevant to the decisions being made. A simple two-parameter thermal model can guide effective load-shifting strategies; a physics-based reservoir model with fitted inflow distributions can support robust planning; a linear model of atmospheric turbulence can enable real-time wavefront correction. The model is a tool for decision-making, not an end in itself.

There is no universally correct way to model a system. The choice depends on what we know, what we can observe, what we care about, and what tools we have. In every case, modeling choices define the structure of the problem and the space of possible solutions. The next chapter turns to numerical trajectory optimization: how to compute optimal trajectories given a dynamics model.

Exercises

Basic

1. Write the 1R1C thermal model $\dot{T} = \frac{1}{RC}(T_{\text{out}} - T) + \frac{1}{C}Q$ in standard state-space form $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{E}\mathbf{d}$. Identify the matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{E}$, and $\mathbf{C}$ (for the output equation $\mathbf{y} = \mathbf{C}\mathbf{x}$).
2. A robot arm has angular position θ and angular velocity $\omega = \dot{\theta}$. The motor applies a torque u , and friction opposes motion proportionally to velocity. Write down a plausible continuous-time state-space model. What is the state vector? What are the system matrices if the model is linear?
3. Convert the continuous-time system $\dot{x} = -ax + bu$ to discrete time using forward Euler with step size Δt . What are the discrete-time system matrices A_d and B_d ?

Conceptual

[resume] Give an example of a system where partial observability matters for control. What is the hidden state? What is observed? Why is it not possible to simply ignore the hidden variables? Explain the

difference between the function-plus-noise view and the transition kernel view of stochastic dynamics. When might you prefer one over the other? The reservoir model assumes that the physics (mass balance) is known exactly, and only the inflows are uncertain. What are the advantages of this structured approach compared to fitting a generic autoregressive model? What are the risks if the physics assumption is wrong? Explain the difference between an analytical model and a simulation-based model. Give an example of a system where you would have access to f directly, and an example where you would only have access to a simulator. A thermostat-controlled HVAC system switches between heating, cooling, and off modes based on temperature thresholds. Is this a discrete-event system, a hybrid system, or an agent-based model? Justify your answer.

Integrative

[resume]Extend the 2R2C building model to include a third thermal mass representing furniture and interior objects. Write down the new state vector and the 3×3 matrix $\mathbf{A}$. Which entries are zero, and why? Consider a drone flying in a plane. Its state includes position (x, y) and velocity (v_x, v_y) . The control inputs are accelerations (a_x, a_y) . Wind adds a random velocity disturbance. Write down a discrete-time stochastic dynamics model. Is this a function-plus-noise model? What would the transition kernel look like? Consider a simple agent-based traffic model with N vehicles on a single-lane road. Each vehicle i has position x_i and velocity v_i , and follows the car in front according to $v_i^+ = \min(v_{\max}, v_{i-1}, (x_{i-1} - x_i - d_{\min})/\tau)$. What is the state space of this system? If you wanted to control a traffic light at one point on the road, how would you formulate the control problem?

Computational

[resume]Modify the 1R1C simulation code to implement a simple thermostat controller: turn the heater on (2 kW) when $T_{\text{in}} < 20^\circ\text{C}$ and off when $T_{\text{in}} > 21^\circ\text{C}$. Simulate 48 hours and plot the results. How does the behavior differ from constant heating? Implement the 2R2C model and simulate it with the same outdoor temperature profile. Use parameters $R_{\text{ia}} = 1$, $R_{\text{wo}} = 3$, $C_{\text{air}} = 2$, $C_{\text{wall}} = 20$ (all in consistent units). Compare the response to the 1R1C model. How does the wall temperature lag behind the air temperature? Implement a simple hybrid system: a bouncing ball with state (h, v) (height and velocity). The ball follows $\dot{h} = v$, $\dot{v} = -g$ until $h = 0$, at which point $v \leftarrow -e \cdot v$ for some coefficient of restitution $e \in (0, 1)$. Simulate 10 seconds with $g = 9.8$, $e = 0.8$, and initial conditions $h_0 = 1$, $v_0 = 0$. Plot the trajectory and observe the mode switches.

3.1.10 Self-checks

4 Numerical Trajectory Optimization

4.1 Discrete-Time Trajectory Optimization

In the previous chapter, we examined different ways to represent dynamical systems: continuous versus discrete time, deterministic versus stochastic, fully versus partially observable, and even simulation-based views such as agent-based or programmatic models. Our focus was on the **structure of models**: how they capture evolution, uncertainty, and information.

In this chapter, we turn to what makes these models useful for **decision-making**. The goal is no longer just to describe how a system behaves, but to leverage that description to **compute actions over time**. This doesn't mean the model prescribes actions on its own. Rather, it provides the scaffolding for optimization: given a model and an objective, we can derive the control inputs that make the modeled system behave well according to a chosen criterion.

Our entry point will be trajectory optimization. By a **trajectory**, we mean the time-indexed sequence of states and controls that the system follows under a plan: the states $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ together with the controls $(\mathbf{u}_1, \dots, \mathbf{u}_{T-1})$. In this chapter, we focus on an **open-loop** viewpoint: starting from a known initial state, we compute the entire sequence of controls in advance and then apply it as-is. This is appealing because, for discrete-time problems, it yields a finite-dimensional optimization over a vector of decisions and cleanly exposes the structure of the constraints. In continuous time, the base formulation is infinite-dimensional; in this course we will rely on direct methods (time discretization and parameterization) to transform it into a finite-dimensional nonlinear program.

Open loop also has a clear limitation: if reality deviates from the model, whether due to disturbances, model mismatch, or unanticipated events, the state you actually reach may differ from the predicted one. The precomputed controls that were optimal for the nominal trajectory can then lead you further off course, and errors can compound over time.

Later, we will study **closed-loop (feedback)** strategies, where the choice of action at time t can depend on the state observed at time t . Instead of a single sequence, we optimize a policy π_t mapping states to controls, $\mathbf{u}_t = \pi_t(\mathbf{x}_t)$. Feedback makes plans resilient to unforeseen situations by adapting on the fly, but it leads to a more challenging problem class. We start with open-loop trajectory optimization to build core concepts and tools before tackling feedback design.

Learning Goals

After studying this chapter, you should be able to:

1. Formulate a discrete-time optimal control problem (DOCP) in Bolza, Lagrange, and Mayer forms, and convert between them.
2. State the KKT conditions for a constrained NLP and explain the role of constraint qualifications.

3. Derive the discrete-time Pontryagin principle from the Lagrangian of a DOCP.
4. Implement single shooting and multiple shooting methods for trajectory optimization.
5. Explain the trade-offs between simultaneous (direct transcription) and sequential (shooting) methods.
6. Compute gradients of the objective with respect to controls using the adjoint (costate) recursion.

Prerequisites

This chapter assumes familiarity with:

- Multivariable calculus (gradients, Jacobians, chain rule)
- Basic optimization (unconstrained minimization, gradient descent)
- Linear algebra (matrix-vector products, linear systems)
- Dynamical systems from the previous chapter

A Motivating Example: Inventory Control Before diving into the general formulation, consider a concrete problem that arises in operations management. A retailer must decide how much inventory to order at the start of each period to meet uncertain demand while balancing holding costs against stockout penalties.

The setup is simple. At the beginning of period k , the retailer observes the current inventory level x_k and places an order of size $u_k \geq 0$. Demand d_k then arrives (assume for now it is known in advance), and the inventory evolves according to

$$x_{k+1} = x_k + u_k - d_k. \quad (41)$$

If inventory is positive at the end of the period, holding costs accrue. If inventory is negative (backorders), penalty costs apply. There may also be a per-unit ordering cost. A typical objective is to minimize total cost over a planning horizon of T periods:

$$\min_{u_0, \dots, u_{T-1}} \sum_{k=0}^{T-1} (h [x_k]_+ + p [-x_k]_+ + c u_k), \quad (42)$$

where $[z]_+ = \max(0, z)$, h is the holding cost rate, p is the backorder penalty rate, and c is the ordering cost per unit. The retailer may also face constraints: orders cannot be negative, and inventory might be bounded above by warehouse capacity.

This problem has a clear structure: a state (x_k) that evolves according to a known rule, a control (u_k) that influences the next state, a cost that accumulates over time, and constraints on the decisions. These ingredients define a **discrete-time optimal control problem** (DOCP). We now formalize this structure in general terms.

Discrete-Time Optimal Control Problems (DOCPs) Consider a system described by a **state** $\mathbf{x}_t \in R^n$, summarizing everything needed to predict its evolution. At each stage t , we can influence the system through a **control input** $\mathbf{u}_t \in R^m$. The dynamics specify how the state evolves:

$$\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t), \quad (43)$$

where $\mathbf{f}_t$ may be nonlinear or time-varying. We assume the initial state $\mathbf{x}_1$ is known.

The goal is to pick a sequence of controls $\mathbf{u}_1, \dots, \mathbf{u}_{T-1}$ that makes the trajectory desirable. But desirable in what sense? That depends on an **objective function**, which often includes two components:

$$(i) \text{ stage cost: } c_t(\mathbf{x}_t, \mathbf{u}_t), \quad (ii) \text{ terminal cost: } c_T(\mathbf{x}_T). \quad (44)$$

The stage cost reflects ongoing penalties such as energy, delay, or risk. The terminal cost measures the value (or cost) of ending in a particular state. Together, these give a discrete-time Bolza problem with path constraints and bounds:

$$\begin{aligned} \text{minimize} \quad & c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\ \text{subject to} \quad & \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \\ & \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t) \leq \mathbf{0} \\ & \mathbf{x}_{\min} \leq \mathbf{x}_t \leq \mathbf{x}_{\max} \\ & \mathbf{u}_{\min} \leq \mathbf{u}_t \leq \mathbf{u}_{\max} \\ \text{given} \quad & \mathbf{x}_1 = \mathbf{x}_0. \end{aligned} \quad (45)$$

Returning to the inventory example, the correspondence is direct: the state $\mathbf{x}_t$ is the inventory level x_k , the control $\mathbf{u}_t$ is the order quantity u_k , and the dynamics $\mathbf{f}_t$ encode the inventory balance equation $x_{k+1} = x_k + u_k - d_k$. The stage cost c_t captures holding, backorder, and ordering costs, while the terminal cost c_T might penalize ending with excess stock. Constraints $\mathbf{g}_t \leq \mathbf{0}$ can enforce non-negativity of orders or warehouse capacity limits. This mapping shows how practical problems fit naturally into the abstract framework.

Written this way, it may seem obvious that the decision variables are the controls $\mathbf{u}_t$. After all, in most intuitive descriptions of control, we think of choosing inputs to influence the system. But notice that in the program above, the entire state trajectory also appears as a set of variables, linked to the controls by the dynamics constraints. This is intentional: it reflects one way of writing the problem that makes the constraints explicit.

Why introduce $\mathbf{x}_t$ as decision variables if they can be simulated forward from the controls? Many readers hesitate here, and the question is natural: *If the model is deterministic and $\mathbf{x}_1$ is known, why not pick $\mathbf{u}_{1:T-1}$ and compute $\mathbf{x}_{2:T}$ on the fly?* That instinct leads to **single shooting**, a method we will return to shortly.

Already in this formulation, though, **the structure of the problem matters**. Ignoring it can make our life much harder. The reason is twofold:

- **Dimensionality grows with the horizon.** For a horizon of length T , the program has roughly $(T - 1)(m + n)$ decision variables.
- **Temporal coupling.** Each control affects all future states and costs. The feasible set is not a simple box but a narrow manifold defined by the dynamics.

Together, these features explain why specialized methods exist and why the way we write the problem influences the algorithms we can use. Whether we keep states explicit or eliminate them through forward simulation determines not just the problem size, but also its conditioning and the trade-offs between robustness and computational effort.

Existence of Solutions and Optimality Conditions Now that we have the optimization problem written down, we can ask: does it always have a solution? And if so, how do we recognize one? These questions lead us to feasibility and optimality conditions.

Existence of Solutions Notice first that nothing in the problem statement required the dynamics

$$\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \quad (46)$$

to be stable. In fact, many problems of interest involve unstable systems; think of balancing a pole or steering a spacecraft. What matters is that the dynamics are **well defined**: given a state-control pair, the rule $\mathbf{f}_t$ produces a valid next state.

In continuous time, one usually requires $\mathbf{f}$ to be continuous (often Lipschitz continuous) in $\mathbf{x}$ so that the ODE has a unique solution on the horizon of interest. In discrete time, the requirement is lighter: we only need the update map to be well posed.

Existence also hinges on **feasibility**. A candidate control sequence must generate a trajectory that respects all constraints: the dynamics, any bounds

on state and control, and any terminal requirements. If no such sequence exists, the feasible set is empty and the problem has no solution. This can happen if the constraints are overly strict, or if the system is uncontrollable from the given initial condition.

Optimality Conditions Assume the feasible set is nonempty. To characterize a point that is not only feasible but **locally optimal**, we use the Lagrange multiplier machinery from nonlinear programming. For a smooth problem

$$\begin{aligned} \min_{\mathbf{z}} \quad & F(\mathbf{z}) \\ \text{s.t.} \quad & G(\mathbf{z}) = \mathbf{0}, \\ & H(\mathbf{z}) \geq \mathbf{0}, \end{aligned} \tag{47}$$

define the **Lagrangian**

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = F(\mathbf{z}) + \boldsymbol{\lambda}^\top G(\mathbf{z}) + \boldsymbol{\mu}^\top H(\mathbf{z}), \quad \boldsymbol{\mu} \geq \mathbf{0}. \tag{48}$$

For an inequality system $H(\mathbf{z}) \geq \mathbf{0}$ and a candidate point $\mathbf{z}$, the **active set** is

$$\mathcal{A}(\mathbf{z}) = \{ i : H_i(\mathbf{z}) = 0 \}, \tag{49}$$

while indices with $H_i(\mathbf{z}) > 0$ are **inactive**. Only active inequalities can carry positive multipliers.

We now make a **constraint qualification** assumption. In plain language, it says the constraints near the solution intersect in a regular way so that the feasible set has a well-defined tangent space and the multipliers exist. Algebraically, this amounts to a **full row rank** condition on the Jacobian of the equalities together with the active inequalities:

$$\text{rows of } [\nabla G(\mathbf{z}^*); \nabla H_{\mathcal{A}}(\mathbf{z}^*)] \text{ are linearly independent.} \tag{50}$$

This is the **LICQ** (Linear Independence Constraint Qualification). In convex problems, **Slater's condition** (existence of a strictly feasible point) plays a similar role. You can think of these as the assumptions that let the linearized KKT equations be solvable; we do not literally invert that Jacobian, but the full-rank property is what would make such an inversion possible in principle.

Under such a constraint qualification, any local minimizer $\mathbf{z}^*$ admits multipliers $(\boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ that satisfy the **Karush–Kuhn–Tucker (KKT) conditions**:

$$\begin{aligned} \text{stationarity:} \quad & \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) = \mathbf{0}, \\ \text{primal feasibility:} \quad & G(\mathbf{z}^*) = \mathbf{0}, \quad H(\mathbf{z}^*) \geq \mathbf{0}, \\ \text{dual feasibility:} \quad & \boldsymbol{\mu}^* \geq \mathbf{0}, \\ \text{complementarity:} \quad & \mu_i^* H_i(\mathbf{z}^*) = 0 \quad \text{for all } i. \end{aligned} \tag{51}$$

Only constraints that are **active** at $\mathbf{z}^*$ can have $\mu_i^* > 0$; inactive ones have $\mu_i^* = 0$. The multipliers quantify marginal costs: λ_j^* measures how the optimal

value changes if the j -th equality is relaxed, and μ_i^* does the same for the i -th inequality. (If you prefer $h(\mathbf{z}) \leq 0$, signs flip accordingly.)

In our trajectory problems, $\mathbf{z}$ stacks state and control trajectories, G enforces the dynamics, and H collects bounds and path constraints. The equalities' multipliers act as **costates** or **shadow prices** for the dynamics. Writing the KKT system stage by stage yields the discrete-time Pontryagin principle, derived next. For convex programs these conditions are also sufficient.

What fails without a CQ? If the active gradients are dependent (for example duplicated or nearly parallel), the Jacobian loses rank; multipliers may then be nonunique or fail to exist, and the linearized equations become ill-posed. In transcribed trajectory problems this shows up as dependent dynamic constraints or redundant path constraints, which leads to fragile solver behavior.

Recap. The KKT conditions provide necessary conditions for a point to be a local minimizer of a constrained optimization problem. They consist of four parts: stationarity (the gradient of the Lagrangian vanishes), primal feasibility (constraints are satisfied), dual feasibility (inequality multipliers are nonnegative), and complementarity (inactive constraints have zero multipliers). The multipliers have an economic interpretation as marginal costs—they tell us how much the optimal value would change if we relaxed a constraint slightly. For convex problems, the KKT conditions are also sufficient, meaning any point satisfying them is globally optimal. In trajectory optimization, these conditions will reappear in structured form as the Pontryagin principle.

From KKT to algorithms The KKT system can be read as the first-order optimality conditions of a **saddle-point** problem. With equalities $G(\mathbf{z}) = \mathbf{0}$ and inequalities $H(\mathbf{z}) \geq \mathbf{0}$, define the Lagrangian

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = F(\mathbf{z}) + \boldsymbol{\lambda}^\top G(\mathbf{z}) + \boldsymbol{\mu}^\top H(\mathbf{z}), \quad \boldsymbol{\mu} \geq \mathbf{0}. \quad (52)$$

Optimality corresponds to a saddle: minimize in $\mathbf{z}$, maximize in $(\boldsymbol{\lambda}, \boldsymbol{\mu})$ (with $\boldsymbol{\mu}$ constrained to the nonnegative orthant).

Primal–dual gradient dynamics (Arrow–Hurwicz) The simplest algorithm mirrors this saddle structure by descending in the primal variables and ascending in the dual variables, with a projection for the inequalities:

$$\begin{aligned} \mathbf{z}^{k+1} &= \mathbf{z}^k - \alpha_k (\nabla F(\mathbf{z}^k) + \nabla G(\mathbf{z}^k)^\top \boldsymbol{\lambda}^k + \nabla H(\mathbf{z}^k)^\top \boldsymbol{\mu}^k), \\ \boldsymbol{\lambda}^{k+1} &= \boldsymbol{\lambda}^k + \beta_k G(\mathbf{z}^k), \\ \boldsymbol{\mu}^{k+1} &= \Pi_{\geq 0}(\boldsymbol{\mu}^k + \beta_k H(\mathbf{z}^k)). \end{aligned} \quad (53)$$

Here $\Pi_{\geq 0}$ is the projection onto $\{\boldsymbol{\mu} \geq 0\}$. In convex settings and with suitable step sizes, these iterates converge to a saddle point. In nonconvex problems (our trajectory optimizations after transcription), these updates are often used inside **augmented Lagrangian** or **penalty** frameworks to improve robustness, for example by replacing $\mathcal{L}$ with

$$\mathcal{L}_\rho(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = \mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}, \boldsymbol{\mu}) + \frac{\rho}{2} \|G(\mathbf{z})\|^2 + \frac{\rho}{2} \|\min\{0, H(\mathbf{z})\}\|^2, \quad (54)$$

which stabilizes the dual ascent when constraints are not yet well satisfied.

SQP as Newton on the KKT system (equality case) With only equality constraints $G(\mathbf{z}) = \mathbf{0}$, write first-order conditions

$$\nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}, \boldsymbol{\lambda}) = \mathbf{0}, \quad G(\mathbf{z}) = \mathbf{0}, \quad \text{where } \mathcal{L} = F + \boldsymbol{\lambda}^\top G. \quad (55)$$

Applying Newton's method to this system gives the linear KKT solve

$$\begin{bmatrix} \nabla_{\mathbf{z}\mathbf{z}}^2 \mathcal{L}(\mathbf{z}^k, \boldsymbol{\lambda}^k) & \nabla G(\mathbf{z}^k)^\top \\ \nabla G(\mathbf{z}^k) & 0 \end{bmatrix} \begin{bmatrix} \Delta \mathbf{z} \\ \Delta \boldsymbol{\lambda} \end{bmatrix} = - \begin{bmatrix} \nabla_{\mathbf{z}} \mathcal{L}(\mathbf{z}^k, \boldsymbol{\lambda}^k) \\ G(\mathbf{z}^k) \end{bmatrix}. \quad (56)$$

This is exactly the step computed by **Sequential Quadratic Programming (SQP)** in the equality-constrained case: it is Newton's method on the KKT equations. For general problems with inequalities, SQP forms a **quadratic subproblem** by quadratically modeling F with $\nabla_{\mathbf{z}\mathbf{z}}^2 \mathcal{L}$ and linearizing the constraints, then solves that QP with line search or trust region. In least-squares-like problems one often uses **Gauss–Newton** (or a Levenberg–Marquardt trust region) as a positive-definite approximation to the Lagrangian Hessian.

In trajectory optimization, the KKT matrix inherits banded/sparse structure from the dynamics. Newton/SQP steps can be computed efficiently by exploiting this structure; in the special case of quadratic models and linearized dynamics, the QP reduces to an LQR solve along the horizon (this is the backbone of iLQR/DDP-style methods). Primal-dual updates provide simpler iterations and are easy to implement; augmented terms are typically needed to obtain stable progress when constraints couple stages.

The choice between methods depends on the context. Primal-dual gradients give lightweight iterations and are suited for warm starts or as inner loops with penalties. SQP/Newton gives rapid local convergence when close to a solution and LICQ holds; trust regions or line search help globalize convergence.

Examples of DOCPs To make things concrete, here are additional problems that are naturally posed as discrete-time OCPs. We already introduced inventory control at the start of this chapter; below are two more examples from different domains, followed by a discussion of how continuous-time problems give rise to DOCPs through discretization.

End-of-Day Portfolio Rebalancing At each trading day k , choose trades u_k to adjust holdings h_k before next-day returns r_k realize. Deterministic planning uses predicted returns μ_k , with dynamics $h_{k+1} = (h_k + u_k) \odot (\mathbf{1} + \mu_k)$ and budget/box constraints. The stage cost can capture transaction costs and risk, e.g., $c_k(h_k, u_k) = \tau \|u_k\|_1 + \frac{\lambda}{2} h_k^\top \Sigma_k h_k$, with a terminal utility or wealth objective.

Daily Ad-Budget Allocation with Carryover Allocate spend $u_k \in [0, U_{\max}]$ to build awareness s_k with carryover dynamics $s_{k+1} = \alpha s_k + \beta u_k$. Conversions/revenue at day k follow a response curve $g(s_k, u_k)$; the goal is $\max \sum_{k=0}^{T-1} g(s_k, u_k) - c u_k$ subject to spend limits. This is naturally discrete because decisions and measurements occur daily.

DOCPs Arising from the Discretization of Continuous-Time OCPs Although many applications are natively discrete-time, it is also common to obtain a DOCP by discretizing a continuous-time formulation. Consider a system on $[0, T_c]$ given by

$$\dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(0) = \mathbf{x}_0. \quad (57)$$

Choose a step size $\Delta > 0$ and grid $t_k = k \Delta$. A one-step integration scheme induces a discrete map $\mathbf{F}_\Delta$ so that

$$\mathbf{x}_{k+1} = \mathbf{F}_\Delta(\mathbf{x}_k, \mathbf{u}_k, t_k), \quad k = 0, \dots, T-1, \quad (58)$$

where, for example, explicit Euler gives $\mathbf{F}_\Delta(\mathbf{x}, \mathbf{u}, t) = \mathbf{x} + \Delta \mathbf{f}(t, \mathbf{x}, \mathbf{u})$. The resulting discrete-time optimal control problem takes the Bolza form with these induced dynamics:

$$\begin{aligned} \min_{\{\mathbf{x}_k, \mathbf{u}_k\}} \quad & c_T(\mathbf{x}_T) + \sum_{k=0}^{T-1} c_k(\mathbf{x}_k, \mathbf{u}_k) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} - \mathbf{F}_\Delta(\mathbf{x}_k, \mathbf{u}_k, t_k) = 0, \quad k = 0, \dots, T-1, \\ & \mathbf{x}_0 = \mathbf{x}_{\text{init}}. \end{aligned} \quad (59)$$

Programs as DOCPs and Differentiable Programming It is often useful to view a computer program itself as a discrete-time dynamical system. Let the **program state** collect memory, buffers, and intermediate variables, and let the **control** represent inputs or tunable decisions at each step. A single execution step defines a transition map

$$\mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k), \quad (60)$$

and a scalar objective (e.g., loss, error, runtime, energy) yields a DOCP:

$$\min_{\{\mathbf{u}_k\}} c_T(\mathbf{x}_T) + \sum_{k=0}^{T-1} c_k(\mathbf{x}_k, \mathbf{u}_k) \quad \text{s.t.} \quad \mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k). \quad (61)$$

In differentiable programming (e.g., JAX, PyTorch), the composed map $\Phi_{T-1} \circ \dots \circ \Phi_0$ is differentiable, enabling reverse-mode automatic differentiation and efficient gradient-based trajectory optimization. When parts of the program are non-differentiable (discrete branches, simulators with events), DOCPs can still be solved using derivative-free or weak-gradient methods (eg. finite

differences, SPSA, Nelder–Mead, CMA-ES, or evolutionary strategies) optionally combined with smoothing, relaxations, or stochastic estimators to navigate non-smooth regions.

Example: Retry Loop Optimization Consider a simple retry loop that attempts an operation up to K times, waiting u_k seconds before the k -th attempt. The server has time-varying availability: it is unreliable early on but recovers after a transient outage. The goal is to find a wait schedule that minimizes expected latency while ensuring eventual success.

We cast this as a DOCP. The state $x_k = (t, k, \text{done})$ tracks elapsed time, attempt count, and success status. The control $u_k \in [0, 2]$ is the wait before attempt k . The transition applies the wait, then flips a biased coin based on current server availability. The cost penalizes total time and failure.

The optimized schedule learns to wait longer before early attempts, giving the server time to recover from the outage. This simple example illustrates how viewing a program as a dynamical system enables trajectory optimization even when the transition involves stochastic branching.

Example: Gradient Descent with Momentum as DOCP To connect this lens to familiar practice, including hyperparameter optimization, treat the learning rate and momentum (or their schedules) as controls. Rather than fixing them a priori, we can optimize them as part of a trajectory optimization. The optimizer itself becomes the dynamical system whose execution we shape to minimize final loss.

Program: gradient descent with momentum on a quadratic loss. We fit $\theta \in R^p$ to data $(\mathbf{A}, \mathbf{b})$ by minimizing

$$\ell(\theta) = \frac{1}{2} \|\mathbf{A}\theta - \mathbf{b}\|_2^2. \quad (62)$$

The program maintains parameters θ_k and momentum $\mathbf{m}_k$. Each iteration does:

1. compute gradient $\mathbf{g}_k = \nabla_{\theta} \ell(\theta_k) = \mathbf{A}^\top (\mathbf{A}\theta_k - \mathbf{b})$
2. update momentum $\mathbf{m}_{k+1} = \beta_k \mathbf{m}_k + \mathbf{g}_k$
3. update parameters $\theta_{k+1} = \theta_k - \alpha_k \mathbf{m}_{k+1}$

State, control, and transition. Define the state $\mathbf{x}_k = \begin{bmatrix} \theta_k \\ \mathbf{m}_k \end{bmatrix} \in R^{2p}$ and the control $\mathbf{u}_k = \begin{bmatrix} \alpha_k \\ \beta_k \end{bmatrix}$. One program step is

$$\Phi_k(\mathbf{x}_k, \mathbf{u}_k) = \begin{bmatrix} \theta_k - \alpha_k (\beta_k \mathbf{m}_k + \mathbf{A}^\top (\mathbf{A}\theta_k - \mathbf{b})) \\ \beta_k \mathbf{m}_k + \mathbf{A}^\top (\mathbf{A}\theta_k - \mathbf{b}) \end{bmatrix}. \quad (63)$$

Executing the program for T iterations gives the trajectory

$$\mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, T-1, \quad \mathbf{x}_0 = \begin{bmatrix} \boldsymbol{\theta}_0 \\ \mathbf{m}_0 \end{bmatrix}. \quad (64)$$

Objective as a DOCP. Choose terminal cost $c_T(\mathbf{x}_T) = \ell(\boldsymbol{\theta}_T)$ and (optionally) stage costs $c_k(\mathbf{x}_k, \mathbf{u}_k) = \rho_\alpha \alpha_k^2 + \rho_\beta (\beta_k - \bar{\beta})^2$. The program-as-control problem is

$$\min_{\{\alpha_k, \beta_k\}} \ell(\boldsymbol{\theta}_T) + \sum_{k=0}^{T-1} (\rho_\alpha \alpha_k^2 + \rho_\beta (\beta_k - \bar{\beta})^2) \quad \text{s.t.} \quad \mathbf{x}_{k+1} = \Phi_k(\mathbf{x}_k, \mathbf{u}_k). \quad (65)$$

Backpropagation = reverse-time costate recursion. Because Φ_k is differentiable, reverse-mode AD computes $\nabla_{\mathbf{u}_0:T-1} (c_T + \sum c_k)$ by propagating a costate $\boldsymbol{\lambda}_k = \partial \mathcal{J} / \partial \mathbf{x}_k$ backward:

$$\boldsymbol{\lambda}_T = \nabla_{\mathbf{x}_T} c_T, \quad \boldsymbol{\lambda}_k = \nabla_{\mathbf{x}_k} c_k + (\nabla_{\mathbf{x}_k} \Phi_k)^\top \boldsymbol{\lambda}_{k+1}, \quad (66)$$

and the gradients with respect to controls are

$$\nabla_{\mathbf{u}_k} \mathcal{J} = \nabla_{\mathbf{u}_k} c_k + (\nabla_{\mathbf{u}_k} \Phi_k)^\top \boldsymbol{\lambda}_{k+1}. \quad (67)$$

Unrolling a tiny horizon ($T = 3$) to see the composition:

$$\begin{aligned} \mathbf{x}_1 &= \Phi_0(\mathbf{x}_0, \mathbf{u}_0), \\ \mathbf{x}_2 &= \Phi_1(\mathbf{x}_1, \mathbf{u}_1), \\ \mathbf{x}_3 &= \Phi_2(\mathbf{x}_2, \mathbf{u}_2), \quad \mathcal{J} = c_T(\mathbf{x}_3) + \sum_{k=0}^2 c_k(\mathbf{x}_k, \mathbf{u}_k). \end{aligned} \quad (68)$$

What if the program branches? Suppose we insert a “skip-small-gradients” branch

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha_k \mathbf{m}_{k+1} \mathbf{1}\{\|\mathbf{g}_k\| > \tau\}, \quad (69)$$

which is non-differentiable because of the indicator. The DOCP view still applies, but gradients are unreliable. Two practical paths: smooth the branch (e.g., replace $\mathbf{1}\{\cdot\}$ with $\sigma((\|\mathbf{g}_k\| - \tau)/\epsilon)$ for small ϵ) and use autodiff; or go derivative-free on $\{\alpha_k, \beta_k, \tau\}$ (e.g., SPSA or CMA-ES) while keeping the inner dynamics exact.

Variants: Lagrange and Mayer Problems The Bolza form is general enough to cover most situations, but two common special cases deserve mention:

- **Lagrange problem (no terminal cost)** If the objective only accumulates stage costs:

$$\min_{\mathbf{u}_{1:T-1}} \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t). \quad (70)$$

Example: *Energy minimization for a delivery drone*. The concern is total battery use, regardless of the final position.

- **Mayer problem (terminal cost only)** If the objective depends only on the final state:

$$\min_{\mathbf{u}_{1:T-1}} c_T(\mathbf{x}_T). \quad (71)$$

Example: *Satellite orbital transfer*. The only goal is to reach a specified orbit, no matter the fuel spent along the way.

These distinctions matter when deriving optimality conditions, but conceptually they fit in the same framework: the system evolves over time, and we choose controls to shape the trajectory.

Reducing to Mayer Form by State Augmentation Although Bolza, Lagrange, and Mayer problems look different, they are equivalent in expressive power. Any problem with running costs can be rewritten as a Mayer problem (one whose objective depends only on the final state) through a simple trick: **augment the state with a running sum of costs**.

The idea is straightforward. Introduce a new variable, y_t , that keeps track of the cumulative cost so far. At each step, we update this running sum along with the system state:

$$\tilde{\mathbf{x}}_{t+1} = \begin{pmatrix} \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \\ y_t + c_t(\mathbf{x}_t, \mathbf{u}_t) \end{pmatrix}, \quad (72)$$

where $\tilde{\mathbf{x}}_t = (\mathbf{x}_t, y_t)$. The terminal cost then becomes:

$$\tilde{c}_T(\tilde{\mathbf{x}}_T) = c_T(\mathbf{x}_T) + y_T. \quad (73)$$

The overall effect is that the explicit sum $\sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t)$ disappears from the objective and is captured implicitly by the augmented state. This lets us write every optimal control problem in Mayer form.

This reduction serves two purposes. First, it often simplifies mathematical derivations, as we will see later when deriving necessary conditions. Second, it can streamline algorithmic implementation: instead of writing separate code paths for Mayer, Lagrange, and Bolza problems, we can reduce everything to one canonical form. That said, this unified approach is not always best in practice. Specialized formulations can sometimes be more efficient computationally, especially when the running cost has simple structure.

The unifying theme is that a DOCP may look like a generic NLP on paper, but its structure matters. Ignoring that structure often leads to impractical

solutions, whereas formulations that expose sparsity and respect temporal coupling allow modern solvers to scale effectively. In the following sections, we will examine how these choices play out in practice through single shooting, multiple shooting, and collocation methods, and why different formulations strike different trade-offs between robustness and computational effort.

4.1.1 Numerical Methods for Solving DOCPs

Before we discuss specific algorithms, it is useful to clarify the goal: we want to recast a discrete-time optimal control problem as a standard nonlinear program (NLP). Collect all decision variables (states, controls, and any auxiliary variables) into a single vector $\mathbf{z} \in R^{n_z}$ and write

$$\begin{aligned} \min_{\mathbf{z} \in R^{n_z}} \quad & F(\mathbf{z}) \\ \text{s.t.} \quad & G(\mathbf{z}) = 0, \\ & H(\mathbf{z}) \geq 0, \end{aligned} \tag{74}$$

with maps $F : R^{n_z} \rightarrow R$, $G : R^{n_z} \rightarrow R^{r_e}$, and $H : R^{n_z} \rightarrow R^{r_h}$. In optimal control, G typically encodes dynamics and boundary conditions, while H captures path and box constraints.

There are multiple ways to arrive at (and benefit from) this NLP:

- Simultaneous (direct transcription / full discretization): keep all states and controls as variables and impose the dynamics as equality constraints. This is straightforward and exposes sparsity, but the problem can be large unless solver-side techniques (e.g., condensing) are exploited.
- Sequential (recursive elimination / single shooting): eliminate states by forward propagation from the initial condition, leaving controls as the main decision variables. This reduces dimension and constraints, but can be sensitive to initialization and longer horizons.
- Multiple shooting: introduce state variables at segment boundaries and enforce continuity between simulated segments. This compromises between size and conditioning and is often more robust than pure single shooting.

The next sections work through these formulations, starting with simultaneous methods, then sequential methods, and finally multiple shooting, before discussing how generic NLP solvers and specialized algorithms leverage the resulting structure in practice.

Simultaneous Methods In the simultaneous (also called direct transcription or full discretization) approach, we keep the entire trajectory explicit and enforce the dynamics as equality constraints. Starting from the Bolza DOCP,

$$\min_{\{\mathbf{x}_t, \mathbf{u}_t\}} c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \quad \text{s.t.} \quad \mathbf{x}_{t+1} - \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) = 0, \quad t = 1, \dots, T-1, \quad (75)$$

collect all variables into a single vector

$$\mathbf{z} := [\mathbf{x}_1^\top \quad \dots \quad \mathbf{x}_T^\top \quad \mathbf{u}_1^\top \quad \dots \quad \mathbf{u}_{T-1}^\top]^\top \in R^{n_z}. \quad (76)$$

Path constraints typically apply only at selected times. Let E index additional equality constraints g_i and I index inequality constraints h_i . For each constraint i , define the set of time indices $K_i \subseteq \{1, \dots, T\}$ where it is enforced (e.g., terminal constraints use $K_i = \{T\}$). The simultaneous transcription is the NLP

$$\begin{aligned} \min_{\mathbf{z}} \quad & F(\mathbf{z}) := c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\ \text{s.t.} \quad & G(\mathbf{z}) = \begin{bmatrix} [g_i(\mathbf{x}_k, \mathbf{u}_k)]_{i \in E, k \in K_i} \\ [\mathbf{x}_{t+1} - \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)]_{t=1:T-1} \\ \mathbf{x}_1 - \mathbf{x}_{\text{init}} \end{bmatrix} = \mathbf{0}, \\ & H(\mathbf{z}) = [h_i(\mathbf{x}_k, \mathbf{u}_k)]_{i \in I, k \in K_i} \geq \mathbf{0}, \end{aligned} \quad (77)$$

optionally with simple bounds $\mathbf{x}_{\text{lb}} \leq \mathbf{x}_t \leq \mathbf{x}_{\text{ub}}$ and $\mathbf{u}_{\text{lb}} \leq \mathbf{u}_t \leq \mathbf{u}_{\text{ub}}$ folded into H or provided to the solver separately. For notational convenience, some constraints may not depend on $\mathbf{u}_k$ at times in K_i ; the indexing still helps specify when each condition is active.

This direct transcription is attractive because it is faithful to the model and exposes sparsity. The Jacobian of G has a block bi-diagonal structure induced by the dynamics, and the KKT matrix is sparse and structured. These properties are exploited by interior-point and SQP methods. The trade-off is size: with state dimension n and control dimension m , the decision vector has $(T \cdot n) + ((T-1) \cdot m)$ entries, and there are roughly $(T-1) \cdot n$ dynamic equalities plus any path and boundary conditions. Techniques such as partial or full condensing eliminate state variables to reduce the equality set (at the cost of denser matrices), while keeping states explicit preserves sparsity and often improves robustness on long horizons and in the presence of state constraints.

Compared to alternatives, simultaneous methods avoid the long nonlinear dependency chains of single shooting and make it easier to impose state/path constraints. They can, however, demand more memory and per-iteration linear algebra, so practical performance hinges on exploiting sparsity and good initialization.

The same logic applies when selecting an optimizer. For small-scale problems, it is common to rely on general-purpose routines such as those in `scipy.optimize.minimize`. Derivative-free methods like Nelder–Mead require no gradients but scale poorly as dimensionality increases. Quasi-Newton schemes such as BFGS work well for

moderate dimensions and can approximate gradients by finite differences, while large-scale trajectory optimization often calls for gradient-based constrained solvers such as interior-point or sequential quadratic programming methods that can exploit sparse Jacobians and benefit from automatic differentiation. Stochastic techniques, including genetic algorithms, simulated annealing, or particle swarm optimization, occasionally appear when gradients are unavailable, but their cost grows rapidly with dimension and they are rarely competitive for structured optimal control problems.

Interactive Eco-Cruise Visualization The precomputed browser demo below compares an energy-aware trajectory with a naive speed profile. Use its controls to inspect position, velocity, acceleration, stage cost, and cumulative energy without rerunning the optimizer.

[Open the Eco-Cruise visualization full-screen.](#)

Sequential Methods The previous section showed how a discrete-time optimal control problem can be solved by treating all states and controls as decision variables and enforcing the dynamics as equality constraints. This produces a nonlinear program that can be passed to solvers such as `scipy.optimize.minimize` with the SLSQP method. For short horizons, this approach is straightforward and works well; the code stays close to the mathematical formulation.

It also has a real advantage: by keeping the states explicit and imposing the dynamics through constraints, we anchor the trajectory at multiple points. This extra structure helps stabilize the optimization, especially for long horizons where small deviations in early steps can otherwise propagate and cause the optimizer to drift or diverge. In that sense, this formulation is better conditioned and more robust than approaches that treat the dynamics implicitly.

The drawback is scale. As the horizon grows, the number of variables and constraints grows with it, and all are coupled by the dynamics. Each iteration of a sequential quadratic programming (SQP) or interior-point method requires building and factorizing large Jacobians and Hessians. These methods have been embedded in reinforcement learning and differentiable programming pipelines, through implicit layers or differentiable convex solvers, but the cost is significant. They remain serial, rely on repeated linear algebra factorizations, and are difficult to parallelize efficiently. When thousands of such problems must be solved inside a learning loop, the overhead becomes prohibitive.

This motivates an alternative that aligns better with the computational model of machine learning. If the dynamics are deterministic and state constraints are absent (or reducible to simple bounds on controls), we can eliminate the equality constraints altogether by making the states implicit. Instead of solving for both states and controls, we fix the initial state and roll the system forward under a candidate control sequence. This is the essence of **single shooting**.

The term “shooting” comes from the idea of *aiming and firing* a trajectory from the initial state: you pick a control sequence, integrate (or step) the system

forward, and see where it lands. If the final state misses the target, you adjust the controls and try again: like adjusting the angle of a shot until it hits the mark. It is called **single** shooting because we compute the entire trajectory in one pass from the starting point, without breaking it into segments. Later, we will contrast this with **multiple shooting**, where the horizon is divided into smaller arcs that are optimized jointly to improve stability and conditioning.

The analogy with deep learning is also immediate: the control sequence plays the role of parameters, the rollout is a forward pass, and the cost is a scalar loss. Gradients can be obtained with reverse-mode automatic differentiation. In the single shooting formulation of the DOCP, the constrained program

$$\min_{\mathbf{x}_{1:T}, \mathbf{u}_{1:T-1}} J(\mathbf{x}_{1:T}, \mathbf{u}_{1:T-1}) \quad \text{s.t.} \quad \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) \quad (78)$$

collapses to

$$\min_{\mathbf{u}_{1:T-1}} c_T(\phi_T(\mathbf{u}, \mathbf{x}_1)) + \sum_{t=1}^{T-1} c_t(\phi_t(\mathbf{u}, \mathbf{x}_1), \mathbf{u}_t), \quad \mathbf{u}_{\text{lb}} \leq \mathbf{u}_t \leq \mathbf{u}_{\text{ub}}. \quad (79)$$

Here ϕ_t denotes the state reached at time t by recursively applying the dynamics to the previous state and current control. This recursion can be written as

$$\phi_{t+1}(\mathbf{u}, \mathbf{x}_1) = \mathbf{f}_t(\phi_t(\mathbf{u}, \mathbf{x}_1), \mathbf{u}_t), \quad \phi_1 = \mathbf{x}_1. \quad (80)$$

Concretely, here is JAX-style pseudocode for defining `phi(u, x_0, t)` using `jax.lax.scan` with a zero-based time index:

```
def phi(u_seq, x0, t):
    """Return \phi_t(u, x0) with 0 -based t (\phi_0 = x0).

    u_seq: controls of length T (or T -1); only first t entries are used
    x0: initial state at time 0
    t: integer >= 0
    """
    if t <= 0:
        return x0

    def step(carry, u):
        x, t_idx = carry
        x_next = f(x, u, t_idx)
        return (x_next, t_idx + 1), None

    (x_t, _), _ = lax.scan(step, (x0, 0), u_seq[:t])
    return x_t
```

The pattern mirrors an RNN unroll: starting from an initial state ($\mathbf{x}_1^*$) and a sequence of controls ($\mathbf{u}_{1:T-1}^*$), we propagate forward through the dynamics, updating the state at each step and accumulating cost along the way. This structural similarity is why single shooting often feels natural to practitioners with a deep learning background: the rollout is a forward pass, and gradients propagate backward through time exactly as in backpropagation through an RNN.

Algorithmically:

In JAX or PyTorch, this loop can be JIT-compiled and differentiated automatically. Any gradient-based optimizer (L-BFGS, Adam, even SGD) can be applied, making the pipeline look very much like training a neural network. In effect, we are “backpropagating through the world model” when computing $\nabla J(\mathbf{u})$.

Single shooting is attractive for its simplicity and compatibility with differentiable programming, but it has limitations. The absence of intermediate constraints makes it sensitive to initialization and prone to numerical instability over long horizons. When state constraints or robustness matter, formulations that keep states explicit, such as multiple shooting or collocation, become preferable. These trade-offs are the focus of the next section.

Before reading on

Consider what happens to single shooting when the horizon T is very large. What numerical difficulties might arise? Think about how small errors in early controls could propagate through the dynamics.

In Between Sequential and Simultaneous The two formulations we have seen so far lie at opposite ends. The **full discretization** approach keeps every state explicit and enforces the dynamics through equality constraints, which makes the structure clear but leads to a large optimization problem. At the other end, **single shooting** removes these constraints by simulating forward from the initial state, leaving only the controls as decision variables. That makes the problem smaller, but it also introduces a long and highly nonlinear dependency from the first control to the last state.

Multiple shooting sits in between. Instead of simulating the entire horizon in one shot, we divide it into smaller segments. For each segment, we keep its starting state as a decision variable and propagate forward using the dynamics for that segment. At the end, we enforce continuity by requiring that the simulated end state of one segment matches the decision variable for the next.

Formally, suppose the horizon of T steps is divided into K segments of length L (with $T = K \cdot L$ for simplicity). We introduce:

- The controls for each step: $\mathbf{u}_{1:T-1}$.
- The state at the start of each segment: $\mathbf{x}_1, \dots, \mathbf{x}_K$.

Given $\mathbf{x}_k$ and the controls in its segment, we compute the predicted terminal state by simulating forward:

$$\hat{\mathbf{x}}_{k+1} = \Phi(\mathbf{x}_k, \mathbf{u}_{\text{segment } k}), \quad (81)$$

where Φ represents L applications of the dynamics. Continuity constraints enforce:

$$\mathbf{x}_{k+1} - \hat{\mathbf{x}}_{k+1} = 0, \quad k = 1, \dots, K-1. \quad (82)$$

The resulting nonlinear program looks like this:

$$\begin{aligned} \min_{\{\mathbf{x}_k, \mathbf{u}_t\}} \quad & c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\ \text{subject to} \quad & \mathbf{x}_{k+1} - \Phi(\mathbf{x}_k, \mathbf{u}_{\text{segment } k}) = 0, \quad k = 1, \dots, K-1, \\ & \mathbf{u}_{\text{lb}} \leq \mathbf{u}_t \leq \mathbf{u}_{\text{ub}}, \\ & \text{boundary conditions on } \mathbf{x}_1 \text{ and } \mathbf{x}_K. \end{aligned} \quad (83)$$

Compared to the full NLP, we no longer introduce every intermediate state as a variable, only the anchors at segment boundaries. Inside each segment, states are reconstructed by simulation. Compared to single shooting, these anchors break the long dependency chain that makes optimization unstable: gradients only have to travel across L steps before they hit a decision variable, rather than the entire horizon. This is the same reason why exploding or vanishing gradients appear in deep recurrent networks: when the chain is too long, information either dies out or blows up. Multiple shooting shortens the chain and improves conditioning.

By adjusting the number of segments K , we can interpolate between the two extremes: $K = 1$ gives single shooting, while $K = T$ recovers the full direct NLP. In practice, a moderate number of segments often strikes a good balance between robustness and complexity.

4.1.2 The Discrete-Time Pontryagin Principle

If we take the Bolza formulation of the DOCP and apply the KKT conditions directly, we obtain an optimization system with many multipliers and constraints. Written in raw form, it looks like any other nonlinear program. But in control, this structure has a long history and a name of its own: the **Pontryagin principle**. In fact, the discrete-time version can be seen as the structured KKT system that results from introducing multipliers for the dynamics and collecting terms stage by stage.

We work with the Bolza program

$$\begin{aligned}
\min_{\{\mathbf{x}_t, \mathbf{u}_t\}} \quad & c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\
\text{s.t.} \quad & \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t), \quad t = 1, \dots, T-1, \\
& \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t) \leq \mathbf{0}, \quad \mathbf{u}_t \in \mathcal{U}_t, \\
& \mathbf{h}(\mathbf{x}_T) = \mathbf{0} \quad (\text{optional terminal equalities}).
\end{aligned} \tag{84}$$

Introduce **costates** $\boldsymbol{\lambda}_{t+1} \in R^n$ for the dynamics, multipliers $\boldsymbol{\mu}_t \geq \mathbf{0}$ for path inequalities, and $\boldsymbol{\nu}$ for terminal equalities. The Lagrangian is

$$\mathcal{L} = c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) + \sum_{t=1}^{T-1} \boldsymbol{\lambda}_{t+1}^\top (\mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{x}_{t+1}) + \sum_{t=1}^{T-1} \boldsymbol{\mu}_t^\top \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\nu}^\top \mathbf{h}(\mathbf{x}_T). \tag{85}$$

It is convenient to package the stagewise terms in a **Hamiltonian**

$$H_t(\mathbf{x}_t, \mathbf{u}_t, \boldsymbol{\lambda}_{t+1}, \boldsymbol{\mu}_t) := c_t(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\lambda}_{t+1}^\top \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\mu}_t^\top \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t). \tag{86}$$

Check your understanding

Verify that the Hamiltonian H_t collects exactly the terms from the Lagrangian that involve stage t : the stage cost, the dynamics constraint (weighted by the next costate), and the path constraints (weighted by their multipliers). Why does $\boldsymbol{\lambda}_{t+1}$ appear rather than $\boldsymbol{\lambda}_t$?

Then

$$\mathcal{L} = c_T(\mathbf{x}_T) + \boldsymbol{\nu}^\top \mathbf{h}(\mathbf{x}_T) + \sum_{t=1}^{T-1} \left[H_t(\mathbf{x}_t, \mathbf{u}_t, \boldsymbol{\lambda}_{t+1}, \boldsymbol{\mu}_t) - \boldsymbol{\lambda}_{t+1}^\top \mathbf{x}_{t+1} \right]. \tag{87}$$

Gradient convention

Throughout this section, we use the **denominator layout** (gradient layout) convention:

- $\nabla_{\mathbf{x}} f(\mathbf{x})$ produces a **column vector** (gradient)
- $\frac{\partial f}{\partial \mathbf{x}}$ produces the Jacobian matrix
- For scalar functions: $\nabla_{\mathbf{x}} f = \left(\frac{\partial f}{\partial \mathbf{x}} \right)^\top$

This is the standard convention in optimization and control theory.

Necessary conditions Taking first-order variations and collecting terms gives the discrete-time adjoint system, control stationarity, and complementarity. At a local minimum $\{\mathbf{x}_t^*, \mathbf{u}_t^*\}$ with multipliers $\{\boldsymbol{\lambda}_t^*, \boldsymbol{\mu}_t^*, \boldsymbol{\nu}^*\}$:

State dynamics (primal feasibility)

$$\mathbf{x}_{t+1}^* = \mathbf{f}_t(\mathbf{x}_t^*, \mathbf{u}_t^*), \quad t = 1, \dots, T-1. \quad (88)$$

Costate recursion (backward “adjoint” equation)

$$\boldsymbol{\lambda}_t^* = \nabla_{\mathbf{x}} H_t(\mathbf{x}_t^*, \mathbf{u}_t^*, \boldsymbol{\lambda}_{t+1}^*, \boldsymbol{\mu}_t^*) = \nabla_{\mathbf{x}} c_t(\mathbf{x}_t^*, \mathbf{u}_t^*) + [\nabla_{\mathbf{x}} \mathbf{f}_t(\mathbf{x}_t^*, \mathbf{u}_t^*)]^\top \boldsymbol{\lambda}_{t+1}^* + [\nabla_{\mathbf{x}} \mathbf{g}_t(\mathbf{x}_t^*, \mathbf{u}_t^*)]^\top \boldsymbol{\mu}_t^*, \quad (89)$$

with the **terminal condition**

$$\boldsymbol{\lambda}_T^* = \nabla_{\mathbf{x}} c_T(\mathbf{x}_T^*) + [\nabla_{\mathbf{x}} \mathbf{h}(\mathbf{x}_T^*)]^\top \boldsymbol{\nu}^* \quad (\text{and } \boldsymbol{\nu}^* = \mathbf{0} \text{ if there are no terminal equalities}). \quad (90)$$

Control stationarity (first-order optimality in $\mathbf{u}_t$) If $\mathcal{U}_t = R^m$ (no explicit set constraint), then

$$\nabla_{\mathbf{u}} H_t(\mathbf{x}_t^*, \mathbf{u}_t^*, \boldsymbol{\lambda}_{t+1}^*, \boldsymbol{\mu}_t^*) = \mathbf{0}. \quad (91)$$

If $\mathcal{U}_t$ imposes bounds or a convex set, the condition becomes the **variational inequality**

$$\mathbf{0} \in \nabla_{\mathbf{u}} H_t(\cdot) + N_{\mathcal{U}_t}(\mathbf{u}_t^*), \quad (92)$$

where $N_{\mathcal{U}_t}(\cdot)$ is the normal cone to $\mathcal{U}_t$. For simple box bounds, this reduces to standard KKT sign and complementarity conditions on the components of $\mathbf{u}_t^*$.

Path-constraint multipliers (primal/dual feasibility and complementarity)

$$\mathbf{g}_t(\mathbf{x}_t^*, \mathbf{u}_t^*) \leq \mathbf{0}, \quad \boldsymbol{\mu}_t^* \geq \mathbf{0}, \quad \mu_{t,i}^* g_{t,i}(\mathbf{x}_t^*, \mathbf{u}_t^*) = 0 \quad \text{for all } i, t. \quad (93)$$

Terminal equalities (if present)

$$\mathbf{h}(\mathbf{x}_T^*) = \mathbf{0}. \quad (94)$$

The triplet “forward state, backward costate, control stationarity” is the discrete-time Euler–Lagrange system tailored to control with dynamics. It is the same KKT logic as before, but organized stagewise through the Hamiltonian.

Proposition 4.1 (Discrete-time Pontryagin necessary conditions (summary)).
At a local minimum of the DOCP

$$\min_{\{\mathbf{x}_t, \mathbf{u}_t\}} c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \quad \text{s.t.} \quad \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t), \quad \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t) \leq \mathbf{0}, \quad \mathbf{h}(\mathbf{x}_T) = \mathbf{0}, \quad (95)$$

there exist multipliers $\{\lambda_{t+1}\}$, $\{\mu_t \geq \mathbf{0}\}$, and (if present) ν such that, for $t = 1, \dots, T-1$:

- *State dynamics:* $\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)$.
- *Backward costate recursion:*

$$\lambda_t = \nabla_{\mathbf{x}} c_t(\mathbf{x}_t, \mathbf{u}_t) + [\nabla_{\mathbf{x}} \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)]^\top \lambda_{t+1} + [\nabla_{\mathbf{x}} \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t)]^\top \mu_t. \quad (96)$$

- *Terminal condition:* $\lambda_T = \nabla_{\mathbf{x}} c_T(\mathbf{x}_T) + [\nabla_{\mathbf{x}} \mathbf{h}(\mathbf{x}_T)]^\top \nu$.
- *Control stationarity (unconstrained control):* $\nabla_{\mathbf{u}} H_t(\cdot) = \mathbf{0}$; with a convex control set $\mathcal{U}_t$, $\mathbf{0} \in \nabla_{\mathbf{u}} H_t(\cdot) + N_{\mathcal{U}_t}(\mathbf{u}_t)$.
- *Path inequalities:* $\mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t) \leq \mathbf{0}$, $\mu_t \geq \mathbf{0}$, and complementarity $\mu_{t,i} g_{t,i}(\mathbf{x}_t, \mathbf{u}_t) = 0$ for all i .
- *Terminal equalities (if present):* $\mathbf{h}(\mathbf{x}_T) = \mathbf{0}$.

Here $H_t(\mathbf{x}_t, \mathbf{u}_t, \lambda_{t+1}, \mu_t) := c_t(\mathbf{x}_t, \mathbf{u}_t) + \lambda_{t+1}^\top \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t) + \mu_t^\top \mathbf{g}_t(\mathbf{x}_t, \mathbf{u}_t)$ is the stage Hamiltonian.

Recap. The discrete-time Pontryagin principle is the KKT system for trajectory optimization, organized to exploit temporal structure. It has a forward-backward decomposition: states propagate forward through the dynamics, while costates propagate backward through the adjoint equation. The Hamiltonian H_t packages together the stage cost, the dynamics (weighted by the next costate), and any path constraints (weighted by their multipliers). Control stationarity says that optimal controls minimize the Hamiltonian at each stage. Complementarity ensures that only binding constraints carry nonzero multipliers. This structure underlies both analytical solution methods (such as LQR) and numerical algorithms (such as the adjoint method for gradient computation).

The adjoint equation as reverse accumulation Optimization needs sensitivities. In trajectory problems we adjust decisions (controls or parameters) to reduce an objective while respecting dynamics and constraints. First-order methods in the unconstrained case (e.g., gradient descent, L-BFGS, Adam) require the gradient of the objective with respect to all controls, and constrained methods (SQP, interior-point) require gradients of the Lagrangian, i.e., of costs and constraints. The discrete-time adjoint equations provide these derivatives in a way that scales to long horizons and many decision variables.

Consider

$$J = c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t), \quad \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t). \quad (97)$$

A single forward rollout computes and stores the trajectory $\mathbf{x}_{1:T}$. A single backward sweep then applies the reverse-mode chain rule stage by stage.

Before reading on

Try applying the chain rule to compute how a perturbation to the state at time t affects the total cost J . Start from $\partial J / \partial \mathbf{x}_T$ and work backward. What pattern emerges?

Defining the costate by

$$\boldsymbol{\lambda}_T = \nabla_{\mathbf{x}} c_T(\mathbf{x}_T), \quad \boldsymbol{\lambda}_t = \nabla_{\mathbf{x}} c_t(\mathbf{x}_t, \mathbf{u}_t) + [\nabla_{\mathbf{x}} \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)]^\top \boldsymbol{\lambda}_{t+1}, \quad t = T-1, \dots, 1, \quad (98)$$

yields exactly the discrete-time adjoint (PMP) recursion.

Check your understanding

What terms would you expect to appear in the gradient $\nabla_{\mathbf{u}_t} J$? The control $\mathbf{u}_t$ affects J in two ways: directly through the stage cost c_t , and indirectly by changing the next state $\mathbf{x}_{t+1}$. How should these contributions combine?

The gradient with respect to each control follows from the same reverse pass:

$$\nabla_{\mathbf{u}_t} J = \nabla_{\mathbf{u}_t} c_t(\mathbf{x}_t, \mathbf{u}_t) + [\nabla_{\mathbf{u}_t} \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)]^\top \boldsymbol{\lambda}_{t+1}. \quad (99)$$

Computationally, this reverse accumulation produces all control gradients with one forward rollout and one backward adjoint pass; its cost is essentially a small constant multiple of simulating the system once. Conceptually, the costate $\boldsymbol{\lambda}_t$ is the marginal effect of perturbing the state at time t on the total objective; the control gradient combines a direct contribution from c_t and an indirect contribution through how $\mathbf{u}_t$ changes the next state. This is the same structure that underlies backpropagation, expressed for dynamical systems.

It is instructive to contrast this with alternatives. Black-box finite differences perturb one decision at a time and re-roll the system, requiring on the order of p rollouts for p decision variables and suffering from step-size and noise issues. This becomes prohibitive when $p = (T-1)m$ for an m -dimensional control over T steps. Forward-mode (tangent) sensitivities propagate Jacobian–vector products for each parameter direction; their work also scales with p . Reverse-mode (the adjoint) instead propagates a single vector $\boldsymbol{\lambda}_t$ backward and then reads off all partial derivatives $\nabla_{\mathbf{u}_t} J$ at once. For a scalar objective, its cost is effectively independent of p , at the price of storing (or checkpointing) the forward trajectory. This scalability is why the adjoint is the method of choice for gradient-based trajectory optimization and for constrained transcriptions via the Hamiltonian.

Recap. The adjoint method computes gradients of the objective with respect to all controls in time proportional to a single forward-backward pass through the dynamics. The forward pass simulates the system and stores the

trajectory. The backward pass propagates costates λ_t from the terminal condition back to the initial time, accumulating sensitivity information along the way. Each control gradient is then read off locally as the sum of a direct cost contribution and an indirect contribution through the dynamics. This is mathematically equivalent to reverse-mode automatic differentiation (backpropagation) applied to the unrolled dynamical system, and it explains why gradient-based trajectory optimization scales gracefully to long horizons and high-dimensional control spaces.

Summary and Outlook This chapter developed the foundations of discrete-time trajectory optimization. We introduced three equivalent formulations of the discrete-time optimal control problem (DOCP)—Bolza, Lagrange, and Mayer—and showed how they reduce to finite-dimensional nonlinear programs once a horizon is fixed.

Three approaches to solving DOCPs were presented, each with distinct trade-offs:

- **Simultaneous methods (direct transcription)** keep all states and controls as decision variables and enforce the dynamics as equality constraints. This produces a large but sparse NLP whose structure can be exploited by specialized solvers. The explicit state variables anchor the trajectory at every time step, improving robustness and making it straightforward to impose state constraints.
- **Sequential methods (single shooting)** eliminate state variables by forward simulation, leaving only the controls as decisions. This yields a smaller, unconstrained (or bound-constrained) problem that integrates naturally with automatic differentiation frameworks. The trade-off is sensitivity to initialization and potential numerical instability over long horizons, since errors compound through the dynamics.
- **Multiple shooting** divides the horizon into segments, treating segment boundaries as decision variables and enforcing continuity between them. This interpolates between the two extremes: it shortens the dependency chains that cause instability in single shooting while avoiding the full size of direct transcription.

The Karush–Kuhn–Tucker (KKT) conditions provide necessary conditions for local optimality in constrained optimization. Applied to the DOCP, they yield the discrete-time Pontryagin principle: a forward-backward system where states propagate forward through the dynamics and costates propagate backward through the adjoint equation. The Hamiltonian packages together the stage cost and dynamics, and control stationarity says that optimal controls minimize the Hamiltonian at each stage.

The adjoint method computes gradients of the objective with respect to all controls in time proportional to one forward pass plus one backward pass,

independent of the number of decision variables. This efficiency is what allows gradient-based trajectory optimization to scale to long horizons and high-dimensional control spaces.

The open-loop perspective developed here is foundational but limited: it assumes the model is accurate and that no disturbances will occur. In practice, plans must be adapted as new information arrives. Several directions extend this chapter’s material:

- **Collocation methods** use polynomial approximations to represent the trajectory between grid points, enabling higher-order accuracy and implicit handling of stiff dynamics. These are the subject of the next chapter.
- **Model predictive control (MPC)** repeatedly solves trajectory optimization problems in a receding-horizon fashion, using the first control from each solution and re-planning as new state measurements arrive. This creates a feedback policy from open-loop building blocks.
- **Feedback and policy optimization** shift the focus from computing a single trajectory to learning a policy $\pi(\mathbf{x})$ that maps states to controls. Dynamic programming, policy gradient methods, and actor-critic algorithms address this problem from different angles, and will be developed in later chapters.

The tools introduced here—KKT conditions, the Pontryagin principle, shooting methods, and adjoint gradients—will reappear throughout the book as we move from open-loop planning to closed-loop control and learning.

Exercises

Self-checks

4.2 Trajectory Optimization in Continuous Time

The previous chapter developed trajectory optimization for discrete-time systems, where the horizon splits naturally into stages. Many physical systems, however, are described by ordinary differential equations in continuous time. This chapter extends the optimization framework to that setting. The decision variables are now functions of time, not finite sequences, so we must convert the infinite-dimensional problem into a finite-dimensional nonlinear program that a numerical solver can handle. The methods that accomplish this conversion are called **direct transcription** or **collocation** methods, and they form the computational backbone of modern trajectory optimization software.

Learning Goals

After studying this chapter, you should be able to:

1. Formulate a continuous-time optimal control problem in Mayer, Lagrange, or Bolza form, and convert between them.
2. Explain how quadrature rules and polynomial interpolation convert infinite-dimensional problems into finite-dimensional NLPs.
3. Derive the Euler, trapezoidal, and Hermite–Simpson collocation schemes from first principles.
4. Implement a direct collocation method for a nonlinear dynamical system.
5. Distinguish Gauss, Radau, and Lobatto node families and explain their implications for boundary conditions.

Prerequisites

This chapter assumes familiarity with:

- Discrete-time trajectory optimization, including single and multiple shooting (see [Discrete-Time Trajectory Optimization](#))
- Basic numerical integration concepts (quadrature rules, interpolation)
- Nonlinear programming fundamentals (KKT conditions, constraint handling)

4.2.1 A Motivating Example: Minimum-Time Braking

Before presenting the formal problem classes, consider a simple scenario that illustrates the structure of continuous-time optimal control. A car traveling

at initial velocity v_0 must come to a complete stop at a target position p_f . The driver controls the braking force, which we normalize to an acceleration $u(t) \in [-a_{\max}, 0]$. The dynamics are:

$$\dot{p}(t) = v(t), \quad \dot{v}(t) = u(t). \quad (100)$$

The objective is to stop in minimum time while respecting the braking limit. This is a **Mayer problem**: the cost depends only on the final time t_f , with no running penalty. If instead we penalized control effort $\int_0^{t_f} u(t)^2 dt$, it would be a **Lagrange problem**. Combining both yields a **Bolza problem**.

Notice the essential structure: a state trajectory $(p(t), v(t))$ evolves according to an ODE, a control function $u(t)$ influences that evolution, and the objective involves the trajectory endpoints and possibly an integral over time. The constraints $u(t) \in [-a_{\max}, 0]$ and $v(t_f) = 0$ must hold. This is a continuous-time optimal control problem, and the three standard formulations below capture all such problems.

4.2.2 Problem Formulations

As in the discrete-time setting, we work with three continuous-time variants that differ only in how the objective is written while sharing the same dynamics, path constraints, and bounds. The path constraints $\mathbf{g}(\mathbf{x}(t), \mathbf{u}(t)) \leq \mathbf{0}$ are pointwise in time, and the bounds $\mathbf{x}_{\min} \leq \mathbf{x}(t) \leq \mathbf{x}_{\max}$ and $\mathbf{u}_{\min} \leq \mathbf{u}(t) \leq \mathbf{u}_{\max}$ are understood in the same pointwise sense.

These three forms are different lenses on the same task. Bolza contains both terminal and running terms. Lagrange can be turned into Mayer by augmenting the state with an accumulator:

$$\dot{z}(t) = c(\mathbf{x}(t), \mathbf{u}(t)), \quad z(t_0) = 0, \quad \text{minimize } z(t_f), \quad (101)$$

with the original dynamics left unchanged. Mayer is a special case of Bolza with zero running cost. We will use these equivalences freely, since a numerical method cares only about what must be evaluated and where those evaluations are taken.

With this catalog in place, we now pass from functions to finite representations.

4.2.3 Direct Transcription Methods

The discrete-time problems developed in [Discrete-Time Trajectory Optimization](#) already suggested how to proceed: we convert a continuous problem into one over finitely many numbers by deciding where to look at the trajectories and how to interpolate between those looks. Unlike dynamic programming (see [Dynamic Programming](#)), which computes a value function over the entire state space, direct transcription finds a single optimal trajectory from a given initial condition. This makes it computationally tractable for high-dimensional systems but sacrifices the feedback properties that come from knowing the value function everywhere.

We place a mesh $t_0 < t_1 < \dots < t_N = t_f$ and, inside each window $[t_k, t_{k+1}]$, select a small set of interior fractions $\{\tau_j\}$ on the reference interval $[0, 1]$. The running cost is additive over windows, so we write it as a sum of local integrals, map each window to $[0, 1]$, and approximate each local integral by a quadrature rule with nodes τ_j and weights w_j . This produces

$$\int_{t_0}^{t_f} c(\mathbf{x}(t), \mathbf{u}(t)) dt \approx \sum_{k=0}^{N-1} h_k \sum_{j=1}^q w_j c(\mathbf{x}(t_k + h_k \tau_j), \mathbf{u}(t_k + h_k \tau_j)), \quad (102)$$

with $h_k = t_{k+1} - t_k$. The dynamics are treated in the same way by the fundamental theorem of calculus,

$$\mathbf{x}(t_{k+1}) - \mathbf{x}(t_k) = \int_{t_k}^{t_{k+1}} \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) dt \approx h_k \sum_{j=1}^q b_j \mathbf{f}(\mathbf{x}(t_k + h_k \tau_j), \mathbf{u}(t_k + h_k \tau_j)), \quad (103)$$

so the places where we “pay” running cost are the same places where we “account” for state changes. Path constraints and bounds are then enforced at the same interior times. In the infinite-horizon discounted case, the same formulas apply with an extra factor $e^{-\rho(t_k + h_k \tau_j)}$ multiplying the weights in the cost.

The values $\mathbf{x}(t_k + h_k \tau_j)$ and $\mathbf{u}(t_k + h_k \tau_j)$ do not exist a priori. We create them by a finite representation. One option is shooting: parameterize $\mathbf{u}$ on the mesh, integrate the ODE across each window with a chosen numerical step, and read interior values from that step. Another is collocation: represent $\mathbf{x}$ inside each window by a local polynomial and choose its coefficients so that the ODE holds at the interior nodes. Both constructions lead to the same structure: a nonlinear program whose objective is a composite quadrature of the running term (plus any terminal term in the Bolza case) and whose constraints are algebraic relations that encode the ODE and the pointwise inequalities at the selected nodes.

Specific choices recover familiar schemes. If we use the left endpoint as the single interior node, we obtain the forward Euler transcription. If we use both endpoints with equal weights, we obtain the trapezoidal transcription. Higher-order rules arise when we include interior nodes and richer polynomials for $\mathbf{x}$. What matters here is the unifying picture: choose nodes, translate integrals into weighted sums, and couple those evaluations to a finite trajectory representation so that cost and physics are enforced at the same places. This is the organizing idea that will guide the rest of the chapter.

Discretizing cost and dynamics together In a continuous-time OCP, integrals appear twice: in the objective, which accumulates running cost over time, and implicitly in the dynamics, since state changes over any interval are the integral of the vector field. To compute, we must approximate both the

integrals and the unknown functions $\mathbf{x}(t)$ and $\mathbf{u}(t)$ with finitely many numbers that an optimizer can manipulate.

A natural way to do this is to lay down a finite set of time points (a mesh) over the horizon. You can think of the mesh as a grid we overlay on the “true” trajectories that exist as mathematical objects but are not directly accessible. Our aim is to approximate those trajectories and their integrals using values and simple models tied to the mesh. Using the same mesh for both the cost and the dynamics keeps the representation coherent: we evaluate what we pay and how the state changes at consistent times.

Concretely, we begin by choosing a mesh

$$t_0 < t_1 < \cdots < t_N = t_f, \quad h_k := t_{k+1} - t_k. \quad (104)$$

The running cost is additive over disjoint intervals. When the horizon $[t_0, t_f]$ is partitioned by the mesh, additivity (linearity) of the integral gives

$$\int_{t_0}^{t_f} c(\mathbf{x}(t), \mathbf{u}(t)) dt = \sum_{k=0}^{N-1} \int_{t_k}^{t_{k+1}} c(\mathbf{x}(t), \mathbf{u}(t)) dt. \quad (105)$$

This identity is exact: it is just the additivity (linearity) of the Lebesgue/Riemann integral over a partition. No approximation has been made yet. Approximations enter only when we later replace each window integral by a quadrature rule: a finite set of nodes and positive weights prescribing an integral approximation. This sets the table for three important ingredients that we will use throughout the chapter.

First, it turns a global object into **local contributions** that live on each $[t_k, t_{k+1}]$. Numerical integration is most effective when it is composite: we approximate each small interval integral and then sum the results. Doing so controls error uniformly, because the global quadrature error is the accumulation of local errors that shrink with the step size. It also allows non-uniform steps $h_k = t_{k+1} - t_k$, which we will use later for mesh refinement.

Second, the split aligns the cost with the **local dynamics constraints**. On each interval the ODE can be written, by the fundamental theorem of calculus, as

$$\mathbf{x}(t_{k+1}) - \mathbf{x}(t_k) = \int_{t_k}^{t_{k+1}} \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) dt. \quad (106)$$

When we approximate this integral, we introduce interior evaluation points $t_{k,j} \in [t_k, t_{k+1}]$. Using the **same points** in the cost and in the dynamics ties $\mathbf{x}$ and $\mathbf{u}$ together coherently: the places where we “pay” for running cost are also the places where we enforce the ODE. This avoids a mismatch between where we approximate the objective and where we impose feasibility.

Third, the decomposition yields a nonlinear program with **sparse structure**. Each interval contributes a small block to the objective and constraints that depends only on variables from that interval (and its endpoints). Modern solvers exploit this banded sparsity to scale to long horizons.

With the split justified, we standardize the approximation. Map each interval to a reference domain via $t = t_k + h_k \tau$ with $\tau \in [0, 1]$ and $dt = h_k d\tau$. A **quadrature rule on** $[0, 1]$ is specified by evaluation points $\{\tau_j\}_{j=1}^q \subset [0, 1]$ and positive weights $\{w_j\}_{j=1}^q$ such that, for a smooth ϕ ,

$$\int_0^1 \phi(\tau) d\tau \approx \sum_{j=1}^q w_j \phi(\tau_j). \quad (107)$$

Applying it on each interval gives

$$\int_{t_k}^{t_{k+1}} c(\mathbf{x}(t), \mathbf{u}(t)) dt \approx h_k \sum_{j=1}^q w_j c(\mathbf{x}(t_k + h_k \tau_j), \mathbf{u}(t_k + h_k \tau_j)). \quad (108)$$

Summing these window contributions gives a composite approximation of the integral over $[t_0, t_f]$:

$$\int_{t_0}^{t_f} c(\mathbf{x}(t), \mathbf{u}(t)) dt \approx \sum_{k=0}^{N-1} h_k \sum_{j=1}^q w_j c(\mathbf{x}(t_{k,j}), \mathbf{u}(t_{k,j})). \quad (109)$$

The outer index k indicates the window; the inner index i indicates the samples within that window; the factor h_k appears from the change of variables.

The dynamics admit the same treatment. By the fundamental theorem of calculus,

$$\mathbf{x}(t_{k+1}) - \mathbf{x}(t_k) = \int_{t_k}^{t_{k+1}} \dot{\mathbf{x}}(t) dt = \int_{t_k}^{t_{k+1}} \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) dt. \quad (110)$$

Replacing this integral by a quadrature rule that uses the **same** nodes produces the window defect relation

$$\mathbf{x}_{k+1} - \mathbf{x}_k \approx h_k \sum_{j=1}^q b_j \mathbf{f}(\mathbf{x}(t_{k,j}), \mathbf{u}(t_{k,j})), \quad (111)$$

where $\{b_j\}$ are the weights used for the ODE. Path constraints $\mathbf{g}(\mathbf{x}(t), \mathbf{u}(t)) \leq 0$ are imposed at selected nodes $t_{k,j}$ in the same spirit. Using the same evaluation points for cost and dynamics keeps the representation coherent: we “pay” running cost and “account” for state changes at the same times.

Recap

We have established the core discretization framework: (1) partition the horizon into mesh intervals, (2) approximate integrals using quadrature rules with nodes and weights, and (3) evaluate both the running cost and the dynamics at the same interior points. This alignment principle ensures that the places where we accumulate cost are the same places where we enforce the ODE, yielding a coherent finite-dimensional representation.

What remains is to choose specific nodes and decide how to represent the trajectory between them.

On the choice of interior points Once we select a mesh $t_0 < \dots < t_N$ and interior fractions $\{\tau_j\}_{j=1}^q$ per window $[t_k, t_{k+1}]$, we need $\mathbf{x}(t_{k,j})$ and $\mathbf{u}(t_{k,j})$ at the evaluation times $t_{k,j} := t_k + h_k \tau_j$. These values do not preexist. They come from one of two constructions that align with the standard quadrature taxonomy: **step-function based** and **interpolating-function based** rules.

Step-function based construction (piecewise constants; rectangle or midpoint) Here we approximate the relevant time functions by step-functions on each window. For controls, a common choice is piecewise-constant:

$$\mathbf{u}(t) = \mathbf{u}_k \quad \text{for } t \in [t_k, t_{k+1}]. \quad (112)$$

For the running cost and the vector field, the corresponding quadrature is a rectangle rule on $[t_k, t_{k+1}]$. Using the left endpoint gives

$$\int_{t_k}^{t_{k+1}} c(\mathbf{x}(t), \mathbf{u}(t)) dt \approx h_k c(\mathbf{x}_k, \mathbf{u}_k), \quad (113)$$

and replacing the dynamics integral by the same step-function idea yields the forward Euler relation

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h_k \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k). \quad (114)$$

If we prefer the midpoint rectangle rule, we sample at $t_{k+\frac{1}{2}} = t_k + \frac{h_k}{2}$. In practice we then generate $\mathbf{x}_{k+\frac{1}{2}}$ by a half-step of the chosen integrator, and set $\mathbf{u}_{k+\frac{1}{2}} = \mathbf{u}_k$ (piecewise-constant) or an average if we allow a short linear segment. Either way, interior values come from **integrating forward** given a step-function model for $\mathbf{u}$ and a rectangle-rule view of the integrals. This is the shooting viewpoint. Single shooting keeps only control parameters as decision variables; multiple shooting adds the window-start states and enforces step consistency.

Interpolating-function based construction (low-order polynomials; trapezoid, Simpson, Gauss/Radau/Lobatto) Here we approximate time functions by **polynomials** on each window. If we interpolate $\mathbf{x}(t)$ linearly between endpoints, the cost naturally uses the trapezoidal rule

$$\int_{t_k}^{t_{k+1}} c dt \approx \frac{h_k}{2} [c(\mathbf{x}_k, \mathbf{u}_k) + c(\mathbf{x}_{k+1}, \mathbf{u}_{k+1})], \quad (115)$$

and the dynamics use the matched trapezoidal defect

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{h_k}{2} [\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k) + \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1})]. \quad (116)$$

With a quadratic interpolation that includes the midpoint, Simpson’s rule appears in the cost and the Hermite–Simpson relations tie $\mathbf{x}_{k+\frac{1}{2}}$ to endpoint values and slopes. More generally, **collocation** chooses interior nodes on $[t_k, t_{k+1}]$ (equally spaced gives Newton–Cotes like trapezoid or Simpson; Gaussian points give Gauss, Radau, or Lobatto schemes) and enforces the ODE at those nodes:

$$\frac{d}{dt}\mathbf{x}(t_{k,j}) = \mathbf{f}(\mathbf{x}(t_{k,j}), \mathbf{u}(t_{k,j}), t_{k,j}), \quad (117)$$

with continuity at endpoints. The interior values $\mathbf{x}(t_{k,j})$ are **evaluations of the decision polynomials**; $\mathbf{u}(t_{k,j})$ follows from the chosen control interpolation (constant, linear, or quadratic). The running cost is evaluated by the same interpolatory quadrature at the same nodes, which keeps “where we pay” aligned with “where we enforce.”

4.2.4 Polynomial Interpolation

We often want to construct a function that passes through a given set of points. For example, suppose we know a function should satisfy:

$$f(x_0) = y_0, \quad f(x_1) = y_1, \quad \dots, \quad f(x_m) = y_m. \quad (118)$$

These are called **interpolation constraints**. Our goal is to find a function $f(x)$ that satisfies all of them exactly.

To make the problem tractable, we restrict ourselves to a class of functions. In polynomial interpolation, we assume that $f(x)$ is a polynomial of degree at most N . That means we are trying to find coefficients $c_0, \dots, c_N$ such that

$$f(x) = \sum_{n=0}^N c_n \phi_n(x), \quad (119)$$

where the functions $\phi_n(x)$ form a basis for the space of polynomials. The most common choice is the **monomial basis**, where $\phi_n(x) = x^n$. This gives:

$$f(x) = c_0 + c_1x + c_2x^2 + \dots + c_Nx^N. \quad (120)$$

Other valid bases include Legendre, Chebyshev, and Lagrange polynomials, each chosen for specific numerical properties. But all span the same function space.

To find a unique solution, we need the number of unknowns (the c_n) to match the number of constraints. Since a degree- N polynomial has $N + 1$ coefficients, we need:

$$N + 1 = m + 1 \quad \Rightarrow \quad N = m. \quad (121)$$

So if we want a function that passes through 4 points, we need a cubic polynomial ($N = 3$). Choosing a higher degree than necessary would give us infinitely many solutions; a lower degree may make the problem unsolvable.

Solving for the Coefficients (Monomial Basis) If we fix the basis functions to be monomials, we can build a system of equations by plugging in each x_i into $f(x)$. This gives:

$$\begin{aligned} f(x_0) &= c_0 + c_1 x_0 + c_2 x_0^2 + \cdots + c_N x_0^N = y_0 \\ f(x_1) &= c_0 + c_1 x_1 + c_2 x_1^2 + \cdots + c_N x_1^N = y_1 \\ &\vdots \\ f(x_m) &= c_0 + c_1 x_m + c_2 x_m^2 + \cdots + c_N x_m^N = y_m \end{aligned} \tag{122}$$

This system can be written in matrix form as:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^N \\ 1 & x_1 & x_1^2 & \cdots & x_1^N \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_m & x_m^2 & \cdots & x_m^N \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_m \end{bmatrix} \tag{123}$$

The matrix on the left is called the **Vandermonde matrix**. Solving this system gives the coefficients c_n that define the interpolating polynomial.

Using a Different Basis We don't have to use monomials. We can pick any set of basis functions $\phi_n(x)$, such as Chebyshev or Fourier modes, and follow the same steps. The interpolating function becomes:

$$f(x) = \sum_{n=0}^N c_n \phi_n(x), \tag{124}$$

and each interpolation constraint becomes:

$$f(x_i) = \sum_{n=0}^N c_n \phi_n(x_i) = y_i. \tag{125}$$

Assembling these into a system gives:

$$\begin{bmatrix} \phi_0(x_0) & \phi_1(x_0) & \cdots & \phi_N(x_0) \\ \phi_0(x_1) & \phi_1(x_1) & \cdots & \phi_N(x_1) \\ \vdots & \vdots & & \vdots \\ \phi_0(x_m) & \phi_1(x_m) & \cdots & \phi_N(x_m) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_m \end{bmatrix} \tag{126}$$

From here, we solve as before and reconstruct $f(x)$.

Derivative Constraints Sometimes, instead of a value constraint $f(x_i) = y_i$, we want to impose a slope constraint $f'(x_i) = s_i$. This is common in applications like spline interpolation or collocation methods, where derivative information is available from an ODE.

Since

$$f(x) = \sum_{n=0}^N c_n \phi_n(x) \quad \Rightarrow \quad f'(x) = \sum_{n=0}^N c_n \phi'_n(x), \quad (127)$$

we can directly write the slope constraint:

$$f'(x_i) = \sum_{n=0}^N c_n \phi'_n(x_i) = s_i. \quad (128)$$

To enforce this, we replace one of the interpolation equations in our system with this slope constraint. The resulting system still has $m + 1$ equations and $N + 1 = m + 1$ unknowns.

Concretely, suppose we have $k+1$ value constraints at nodes $X = \{x_0, \dots, x_k\}$ with values $Y = \{y_0, \dots, y_k\}$ and r slope constraints at nodes $Z = \{z_1, \dots, z_r\}$ with slopes $S = \{s_1, \dots, s_r\}$, with $k + 1 + r = N + 1$. The linear system for the coefficients $\mathbf{c} = [c_0, \dots, c_N]^\top$ is

$$\begin{bmatrix} \phi_0(x_0) & \phi_1(x_0) & \cdots & \phi_N(x_0) \\ \vdots & \vdots & & \vdots \\ \phi_0(x_k) & \phi_1(x_k) & \cdots & \phi_N(x_k) \\ \phi'_0(z_1) & \phi'_1(z_1) & \cdots & \phi'_N(z_1) \\ \vdots & \vdots & & \vdots \\ \phi'_0(z_r) & \phi'_1(z_r) & \cdots & \phi'_N(z_r) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} y_0 \\ \vdots \\ y_k \\ s_1 \\ \vdots \\ s_r \end{bmatrix}. \quad (129)$$

If a value and a slope are imposed at the same node, take $z_j = x_i$ and include both the value row and the derivative row; the system remains square. In the monomial basis, the top block is the Vandermonde matrix and the derivative block has entries $n x^{n-1}$. Once solved for $\mathbf{c}$, reconstruct

$$f(x) = \sum_{n=0}^N c_n \phi_n(x). \quad (130)$$

Interpolating ODE Trajectories (Collocation) Having established the general framework of interpolation, we now apply these concepts to the specific context of approximating trajectories governed by ordinary differential equations. The idea of applying polynomial interpolation with derivative constraints yields a method known as “collocation”. More precisely, a **degree- s collocation method** is a way to discretize an ordinary differential equation (ODE) by approximating the solution on each time interval with a polynomial of degree s , and then enforcing that this polynomial satisfies the ODE exactly at s carefully chosen points (the *collocation nodes*).

Consider a dynamical system described by the ordinary differential equation:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), t) \quad (131)$$

where $\mathbf{x}(t)$ represents the state trajectory and $\mathbf{u}(t)$ denotes the control input. Let us focus on a single mesh interval $[t_k, t_{k+1}]$ with step size $h_k := t_{k+1} - t_k$. To work with a standardized domain, we introduce the transformation $t = t_k + h_k \tau$ that maps the physical interval $[t_k, t_{k+1}]$ to the reference interval $[0, 1]$. On this reference interval, we select a set of collocation nodes $\{\tau_j\}_{j=0}^K \subset [0, 1]$.

Our goal is now to approximate the unknown trajectory using a polynomial of degree K . Using a monomial basis, we represent (parameterize) our trajectory as:

$$\mathbf{x}_h(\tau) := \sum_{n=0}^K \mathbf{a}_n \tau^n \quad (132)$$

where $\mathbf{a}_n \in R^d$ are coefficient vectors to be determined. Collocation enforces the differential equation at a chosen set of nodes on $[0, 1]$. Depending on the node family, these nodes may be interior-only or may include one or both endpoints. With the polynomial state model, we can differentiate analytically. Using the change of variables $t = t_k + h_k \tau$, we obtain:

$$\dot{\mathbf{x}}_h(t_k + h_k \tau_j) = \frac{1}{h_k} \sum_{n=1}^K n \mathbf{a}_n \tau_j^{n-1} \quad (133)$$

The collocation condition requires that this polynomial derivative equals the right-hand side of the ODE at each collocation node τ_j :

$$\frac{1}{h_k} \sum_{n=1}^K n \mathbf{a}_n \tau_j^{n-1} = \mathbf{f} \left(\sum_{n=0}^K \mathbf{a}_n \tau_j^n, \mathbf{u}_j, t_k + h_k \tau_j \right), \quad \text{for each collocation node } \tau_j. \quad (134)$$

where $\mathbf{u}_j$ represents the control value at node τ_j .

Boundary Conditions and Node Families The choice of collocation nodes determines how boundary conditions are handled and affects the resulting discretization properties. Three standard families are commonly used: Labatto, Radau and Gauss.

Let's consider these three setup with more generality over any given basis. We start again by taking a mesh interval $[t_k, t_{k+1}]$ of length $h_k = t_{k+1} - t_k$ and reparametrize time by

$$t = t_k + h_k \tau, \quad \tau \in [0, 1]. \quad (135)$$

We then choose to represent the (unknown) state by a degree- K polynomial

$$\mathbf{x}_h(\tau) = \sum_{n=0}^K \mathbf{a}_n \phi_n(\tau), \quad (136)$$

where $\{\phi_n\}_{n=0}^K$ is any linearly independent basis of polynomials of degree $\leq K$, and $\mathbf{a}_0, \dots, \mathbf{a}_K$ are vector coefficients to be determined.

The **collocation condition** mean that we require for the chosen K collocation points that:

$$\frac{d}{dt}\mathbf{x}_h(\tau_j) = \mathbf{f}(\mathbf{x}_h(\tau_j), \mathbf{u}_h(\tau_j), t_k + h_k\tau_j), \quad j = 0, 1, \dots, K, \quad (137)$$

which, using $\frac{d}{dt} = (1/h_k)\frac{d}{d\tau}$, becomes

$$\underbrace{\frac{1}{h_k}\mathbf{x}'_h(\tau_j)}_{\text{polynomial slope at node}} = \underbrace{\mathbf{f}(\mathbf{x}_h(\tau_j), \mathbf{u}_h(\tau_j), t_k + h_k\tau_j)}_{\text{ODE slope at same point}}, \quad j = 0, \dots, K. \quad (138)$$

Put simply: choose some nodes inside the interval, and at each of those nodes force the slope of the polynomial approximation to match the slope prescribed by the ODE. What we mean by the expression “collocation conditions” is simply to say that we want to satisfy a set of “slope-matching equations” at the chosen nodes.

By **definition of the mesh variables**,

$$\mathbf{x}_k := \mathbf{x}_h(0), \quad \mathbf{x}_{k+1} := \mathbf{x}_h(1), \quad (139)$$

and (if you sample the control at endpoints)

$$\mathbf{u}_k := \mathbf{u}_h(0), \quad \mathbf{u}_{k+1} := \mathbf{u}_h(1). \quad (140)$$

With the monomial basis,

$$\phi_n(0) = \delta_{n0} \Rightarrow \mathbf{x}_h(0) = \sum_{n=0}^K \mathbf{a}_n \phi_n(0) = \mathbf{a}_0 = \mathbf{x}_k, \quad (141)$$

$$\phi_n(1) = 1 \Rightarrow \mathbf{x}_h(1) = \sum_{n=0}^K \mathbf{a}_n = \mathbf{x}_{k+1}. \quad (142)$$

For the derivative, $\phi'_n(\tau) = n\tau^{n-1}$, so

$$\mathbf{x}'_h(0) = \sum_{n=0}^K \mathbf{a}_n \phi'_n(0) = \mathbf{a}_1, \quad \mathbf{x}'_h(1) = \sum_{n=1}^K n \mathbf{a}_n. \quad (143)$$

When chaining intervals into a global trajectory, **direct collocation enforces state continuity by construction**: the variable $\mathbf{x}_{k+1}$ at the end of one interval is the same as the starting variable of the next. What is *not* enforced automatically is **slope continuity**; the derivative at the end of one interval generally does not match the derivative at the start of the next. Different collocation methods may have different slope continuity properties depending on the chosen collocation nodes.

Lobatto Nodes (endpoints included):

The family of Lobatto methods correspond to any set of so-called **Lobatto nodes** $\{\tau_j\}_{j=0}^K$ with the specificity that we require $\tau_0 = 0$ and $\tau_K = 1$. Let's assume that we work with the **power (monomial) basis** $\phi_n(\tau) = \tau^n$, so that

$$\mathbf{x}_h(\tau) = \sum_{n=0}^K \mathbf{a}_n \tau^n, \quad (144)$$

Differentiating $\mathbf{x}_h$ with respect to τ gives

$$\frac{d\mathbf{x}_h}{d\tau}(\tau) = \sum_{n=0}^K \mathbf{a}_n \phi'_n(\tau), \quad (145)$$

Since we have the chain rule $\frac{d}{dt} = \frac{1}{h_k} \frac{d}{d\tau}$ from the time transformation $t = t_k + h_k \tau$, the time derivative becomes

$$\frac{d\mathbf{x}_h}{dt}(t_k + h_k \tau) = \frac{1}{h_k} \frac{d\mathbf{x}_h}{d\tau}(\tau) = \frac{1}{h_k} \sum_{n=0}^K \mathbf{a}_n \phi'_n(\tau). \quad (146)$$

so the **collocation equations** at Lobatto nodes are

$$\frac{1}{h_k} \sum_{n=0}^K \mathbf{a}_n \phi'_n(\tau_j) = \mathbf{f}\left(\sum_{n=0}^K \mathbf{a}_n \phi_n(\tau_j), \mathbf{u}_h(\tau_j), t_k + h_k \tau_j\right), \quad j = 0, 1, \dots, K. \quad (147)$$

For $j = 0$ and $j = K$, these conditions become:

$$\frac{1}{h_k} \sum_{n=0}^K \mathbf{a}_n \phi'_n(0) = \mathbf{f}\left(\sum_{n=0}^K \mathbf{a}_n \phi_n(0), \mathbf{u}_h(0), t_k\right), \quad (\text{left endpoint}) \quad (148)$$

$$\frac{1}{h_k} \sum_{n=0}^K \mathbf{a}_n \phi'_n(1) = \mathbf{f}\left(\sum_{n=0}^K \mathbf{a}_n \phi_n(1), \mathbf{u}_h(1), t_{k+1}\right), \quad (\text{right endpoint}) \quad (149)$$

With the monomial basis $\phi_n(\tau) = \tau^n$, we have $\phi'_n(0) = n\delta_{n,1}$ (only $\phi'_1 = 1$, others vanish) and $\phi'_n(1) = n$. Also, $\phi_n(0) = \delta_{n,0}$ and $\phi_n(1) = 1$ for all n . This simplifies the endpoint conditions to:

$$\frac{\mathbf{a}_1}{h_k} = \mathbf{f}(\mathbf{a}_0, \mathbf{u}_h(0), t_k) = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k), \quad (\text{left endpoint slope}) \quad (150)$$

$$\frac{1}{h_k} \sum_{n=1}^K n \mathbf{a}_n = \mathbf{f}\left(\sum_{n=0}^K \mathbf{a}_n, \mathbf{u}_h(1), t_{k+1}\right) = \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}), \quad (\text{right endpoint slope}) \quad (151)$$

These equations enforce that the polynomial's slope at both endpoints matches the ODE's prescribed slope, which is why the figure shows red tangent lines at both endpoints for Lobatto methods.

Radau Nodes (one endpoint included): Radau points include only one endpoint. Radau-I includes the left endpoint ($\tau_0 = 0$) while Radau-II includes the right endpoint ($\tau_K = 1$). This means that a radau collocation is defined by any set of collocation nodes such that $\tau_K = 1$. This translates into requiring that we match the ODE over the mesh only at the right endpoint in addition to the interior nodes. As a consequence, we leave the solution unconstrained to take any value on the left endpoint. When chaining up multiple intervals across a global solution, this may pose some complication as we will no longer be able to ensure continuity as the slope at one endpoint need not match that of the next endpoint. (But could you have a situation where slopes match but the states don't line up?)

At the included endpoint the ODE is enforced (slope shown in the figure), while at the other endpoint continuity links adjacent intervals. For Radau-I with $K + 1$ points:

$$\mathbf{x}_k = \mathbf{x}_h(0) = \mathbf{a}_0 \quad (152)$$

The endpoint $\mathbf{x}_{k+1} = \mathbf{x}_h(1) = \sum_{n=0}^K \mathbf{a}_n$ is not directly constrained by a collocation condition, requiring separate continuity enforcement between intervals.

Gauss Nodes (endpoints excluded): Gauss points exclude both endpoints, using only interior points $\tau_j \in (0, 1)$ for $j = 1, \dots, K$. The ODE is enforced only at interior nodes; both endpoints are handled through separate continuity constraints:

$$\mathbf{x}_k = \mathbf{x}_h(0) = \mathbf{a}_0 \quad (153)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_h(1) = \sum_{n=0}^K \mathbf{a}_n \quad (154)$$

Origins and Selection Criteria: These node families derive from orthogonal polynomial theory. Gauss nodes correspond to roots of Legendre polynomials and provide optimal quadrature accuracy for smooth integrands. Radau nodes are roots of modified Legendre polynomials with one endpoint constraint, while Lobatto nodes include both endpoints and correspond to roots of derivatives of Legendre polynomials.

For optimal control applications, Radau-II nodes are often preferred because they provide implicit time-stepping behavior and good stability properties. Lobatto nodes simplify boundary condition handling but may require smaller time steps. Gauss nodes offer highest quadrature accuracy but complicate endpoint treatment.

Recap

The three standard node families differ in how they treat interval endpoints. **Lobatto** nodes include both endpoints, so the ODE is enforced there and slopes match automatically at mesh boundaries. **Radau** nodes include one endpoint (left for Radau-I, right for Radau-II), leaving the other free for continuity constraints. **Gauss** nodes exclude both endpoints entirely, requiring separate continuity conditions at each mesh point. The choice affects stability, accuracy, and how boundary conditions propagate through the transcription.

Control Parameterization and Cost Integration The control inputs can be handled with similar polynomial approximations. We may use piecewise-constant controls, piecewise-linear controls, or higher-order polynomial parameterizations of the form:

$$\mathbf{u}_h(\tau) = \sum_{n=0}^{K_u} \mathbf{b}_n \tau^n \quad (155)$$

where $\mathbf{u}_j = \mathbf{u}_h(\tau_j)$ represents the control values at each collocation point. This polynomial framework extends to cost function evaluation, where running costs are integrated using the same quadrature nodes and weights:

$$\int_{t_k}^{t_{k+1}} c \, dt \approx h_k \sum_{j=0}^K w_j c(\mathbf{x}_h(\tau_j), \mathbf{u}_h(\tau_j), t_k + h_k \tau_j) \quad (156)$$

4.2.5 A Compendium of Direct Transcription Methods in Trajectory Optimization

The mesh and interior nodes are the common scaffold. What distinguishes one transcription from another is how we obtain values at those nodes and how we approximate the two integrals that appear implicitly and explicitly: the integral of the running cost and the integral that carries the state forward. In other words, we now commit to two design choices that mirror the previous section: a finite representation for $\mathbf{x}(t)$ and $\mathbf{u}(t)$ over each interval $[t_i, t_{i+1}]$, and a quadrature rule whose nodes and weights are used consistently for both cost and dynamics. The result is always a sparse nonlinear program; the differences are in where we sample and how we tie samples together.

Below, each transcription should be read as “same grid, same interior points, same evaluations for cost and physics,” with only the local representation changing.

Euler Collocation Work on one interval $[t_k, t_{k+1}]$ of length h_k with the reparametrization $t = t_k + h_k \tau$, $\tau \in [0, 1]$. Assume a degree 1 polynomial:

$$\mathbf{x}_h(\tau) = \sum_{n=0}^1 \mathbf{a}_n \phi_n(\tau), \quad (157)$$

for any basis $\{\phi_0, \phi_1\}$ of linear polynomials. Endpoint conditions give

$$\mathbf{x}_h(0) = \mathbf{x}_k \Rightarrow \mathbf{a}_0 = \mathbf{x}_k, \quad \mathbf{x}_h(1) = \mathbf{x}_{k+1} \Rightarrow \mathbf{a}_1 = \mathbf{x}_{k+1} - \mathbf{x}_k. \quad (158)$$

by backsubstitution and because

$$\mathbf{x}_h(\tau) = \mathbf{a}_0 + \mathbf{a}_1 \tau. \quad (159)$$

Furthermore, the derivative with respect to τ is:

$$\frac{d}{d\tau} \mathbf{x}_h(\tau) = \mathbf{a}_1 = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad \frac{d}{dt} = \frac{1}{h_k} \frac{d}{d\tau} \Rightarrow \frac{d}{dt} \mathbf{x}_h = \frac{1}{h_k} (\mathbf{x}_{k+1} - \mathbf{x}_k). \quad (160)$$

Because from the mapping $t(\tau) = t_k + h_k \tau$ we can invert: $\tau(t) = \frac{t-t_k}{h_k}$ and differentiating gives

$$\frac{d\tau}{dt} = \frac{1}{h_k}. \quad (161)$$

The **collocation condition** at a single **Radau-II node** $\tau = 1$:

$$\left. \frac{1}{h_k} \mathbf{x}'_h(\tau) \right|_{\tau=1} = \mathbf{f}(\mathbf{x}_h(1), \mathbf{u}_h(1), t_{k+1}) = \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}). \quad (162)$$

Because $\mathbf{x}_h$ is linear, $\mathbf{x}'_h(\tau)$ is constant in τ , so $\mathbf{x}'_h(1) = \mathbf{x}'_h(\tau)$ for all τ . Moreover, linear interpolation between the two endpoints gives

$$\mathbf{x}'_h(\tau) = \mathbf{x}_{k+1} - \mathbf{x}_k. \quad (163)$$

Substitute this into the collocation condition:

$$\frac{1}{h_k} (\mathbf{x}_{k+1} - \mathbf{x}_k) = \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}), \quad (164)$$

which is exactly the **implicit Euler** step

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h_k \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}). \quad (165)$$

The overall direct transcription is then:

Definition 4.1 (Implicit–Euler Collocation (Radau-II, degree 1)). *Let $t_0 < \dots < t_N$ with $h_i := t_{i+1} - t_i$. Decision variables are $\{\mathbf{x}_i\}_{i=0}^N$, $\{\mathbf{u}_i\}_{i=0}^N$. Solve*

$$\begin{aligned}
\min \quad & c_T(\mathbf{x}_N) + \sum_{i=0}^{N-1} h_i c(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \\
s.t. \quad & \mathbf{x}_{i+1} - \mathbf{x}_i - h_i \mathbf{f}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) = \mathbf{0}, \quad i = 0, \dots, N-1, \\
& \mathbf{g}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \leq \mathbf{0}, \\
& \mathbf{x}_{\min} \leq \mathbf{x}_i \leq \mathbf{x}_{\max}, \quad \mathbf{u}_{\min} \leq \mathbf{u}_i \leq \mathbf{u}_{\max}, \\
& \mathbf{x}_0 = \mathbf{x}(t_0).
\end{aligned} \tag{166}$$

Note that:

- The running cost and path constraints are evaluated at the **same** right-endpoint where the dynamics are enforced, keeping “where we pay” aligned with “where we enforce.”
- State continuity is automatic because $\mathbf{x}_{i+1}$ is a shared variable between adjacent intervals; slope continuity is not enforced unless you add it. Here’s an updated subsection that explicitly says **what collocation nodes are chosen** and why the trapezoidal defect uses them the way it does.

Side remark. If you instead collocate at the **left** endpoint (Radau-I with $\tau = 0$) with the same linear model, you obtain $\frac{1}{h_k}(\mathbf{x}_{k+1} - \mathbf{x}_k) = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k)$, i.e., the **explicit Euler** step. In that very precise sense, explicit Euler can be viewed as a (left-endpoint) degree-1 collocation scheme.

Definition 4.2 (Explicit–Euler Collocation (Radau-I, degree 1)). *Let $t_0 < \dots < t_N$ with $h_i := t_{i+1} - t_i$. Decision variables are $\{\mathbf{x}_i\}_{i=0}^N$ and $\{\mathbf{u}_i\}_{i=0}^N$. Solve*

$$\begin{aligned}
\min \quad & c_T(\mathbf{x}_N) + \sum_{i=0}^{N-1} h_i c(\mathbf{x}_i, \mathbf{u}_i) \\
s.t. \quad & \mathbf{x}_{i+1} - \mathbf{x}_i - h_i \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) = \mathbf{0}, \quad i = 0, \dots, N-1, \\
& \mathbf{g}(\mathbf{x}_i, \mathbf{u}_i) \leq \mathbf{0}, \quad i = 0, \dots, N-1, \\
& \mathbf{x}_{\min} \leq \mathbf{x}_i \leq \mathbf{x}_{\max}, \quad \mathbf{u}_{\min} \leq \mathbf{u}_i \leq \mathbf{u}_{\max}, \\
& \mathbf{x}_0 = \mathbf{x}(t_0).
\end{aligned} \tag{167}$$

Trapezoidal collocation In this scheme we take the **two endpoints as the nodes** on each interval:

$$\tau_0 = 0, \quad \tau_1 = 1 \quad (\text{“Lobatto with } K = 1\text{”}). \tag{168}$$

We approximate $\mathbf{x}$ **linearly** over $[t_i, t_{i+1}]$, and we evaluate both the running cost and the dynamics at these two nodes with **equal weights**. Because a linear polynomial has a **constant** derivative, we do **not** try to match the ODE’s slope at both endpoints (that would overconstrain a linear function). Instead, we enforce the ODE in its **integral form** over the interval and approximate the

integral of $\mathbf{f}$ by the **trapezoid rule** using those two nodes. This makes the cost quadrature and the state-update (“defect”) use the **same nodes and weights**.

Definition 4.3 (Trapezoidal Collocation). *Let $t_0 < \dots < t_N$ with $h_i := t_{i+1} - t_i$. Decision variables are $\{\mathbf{x}_i\}_{i=0}^N$, $\{\mathbf{u}_i\}_{i=0}^N$. Solve*

$$\begin{aligned} \min_{\{\mathbf{x}_i, \mathbf{u}_i\}} \quad & c_T(\mathbf{x}_N) + \sum_{i=0}^{N-1} \frac{h_i}{2} \left[c(\mathbf{x}_i, \mathbf{u}_i) + c(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \right] \\ \text{s.t.} \quad & \mathbf{x}_{i+1} - \mathbf{x}_i - \frac{h_i}{2} \left[\mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) + \mathbf{f}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \right] = \mathbf{0}, \quad i = 0, \dots, N-1, \\ & \mathbf{g}(\mathbf{x}_i, \mathbf{u}_i) \leq \mathbf{0}, \quad \mathbf{g}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \leq \mathbf{0}, \\ & \mathbf{x}_{\min} \leq \mathbf{x}_i \leq \mathbf{x}_{\max}, \quad \mathbf{u}_{\min} \leq \mathbf{u}_i \leq \mathbf{u}_{\max}, \\ & \mathbf{x}_0 = \mathbf{x}(t_0). \end{aligned} \tag{169}$$

Summary: the **collocation nodes** for trapezoidal are the **two endpoints**; the state is **linear** on each interval; and the dynamics are enforced via the **integrated ODE** with the **trapezoid rule** at those two nodes, yielding the familiar trapezoidal defect.

Hermite–Simpson (quadratic interpolation; midpoint included) On each interval $[t_i, t_{i+1}]$ we pick **three collocation nodes** on the reference domain $\tau \in [0, 1]$:

$$\tau_0 = 0, \quad \tau_{1/2} = \frac{1}{2}, \quad \tau_1 = 1. \tag{170}$$

So we evaluate at **left, midpoint, right**. These are the same three nodes used by Simpson’s rule (weights 1:4:1) for numerical quadrature. We let $\mathbf{x}_h$ be **quadratic** in τ . Two things then happen:

1. **Integral (defect) enforcement with Simpson’s rule.** We enforce the ODE in integral form over the interval and approximate the integral of $\mathbf{f}$ with Simpson’s rule using the three nodes above. This yields the first constraint (the “Simpson defect”), which uses $\mathbf{f}$ evaluated at left, midpoint, and right.
2. **Slope matching at the midpoint (collocation).** Because a quadratic has limited shape, we don’t try to match slopes at both endpoints. Instead, we **introduce midpoint variables** $(\mathbf{x}_{i+\frac{1}{2}}, \mathbf{u}_{i+\frac{1}{2}})$ and **match the ODE at the midpoint**. The second constraint below is exactly the midpoint collocation condition written in an equivalent Hermite form: it pins the midpoint state to the average of the endpoints plus a correction based on endpoint slopes, ensuring that the polynomial’s derivative is consistent with the ODE at $\tau = \frac{1}{2}$.

This way, **where we pay** (Simpson quadrature) and **where we enforce** (midpoint collocation + Simpson defect) are aligned at the same three nodes, which is why the method is both accurate and well conditioned on smooth problems.

Definition 4.4 (Hermite–Simpson Transcription). *Let $t_0 < \dots < t_N$ with $h_i := t_{i+1} - t_i$ and midpoints $t_{i+\frac{1}{2}}$. Decision variables are $\{\mathbf{x}_i\}_{i=0}^N$, $\{\mathbf{u}_i\}_{i=0}^N$, plus midpoint variables $\{\mathbf{x}_{i+\frac{1}{2}}, \mathbf{u}_{i+\frac{1}{2}}\}_{i=0}^{N-1}$. Solve*

$$\begin{aligned}
\min \quad & c_T(\mathbf{x}_N) + \sum_{i=0}^{N-1} \frac{h_i}{6} \left[c(\mathbf{x}_i, \mathbf{u}_i) + 4c(\mathbf{x}_{i+\frac{1}{2}}, \mathbf{u}_{i+\frac{1}{2}}) + c(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \right] \\
\text{s.t.} \quad & \underbrace{\mathbf{x}_{i+1} - \mathbf{x}_i - \frac{h_i}{6} \left[\mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) + 4\mathbf{f}(\mathbf{x}_{i+\frac{1}{2}}, \mathbf{u}_{i+\frac{1}{2}}) + \mathbf{f}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \right]}_{\text{Simpson defect over } [t_i, t_{i+1}]} = \mathbf{0}, \\
& \underbrace{\mathbf{x}_{i+\frac{1}{2}} - \frac{\mathbf{x}_i + \mathbf{x}_{i+1}}{2} - \frac{h_i}{8} \left[\mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) - \mathbf{f}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \right]}_{\text{midpoint collocation (slope matching at } t_{i+\frac{1}{2}})} = \mathbf{0}, \\
& \mathbf{g}(\mathbf{x}_i, \mathbf{u}_i) \leq \mathbf{0}, \quad \mathbf{g}(\mathbf{x}_{i+\frac{1}{2}}, \mathbf{u}_{i+\frac{1}{2}}) \leq \mathbf{0}, \quad \mathbf{g}(\mathbf{x}_{i+1}, \mathbf{u}_{i+1}) \leq \mathbf{0}, \\
& \mathbf{x}_{\min} \leq \mathbf{x}_i, \mathbf{x}_{i+\frac{1}{2}} \leq \mathbf{x}_{\max}, \quad \mathbf{u}_{\min} \leq \mathbf{u}_i, \mathbf{u}_{i+\frac{1}{2}} \leq \mathbf{u}_{\max}, \\
& \mathbf{x}_0 = \mathbf{x}(t_0), \quad i = 0, \dots, N-1.
\end{aligned} \tag{171}$$

Collocation nodes recap: $\tau = 0, \frac{1}{2}, 1$.

- The **midpoint** is where we explicitly **match the ODE slope** (collocation).
- The **three nodes together** are used for the **Simpson integral** of $\mathbf{f}$ (state update) and of c (cost), keeping physics and objective synchronized.

4.2.6 Examples

Compressor Surge Problem A compressor is a machine that raises the pressure of a gas by squeezing it into a smaller volume. You find them in natural gas pipelines, jet engines, and factories. But compressors can run into trouble if the flow of gas becomes too small. In that case, the machine can “stall” much like an airplane wing at too high an angle. Instead of moving forward, the gas briefly pushes back, creating strong pressure oscillations that can damage the compressor and anything connected to it.

To prevent this, engineers often add a close-coupled valve (CCV) at the outlet. The valve can quickly adjust the flow to keep the compressor away from these unstable conditions. Our goal is to design a control strategy for operating this valve so that the compressor never enters surge.

Following [Gravdahl and Egeland, 1997] and Grancharova and Johansen [2012], we model the compressor using a simplified second-order representation:

$$\begin{aligned}\dot{x}_1 &= B(\Psi_e(x_1) - x_2 - u) \\ \dot{x}_2 &= \frac{1}{B}(x_1 - \Phi(x_2))\end{aligned}\tag{172}$$

Here, $\mathbf{x} = [x_1, x_2]^T$ represents the state variables:

- x_1 is the normalized mass flow through the compressor.
- x_2 is the normalized pressure ratio across the compressor.

The control input u denotes the normalized mass flow through a CCV. The functions $\Psi_e(x_1)$ and $\Phi(x_2)$ represent the characteristics of the compressor and valve, respectively:

$$\begin{aligned}\Psi_e(x_1) &= \psi_{c0} + H \left(1 + 1.5 \left(\frac{x_1}{W} - 1 \right) - 0.5 \left(\frac{x_1}{W} - 1 \right)^3 \right) \\ \Phi(x_2) &= \gamma \operatorname{sign}(x_2) \sqrt{|x_2|}\end{aligned}\tag{173}$$

The system parameters are given as $\gamma = 0.5$, $B = 1$, $H = 0.18$, $\psi_{c0} = 0.3$, and $W = 0.25$.

One possible way to pose the problem [Grancharova and Johansen \[2012\]](#) is by penalizing deviations from the setpoints using a quadratic penalty in the instantaneous cost function as well as in the terminal one. Furthermore, we also penalize taking large actions (which are energy hungry and potentially unsafe) within the integral term. The idea of penalizing deviations throughout is a natural way of posing the problem when solving it via single shooting. Another alternative, which we will explore below, is to set the desired setpoint as a hard terminal constraint.

The control objective is to stabilize the system and prevent surge, formulated as a continuous-time optimal control problem (COCP) in the Bolza form:

$$\begin{aligned}\text{minimize} \quad & \left[\int_0^T \alpha(\mathbf{x}(t) - \mathbf{x}^*)^T (\mathbf{x}(t) - \mathbf{x}^*) + \kappa u(t)^2 dt \right] + \beta(\mathbf{x}(T) - \mathbf{x}^*)^T (\mathbf{x}(T) - \mathbf{x}^*) + Rv^2 \\ \text{subject to} \quad & \dot{x}_1(t) = B(\Psi_e(x_1(t)) - x_2(t) - u(t)) \\ & \dot{x}_2(t) = \frac{1}{B}(x_1(t) - \Phi(x_2(t))) \\ & u_{\min} \leq u(t) \leq u_{\max} \\ & -x_2(t) + 0.4 \leq v \\ & -v \leq 0 \\ & \mathbf{x}(0) = \mathbf{x}_0\end{aligned}\tag{174}$$

The parameters α , β , κ , and R are non-negative weights that allow the designer to prioritize different aspects of performance (e.g., tight setpoint tracking

vs. smooth control actions). We also constraint the control input to be within $0 \leq u(t) \leq 0.3$ due to the physical limitations of the valve.

The authors in [Grancharova and Johansen \[2012\]](#) also add a soft path constraint $x_2(t) \geq 0.4$ to ensure that we maintain a minimum pressure at all time. This is implemented as a soft constraint using slack variables. The reason that we have the term Rv^2 in the objective is to penalizes violations of the soft constraint: we allow for deviations, but don't want to do it too much.

In the experiment below, we choose the setpoint $\mathbf{x}^* = [0.40, 0.60]^T$ as it corresponds to an unstable equilibrium point. If we were to run the system without applying any control, we would see that the system starts to oscillate.

Solution by Trapezoidal Collocation Another way to pose the problem is by imposing a terminal state constraint on the system rather than through a penalty in the integral term. In the following experiment, we use a problem formulation of the form:

$$\begin{aligned} & \text{minimize} && \left[\int_0^T \kappa u(t)^2 dt \right] \\ & \text{subject to} && \dot{x}_1(t) = B(\Psi_e(x_1(t)) - x_2(t) - u(t)) \\ & && \dot{x}_2(t) = \frac{1}{B}(x_1(t) - \Phi(x_2(t))) \\ & && u_{\min} \leq u(t) \leq u_{\max} \\ & && \mathbf{x}(0) = \mathbf{x}_0 \\ & && \mathbf{x}(T) = \mathbf{x}^* \end{aligned} \tag{175}$$

We then find a control function $u(t)$ and state trajectory $x(t)$ using the trapezoidal collocation method.

You can try to vary the number of collocation points in the code and observe how the state trajectory progressively matches the ground truth (the line denoted “integrated solution”). Note that this version of the code also lacks bound constraints on the variable x_2 to ensure a minimum pressure, as we did earlier. Consider this a good exercise to try on your own.

System Identification as Trajectory Optimization (Compressor Surge)

We now turn the compressor surge model into a simple system identification task: estimate unknown parameters (here, the scalar B) from measured trajectories. This can be viewed as a trajectory optimization problem: choose parameters (and optionally states) to minimize reconstruction error while enforcing the dynamics.

Given time-aligned data $\{(\mathbf{u}_k, \mathbf{y}_k)\}_{k=0}^N$, model states $\mathbf{x}_k \in R^d$, outputs $\mathbf{y}_k \approx \mathbf{h}(\mathbf{x}_k; \boldsymbol{\theta})$, step Δt , and dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u}; \boldsymbol{\theta})$, the simultaneous (full-discretization) viewpoint is

$$\begin{aligned}
& \min_{\boldsymbol{\theta}, \{\mathbf{x}_k\}} \sum_{k \in K} \|\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k; \boldsymbol{\theta})\|_2^2 \\
& \text{s.t. } \mathbf{x}_{k+1} - \mathbf{x}_k - \Delta t \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k; \boldsymbol{\theta}) = \mathbf{0}, \quad k = 0, \dots, N-1, \\
& \mathbf{x}_0 \text{ given,}
\end{aligned} \tag{176}$$

while the single-shooting (recursive elimination) variant eliminates the states by simulating forward from $\mathbf{x}_0$:

$$J(\boldsymbol{\theta}) := \sum_{k \in K} \|\mathbf{y}_k - \mathbf{h}(\boldsymbol{\phi}_k(\boldsymbol{\theta}; \mathbf{x}_0, \mathbf{u}_{0:N-1}))\|_2^2, \quad \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \tag{177}$$

where $\boldsymbol{\phi}_k$ denotes the state reached at step k by an RK4 rollout under parameter $\boldsymbol{\theta}$. In our demo the data grid and rollout grid coincide, so $\boldsymbol{\phi}_k = \mathbf{x}_k$ and no interpolation is required. We will identify B by fitting the model to data generated from the ground-truth $B = 1$ system under randomized initial conditions and small input perturbations.

Flight Trajectory Optimization We consider a concrete task: computing a fuel-optimal trajectory between Montréal–Trudeau (CYUL) and Toronto Pearson (CYYZ), taking into account both aircraft dynamics and wind conditions along the route. For this demo, we leverage the excellent library [OpenAP.top](https://openap.top) which provides direct transcription methods and airplane dynamics models [Sun, 2022]. Furthermore, it allows us to import a wind field comes from **ERA5** [C3S, 2018], a global atmospheric dataset. It combines historical observations from satellites, aircraft, and surface stations with a weather model to reconstruct the state of the atmosphere across space and time. In climate science, this is called a *reanalysis*.

ERA5 data is stored in **GRIB files**, a compact format widely used in meteorology. Each file contains a **gridded field**: values of wind and other variables arranged on a regular 4D lattice over longitude, latitude, altitude, and time. Since the aircraft rarely sits exactly on a grid point, we interpolate the wind components it sees as it moves.

The aircraft is modeled as a point mass with state

$$\mathbf{x}(t) = (x(t), y(t), h(t), m(t)), \tag{178}$$

where (x, y) is horizontal position, h is altitude, and m is remaining mass. Controls are Mach number $M(t)$, vertical speed $v_s(t)$, and heading angle $\psi(t)$. The equations of motion combine airspeed and wind:

$$\begin{aligned}
\dot{x} &= v(M, h) \cos \psi \cos \gamma + u_w(x, y, h, t), \\
\dot{y} &= v(M, h) \sin \psi \cos \gamma + v_w(x, y, h, t), \\
\dot{h} &= v_s, \\
\dot{m} &= -\text{FF}(T(h, M, v_s), h, M, v_s),
\end{aligned} \tag{179}$$

where $\gamma = \arcsin(v_s/v)$ is the flight path angle and FF is the fuel flow rate based on current conditions. The wind terms u_w and v_w are taken from ERA5 and interpolated in space and time.

The optimization minimizes fuel burn over the CYUL–CYYZ leg. But the same setup could be used to minimize arrival time, or some weighted combination of time, cost, and emissions.

We use `OpenAP.top`, which solves the problem using direct collocation at **Legendre–Gauss–Lobatto (LGL)** points. Each trajectory segment is mapped to the unit interval, the state is interpolated by Lagrange polynomials at nonuniform LGL nodes, and the dynamics are enforced at those points. Integration is done with matching quadrature weights.

This setup lets us optimize trajectories under realistic conditions by feeding in the appropriate ERA5 GRIB file (e.g., `era5_mtl_20230601_12.grib`). The result accounts for wind patterns (eg. headwinds, tailwinds, shear) along the corridor between Montréal and Toronto.

Hydro Cascade Scheduling with Physical Routing and Multiple Shooting In [Dynamic Programming](#), we introduced a simplified view of hydro reservoir control, where the water level evolves in discrete time by accounting for inflow and outflow, with precipitation treated as a noisy input. While useful for learning and control design, that model abstracts away much of the physical behavior of actual rivers and dams.

In this chapter, we move toward a more realistic setup inspired by [\[Savorgnan et al., 2011\]](#). We consider a series of dams arranged in a cascade, where the actions taken upstream influence downstream levels with a delay. The amount of power produced depends not only on how much water flows through the turbines, but also on the head (the vertical distance between the reservoir surface and the turbine outlet). The larger the head, the more potential energy is available for conversion into electricity, and the higher the power output.

To capture these effects, we follow a modeling approach inspired by the Saint-Venant equations, which describe how water levels and flows evolve in open channels. Instead of solving the full PDEs, we use a reduced model that approximates each dammed section of river (called a reach) as a lumped system governed by an ordinary differential equation. The main variable of interest is the water level $h_r(t)$, which changes over time depending on how much water enters, how much is discharged through the turbines $q_r(t)$, and how much is spilled $s_r(t)$. The mass balance for reach r is written as:

$$\frac{dh_r(t)}{dt} = \frac{1}{A_r} (z_r(t) - q_r(t) - s_r(t)), \quad (180)$$

where A_r is the surface area of the reservoir, assumed constant. The inflow $z_r(t)$ to a reach either comes from nature (for the first dam), or from the upstream turbine and spill discharge, delayed by a travel time τ_{r-1} :

$$z_1(t) = \text{inflow}(t), \quad z_r(t) = q_{r-1}(t - \tau_{r-1}) + s_{r-1}(t - \tau_{r-1}), \quad \text{for } r > 1. \quad (181)$$

Power generation at each reach depends on how much water is discharged and the available head:

$$P_r(t) = \rho g \eta q_r(t) H_r(h_r(t)), \quad (182)$$

where ρ is water density, g is gravitational acceleration, η is turbine efficiency, and $H_r(h_r(t))$ denotes the head as a function of the water level. In some models, the head is approximated as the difference between the current level and a fixed tailwater height (the water level downstream of the dam, after it has passed through the turbine).

The operator's goal is to meet a target generation profile $P^{\text{ref}}(t)$, such as one dictated by a market dispatch or load-following constraint. This leads to an objective that minimizes the deviation from the target over the full horizon:

$$\min_{\{q_r(t), s_r(t)\}} \int_0^T \left(\sum_{r=1}^R P_r(t) - P^{\text{ref}}(t) \right)^2 dt. \quad (183)$$

In practice, this is combined with operational constraints: turbine capacity $0 \leq q_r(t) \leq \bar{q}_r$, spillway limits $0 \leq s_r(t) \leq \bar{s}_r$, and safe level bounds $h_r^{\min} \leq h_r(t) \leq h_r^{\max}$. Depending on the use case, one may also penalize spill to encourage water conservation, or penalize fast changes in levels for ecological reasons.

What makes this problem particularly interesting is the coupling across space and time. An upstream reach cannot simply act in isolation: if the operator wants reach r to produce power at a specific time, the water must be released by reach $r - 1$ sufficiently in advance. This coordination is further complicated by delays, nonlinearities in head-dependent power, and limited storage capacity.

We solve the problem using **multiple shooting**. Each reach is divided into local simulation segments over short time windows. Within each segment, the dynamics are integrated forward using the ODEs, and continuity constraints are added to ensure that the water levels match across segment boundaries. At the same time, the inflows passed from upstream reaches must arrive at the right time and be consistent with previous decisions. In discrete time, this gives rise to a set of state-update equations:

$$h_r^{k+1} = h_r^k + \Delta t \cdot \frac{1}{A_r} (z_r^k - q_r^k - s_r^k), \quad (184)$$

with delays handled by shifting z_r^k according to the appropriate travel time. These constraints are enforced as part of a nonlinear program, alongside the power tracking objective and control bounds.

Compared to the earlier inflow-outflow model, this richer setup introduces more structure, but also more opportunity. The cascade now behaves like a coordinated team: upstream reservoirs can store water in anticipation of future needs, while downstream dams adjust their output to match arrivals and avoid overflows. The optimization produces not just a schedule, but a strategy for how the entire system should act together to meet demand.

The figure shows the result of a multiple-shooting optimization applied to a three-reach hydroelectric cascade. The time horizon is discretized into 16 intervals, and SciPy’s `trust-constr` solver is used to find a feasible control sequence that satisfies mass balance, turbine and spillway limits, and Muskingum-style routing dynamics. Each reach integrates its own local ODE, with continuity constraints linking the flows between reaches.

The top-left panel shows the water levels in each reservoir. We observe that upstream reservoirs tend to increase their levels ahead of discharge events, building potential energy before releasing water downstream. The top-right panel shows turbine discharges for each reach. These vary smoothly and are temporally coordinated across the system. The bottom-right panel compares the total generation to a synthetic demand profile, which is generated by a sum of time-shifted sigmoids and normalized to be feasible given turbine capacities. The optimized schedule (orange) tracks this demand closely, while the initial guess (blue) lags behind. The bottom-left panel plots the routed inflows between reaches, which display the expected lag and smoothing effects from Muskingum routing. The interplay between these plots shows how the system anticipates, stores, and routes water to meet time-varying generation targets within physical and operational limits.

4.2.7 Exercises

4.2.8 Summary and Outlook

This chapter developed **direct transcription methods** for continuous-time optimal control problems. The central idea is to convert an infinite-dimensional optimization over functions into a finite-dimensional nonlinear program by:

1. Partitioning the time horizon into mesh intervals
2. Representing trajectories using polynomials on each interval
3. Enforcing the dynamics at selected collocation nodes
4. Approximating integrals using quadrature rules at the same nodes

We derived three transcription schemes of increasing sophistication:

- **Euler collocation** uses degree-1 polynomials and a single node per interval, recovering implicit or explicit Euler depending on whether the node is at the right or left endpoint.
- **Trapezoidal collocation** uses degree-1 polynomials with both endpoints as nodes, applying the trapezoid rule for integration.
- **Hermite–Simpson collocation** uses degree-2 polynomials with left, mid-point, and right nodes, achieving higher accuracy through Simpson’s rule.

The choice of **node family** (Gauss, Radau, or Lobatto) determines how boundary conditions are handled. Lobatto nodes include both endpoints and enforce slopes there automatically. Radau nodes include one endpoint, and Gauss nodes exclude both, requiring separate continuity constraints.

These methods form the computational core of **model predictive control** ([Model Predictive Control](#)), where the trajectory optimization is solved repeatedly from the current measured state. The sparse structure of the resulting NLP, where each interval contributes only local blocks, enables efficient solution even for long horizons.

Beyond the methods presented here, several extensions are important in practice:

- **Mesh refinement** and **hp-adaptive methods** adjust the number and placement of mesh points based on solution accuracy
- **Higher-order collocation** uses more nodes per interval for increased accuracy
- **Indirect methods** solve the optimality conditions (Pontryagin’s principle) directly rather than discretizing the primal problem

The examples in this chapter (compressor surge control, flight trajectory optimization, and hydro cascade scheduling) illustrate how direct collocation applies to realistic engineering systems with nonlinear dynamics, path constraints, and complex objectives.

4.2.9 Self-checks

5 From Trajectories to Policies

5.1 Model Predictive Control

The trajectory optimization methods presented so far compute a complete control trajectory from an initial state to a final time or state. Once computed, this trajectory is executed without modification, making these methods fundamentally open-loop. The control function, $\mathbf{u}[k]$ in discrete time or $\mathbf{u}(t)$ in continuous time, depends only on the clock, reading off precomputed values from memory or interpolating between them. This approach assumes perfect models and no disturbances. Under these idealized conditions, repeating the same control sequence from the same initial state would always produce identical results.

Real systems face modeling errors, external disturbances, and measurement noise that accumulate over time. A precomputed trajectory becomes increasingly irrelevant as these perturbations push the actual system state away from the predicted path. The solution is to incorporate feedback, making control decisions that respond to the current state rather than blindly following a predetermined schedule. While dynamic programming provides the theoretical framework for deriving feedback policies through value functions and Bellman equations, there exists a more direct approach that leverages the trajectory optimization methods already developed.

Closing the Loop by Replanning Model Predictive Control creates a feedback controller by repeatedly solving trajectory optimization problems. Rather than computing a single trajectory for the entire task duration, MPC solves a finite-horizon problem at each time step, starting from the current measured state. The controller then applies only the first control action from this solution before repeating the entire process. This strategy transforms any trajectory optimization method into a feedback controller.

The Receding Horizon Principle The defining characteristic of MPC is its receding horizon strategy. At each time step, the controller solves an optimization problem looking a fixed duration into the future, but this prediction window constantly moves forward in time. The horizon “recedes” because it always starts from the current time and extends forward by the same amount.

Consider the discrete-time optimal control problem in Bolza form:

$$\begin{aligned}
 & \text{minimize} && c_T(\mathbf{x}_N) + \sum_{k=0}^{N-1} c(\mathbf{x}_k, \mathbf{u}_k) \\
 & \text{subject to} && \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \\
 & && \mathbf{g}(\mathbf{x}_k, \mathbf{u}_k) \leq \mathbf{0} \\
 & && \mathbf{u}_{\min} \leq \mathbf{u}_k \leq \mathbf{u}_{\max} \\
 & \text{given} && \mathbf{x}_0 = \mathbf{x}_{\text{current}}
 \end{aligned} \tag{185}$$

At time step t , this problem optimizes over the interval $[t, t + N]$. At the next time step $t + 1$, the horizon shifts to $[t + 1, t + N + 1]$. What makes this work is that only the first control $\mathbf{u}_0^*$ from each optimization is applied. The

remaining controls $\mathbf{u}_1^*, \dots, \mathbf{u}_{N-1}^*$ are discarded, though they may initialize the next optimization through warm-starting.

This receding horizon principle enables feedback without computing an explicit policy. By constantly updating predictions based on current measurements, MPC naturally corrects for disturbances and model errors. The apparent waste of computing but not using most of the trajectory is actually the mechanism that provides robustness.

Horizon Selection and Problem Formulation The choice of prediction horizon depends on the control objective. We distinguish between three cases, each requiring different mathematical formulations.

Infinite-Horizon Regulation For stabilization problems where the system must operate indefinitely around an equilibrium, the true objective is:

$$J_\infty = \sum_{k=0}^{\infty} c(\mathbf{x}_k, \mathbf{u}_k) \quad (186)$$

Since this cannot be solved directly, MPC approximates it with:

$$\begin{aligned} & \text{minimize} && V_f(\mathbf{x}_N) + \sum_{k=0}^{N-1} c(\mathbf{x}_k, \mathbf{u}_k) \\ & \text{subject to} && \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \\ & && \mathbf{x}_N \in \mathcal{X}_f \\ & && \text{other constraints} \end{aligned} \quad (187)$$

The terminal cost $V_f(\mathbf{x}_N)$ approximates $\sum_{k=N}^{\infty} c(\mathbf{x}_k, \mathbf{u}_k)$, the cost-to-go beyond the horizon. The terminal constraint $\mathbf{x}_N \in \mathcal{X}_f$ ensures the state reaches a region where a known stabilizing controller exists. Without these terminal ingredients, the finite-horizon approximation may produce unstable behavior, as the controller ignores consequences beyond the horizon.

Finite-Duration Tasks For tasks ending at time t_f , the true objective spans from current time t to t_f :

$$J_{[t, t_f]} = c_f(\mathbf{x}(t_f)) + \sum_{k=t}^{t_f-1} c(\mathbf{x}_k, \mathbf{u}_k) \quad (188)$$

The MPC formulation must adapt as time progresses:

$$\begin{aligned}
& \text{minimize} && c_{T,k}(\mathbf{x}_{N_k}) + \sum_{j=0}^{N_k-1} c(\mathbf{x}_j, \mathbf{u}_j) \\
& \text{where} && N_k = \min(N, t_f - t_k) \\
& && c_{T,k} = \begin{cases} c_f & \text{if } t_k + N_k = t_f \\ c_T & \text{otherwise} \end{cases}
\end{aligned} \tag{189}$$

As the task approaches completion, the horizon shrinks and the terminal cost switches from the approximation c_T to the true final cost c_f . This prevents the controller from optimizing beyond task completion, which would produce meaningless or aggressive control actions.

Periodic Tasks Some systems operate on repeating cycles where the optimal behavior depends on the time of day, week, or season. Consider a commercial building where heating costs are higher at night, electricity prices vary hourly, and occupancy patterns repeat daily. The MPC controller must account for these periodic patterns while planning over a finite horizon.

For tasks with period T_p , such as daily building operations, the formulation accounts for transitions across period boundaries:

$$\begin{aligned}
& \text{minimize} && \sum_{k=0}^{N-1} c_k(\mathbf{x}_k, \mathbf{u}_k, \phi_k) \\
& \text{where} && \phi_k = (t + k) \bmod T_p \\
& && c_k(\cdot, \cdot, \phi) = \begin{cases} c_{\text{day}}(\cdot, \cdot) & \text{if } \phi \in [6\text{am}, 6\text{pm}] \\ c_{\text{night}}(\cdot, \cdot) & \text{otherwise} \end{cases}
\end{aligned} \tag{190}$$

The cost function changes based on the phase ϕ within the period. Constraints may similarly depend on the phase, reflecting different operational requirements at different times.

The MPC Algorithm The complete MPC procedure implements the receding horizon principle through repeated optimization:

Successive Linearization and Quadratic Approximations For many regulation and tracking problems, the nonlinear dynamics and costs we encounter can be approximated locally by linear and quadratic functions. The basic idea is to linearize the system around the current operating point and approximate the cost with a quadratic form. This reduces each MPC subproblem to a **quadratic program (QP)**, which can be solved reliably and very quickly using standard solvers.

Suppose the true dynamics are nonlinear,

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k). \tag{191}$$

Around a nominal trajectory $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$, we take a first-order expansion:

$$\mathbf{x}_{k+1} \approx f(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k) + \mathbf{A}_k(\mathbf{x}_k - \bar{\mathbf{x}}_k) + \mathbf{B}_k(\mathbf{u}_k - \bar{\mathbf{u}}_k), \quad (192)$$

with Jacobians

$$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k), \quad \mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k). \quad (193)$$

Similarly, if the stage cost is nonlinear,

$$c(\mathbf{x}_k, \mathbf{u}_k), \quad (194)$$

we approximate it quadratically near the nominal point:

$$c(\mathbf{x}_k, \mathbf{u}_k) \approx \|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_{\mathbf{Q}_k}^2 + \|\mathbf{u}_k - \mathbf{u}_k^{\text{ref}}\|_{\mathbf{R}_k}^2, \quad (195)$$

with positive semidefinite weighting matrices $\mathbf{Q}_k$ and $\mathbf{R}_k$.

The resulting MPC subproblem has the form

$$\begin{aligned} \min_{\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}} \quad & \|\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}\|_{\mathbf{P}}^2 + \sum_{k=0}^{N-1} (\|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_{\mathbf{Q}_k}^2 + \|\mathbf{u}_k - \mathbf{u}_k^{\text{ref}}\|_{\mathbf{R}_k}^2) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{A}_k \mathbf{x}_k + \mathbf{B}_k \mathbf{u}_k + \mathbf{d}_k, \\ & \mathbf{u}_{\min} \leq \mathbf{u}_k \leq \mathbf{u}_{\max}, \\ & \mathbf{x}_{\min} \leq \mathbf{x}_k \leq \mathbf{x}_{\max}, \\ & \mathbf{x}_0 = \mathbf{x}_{\text{current}}, \end{aligned} \quad (196)$$

where $\mathbf{d}_k = f(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k) - \mathbf{A}_k \bar{\mathbf{x}}_k - \mathbf{B}_k \bar{\mathbf{u}}_k$ captures the local affine offset.

Because the dynamics are now linear and the cost quadratic, this optimization problem is a convex quadratic program. Quadratic programs are attractive in practice: they can be solved at kilohertz rates with mature numerical methods, making them the backbone of many real-time MPC implementations.

At each MPC step, the controller updates its linearization around the new operating point, constructs the local QP, and solves it. The process repeats, with the linear model and quadratic cost refreshed at every reoptimization. Despite the approximation, this yields a closed-loop controller that inherits the fast computation of QPs while retaining the ability to track trajectories of the underlying nonlinear system.

Theoretical Guarantees The finite-horizon approximation in MPC brings a new challenge: the controller cannot see consequences beyond the horizon. Without proper design, this myopia can destabilize even simple systems. The solution is to carefully encode information about the infinite-horizon problem into the finite-horizon optimization through its terminal conditions.

Before diving into the mathematics, we should first establish what “stability” means and which tasks these theoretical guarantees address, as the notion of stability varies significantly across different control objectives.

Stability Notions Across Control Tasks The terminal conditions provide different types of guarantees depending on the control objective. For regulation problems, where the task is to drive the state to a fixed equilibrium $(\mathbf{x}_{\text{eq}}, \mathbf{u}_{\text{eq}})$ (often shifted to the origin), the stability guarantee is **asymptotic stability**: starting sufficiently close to the equilibrium, we have $\mathbf{x}_k \rightarrow \mathbf{x}_{\text{eq}}$ while constraints remain satisfied throughout the trajectory (**recursive feasibility**). This requires the stage cost $\ell(\mathbf{x}, \mathbf{u})$ to be positive definite in the deviation from equilibrium.

When tracking a constant setpoint, the task becomes following a constant reference $(\mathbf{x}_{\text{ref}}, \mathbf{u}_{\text{ref}})$ that solves the steady-state equations. This problem is handled by working in **error coordinates** $\tilde{\mathbf{x}} = \mathbf{x} - \mathbf{x}_{\text{ref}}$ and $\tilde{\mathbf{u}} = \mathbf{u} - \mathbf{u}_{\text{ref}}$, transforming the tracking problem into a regulation problem for the error system. The stability guarantee becomes asymptotic **tracking**, meaning $\tilde{\mathbf{x}}_k \rightarrow 0$, again with recursive feasibility.

The terminal conditions we discuss below primarily address regulation and constant reference tracking. Time-varying tracking and economic MPC require additional techniques such as tube MPC and dissipativity theory.

MPC with Stability Guarantees To provide theoretical guarantees, the finite-horizon MPC problem is augmented with three interconnected components. The **terminal cost** $V_f(\mathbf{x})$ approximates the cost-to-go beyond the horizon, providing a surrogate for the infinite-horizon tail that cannot be explicitly optimized. The **terminal constraint set** $\mathcal{X}_f$ defines a region where we have local knowledge of how to stabilize the system. Finally, the **terminal controller** $\kappa_f(\mathbf{x})$ provides a local stabilizing control law that remains valid within $\mathcal{X}_f$.

These components must satisfy specific compatibility conditions to provide theoretical guarantees:

Theorem 5.1 (Recursive Feasibility and Asymptotic Stability). *Consider the MPC problem with terminal cost V_f , terminal set $\mathcal{X}_f$, and local controller κ_f . If the following conditions hold:*

Control invariance: *For all $\mathbf{x} \in \mathcal{X}_f$, we have $\mathbf{f}(\mathbf{x}, \kappa_f(\mathbf{x})) \in \mathcal{X}_f$ (the set is invariant) and $\mathbf{g}(\mathbf{x}, \kappa_f(\mathbf{x})) \leq \mathbf{0}$ (constraints remain satisfied).*

Lyapunov decrease: *For all $\mathbf{x} \in \mathcal{X}_f$:*

$$V_f(\mathbf{f}(\mathbf{x}, \kappa_f(\mathbf{x}))) - V_f(\mathbf{x}) \leq -\ell(\mathbf{x}, \kappa_f(\mathbf{x})) \quad (197)$$

where ℓ is the stage cost.

Then the MPC controller achieves recursive feasibility (if the problem is feasible at time k , it remains feasible at time $k+1$), asymptotic stability to the target equilibrium for regulation problems, and monotonic cost decrease along trajectories until the target is reached.

Suboptimality Bounds The finite-horizon MPC value $V_N(\mathbf{x})$ provides an upper bound approximation of the true infinite-horizon value $V_\infty(\mathbf{x})$. Un-

derstanding how close this approximation can be tells us about the effectiveness of short-horizon MPC.

The upper bound $V_N(\mathbf{x}) \geq V_\infty(\mathbf{x})$ follows immediately from the fact that MPC considers fewer control choices. The infinite-horizon controller can choose any sequence $(\mathbf{u}_0, \mathbf{u}_1, \mathbf{u}_2, \dots)$, while the N -horizon controller is restricted to sequences of the form $(\mathbf{u}_0, \dots, \mathbf{u}_{N-1}, \kappa_f(\mathbf{x}_N), \kappa_f(\mathbf{x}_{N+1}), \dots)$ where the tail follows the fixed terminal controller. Since the infinite-horizon problem optimizes over a larger feasible set, its optimal value cannot exceed that of the finite-horizon problem.

Deriving the Approximation Error The interesting question is bounding the approximation error $\varepsilon_N = V_N(\mathbf{x}) - V_\infty(\mathbf{x})$. This error represents the cost of being forced to use κ_f beyond the horizon rather than continuing to optimize.

Let $(\mathbf{u}_0^*, \mathbf{u}_1^*, \dots)$ denote the infinite-horizon optimal control sequence with corresponding state trajectory $(\mathbf{x}_0^*, \mathbf{x}_1^*, \dots)$ where $\mathbf{x}_0^* = \mathbf{x}$. The infinite-horizon cost is:

$$V_\infty(\mathbf{x}) = \sum_{k=0}^{\infty} \ell(\mathbf{x}_k^*, \mathbf{u}_k^*) \quad (198)$$

Now consider what happens when we truncate this optimal sequence at horizon N and continue with the terminal controller. The cost becomes:

$$\tilde{V}_N(\mathbf{x}) = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k^*, \mathbf{u}_k^*) + V_f(\mathbf{x}_N^*) \quad (199)$$

where $V_f(\mathbf{x}_N^*)$ approximates the tail cost $\sum_{k=N}^{\infty} \ell(\mathbf{x}_k^*, \mathbf{u}_k^*)$.

Since $V_N(\mathbf{x})$ is the optimal N -horizon cost (which may do better than this particular truncated sequence), we have $V_N(\mathbf{x}) \leq \tilde{V}_N(\mathbf{x})$. The approximation error therefore satisfies:

$$\varepsilon_N \leq \tilde{V}_N(\mathbf{x}) - V_\infty(\mathbf{x}) = V_f(\mathbf{x}_N^*) - \sum_{k=N}^{\infty} \ell(\mathbf{x}_k^*, \mathbf{u}_k^*) \quad (200)$$

This bound shows that the approximation error depends on how well the terminal cost V_f approximates the true tail cost along the infinite-horizon optimal trajectory.

5.1.1 The Landscape of MPC Variants

Once the basic idea of receding-horizon control is clear, it is helpful to see how the same backbone accommodates many variations. In every case, we transcribe the continuous-time optimal control problem into a nonlinear program of the form

$$\begin{aligned}
& \text{minimize} && c(\mathbf{x}_N) + \sum_{k=0}^{N-1} w_k c(\mathbf{x}_k, \mathbf{u}_k) \\
& \text{subject to} && \mathbf{x}_{k+1} = \mathbf{F}_k(\mathbf{x}_k, \mathbf{u}_k) \\
& && \mathbf{g}(\mathbf{x}_k, \mathbf{u}_k) \leq \mathbf{0} \\
& && \mathbf{x}_{\min} \leq \mathbf{x}_k \leq \mathbf{x}_{\max} \\
& && \mathbf{u}_{\min} \leq \mathbf{u}_k \leq \mathbf{u}_{\max} \\
& \text{given} && \mathbf{x}_0 = \hat{\mathbf{x}}(t) .
\end{aligned} \tag{201}$$

The components in this NLP come from discretizing the continuous-time problem with a fixed horizon $[t, t+T]$ and step size Δt . The stage weights w_k and discrete dynamics $\mathbf{F}_k$ are determined by the choice of quadrature and integration scheme. With this blueprint in place, the rest is a matter of interpretation: how we define the cost, how we handle uncertainty, how we treat constraints, and what structure we exploit.

Tracking MPC The most common setup is reference tracking. Here, we are given time-varying target trajectories $(\mathbf{x}_k^{\text{ref}}, \mathbf{u}_k^{\text{ref}})$, and the controller's job is to keep the system close to these. The cost is typically quadratic:

$$\begin{aligned}
c(\mathbf{x}_k, \mathbf{u}_k) &= \|\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_k - \mathbf{u}_k^{\text{ref}}\|_{\mathbf{R}}^2 \\
c(\mathbf{x}_N) &= \|\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}\|_{\mathbf{P}}^2 .
\end{aligned} \tag{202}$$

When dynamics are linear and constraints are polyhedral, this yields a convex quadratic program at each time step.

Regulatory MPC In regulation tasks, we aim to bring the system back to an equilibrium point $(\mathbf{x}^e, \mathbf{u}^e)$, typically in the presence of disturbances. This is simply tracking MPC with constant references:

$$\begin{aligned}
c(\mathbf{x}_k, \mathbf{u}_k) &= \|\mathbf{x}_k - \mathbf{x}^e\|_{\mathbf{Q}}^2 + \|\mathbf{u}_k - \mathbf{u}^e\|_{\mathbf{R}}^2 \\
c(\mathbf{x}_N) &= \|\mathbf{x}_N - \mathbf{x}^e\|_{\mathbf{P}}^2 .
\end{aligned} \tag{203}$$

To guarantee stability, it is common to include a terminal constraint $\mathbf{x}_N \in \mathcal{X}_f$, where $\mathcal{X}_f$ is a control-invariant set under a known feedback law.

Economic MPC Not all systems operate around a reference. Sometimes the goal is to optimize a true economic objective (eg. energy cost, revenue, efficiency) directly. This gives rise to **economic MPC**, where the cost functions reflect real operational performance:

$$c(\mathbf{x}_k, \mathbf{u}_k) = c_{\text{op}}(\mathbf{x}_k, \mathbf{u}_k), \quad c(\mathbf{x}_N) = c_{\text{op},T}(\mathbf{x}_N) . \tag{204}$$

There is no reference trajectory here. The cost function determines the optimal behavior. In this setting, standard stability arguments no longer apply

automatically, and one must be careful to add terminal penalties or constraints that ensure the closed-loop system remains well-behaved.

Robust MPC Some systems are exposed to external disturbances or small errors in the model. In those cases, we want the controller to make decisions that will still work no matter what happens, as long as the disturbances stay within some known bounds. This is the idea behind **robust MPC**.

Instead of planning a single trajectory, the controller plans a “nominal” path (what would happen in the absence of any disturbance) and then adds a feedback correction to react to whatever disturbances actually occur. This looks like:

$$\mathbf{u}_k = \bar{\mathbf{u}}_k + \mathbf{K}(\mathbf{x}_k - \bar{\mathbf{x}}_k) , \quad (205)$$

where $\bar{\mathbf{u}}_k$ is the planned input and $\mathbf{K}$ is a feedback gain that pulls the system back toward the nominal path if it deviates.

Because we know the worst-case size of the disturbance, we can estimate how far the real state might drift from the plan, and “shrink” the constraints accordingly. The result is that the nominal plan is kept safely away from constraint boundaries, so even if the system gets pushed around, it stays inside limits. This is often called **tube MPC** because the true trajectory stays inside a tube around the nominal one.

The main benefit is that we can handle uncertainty without solving a complicated worst-case optimization at every time step. All the uncertainty is accounted for in the design of the feedback $\mathbf{K}$ and the tightened constraints.

Stochastic MPC If disturbances are random rather than adversarial, a natural goal is to optimize expected cost while enforcing constraints probabilistically. This gives rise to **stochastic MPC**, in which:

- The cost becomes an expectation:

$$E \left[c(\mathbf{x}_N) + \sum_{k=0}^{N-1} w_k c(\mathbf{x}_k, \mathbf{u}_k) \right] \quad (206)$$

- Constraints are allowed to be violated with small probability:

$$P[\mathbf{g}(\mathbf{x}_k, \mathbf{u}_k) \leq \mathbf{0}] \geq 1 - \varepsilon \quad (207)$$

In practice, expectations are approximated using a finite set of disturbance scenarios drawn ahead of time. For each scenario, the system dynamics are simulated forward using the same control inputs $\mathbf{u}_k$, which are shared across all scenarios to respect non-anticipativity. The result is a single deterministic optimization problem with multiple parallel copies of the dynamics, one per sampled future. This retains the standard MPC structure, with only moderate growth in problem size.

Despite appearances, this is not dynamic programming. There is no value function or tree of all possible paths. There is only a finite set of futures chosen a priori, and optimized over directly. This scenario-based approach is common in energy systems such as hydro scheduling, where inflows are uncertain but sample trajectories can be generated from forecasts.

Risk constraints are typically enforced across all scenarios or encoded using risk measures like CVaR. For example, one might penalize violations that occur in the worst $(1 - \alpha)\%$ of samples, while still optimizing expected performance overall.

Hybrid and Mixed-Integer MPC When systems involve discrete switches (eg. on/off valves, mode selection, or combinatorial logic) the MPC problem must include integer or binary variables. These show up in constraints like

$$\delta_k \in \{0, 1\}^m, \quad \mathbf{u}_k \in \mathcal{U}(\delta_k) \quad (208)$$

along with mode-dependent dynamics and costs. The resulting formulation is a **mixed-integer nonlinear program** (MINLP). The receding-horizon idea is the same, but each solve is more expensive due to the combinatorial nature of the decision space.

Distributed and Decentralized MPC Large-scale systems often consist of interacting subsystems. Distributed MPC decomposes the global NLP into smaller ones that run in parallel, with coordination constraints enforcing consistency across shared variables:

$$\sum_i \mathbf{H}^i \mathbf{z}_k^i = \mathbf{0} \quad (\text{coupling constraint}) \quad (209)$$

Each subsystem solves a local problem over its own state and input variables, then exchanges information with neighbors. Coordination can be done via primal-dual methods, ADMM, or consensus schemes, but each local block looks like a standard MPC problem.

Adaptive and Learning-Based MPC In practice, we may not know the true model $\mathbf{F}_k$ or cost function c precisely. In **adaptive MPC**, these are updated online from data:

$$\mathbf{x}_{k+1} = \mathbf{F}_k(\mathbf{x}_k, \mathbf{u}_k; \boldsymbol{\theta}_t), \quad c(\mathbf{x}_k, \mathbf{u}_k) = c(\mathbf{x}_k, \mathbf{u}_k; \phi_t) \quad (210)$$

The parameters $\boldsymbol{\theta}_t$ and ϕ_t are learned in real time. When combined with policy distillation, value approximation, or trajectory imitation, this leads to overlaps with reinforcement learning where the MPC solutions act as supervision for a reactive policy.

5.1.2 Robustness in Real-Time MPC

The trajectory optimization methods we have studied assume perfect models and deterministic dynamics. In practice, however, MPC controllers must operate in environments where models are approximate, disturbances are unpredictable, and computational resources are limited. The mathematical elegance of optimal control must always yield to the engineering reality of robust operation as **perfect optimality is less important than reliable operation**. This philosophy permeates industrial MPC applications. A controller that achieves 95% performance 100% of the time is superior to one that achieves 100% performance 95% of the time and fails catastrophically the remaining 5%. Airlines accept suboptimal fuel consumption over missed approaches, power grids tolerate efficiency losses to prevent blackouts, and chemical plants sacrifice yield for safety. By designing for failure, we want to create MPC systems that degrade gracefully rather than fail catastrophically, maintaining safety and stability even when the impossible is asked of them.

Example: Wind Farm Yield Optimization Consider a wind farm where MPC controllers coordinate individual turbines to maximize overall power production while minimizing wake interference. Each turbine can adjust both its thrust coefficient (through blade pitch) and yaw angle to redirect its wake away from downstream turbines. At time t_k , the MPC controller solves the optimization problem:

$$\begin{aligned}
& \min_{\mathbf{u}_{0:N-1}} \quad \sum_{i=0}^{N-1} \|\mathbf{x}_i - \mathbf{x}_i^{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_i\|_{\mathbf{R}}^2 \\
& \text{s.t.} \quad \mathbf{x}_{i+1} = \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) \\
& \quad \mathbf{x}_i \in \mathcal{X}_{\text{safe}} \\
& \quad \|\mathbf{u}_i\|_{\infty} \leq u_{\text{max}} \\
& \quad \mathbf{x}_0 = \mathbf{x}_{\text{current}}
\end{aligned} \tag{211}$$

Now suppose an unexpected wind direction change occurs, shifting the incoming wind vector by 30 degrees. The current state $\mathbf{x}_{\text{current}}$ reflects wake patterns that no longer align with the new wind direction, and the optimizer discovers that no feasible trajectory exists that can redirect all wakes appropriately within the physical limits of yaw rate and thrust adjustment. The solver reports infeasibility.

This scenario illustrates the fundamental challenge of real-time MPC: **constraint incompatibility**. When disturbances push the system into states from which recovery appears impossible, or when reference trajectories demand physically impossible maneuvers, the intersection of all constraint sets becomes empty. Model mismatch compounds this problem as prediction errors accumulate over the horizon.

Even when feasible solutions exist, **computational constraints** can prevent their discovery. A control loop running at 100 Hz allows only 10 millisec-

onds per iteration. If the solver requires 15 milliseconds to converge, we face an impossible choice: delay the control action and risk destabilizing the system, or apply an unconverged iterate that may violate critical constraints.

A third failure mode involves **numerical instabilities**: ill-conditioned matrices, rank deficiency, or division by zero in the linear algebra routines. These failures are particularly problematic because they occur sporadically, triggered by specific state configurations that create near-singular conditions in the optimization problem.

Softening Constraints Through Slack Variables The first approach to handling infeasibility recognizes that not all constraints carry equal importance. A chemical reactor’s temperature must never exceed the runaway threshold: this is a hard constraint that cannot be violated. However, maintaining temperature within an optimal efficiency band is merely desirable. This can be treated as a soft constraint that we prefer to satisfy but can relax when necessary.

This hierarchy motivates reformulating the optimization problem using **slack variables**:

$$\begin{aligned}
\min_{\mathbf{u}, \boldsymbol{\epsilon}} \quad & \sum_{i=0}^{N-1} \|\mathbf{x}_i - \mathbf{x}_i^{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_i\|_{\mathbf{R}}^2 + \boldsymbol{\rho}^T \boldsymbol{\epsilon}_i \\
\text{s.t.} \quad & \mathbf{x}_{i+1} = \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) \\
& \mathbf{g}_{\text{hard}}(\mathbf{x}_i, \mathbf{u}_i) \leq \mathbf{0} \\
& \mathbf{g}_{\text{soft}}(\mathbf{x}_i, \mathbf{u}_i) \leq \boldsymbol{\epsilon}_i \\
& \boldsymbol{\epsilon}_i \geq \mathbf{0}
\end{aligned} \tag{212}$$

The penalty weights $\boldsymbol{\rho}$ encode our priorities. Safety constraints might use $\rho_j = 10^6$, while comfort constraints use $\rho_j = 1$. This reformulated problem is always feasible as long as the hard constraints alone admit a solution. That is: we can always make the slack variables $\boldsymbol{\epsilon}$ sufficiently large to satisfy the soft constraints.

Rather than treating constraints as binary hard/soft categories, we can establish a **constraint hierarchy** that enables graceful degradation:

$$\begin{array}{ll}
\text{Safety:} & T_{\text{reactor}} \leq T_{\text{runaway}} - 10 \quad \rho = \infty \text{ (hard)} \\
\text{Equipment:} & 0 \leq u_{\text{valve}} \leq 100 \quad \rho = 10^4 \\
\text{Efficiency:} & T_{\text{optimal}} - 5 \leq T \leq T_{\text{optimal}} + 5 \quad \rho = 10^2 \\
\text{Comfort:} & |T - T_{\text{setpoint}}| \leq 1 \quad \rho = 1
\end{array} \tag{213}$$

As conditions deteriorate, the controller abandons objectives in reverse priority order, maintaining safety even when optimality becomes impossible.

Feasibility Restoration When even soft constraints prove insufficient (perhaps due to catastrophic solver failure or corrupted problem structure) we need **feasibility restoration** that finds any feasible point regardless of optimality:

$$\begin{aligned}
& \min_{\mathbf{u}, \mathbf{s}} \quad \|\mathbf{s}\|_1 \\
& \text{s.t.} \quad \mathbf{x}_{i+1} = \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i) + \mathbf{s}_i \\
& \quad \mathbf{x}_{\min} - \mathbf{s}_{x,i} \leq \mathbf{x}_i \leq \mathbf{x}_{\max} + \mathbf{s}_{x,i} \\
& \quad \mathbf{u}_{\min} \leq \mathbf{u}_i \leq \mathbf{u}_{\max} \\
& \quad \mathbf{s} \geq \mathbf{0}
\end{aligned} \tag{214}$$

This formulation temporarily relaxes even the dynamics constraints, finding the “least infeasible” solution. It answers the question: if we must violate something, what is the minimal violation required? Once feasibility is restored, we can warm-start the original problem from this point.

Reference Governors Rather than reacting to infeasibility after it occurs, we can prevent it by filtering references through a **reference governor**. Consider an aircraft following waypoints. Instead of passing waypoints directly to the MPC, the governor asks: what is the closest approachable reference from our current state?

$$\mathbf{r}_{\text{filtered}} = \arg \max_{\kappa \in [0,1]} \kappa \quad \text{s.t.} \quad \text{MPC}(\mathbf{x}_{\text{current}}, \kappa \mathbf{r}_{\text{desired}} + (1 - \kappa) \mathbf{x}_{\text{current}}) \text{ is feasible} \tag{215}$$

The governor performs a line search between the current state (always feasible since staying put requires no action) and the desired reference (potentially infeasible). This guarantees the MPC always receives feasible problems while making maximum progress toward the goal.

For computational efficiency, we can pre-compute the **maximal output admissible set**:

$$\mathcal{O}_{\infty} = \{\mathbf{r} : \exists \text{ feasible trajectory from } \mathbf{x} \text{ to } \mathbf{r} \text{ respecting all constraints}\} \tag{216}$$

Online, the governor simply projects the desired reference onto $\mathcal{O}_{\infty}$.

Backup Controllers When MPC fails entirely (due to solver crashes, timeouts, or numerical failures) we need backup controllers that require minimal computation while guaranteeing stability and keeping the system away from dangerous regions.

The standard approach uses a pre-computed **local LQR controller** around the equilibrium:

$$\mathbf{K}_{\text{LQR}}, \mathbf{P} = \text{LQR}(\mathbf{A}, \mathbf{B}, \mathbf{Q}, \mathbf{R}) \tag{217}$$

where $(\mathbf{A}, \mathbf{B})$ are the linearized dynamics at equilibrium. When MPC fails:

$$\mathbf{u}_{\text{backup}} = \begin{cases} \mathbf{K}_{\text{LQR}}(\mathbf{x} - \mathbf{x}_{\text{eq}}) & \text{if } \mathbf{x} \in \mathcal{X}_{\text{LQR}} \\ \mathbf{u}_{\text{safe}} & \text{otherwise} \end{cases} \tag{218}$$

The region $\mathcal{X}_{\text{LQR}} = \{\mathbf{x} : (\mathbf{x} - \mathbf{x}_{\text{eq}})^T \mathbf{P} (\mathbf{x} - \mathbf{x}_{\text{eq}}) \leq \alpha\}$ represents the largest invariant set where LQR is guaranteed to work.

Cascade Architectures Production MPC systems rarely rely on a single solver. Instead, they implement a **cascade of increasingly conservative controllers** that trade optimality for reliability:

```
def get_control(self, x, time_budget):
    """
    Multi -level cascade for robust real -time control
    """
    time_remaining = time_budget

    # Level 1: Full nonlinear MPC
    if time_remaining > 5e -3: # 5ms minimum
        try:
            u, solve_time = self.solve_nmpc(x, time_remaining)
            if converged:
                return u
        except:
            pass
        time_remaining -= solve_time

    # Level 2: Simplified linear MPC
    if time_remaining > 1e -3: # 1ms minimum
        try:
            # Linearize around current state
            A, B = self.linearize_dynamics(x)
            u, solve_time = self.solve_lmpc(x, A, B, time_remaining)
            return u
        except:
            pass
        time_remaining -= solve_time

    # Level 3: Explicit MPC lookup
    if time_remaining > 1e -4: # 0.1ms minimum
        region = self.find_critical_region(x)
        if region is not None:
            return self.explicit_control_law[region](x)

    # Level 4: LQR backup
    if self.in_lqr_region(x):
        return self.K_lqr @ (x - self.x_eq)

    # Level 5: Emergency safe mode
    return self.emergency_stop(x)
```

Each level trades optimality for reliability: Level 1 provides optimal but computationally expensive control, Level 2 offers suboptimal but faster solutions, Level 3 provides pre-computed instant evaluation, Level 4 ensures stabilizing control without tracking, and Level 5 implements safe shutdown.

Even when using backup controllers, we can maintain solution continuity through **persistent warm-starting**:

$$\mathbf{z}_{\text{warm}}^{(k+1)} = \begin{cases} \text{shift}(\mathbf{z}^{(k)}) & \text{if MPC succeeded at time } k \\ \text{lift}(\mathbf{u}_{\text{backup}}^{(k)}) & \text{if backup controller used} \\ \text{propagate}(\mathbf{z}_{\text{warm}}^{(k)}) & \text{if maintaining virtual solution} \end{cases} \quad (219)$$

The **shift** operation takes a successful MPC solution and moves it forward by one time step, appending a terminal action: $[\mathbf{u}_1^{(k)}, \mathbf{u}_2^{(k)}, \dots, \mathbf{u}_{N-1}^{(k)}, \kappa_f(\mathbf{x}_N^{(k)})]$. This shifted sequence provides natural temporal continuity for the next optimization.

When MPC fails and backup control is applied, the **lift** operation extends the single backup action $\mathbf{u}_{\text{backup}}^{(k)}$ into a full horizon-length sequence, either by repetition or by simulating the backup controller forward. This creates a reasonable warm-start guess from limited information.

The **propagate** operation maintains a “virtual” trajectory by continuing to evolve the previous solution as if it were still being executed, even when the actual system follows backup control. This forward simulation keeps the warm-start temporally aligned and relevant for when MPC recovers.

Example: Chemical Reactor Control Under Failure Consider a continuous stirred tank reactor (CSTR) where an exothermic reaction must be controlled:

$$\begin{aligned} \dot{C}_A &= \frac{q}{V}(C_{A,\text{in}} - C_A) - k_0 e^{-E/RT} C_A \\ \dot{T} &= \frac{q}{V}(T_{\text{in}} - T) + \frac{\Delta H}{\rho c_p} k_0 e^{-E/RT} C_A - \frac{UA}{\rho c_p V}(T - T_c) \end{aligned} \quad (220)$$

The MPC must maintain temperature below the runaway threshold T_{runaway} while maximizing conversion. Under normal operation, it solves:

$$\begin{aligned} \min \quad & -C_A(t_f) + \int_0^{t_f} \|T - T_{\text{optimal}}\|^2 dt \\ \text{s.t.} \quad & T \leq T_{\text{runaway}} - \Delta T_{\text{safety}} \\ & q_{\min} \leq q \leq q_{\max} \end{aligned} \quad (221)$$

When the cooling system partially fails, T_c suddenly increases. The MPC cannot maintain T_{optimal} within safety limits. The cascade activates: soft constraints allow T to exceed T_{optimal} with penalty, the reference governor reduces the production target $C_{A,\text{target}}$, and if still infeasible, the backup controller switches to maximum cooling $q = q_{\max}$. If temperature approaches runaway, emergency shutdown stops the feed with $q = 0$.

5.1.3 Computational Efficiency via Parametric Programming

Real-time model predictive control places strict limits on computation. In applications such as adaptive optics, the controller must run at kilohertz rates. A sampling frequency of 1000 Hz allows only one millisecond per step to compute and apply a control input. This makes efficiency a first-class concern.

The structure of MPC lends itself naturally to optimization reuse. Each time step requires solving a problem with the same dynamics and constraints. Only the initial state, forecasts, or reference signals change. Instead of treating each instance as a new problem, we can frame MPC as a *parametric optimization problem* and focus on how the solution evolves with the parameter.

General Framework: Parametric Optimization We begin with a general optimization problem indexed by a parameter $\theta \in \Theta \subset R^p$:

$$\begin{aligned} \min_{\mathbf{x} \in R^n} \quad & f(\mathbf{x}; \theta) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x}; \theta) \leq \mathbf{0}, \\ & \mathbf{h}(\mathbf{x}; \theta) = \mathbf{0}. \end{aligned} \tag{222}$$

For each value of θ , we obtain a concrete optimization problem. The goal is to understand how the optimizer $\mathbf{x}^*(\theta)$ and value function

$$v(\theta) := \inf \{ f(\mathbf{x}; \theta) : \mathbf{x} \text{ feasible at } \theta \} \tag{223}$$

depend on θ .

When the problem is smooth and regular, the Karush–Kuhn–Tucker (KKT) conditions characterize optimality:

$$\begin{aligned} \nabla_{\mathbf{x}} f(\mathbf{x}; \theta) + \nabla_{\mathbf{x}} \mathbf{g}(\mathbf{x}; \theta)^\top \boldsymbol{\lambda} + \nabla_{\mathbf{x}} \mathbf{h}(\mathbf{x}; \theta)^\top \boldsymbol{\nu} &= \mathbf{0}, \\ \mathbf{g}(\mathbf{x}; \theta) &\leq \mathbf{0}, \quad \boldsymbol{\lambda} \geq \mathbf{0}, \quad \lambda_i g_i(\mathbf{x}; \theta) = 0, \\ \mathbf{h}(\mathbf{x}; \theta) &= \mathbf{0}. \end{aligned} \tag{224}$$

If the active set remains fixed over changes in θ , the implicit function theorem ensures that the mappings

$$\theta \mapsto \mathbf{x}^*(\theta), \quad \theta \mapsto \boldsymbol{\lambda}^*(\theta), \quad \theta \mapsto \boldsymbol{\nu}^*(\theta) \tag{225}$$

are differentiable.

In linear and quadratic programming, this structure becomes even more tractable. Consider a linear program with affine dependence on θ :

$$\min_{\mathbf{x}} \quad \mathbf{c}(\theta)^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A}(\theta) \mathbf{x} \leq \mathbf{b}(\theta). \tag{226}$$

Each active set determines a basis and thus a region in Θ where the solution is affine in θ . The feasible parameter space is partitioned into polyhedral regions, each with its own affine law.

Similarly, in strictly convex quadratic programs

$$\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{q}(\boldsymbol{\theta})^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A} \mathbf{x} \leq \mathbf{b}(\boldsymbol{\theta}), \quad \mathbf{H} \succ 0, \quad (227)$$

each active set again leads to an affine optimizer, with piecewise-affine global structure and a piecewise-quadratic value function.

Parametric programming focuses on the structure of the map $\boldsymbol{\theta} \mapsto \mathbf{x}^*(\boldsymbol{\theta})$, and the regions over which this map takes a simple form.

Solution Sensitivity via the Implicit Function Theorem We often meet equations of the form

$$F(y, \boldsymbol{\theta}) = 0, \quad (228)$$

where $y \in R^m$ are unknowns and $\boldsymbol{\theta} \in R^p$ are parameters. The **implicit function theorem** says that, if F is smooth and the Jacobian with respect to y ,

$$\frac{\partial F}{\partial y}(y^*, \boldsymbol{\theta}^*), \quad (229)$$

is invertible at a solution $(y^*, \boldsymbol{\theta}^*)$, then in a neighborhood of $\boldsymbol{\theta}^*$ there exists a unique smooth mapping $y(\boldsymbol{\theta})$ with $F(y(\boldsymbol{\theta}), \boldsymbol{\theta}) = 0$ and $y(\boldsymbol{\theta}^*) = y^*$. Moreover, its derivative is

$$\frac{dy}{d\boldsymbol{\theta}}(\boldsymbol{\theta}^*) = -\left(\frac{\partial F}{\partial y}(y^*, \boldsymbol{\theta}^*)\right)^{-1} \frac{\partial F}{\partial \boldsymbol{\theta}}(y^*, \boldsymbol{\theta}^*). \quad (230)$$

In words: if the square Jacobian in y is nonsingular, the solution varies smoothly with the parameter, and we can differentiate it by solving one linear system.

Return to (P_θ) and its KKT system. Collect the primal and dual variables into

$$y := (\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}), \quad (231)$$

and write the KKT equations as a single residual

$$F(y, \boldsymbol{\theta}) = \begin{bmatrix} \nabla_{\mathbf{x}} f(\mathbf{x}; \boldsymbol{\theta}) + \nabla_{\mathbf{x}} \mathbf{g}(\mathbf{x}; \boldsymbol{\theta})^\top \boldsymbol{\lambda} + \nabla_{\mathbf{x}} \mathbf{h}(\mathbf{x}; \boldsymbol{\theta})^\top \boldsymbol{\nu} \\ \mathbf{h}(\mathbf{x}; \boldsymbol{\theta}) \\ \mathbf{g}_{\mathcal{A}}(\mathbf{x}; \boldsymbol{\theta}) \end{bmatrix} = \mathbf{0}. \quad (232)$$

Here $\mathcal{A}$ denotes the set of inequality constraints active at the solution (the complementarity part is encoded by keeping $\mathcal{A}$ fixed; see below).

To invoke IFT, we need the Jacobian $\partial F / \partial y$ to be invertible at $(y^*, \boldsymbol{\theta}^*)$. Standard regularity conditions that ensure this are:

- **LICQ (Linear Independence Constraint Qualification)** at $(\mathbf{x}^*, \boldsymbol{\theta}^*)$: the gradients of all active constraints are linearly independent.

- **Second-order sufficiency** on the critical cone (the Lagrangian Hessian is positive definite on feasible directions).
- **Strict complementarity** (optional but convenient): each active inequality has strictly positive multiplier.

Under these, the **KKT matrix**,

$$K = \frac{\partial F}{\partial y}(y^*, \theta^*) = \begin{bmatrix} \nabla_{\mathbf{x}\mathbf{x}}^2 \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*; \theta^*) & \nabla_{\mathbf{x}} \mathbf{g}_{\mathcal{A}}(\mathbf{x}^*; \theta^*)^\top & \nabla_{\mathbf{x}} \mathbf{h}(\mathbf{x}^*; \theta^*)^\top \\ \nabla_{\mathbf{x}} \mathbf{g}_{\mathcal{A}}(\mathbf{x}^*; \theta^*) & 0 & 0 \\ \nabla_{\mathbf{x}} \mathbf{h}(\mathbf{x}^*; \theta^*) & 0 & 0 \end{bmatrix}, \quad (233)$$

is nonsingular. Here $\mathcal{L} = f + \boldsymbol{\lambda}^\top \mathbf{g} + \boldsymbol{\nu}^\top \mathbf{h}$.

The right-hand side sensitivity to parameters is

$$G = \frac{\partial F}{\partial \theta}(y^*, \theta^*) = \begin{bmatrix} \nabla_{\theta} \nabla_{\mathbf{x}} f + \sum_{i \in \mathcal{A}} \lambda_i^* \nabla_{\theta} \nabla_{\mathbf{x}} g_i + \sum_j \nu_j^* \nabla_{\theta} \nabla_{\mathbf{x}} h_j \\ \nabla_{\theta} \mathbf{h} \\ \nabla_{\theta} \mathbf{g}_{\mathcal{A}} \end{bmatrix}_{(\mathbf{x}^*, \theta^*)}. \quad (234)$$

IFT then gives **local differentiability of the optimizer and multipliers**:

$$\frac{dy^*}{d\theta}(\theta^*) = -K^{-1}G. \quad (235)$$

The formula above is valid **as long as the active set $\mathcal{A}$ does not change**. If a constraint switches between active/inactive, the mapping remains piecewise smooth, but the derivative may jump. In MPC, this is exactly why warm-starts are very effective most of the time and occasionally require a refactorization when the active set flips.

In parametric MPC, θ gathers the current state, references, and forecasts. The IFT tells us that, under regularity and a stable active set, the optimal trajectory and first input vary smoothly with θ . The linear map $-K^{-1}G$ is exactly the object used in sensitivity-based warm starts and real-time iterations: small changes in θ can be propagated through a single KKT solve to update the primal-dual guess before taking one or two Newton/SQP steps.

Predictor-Corrector MPC We start with a smooth root-finding problem

$$F(y) = 0, \quad F : R^m \rightarrow R^m. \quad (236)$$

Newton's method iterates

$$y^{(t+1)} = y^{(t)} - [\nabla F(y^{(t)})]^{-1} F(y^{(t)}), \quad (237)$$

or equivalently solves the linearized system

$$\nabla F(y^{(t)}) \Delta y^{(t)} = -F(y^{(t)}), \quad y^{(t+1)} = y^{(t)} + \Delta y^{(t)}. \quad (238)$$

Convergence is local and fast when the Jacobian is nonsingular and the initial guess is close.

Now suppose the root depends on a parameter:

$$F(y, \theta) = 0, \quad \theta \in R. \quad (239)$$

We want the solution path $\theta \mapsto y^*(\theta)$. **Numerical continuation** advances θ in small steps and uses the previous solution as a warm start for the next Newton solve. This is the simplest and most effective way to “track” solutions of parametric systems.

At a known solution (y^*, θ^*) , differentiate $F(y^*(\theta), \theta) = 0$ with respect to θ :

$$\nabla_y F(y^*, \theta^*) \frac{dy^*}{d\theta}(\theta^*) + \nabla_\theta F(y^*, \theta^*) = 0. \quad (240)$$

If $\nabla_y F$ is invertible (IFT conditions), the **tangent** is

$$\frac{dy^*}{d\theta}(\theta^*) = -[\nabla_y F(y^*, \theta^*)]^{-1} \nabla_\theta F(y^*, \theta^*). \quad (241)$$

This is exactly the **implicit differentiation** formula. Continuation uses it as a **predictor**:

$$y_{\text{pred}} = y^*(\theta^*) + \Delta\theta \frac{dy^*}{d\theta}(\theta^*). \quad (242)$$

Then a few **corrector** steps apply Newton to $F(\cdot, \theta^* + \Delta\theta) = 0$ starting from y_{pred} . If Newton converges quickly, the step $\Delta\theta$ was appropriate; otherwise reduce $\Delta\theta$ and retry.

For parametric KKT systems, set $y = (\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu})$ where $\mathbf{x}$ stacks the primal decision variables (states and inputs), and $F(y, \theta) = 0$ the KKT residual with θ collecting state, references, forecasts. The **KKT matrix** $K = \partial F / \partial y$ and **parameter sensitivity** $G = \partial F / \partial \theta$ give the tangent

$$\frac{dy^*}{d\theta} = -K^{-1}G. \quad (243)$$

Continuation then becomes:

1. **Predictor:** $y_{\text{pred}} = y^* + (\Delta\theta)(-K^{-1}G)$.
2. **Corrector:** a few Newton/SQP steps on the KKT equations at the new θ .

In MPC, this yields efficient **warm starts** across time: as the parameter θ_t (current state, references) changes slightly, we predict the new primal–dual point and correct with 1–2 iterations—often enough to hit tolerance in real time.

Amortized Optimization and Neural Approximation of Controllers

The idea of reusing structure across similar optimization problems is not exclusive to parametric programming. In machine learning, a related concept known as **amortized optimization** aims to reduce the cost of repeated inference by replacing explicit optimization with a function that has been *learned* to approximate the solution map. This approach shifts the computational burden from online solving to offline training.

The goal is to construct a function $\hat{\pi}_\phi(\boldsymbol{\theta})$, typically parameterized by a neural network, that maps the input $\boldsymbol{\theta}$ to an approximate solution $\hat{z}^*(\boldsymbol{\theta})$ or control action $\hat{\mathbf{u}}_0^*(\boldsymbol{\theta})$. Once trained, this map can be evaluated quickly at runtime, with no need to solve an optimization problem explicitly.

Amortized optimization has emerged in several contexts:

- In **probabilistic inference**, where variational autoencoders (VAEs) amortize the computation of posterior distributions across a dataset.
- In **meta-learning**, where the objective is to learn a model that generalizes across tasks by internalizing how to adapt.
- In **hyperparameter optimization**, where learning a surrogate model can guide the search over configuration space efficiently.

This perspective has also begun to influence control. Recent work investigates how to **amortize nonlinear MPC (NMPC)** policies into neural networks. The training data come from solving many instances of the underlying optimal control problem offline. The resulting neural policy $\hat{\pi}_\phi$ acts as a differentiable, low-latency controller that can generalize to new situations within the training distribution.

Compared to explicit MPC, which partitions the parameter space and stores exact solutions region by region, amortized control smooths over the domain by learning an approximate policy globally. It is less precise, but scalable to high-dimensional problems where enumeration of regions is impossible.

Neural network amortization is advantageous due to the expressivity of these models. However, the challenge is ensuring **constraint satisfaction and safety**, which are hard to guarantee with unconstrained neural approximators. Hybrid approaches attempt to address this by combining a neural warm-start policy with a final projection step, or by embedding the network within a constrained optimization layer. Other strategies include learning structured architectures that respect known physics or control symmetries.

Imitation Learning Framework Consider a fixed horizon N and parameter vector $\boldsymbol{\theta}$ encoding the current state, references, and forecasts. The oracle MPC controller solves

$$\begin{aligned}
z^*(\boldsymbol{\theta}) \in \arg \min_{z=(\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1})} J(z; \boldsymbol{\theta}) \\
\text{s.t. } \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k; \boldsymbol{\theta}), \quad k = 0..N-1, \\
g(\mathbf{x}_k, \mathbf{u}_k; \boldsymbol{\theta}) \leq 0, \quad h(\mathbf{x}_N; \boldsymbol{\theta}) = 0.
\end{aligned} \tag{244}$$

The applied action is $\pi^*(\boldsymbol{\theta}) := \mathbf{u}_0^*(\boldsymbol{\theta})$. Our goal is to learn a fast surrogate mapping $\hat{\pi}_\phi : \boldsymbol{\theta} \mapsto \hat{\mathbf{u}}_0 \approx \pi^*(\boldsymbol{\theta})$ that can be evaluated in microseconds, optionally followed by a safety projection layer.

Supervised learning from oracle solutions. One first samples parameters $\boldsymbol{\theta}^{(i)}$ from the operational domain and solves the corresponding NMPC problems offline. The resulting dataset

$$\mathcal{D} = \{(\boldsymbol{\theta}^{(i)}, \mathbf{u}_0^*(\boldsymbol{\theta}^{(i)}))\}_{i=1}^M \tag{245}$$

is then used to train a neural network $\hat{\pi}_\phi$ by minimizing

$$\min_{\phi} \frac{1}{M} \sum_{i=1}^M \|\hat{\pi}_\phi(\boldsymbol{\theta}^{(i)}) - \mathbf{u}_0^*(\boldsymbol{\theta}^{(i)})\|^2. \tag{246}$$

Once trained, the network acts as a surrogate for the optimizer, providing instantaneous evaluations that approximate the MPC law.

5.1.4 Example: Propofol Infusion Control

This problem explores the control of propofol infusion in total intravenous anesthesia (TIVA). Our presentation follows the problem formulation developed by [Sawaguchi et al. \[2008\]](#). The primary objective is to maintain the desired level of unconsciousness while minimizing adverse reactions and ensuring quick recovery after surgery.

The level of unconsciousness is measured by the Bispectral Index (BIS), which is obtained using an electroencephalography (EEG) device. The BIS ranges from 0 (complete suppression of brain activity) to 100 (fully awake), with the target range for general anesthesia typically between 40 and 60.

The goal is to design a control system that regulates the infusion rate of propofol to maintain the BIS within the target range. This can be formulated as an optimal control problem:

$$\min_{u(t)} \int_0^T (BIS(t) - BIS_{\text{target}})^2 + \lambda u(t)^2 dt$$

subject to:

$$\dot{x}_1 = -(k_{10} + k_{12} + k_{13})x_1 + k_{21}x_2 + k_{31}x_3 + \frac{u(t)}{V_1}$$

$$\dot{x}_2 = k_{12}x_1 - k_{21}x_2$$

$$\dot{x}_3 = k_{13}x_1 - k_{31}x_3$$

$$\dot{x}_e = k_{e0}(x_1 - x_e)$$

$$BIS(t) = E_0 - E_{\max} \frac{x_e^\gamma}{x_e^\gamma + EC_{50}^\gamma}$$

Where:

- $u(t)$ is the propofol infusion rate (mg/kg/h)
- x_1 , x_2 , and x_3 are the drug concentrations in different body compartments
- x_e is the effect-site concentration
- k_{ij} are rate constants for drug transfer between compartments
- $BIS(t)$ is the Bispectral Index
- λ is a regularization parameter penalizing excessive drug use
- E_0 , $E_{\max}$, EC_{50} , and γ are parameters of the pharmacodynamic model

The specific dynamics model used in this problem is so-called “Pharmacokinetic-Pharmacodynamic Model” and consists of three main components:

1. **Pharmacokinetic Model**, which describes how the drug distributes through the body over time. It’s based on a three-compartment model:
 - Central compartment (blood and well-perfused organs)
 - Shallow peripheral compartment (muscle and other tissues)
 - Deep peripheral compartment (fat)
2. **Effect Site Model**, which represents the delay between drug concentration in the blood and its effect on the brain.
3. **Pharmacodynamic Model** that relates the effect-site concentration to the observed BIS.

The propofol infusion control problem presents several interesting challenges from a research perspective. First, there is a delay in how fast the drug can reach a different compartments in addition to the BIS measurements which can lag. This could lead to instability if not properly addressed in the control design.

Furthermore, every patient is different from another. Hence, we cannot simply learn a single controller offline and hope that it will generalize to an entire patient population. We will account for this variability through Model Predictive Control (MPC) and dynamically adapt to the model mismatch through replanning. How a patient will react to a given dose of drug also varies and must be carefully controlled to avoid overdoses. This adds an additional layer of complexity since we have to incorporate safety constraints. Finally, the patient might suddenly change state, for example due to surgical stimuli, and the controller must be able to adapt quickly to compensate for the disturbance to the system.

Self-checks

5.2 Dynamic Programming

Unlike the methods we've discussed so far, dynamic programming takes a step back and considers an entire family of related problems rather than a single optimization problem. This approach, while seemingly more complex at first glance, can often lead to efficient solutions.

Dynamic programming leverage the solution structure underlying many control problems that allows for a decomposition it into smaller, more manageable subproblems. Each subproblem is itself an optimization problem, embedded within the larger whole. This recursive structure is the foundation upon which dynamic programming constructs its solutions.

To ground our discussion, let us return to the domain of discrete-time optimal control problems (DOCPs). These problems frequently arise from the discretization of continuous-time optimal control problems. While the focus here will be on deterministic problems, these concepts extend naturally to stochastic problems by taking the expectation over the random quantities.

Consider a typical DOCP of Bolza type:

$$\begin{aligned} & \text{minimize} && Jc_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \\ & \text{subject to} && \mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t), \quad t = 1, \dots, T-1, \\ & && \mathbf{u}_{lb} \leq \mathbf{u}_t \leq \mathbf{u}_{ub}, \quad t = 1, \dots, T, \\ & && \mathbf{x}_{lb} \leq \mathbf{x}_t \leq \mathbf{x}_{ub}, \quad t = 1, \dots, T, \\ & \text{given} && \mathbf{x}_1 \end{aligned}$$

Rather than considering only the total cost from the initial time to the final time, dynamic programming introduces the concept of cost from an arbitrary point in time to the end. This leads to the definition of the “cost-to-go” or “value function” $J_k(\mathbf{x}_k)$:

$$J_k(\mathbf{x}_k)c_T(\mathbf{x}_T) + \sum_{t=k}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t) \quad (247)$$

This function represents the total cost incurred from stage k onwards to the end of the time horizon, given that the system is initialized in state $\mathbf{x}_k$ at stage k . Suppose the problem has been solved from stage $k+1$ to the end, yielding the optimal cost-to-go $J_{k+1}^*(\mathbf{x}_{k+1})$ for any state $\mathbf{x}_{k+1}$ at stage $k+1$. The question then becomes: how does this information inform the decision at stage k ?

Given knowledge of the optimal behavior from $k+1$ onwards, the task reduces to determining the optimal action $\mathbf{u}_k$ at stage k . This control should minimize the sum of the immediate cost $c_k(\mathbf{x}_k, \mathbf{u}_k)$ and the optimal future cost $J_{k+1}^*(\mathbf{x}_{k+1})$, where $\mathbf{x}_{k+1}$ is the resulting state after applying action $\mathbf{u}_k$. Mathematically, this is expressed as:

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} [c_k(\mathbf{x}_k, \mathbf{u}_k) + J_{k+1}^*(\mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k))] \quad (248)$$

This equation is known as Bellman’s equation, named after Richard Bellman, who formulated the principle of optimality:

An optimal policy has the property that whatever the previous state and decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the previous decision.

In other words, any sub-path of an optimal path, from any intermediate point to the end, must itself be optimal. This principle is the basis for the backward induction procedure which computes the optimal value function and provides closed-loop control capabilities without having to use an explicit NLP solver.

Dynamic programming can handle nonlinear systems and non-quadratic cost functions naturally. It provides a global optimal solution, when one exists, and can incorporate state and control constraints with relative ease. However, as the dimension of the state space increases, this approach suffers from what Bellman termed the “curse of dimensionality.” The computational complexity and memory requirements grow exponentially with the state dimension, rendering direct application of dynamic programming intractable for high-dimensional problems.

Fortunately, learning-based methods offer efficient tools to combat the curse of dimensionality on two fronts: by using function approximation (e.g., neural networks) to avoid explicit discretization, and by leveraging randomization through Monte Carlo methods inherent in the learning paradigm. Most of this course is dedicated to those ideas.

Backward Recursion The principle of optimality provides a methodology for solving optimal control problems. Beginning at the final time horizon and working backwards, at each stage the local optimization problem given by Bellman’s equation is solved. This process, termed backward recursion or backward induction, constructs the optimal value function stage by stage.

Upon completion of this backward pass, we now have access to the optimal control to take at any stage and in any state. Furthermore, we can simulate optimal trajectories from any initial state and applying the optimal policy at each stage to generate the optimal trajectory.

Theorem 5.2 (Backward induction solves deterministic Bolza DOCP). ***Setting.** Let $\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t)$ for $t = 1, \dots, T - 1$, with admissible action sets $\mathcal{U}_t(\mathbf{x}) \neq \emptyset$. Let stage costs $c_t(\mathbf{x}, \mathbf{u})$ and terminal cost $c_T(\mathbf{x})$ be real-valued and bounded below. Assume for every $(t, \mathbf{x})$ the one-step problem*

$$\min_{\mathbf{u} \in \mathcal{U}_t(\mathbf{x})} \{c_t(\mathbf{x}, \mathbf{u}) + J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \mathbf{u}))\} \quad (249)$$

admits a minimizer (e.g., compact $\mathcal{U}_t(\mathbf{x})$ and continuity suffice).

Define $J_T^(\mathbf{x}) \equiv c_T(\mathbf{x})$ and for $t = T - 1, \dots, 1$*

$$J_t^*(\mathbf{x}) \equiv \min_{\mathbf{u} \in \mathcal{U}_t(\mathbf{x})} [c_t(\mathbf{x}, \mathbf{u}) + J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \mathbf{u}))], \quad (250)$$

and select any minimizer $\pi_t^*(\mathbf{x}) \in \arg \min(\cdot)$.

Claim. For every initial state $\mathbf{x}_1$, the control sequence $\pi_1^*(\mathbf{x}_1), \dots, \pi_{T-1}^*(\mathbf{x}_{T-1})$ generated by these selectors is optimal for the Bolza problem, and $J_1^*(\mathbf{x}_1)$ equals the optimal cost. Moreover, $J_t^*(\cdot)$ is the optimal cost-to-go from stage t for every state, i.e., backward induction recovers the entire value function.

Proof. We give a direct proof by backward induction. The general idea is that any feasible sequence can be improved by replacing its tail with an optimal continuation, so optimal solutions can be built stage by stage. This is sometimes called a “cut-and-paste” argument.

Step 1 (verification of the recursion at a fixed stage).

Fix $t \in \{1, \dots, T-1\}$ and $\mathbf{x} \in X$. Consider any admissible control sequence $\mathbf{u}_t, \dots, \mathbf{u}_{T-1}$ starting from $\mathbf{x}_t = \mathbf{x}$ and define the induced states $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k)$. Its total cost from t is

$$c_t(\mathbf{x}_t, \mathbf{u}_t) + \sum_{k=t+1}^{T-1} c_k(\mathbf{x}_k, \mathbf{u}_k) + c_T(\mathbf{x}_T). \quad (251)$$

By definition of J_{t+1}^* , the tail cost satisfies

$$\sum_{k=t+1}^{T-1} c_k(\mathbf{x}_k, \mathbf{u}_k) + c_T(\mathbf{x}_T) \geq J_{t+1}^*(\mathbf{x}_{t+1}) = J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \mathbf{u}_t)). \quad (252)$$

Hence the total cost is bounded below by

$$c_t(\mathbf{x}, \mathbf{u}_t) + J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \mathbf{u}_t)). \quad (253)$$

Taking the minimum over $\mathbf{u}_t \in \mathcal{U}_t(\mathbf{x})$ yields

$$(\text{any admissible cost from } t) \geq J_t^*(\mathbf{x}). \quad (*)$$

Step 2 (existence of an optimal prefix at stage t).

By assumption, there exists $\pi_t^*(\mathbf{x})$ attaining the minimum in the definition of $J_t^*(\mathbf{x})$. If we now **paste** to $\pi_t^*(\mathbf{x})$ an optimal tail policy from $t+1$ (whose existence we will establish inductively), the resulting sequence attains cost exactly

$$c_t(\mathbf{x}, \pi_t^*(\mathbf{x})) + J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \pi_t^*(\mathbf{x}))) = J_t^*(\mathbf{x}), \quad (254)$$

which matches the lower bound $(*)$; hence it is optimal from t .

Step 3 (backward induction over time).

Base case $t = T$. The statement holds because $J_T^*(\mathbf{x}) = c_T(\mathbf{x})$ and there is no control to choose.

Inductive step. Assume the tail statement holds for $t+1$: from any state $\mathbf{x}_{t+1}$ there exists an optimal control sequence realizing $J_{t+1}^*(\mathbf{x}_{t+1})$. Then by Steps 1–2, selecting $\pi_t^*(\mathbf{x}_t)$ at stage t and concatenating the optimal tail from $t+1$ yields an optimal sequence from t with value $J_t^*(\mathbf{x}_t)$.

By backward induction, the claim holds for all t , in particular for $t = 1$ and any initial $\mathbf{x}_1$. Therefore the backward recursion both **certifies** optimality (verification) and **constructs** an optimal policy (synthesis), while recovering the full family $\{J_t^*\}_{t=1}^T$. $\square$

Remark 5.1 (No “big NLP” required). *The Bolza DOCP over the whole horizon couples all controls through the dynamics and is typically posed as a single large nonlinear program. The proof shows you can solve $T - 1$ **sequences of one-step problems** instead: at each $(t, \mathbf{x})$ minimize*

$$\mathbf{u} \mapsto c_t(\mathbf{x}, \mathbf{u}) + J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \mathbf{u})). \quad (255)$$

In finite state-action spaces this becomes pure table lookup and argmin. In continuous spaces you still solve local one-step minimizations, but you avoid a monolithic horizon-coupled NLP.

Remark 5.2 (Graph interpretation (optional intuition)). *Unroll time to form a DAG whose nodes are $(t, \mathbf{x})$ and whose edges correspond to feasible controls with edge weight $c_t(\mathbf{x}, \mathbf{u})$. The terminal node cost is $c_T(\cdot)$. The Bolza problem is a shortest-path problem on this DAG. The equation*

$$J_t^*(\mathbf{x}) = \min_{\mathbf{u}} \{c_t(\mathbf{x}, \mathbf{u}) + J_{t+1}^*(\mathbf{f}_t(\mathbf{x}, \mathbf{u}))\} \quad (256)$$

is exactly the dynamic programming recursion for shortest paths on acyclic graphs, hence backward induction is optimal.

Example: Optimal Harvest in Resource Management Dynamic programming is often used in resource management and conservation biology to devise policies to be implemented by decision makers and stakeholders : for eg. in fisheries, or timber harvesting. Per [Conroy and Peterson \[2013\]](#), we consider a population of a particular species, whose abundance we denote by x_t , where t represents discrete time steps. Our objective is to maximize the cumulative harvest over a finite time horizon, while also considering the long-term sustainability of the population. This optimization problem can be formulated as:

$$\text{maximize} \quad \sum_{t=t_0}^{t_f} F(x_t \cdot h_t) + F_T(x_{t_f}) \quad (257)$$

Here, $F(\cdot)$ represents the immediate reward function associated with harvesting, h_t is the harvest rate at time t , and $F_T(\cdot)$ denotes a terminal value function that could potentially assign value to the final population state. In this particular problem, we assign no terminal value to the final population state, setting $F_T(x_{t_f}) = 0$ and allowing us to focus solely on the cumulative harvest over the time horizon.

In our model population model, the abundance of a species x ranges from 1 to 100 individuals. The decision variable is the harvest rate h , which can take values from the set $D = \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$. The population dynamics are governed by a modified logistic growth model:

$$x_{t+1} = x_t + 0.3x_t(1 - x_t/125) - h_t x_t \quad (258)$$

where the 0.3 represents the growth rate and 125 is the carrying capacity (the maximum population size given the available resources). The logistic growth model returns continuous values; however our DP formulation uses a discrete state space. Therefore, we also round the the outcomes to the nearest integer.

Applying the principle of optimality, we can express the optimal value function $J^*(x_t, t)$ recursively:

$$J^*(x_t, t) = \max_{h_t \in D} (F(x, h, t) + J^*(x_{t+1}, t + 1)) \quad (259)$$

with the boundary condition $J^*(x_{t_f}) = 0$.

It's worth noting that while this example uses a relatively simple model, the same principles can be applied to more complex scenarios involving stochasticity, multiple species interactions, or spatial heterogeneity.

Handling Continuous Spaces with Interpolation In many real-world problems, such as our resource management example, the state space is inherently continuous. Dynamic programming, however, is usually defined on discrete state spaces. To reconcile this, we approximate the value function on a finite grid of points and use interpolation to estimate its value elsewhere.

In our earlier example, we acted as if population sizes could only be whole numbers: 1 fish, 2 fish, 3 fish. But real measurements don't fit neatly. What do you do with a survey that reports 42.7 fish? Our reflex in the code example was to round to the nearest integer, effectively saying "let's just call it 43." This corresponds to **nearest-neighbor interpolation**, also known as discretization. It's the zeroth-order case: you assume the value between grid points is constant and equal to the closest one. In practice, this amounts to overlaying a grid on the continuous landscape and forcing yourself to stand at the intersections. In our demo code, this step was carried out with `numpy.searchsorted`.

While easy to implement, nearest-neighbor interpolation can introduce artifacts:

1. Decisions may change abruptly, even if the state only shifts slightly.
2. Precision is lost, especially in regimes where small variations matter.
3. The curse of dimensionality forces an impractically fine grid if many state variables are added.

To address these issues, we can use **higher-order interpolation**. Instead of taking the nearest neighbor, we estimate the value at off-grid points by leveraging multiple nearby values.

Backward Recursion with Interpolation Suppose we have computed $J_{k+1}^*(\mathbf{x})$ only at grid points $\mathbf{x} \in \mathcal{X}_{\text{grid}}$. To evaluate Bellman’s equation at an arbitrary $\mathbf{x}_{k+1}$, we interpolate. Formally, let $I_{k+1}(\mathbf{x})$ be the interpolation operator that extends the value function from $\mathcal{X}_{\text{grid}}$ to the continuous space. Then:

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} \left[c_k(\mathbf{x}_k, \mathbf{u}_k) + I_{k+1}(\mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k)) \right]. \quad (260)$$

For instance, in one dimension, linear interpolation gives:

$$I_{k+1}(x) = J_{k+1}^*(x_l) + \frac{x - x_l}{x_u - x_l} (J_{k+1}^*(x_u) - J_{k+1}^*(x_l)), \quad (261)$$

where x_l and x_u are the nearest grid points bracketing x . Linear interpolation is often sufficient, but higher-order methods (cubic splines, radial basis functions) can yield smoother and more accurate estimates. The choice of interpolation scheme and grid layout both affect accuracy and efficiency. A finer grid improves resolution but increases computational cost, motivating strategies like adaptive grid refinement or replacing interpolation altogether with parametric function approximation which we are going to see later in this book.

In higher-dimensional spaces, naive interpolation becomes prohibitively expensive due to the curse of dimensionality. Several approaches such as tensorized multilinear interpolation, radial basis functions, and machine learning models address this challenge by extending a common principle: they approximate the value function at unobserved points using information from a finite set of evaluations. However, as dimensionality continues to grow, even tensor methods face scalability limits, which is why flexible parametric models like neural networks have become essential tools for high-dimensional function approximation.

Example: Optimal Harvest with Linear Interpolation Here is a demonstration of the backward recursion procedure using linear interpolation.

Due to pedagogical considerations, this example is using our own implementation of the linear interpolation procedure. However, a more general and practical approach would be to use a built-in interpolation procedure in Numpy. Because our state space has a single dimension, we can simply use [scipy.interpolate.interp1d](#) which offers various interpolation methods through its `kind` argument, which can take values in ‘linear’, ‘nearest’, ‘nearest-up’, ‘zero’, ‘slinear’, ‘quadratic’, ‘cubic’, ‘previous’, or ‘next’. ‘zero’, ‘slinear’, ‘quadratic’ and ‘cubic’.

Here’s a more general implementation which here uses cubic interpolation through the `scipy.interpolate.interp1d` function:

5.2.1 Stochastic Dynamic Programming and Markov Decision Processes

While our previous discussion centered on deterministic systems, many real-world problems involve uncertainty. Stochastic Dynamic Programming (SDP) extends our framework to handle stochasticity in both the objective function and

system dynamics. This extension naturally leads us to consider more general policy classes and to formalize when simpler policies suffice.

Decision Rules and Policies Before diving into stochastic systems, we need to establish terminology for the different types of strategies a decision maker might employ. In the deterministic setting, we implicitly used feedback controllers of the form $u(\mathbf{x}, t)$. In the stochastic setting, we must be more precise about what information policies can use and how they select actions.

A **decision rule** is a prescription for action selection in each state at a specified decision epoch. These rules can vary in their complexity based on two main criteria:

1. **Dependence on history:** Markovian or History-dependent
2. **Action selection method:** Deterministic or Randomized

Markovian decision rules depend only on the current state, while **history-dependent rules** consider the entire sequence of past states and actions. Formally, a history h_t at time t is:

$$h_t = (s_1, a_1, \dots, s_{t-1}, a_{t-1}, s_t) \quad (262)$$

The set of all possible histories at time t , denoted H_t , grows exponentially with t :

- $H_1 = \mathcal{S}$ (just the initial state)
- $H_2 = \mathcal{S} \times \mathcal{A} \times \mathcal{S}$
- $H_t = \mathcal{S} \times (\mathcal{A} \times \mathcal{S})^{t-1}$

Deterministic rules select an action with certainty, while **randomized rules** specify a probability distribution over the action space.

These classifications lead to four types of decision rules:

1. **Markovian Deterministic (MD):** $\pi_t : \mathcal{S} \rightarrow \mathcal{A}_s$
2. **Markovian Randomized (MR):** $\pi_t : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A}_s)$
3. **History-dependent Deterministic (HD):** $\pi_t : H_t \rightarrow \mathcal{A}_s$
4. **History-dependent Randomized (HR):** $\pi_t : H_t \rightarrow \mathcal{P}(\mathcal{A}_s)$

where $\mathcal{P}(\mathcal{A}_s)$ denotes the set of probability distributions over $\mathcal{A}_s$.

A **policy** $\boldsymbol{\pi}$ is a sequence of decision rules, one for each decision epoch:

$$\boldsymbol{\pi} = (\pi_1, \pi_2, \dots, \pi_{N-1}) \quad (263)$$

The set of all policies of class K (where $K \in \{HR, HD, MR, MD\}$) is denoted as Π^K . These policy classes form a hierarchy:

$$\Pi^{MD} \subset \Pi^{MR} \subset \Pi^{HR}, \quad \Pi^{MD} \subset \Pi^{HD} \subset \Pi^{HR} \quad (264)$$

The largest set Π^{HR} contains all possible policies. We ask: under what conditions can we restrict attention to the much simpler set Π^{MD} without loss of optimality?

Notation: rules vs. policies

- **Decision rule (kernel).** A map from information to action distributions:
 - Markov, deterministic: $\pi_t : \mathcal{S} \rightarrow \mathcal{A}_s$
 - Markov, randomized: $\pi_t(\cdot | s) \in \Delta(\mathcal{A}_s)$
 - History-dependent: $\pi_t(\cdot | h_t) \in \Delta(\mathcal{A}_{s_t})$
- **Policy (sequence).** $\pi = (\pi_1, \pi_2, \dots)$.
- **Stationary policy.** $\pi = \text{const}(\pi)$ with $\pi_t \equiv \pi \forall t$.
By convention, we identify π with its stationary policy $\text{const}(\pi)$ when no confusion arises.

Stochastic System Dynamics In the stochastic setting, our system evolution takes the form:

$$\mathbf{x}_{t+1} = \mathbf{f}_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t) \quad (265)$$

Here, $\mathbf{w}_t$ represents a random disturbance or noise term at time t due to the inherent uncertainty in the system's behavior. The stage cost function may also incorporate stochastic influences:

$$c_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t) \quad (266)$$

In this context, our objective shifts from minimizing a deterministic cost to minimizing the expected total cost:

$$E \left[c_T(\mathbf{x}_T) + \sum_{t=1}^{T-1} c_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t) \right] \quad (267)$$

where the expectation is taken over the distributions of the random variables $\mathbf{w}_t$. The principle of optimality still holds in the stochastic case, but Bellman's optimality equation now involves an expectation:

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} E_{\mathbf{w}_k} [c_k(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k) + J_{k+1}^*(\mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k))] \quad (268)$$

In practice, this expectation is often computed by discretizing the distribution of $\mathbf{w}_k$ when the set of possible disturbances is very large or even continuous. Let's say we approximate the distribution with K discrete values $\mathbf{w}_k^i$, each occurring with probability p_k^i . Then our Bellman equation becomes:

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} \sum_{i=1}^K p_k^i (c_k(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k^i) + J_{k+1}^*(\mathbf{f}_k(\mathbf{x}_k, \mathbf{u}_k, \mathbf{w}_k^i))) \quad (269)$$

Optimality Equations in the Stochastic Setting When dealing with stochastic systems, a central question arises: what information should our control policy use? In the most general case, a policy might use the entire history of observations and actions. However, as we'll see, the Markovian structure of our problems allows for dramatic simplifications.

Let $h_t = (s_1, a_1, s_2, a_2, \dots, s_{t-1}, a_{t-1}, s_t)$ denote the complete history up to time t . In the stochastic setting, the history-based optimality equations become:

$$u_t(h_t) = \sup_{a \in A_{s_t}} \left\{ r_t(s_t, a) + \sum_{j \in S} p_t(j | s_t, a) u_{t+1}(h_t, a, j) \right\}, \quad u_N(h_N) = r_N(s_N) \quad (270)$$

where we now explicitly use the transition probabilities $p_t(j|s_t, a)$ rather than a deterministic dynamics function.

Theorem 5.3 (Principle of optimality for stochastic systems). *Let u_t^* be the optimal expected return from epoch t onward. Then:*

a. u_t^ satisfies the optimality equations:*

$$u_t^*(h_t) = \sup_{a \in A_{s_t}} \left\{ r_t(s_t, a) + \sum_{j \in S} p_t(j | s_t, a) u_{t+1}^*(h_t, a, j) \right\} \quad (271)$$

with boundary condition $u_N^(h_N) = r_N(s_N)$.*

b. Any policy π^ that selects actions attaining the supremum (or maximum) in the above equation at each history is optimal.*

Intuition: This formalizes Bellman's principle of optimality: "An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision." The recursive structure means that optimal local decisions (choosing the best action at each step) lead to global optimality, even with uncertainty captured by the transition probabilities.

A simplification occurs when we examine these history-based equations more closely. The Markov property of our system dynamics and rewards means that the optimal return actually depends on the history only through the current state:

Proposition 5.1 (State sufficiency for stochastic MDPs). *In finite-horizon stochastic MDPs with Markovian dynamics and rewards, the optimal return $u_t^*(h_t)$ depends on the history only through the current state s_t . Thus we can write $u_t^*(h_t) = v_t^*(s_t)$ for some function v_t^* that depends only on state and time.*

Proof. Following Puterman [1994] Theorem 4.4.2. We proceed by backward induction.

Base case: At the terminal time N , we have $u_N^*(h_N) = r_N(s_N)$ by the boundary condition. Since the terminal reward depends only on the final state s_N and not on how we arrived there, $u_N^*(h_N) = u_N^*(s_N)$.

Inductive step: Assume $u_{t+1}^*(h_{t+1})$ depends on h_{t+1} only through s_{t+1} for all $t+1, \dots, N$. Then from the optimality equation:

$$u_t^*(h_t) = \sup_{a \in A_{s_t}} \left\{ r_t(s_t, a) + \sum_{j \in S} p_t(j|s_t, a) u_{t+1}^*(h_t, a, j) \right\} \quad (272)$$

By the induction hypothesis, $u_{t+1}^*(h_t, a, j)$ depends only on the next state j , so:

$$u_t^*(h_t) = \sup_{a \in A_{s_t}} \left\{ r_t(s_t, a) + \sum_{j \in S} p_t(j|s_t, a) u_{t+1}^*(j) \right\} \quad (273)$$

Since the expression in brackets depends on h_t only through the current state s_t (the rewards and transition probabilities are Markovian), we conclude that $u_t^*(h_t) = u_t^*(s_t)$. □

Intuition: The Markov property means that the current state contains all information needed to predict future evolution. The past provides no additional value for decision-making. This result allows us to work with value functions $v_t^*(s)$ indexed only by state and time, dramatically simplifying both theory and computation.

This state-sufficiency result, combined with the fact that randomization never helps when maximizing expected returns, leads to a dramatic simplification of the policy space:

Theorem 5.4 (Policy reduction for stochastic MDPs). *For finite-horizon stochastic MDPs with finite state and action sets:*

$$\sup_{\pi \in \Pi^{\text{HR}}} v_\pi(s, t) = \max_{\pi \in \Pi^{\text{MD}}} v_\pi(s, t) \quad (274)$$

That is, there exists an optimal policy that is both deterministic and Markovian.

Proof. Sketch following Puterman [1994] Lemma 4.3.1 and Theorem 4.4.2. First, Lemma 4.3.1 shows that for any function w and any distribution q over actions, $\sup_a w(a) \geq \sum_a q(a)w(a)$. Thus randomization cannot improve the expected

value over choosing a single maximizing action. Second, by state sufficiency (Proposition 5.1 and Puterman [1994] Thm. 4.4.2(a)), the optimal return depends on the history only through (s_t, t) . Therefore, selecting at each (s_t, t) an action that attains the maximum yields a deterministic Markov decision rule which is optimal whenever the maximum is attained. If only a supremum exists, ε -optimal selectors exist by choosing actions within ε of the supremum (see Puterman [1994] Thm. 4.3.4). □

Intuition: Even in stochastic systems, randomization in the policy doesn't help when maximizing expected returns: you should always choose the action with the highest expected value. Combined with state sufficiency, this means simple state-to-action mappings are optimal.

These results justify focusing on deterministic Markov policies and lead to the backward recursion algorithm for stochastic systems:

While SDP provides us with a framework to for handling uncertainty, it makes the curse of dimensionality even more difficult to handle in practice. Both the state space and the disturbance space must be discretized. This can lead to a combinatorial explosion in the number of scenarios to be evaluated at each stage.

However, just as we tackled the challenges of continuous state spaces with discretization and interpolation, we can devise efficient methods to handle the additional complexity of evaluating expectations. This problem essentially becomes one of numerical integration. When the set of disturbances is continuous (as is often the case with continuous state spaces), we enter a domain where numerical quadrature methods could be applied. But these methods tend to scale poorly as the number of dimensions grows. This is where more efficient techniques, often rooted in Monte Carlo methods, come into play. Two ingredients tackle the curse of dimensionality:

1. Function approximation (through discretization, interpolation, neural networks, etc.)
2. Monte Carlo integration (simulation)

These two elements essentially distill the key ingredients of machine learning, which is the direction we'll be exploring in this course.

Example: Stochastic Optimal Harvest in Resource Management Building upon our previous deterministic model, we now introduce stochasticity to more accurately reflect the uncertainties inherent in real-world resource management scenarios [Conroy and Peterson, 2013]. As before, we consider a population of a particular species, whose abundance we denote by x_t , where t represents discrete time steps. Our objective remains to maximize the cumulative harvest over a finite time horizon, while also considering the long-term sustainability of the population. However, we now account for two sources of stochasticity:

partial controllability of harvest and environmental variability affecting growth rates. The optimization problem can be formulated as:

$$\text{maximize } E \left[\sum_{t=t_0}^{t_f} F(x_t \cdot h_t) \right] \quad (275)$$

Here, $F(\cdot)$ represents the immediate reward function associated with harvesting, and h_t is the realized harvest rate at time t . The expectation $E[\cdot]$ over both harvest and growth rates, which we view as random variables. In our stochastic model, the abundance x still ranges from 1 to 100 individuals. The decision variable is now the desired harvest rate d_t , which can take values from the set $D = 0, 0.1, 0.2, 0.3, 0.4, 0.5$. However, the realized harvest rate h_t is stochastic and follows a discrete distribution:

$$h_t = \begin{cases} 0.75d_t & \text{with probability 0.25} \\ d_t & \text{with probability 0.5} \\ 1.25d_t & \text{with probability 0.25} \end{cases} \quad (276)$$

By expressing the harvest rate as a random variable, we mean to capture the fact that harvesting is not completely under our control: we might obtain more or less what we had intended to. Furthermore, we generalize the population dynamics to the stochastic case via:

\$\$

$$x_{t+1} = x_t + r_t x_t (1 - x_t/K) - h_t x_t \quad (277)$$

where $K = 125$ is the carrying capacity. The growth rate r_t is now stochastic and follows a discrete distribution:

$$r_t = \begin{cases} 0.85r_{\max} & \text{with probability 0.25} \\ 1.05r_{\max} & \text{with probability 0.5} \\ 1.15r_{\max} & \text{with probability 0.25} \end{cases} \quad (277)$$

where $r_{\max} = 0.3$ is the maximum growth rate. Applying the principle of optimality, we can express the optimal value function $J^*(x_t, t)$ recursively:

$$J^*(x_t, t) = \max_{d(t) \in D} E [F(x_t \cdot h_t) + J^*(x_{t+1}, t+1)] \quad (278)$$

where the expectation is taken over the harvest and growth rate random variables. The boundary condition remains $J^*(x_{t_f}) = 0$. We can now adapt our previous code to account for the stochasticity in our model. One important difference is that simulating a solution in this context requires multiple realizations of our process. This is an important consideration when evaluating reinforcement learning methods in practice, as success cannot be claimed based on a single successful trajectory.

Linear Quadratic Regulator via Dynamic Programming We now examine a special case where the backward recursion admits a closed-form solution. When the system dynamics are linear and the cost function is quadratic, the optimization at each stage can be solved analytically. Moreover, the value function itself maintains a quadratic structure throughout the recursion, and the optimal policy reduces to a simple linear feedback law. This result eliminates the need for discretization, interpolation, or any function approximation. The infinite-dimensional problem collapses to tracking a finite set of matrices.

Consider a discrete-time linear system:

$$\mathbf{x}_{t+1} = A_t \mathbf{x}_t + B_t \mathbf{u}_t \quad (279)$$

where $\mathbf{x}_t \in R^n$ is the state and $\mathbf{u}_t \in R^m$ is the control input. The matrices $A_t \in R^{n \times n}$ and $B_t \in R^{n \times m}$ describe the system dynamics at time t .

The cost function to be minimized is quadratic:

$$J = \frac{1}{2} \mathbf{x}_T^\top Q_T \mathbf{x}_T + \frac{1}{2} \sum_{t=0}^{T-1} (\mathbf{x}_t^\top Q_t \mathbf{x}_t + \mathbf{u}_t^\top R_t \mathbf{u}_t) \quad (280)$$

where $Q_T \succeq 0$ (positive semidefinite), $Q_t \succeq 0$, and $R_t \succ 0$ (positive definite) are symmetric matrices of appropriate dimensions. The positive definiteness of R_t ensures the minimization problem is well-posed.

What we now have to observe is that if the terminal cost is quadratic, then the value function at every earlier stage remains quadratic. This is not immediately obvious, but it follows from the structure of Bellman's equation combined with the linearity of the dynamics.

We claim that the optimal cost-to-go from any stage t takes the form:

$$J_t^*(\mathbf{x}_t) = \frac{1}{2} \mathbf{x}_t^\top P_t \mathbf{x}_t \quad (281)$$

for some positive semidefinite matrix P_t . At the terminal time, this is true by definition: $P_T = Q_T$.

Let's verify this structure and derive the recursion for P_t using backward induction. Suppose we've established that $J_{t+1}^*(\mathbf{x}_{t+1}) = \frac{1}{2} \mathbf{x}_{t+1}^\top P_{t+1} \mathbf{x}_{t+1}$. Bellman's equation at stage t states:

$$J_t^*(\mathbf{x}_t) = \min_{\mathbf{u}_t} \left[\frac{1}{2} \mathbf{x}_t^\top Q_t \mathbf{x}_t + \frac{1}{2} \mathbf{u}_t^\top R_t \mathbf{u}_t + J_{t+1}^*(\mathbf{x}_{t+1}) \right] \quad (282)$$

Substituting the dynamics $\mathbf{x}_{t+1} = A_t \mathbf{x}_t + B_t \mathbf{u}_t$ and the quadratic form for J_{t+1}^* :

$$J_t^*(\mathbf{x}_t) = \min_{\mathbf{u}_t} \left[\frac{1}{2} \mathbf{x}_t^\top Q_t \mathbf{x}_t + \frac{1}{2} \mathbf{u}_t^\top R_t \mathbf{u}_t + \frac{1}{2} (A_t \mathbf{x}_t + B_t \mathbf{u}_t)^\top P_{t+1} (A_t \mathbf{x}_t + B_t \mathbf{u}_t) \right] \quad (283)$$

Expanding the last term:

$$(A_t \mathbf{x}_t + B_t \mathbf{u}_t)^\top P_{t+1} (A_t \mathbf{x}_t + B_t \mathbf{u}_t) = \mathbf{x}_t^\top A_t^\top P_{t+1} A_t \mathbf{x}_t + 2\mathbf{x}_t^\top A_t^\top P_{t+1} B_t \mathbf{u}_t + \mathbf{u}_t^\top B_t^\top P_{t+1} B_t \mathbf{u}_t \quad (284)$$

The expression inside the minimization becomes:

$$\frac{1}{2} \mathbf{x}_t^\top Q_t \mathbf{x}_t + \frac{1}{2} \mathbf{u}_t^\top R_t \mathbf{u}_t + \frac{1}{2} \mathbf{x}_t^\top A_t^\top P_{t+1} A_t \mathbf{x}_t + \mathbf{x}_t^\top A_t^\top P_{t+1} B_t \mathbf{u}_t + \frac{1}{2} \mathbf{u}_t^\top B_t^\top P_{t+1} B_t \mathbf{u}_t \quad (285)$$

Collecting terms involving $\mathbf{u}_t$:

$$= \frac{1}{2} \mathbf{x}_t^\top (Q_t + A_t^\top P_{t+1} A_t) \mathbf{x}_t + \mathbf{x}_t^\top A_t^\top P_{t+1} B_t \mathbf{u}_t + \frac{1}{2} \mathbf{u}_t^\top (R_t + B_t^\top P_{t+1} B_t) \mathbf{u}_t \quad (286)$$

This is a quadratic function of $\mathbf{u}_t$. To find the minimizer, we take the gradient with respect to $\mathbf{u}_t$ and set it to zero:

$$\frac{\partial}{\partial \mathbf{u}_t} = (R_t + B_t^\top P_{t+1} B_t) \mathbf{u}_t + B_t^\top P_{t+1} A_t \mathbf{x}_t = 0 \quad (287)$$

Since $R_t + B_t^\top P_{t+1} B_t$ is positive definite (both R_t and P_{t+1} are positive semidefinite with R_t strictly positive), we can solve for the optimal control:

$$\mathbf{u}_t^* = -(R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t \mathbf{x}_t \quad (288)$$

Define the gain matrix:

$$K_t = (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t \quad (289)$$

so that $\mathbf{u}_t^* = -K_t \mathbf{x}_t$. This is a **linear feedback policy**: the optimal control is simply a linear function of the current state.

Substituting $\mathbf{u}_t^*$ back into the cost-to-go expression and simplifying (by completing the square), we obtain:

$$J_t^*(\mathbf{x}_t) = \frac{1}{2} \mathbf{x}_t^\top P_t \mathbf{x}_t \quad (290)$$

where P_t satisfies the **discrete-time Riccati equation**:

$$P_t = Q_t + A_t^\top P_{t+1} A_t - A_t^\top P_{t+1} B_t (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t \quad (291)$$

Putting everything together, the backward induction procedure under the LQR setting then becomes:

5.2.2 Markov Decision Process Formulation

Rather than expressing the stochasticity in our system through a disturbance term as a parameter to a deterministic difference equation, we often work with an alternative representation (more common in operations research) which uses the Markov Decision Process formulation. The idea is that when we model our system in this way with the disturbance term being drawn independently of the previous stages, the induced trajectory are those of a Markov chain. Hence, we can re-cast our control problem in that language, leading to the so-called Markov Decision Process framework in which we express the system dynamics in terms of transition probabilities rather than explicit state equations. In this framework, we express the probability that the system is in a given state using the transition probability function:

$$p_t(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) \quad (292)$$

This function gives the probability of transitioning to state $\mathbf{x}_{t+1}$ at time $t+1$, given that the system is in state $\mathbf{x}_t$ and action $\mathbf{u}_t$ is taken at time t . Therefore, p_t specifies a conditional probability distribution over the next states: namely, the sum (for discrete state spaces) or integral over the next state should be 1.

Given the control theory formulation of our problem via a deterministic dynamics function and a noise term, we can derive the corresponding transition probability function through the following relationship:

$$\begin{aligned} p_t(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) &= P(\mathbf{W}_t \in \{\mathbf{w} \in \mathbf{W} : \mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w})\}) \\ &= \sum_{\{\mathbf{w} \in \mathbf{W} : \mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w})\}} q_t(\mathbf{w}) \end{aligned} \quad (293)$$

Here, $q_t(\mathbf{w})$ represents the probability density or mass function of the disturbance $\mathbf{W}_t$ (assuming discrete state spaces). When dealing with continuous spaces, the above expression simply contains an integral rather than a summation.

For a system with deterministic dynamics and no disturbance, the transition probabilities become much simpler and be expressed using the indicator function. Given a deterministic system with dynamics:

$$\mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t) \quad (294)$$

The transition probability function can be expressed as:

$$p_t(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \begin{cases} 1 & \text{if } \mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t) \\ 0 & \text{otherwise} \end{cases} \quad (295)$$

With this transition probability function, we can recast our Bellman optimality equation:

$$J_t^*(\mathbf{x}_t) = \max_{\mathbf{u}_t \in \mathbf{U}} \left\{ c_t(\mathbf{x}_t, \mathbf{u}_t) + \sum_{\mathbf{x}_{t+1}} p_t(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t) J_{t+1}^*(\mathbf{x}_{t+1}) \right\} \quad (296)$$

Here, $c(\mathbf{x}_t, \mathbf{u}_t)$ represents the expected immediate reward (or negative cost) when in state $\mathbf{x}_t$ and taking action $\mathbf{u}_t$ at time t . The summation term computes the expected optimal value for the future states, weighted by their transition probabilities.

This formulation offers several advantages:

1. It makes the Markovian nature of the problem explicit: the future state depends only on the current state and action, not on the history of states and actions.
2. For discrete-state problems, the entire system dynamics can be specified by a set of transition matrices, one for each possible action.
3. It allows us to bridge the gap with the wealth of methods in the field of probabilistic graphical models and statistical machine learning techniques for modelling and analysis.

Notation in Operations Research The presentation above was intended to bridge the gap between the control-theoretic perspective and the world of closed-loop control through the idea of determining the value function of a parametric optimal control problem. We then saw how the backward induction procedure was applicable to both the deterministic and stochastic cases by taking the expectation over the disturbance variable. We then said that we can alternatively work with a representation of our system where instead of writing our model as a deterministic dynamics function taking a disturbance as an input, we would rather work directly via its transition probability function, which gives rise to the Markov chain interpretation of our system in simulation.

Note that the notation used in control theory tends to differ from that found in operations research communities, in which the field of dynamic programming flourished. We summarize those (purely notational) differences in this section.

In operations research, the system state at each decision epoch is typically denoted by $s \in \mathcal{S}$, where $\mathcal{S}$ is the set of possible system states. When the system is in state s , the decision maker may choose an action a from the set of allowable actions $\mathcal{A}_s$. The union of all action sets is denoted as $\mathcal{A} = \bigcup_{s \in \mathcal{S}} \mathcal{A}_s$.

The dynamics of the system are described by a transition probability function $p_t(j|s, a)$, which represents the probability of transitioning to state $j \in \mathcal{S}$ at time $t + 1$, given that the system is in state s at time t and action $a \in \mathcal{A}_s$ is chosen. This transition probability function satisfies:

$$\sum_{j \in \mathcal{S}} p_t(j|s, a) = 1 \quad (297)$$

It's worth noting that in operations research, we typically work with reward maximization rather than cost minimization, which is more common in control theory. However, we can easily switch between these perspectives by simply negating the quantity. That is, maximizing a reward function is equivalent to minimizing its negative, which we would then call a cost function.

The reward function is denoted by $r_t(s, a)$, representing the reward received at time t when the system is in state s and action a is taken. In some cases, the reward may also depend on the next state, in which case it is denoted as $r_t(s, a, j)$. The expected reward can then be computed as:

$$r_t(s, a) = \sum_{j \in \mathcal{S}} r_t(s, a, j) p_t(j|s, a) \quad (298)$$

Combined together, these elements specify a Markov decision process, which is fully described by the tuple:

$$\{T, \mathcal{S}, \mathcal{A}_s, p_t(\cdot|s, a), r_t(s, a)\} \quad (299)$$

where T represents the set of decision epochs (the horizon).

What is an Optimal Policy? Let's go back to the starting point and define what it means for a policy to be optimal in a Markov Decision Problem. For this, we will be considering different possible search spaces (policy classes) and compare policies based on the ordering of their value from any possible start state. The value of a policy π (optimal or not) is defined as the expected total reward obtained by following that policy from a given starting state. Formally, for a finite-horizon MDP with N decision epochs, we define the value function $v^\pi(s, t)$ as:

$$v^\pi(s, t) = E \left[\sum_{k=t}^{N-1} r_t(S_k, A_k) + r_N(S_N) \mid S_t = s \right] \quad (300)$$

where S_t is the state at time t , A_t is the action taken at time t , and r_t is the reward function. For simplicity, we write $v^\pi(s)$ to denote $v^\pi(s, 1)$, the value of following policy π from state s at the first stage over the entire horizon N .

In finite-horizon MDPs, our goal is to identify an optimal policy, denoted by π^* , that maximizes total expected reward over the horizon N . Specifically:

$$v^{\pi^*}(s) \geq v^\pi(s), \quad \forall s \in \mathcal{S}, \quad \forall \pi \in \Pi^{\text{HR}} \quad (301)$$

We call π^* an **optimal policy** because it yields the highest possible value across all states and all policies within the policy class Π^{HR} . We denote by v^* the maximum value achievable by any policy:

$$v^*(s) = \max_{\pi \in \Pi^{\text{HR}}} v^\pi(s), \quad \forall s \in \mathcal{S} \quad (302)$$

In reinforcement learning literature, v^* is typically referred to as the “optimal value function,” while in some operations research references, it might be called

the “value of an MDP.” An optimal policy π^* is one for which its value function equals the optimal value function:

$$v^{\pi^*}(s) = v^*(s), \quad \forall s \in \mathcal{S} \quad (303)$$

This notion of optimality applies to every state. Policies optimal in this sense are sometimes called “uniformly optimal policies.” A weaker notion of optimality, often encountered in reinforcement learning practice, is optimality with respect to an initial distribution of states. In this case, we seek a policy $\pi \in \Pi^{\text{HR}}$ that maximizes:

$$\sum_{s \in \mathcal{S}} v^\pi(s) P_1(S_1 = s) \quad (304)$$

where $P_1(S_1 = s)$ is the probability of starting in state s .

The maximum value can be achieved by searching over the space of deterministic Markovian Policies. Consequently:

$$v^*(s) = \max_{\pi \in \Pi^{\text{HR}}} v^\pi(s) = \max_{\pi \in \Pi^{\text{MD}}} v^\pi(s), \quad s \in \mathcal{S} \quad (305)$$

This equality significantly simplifies the computational complexity of our algorithms, as the search problem can now be decomposed into N sub-problems in which we only have to search over the set of possible actions. This is the backward induction algorithm, which we present a second time, but departing this time from the control-theoretic notation and using the MDP formalism:

Note that the same procedure can also be used for finding the value of a policy with minor changes;

This code could also finally be adapted to support randomized policies using:

$$v^\pi(s_t, t) = \sum_{a_t \in \mathcal{A}_{s_t}} \pi_t(a_t | s_t) \left(r_t(s_t, a_t) + \sum_{j \in \mathcal{S}} p_t(j | s_t, a_t) v^\pi(j, t+1) \right) \quad (306)$$

Example: Sample Size Determination in Pharmaceutical Development Pharmaceutical development is the process of bringing a new drug from initial discovery to market availability. This process is lengthy, expensive, and risky, typically involving several stages:

1. **Drug Discovery:** Identifying a compound that could potentially treat a disease.
2. **Preclinical Testing:** Laboratory and animal testing to assess safety and efficacy. . **Clinical Trials:** Testing the drug in humans, divided into phases:
 - Phase I: Testing for safety in a small group of healthy volunteers.

- Phase II: Testing for efficacy and side effects in a larger group with the target condition.
 - Phase III: Large-scale testing to confirm efficacy and monitor side effects.
3. **Regulatory Review:** Submitting a New Drug Application (NDA) for approval.
 4. **Post-Market Safety Monitoring:** Continuing to monitor the drug's effects after market release.

This process can take 10-15 years and cost over \$1 billion [Adams and Brantner \[2009\]](#). The high costs and risks involved call for a principled approach to decision making. We'll focus on the clinical trial phases and NDA approval, per the MDP model presented by [Chang \[2010\]](#):

1. **States (S):** Our state space is $S = \{s_1, s_2, s_3, s_4\}$, where:
 - s_1 : Phase I clinical trial
 - s_2 : Phase II clinical trial
 - s_3 : Phase III clinical trial
 - s_4 : NDA approval
2. **Actions (A):** At each state, the action is choosing the sample size n_i for the corresponding clinical trial. The action space is $A = \{10, 11, \dots, 1000\}$, representing possible sample sizes.
3. **Transition Probabilities (P):** The probability of moving from one state to the next depends on the chosen sample size and the inherent properties of the drug. We define:
 - $P(s_2|s_1, n_1) = p_{12}(n_1) = \sum_{i=0}^{\lfloor \eta_1 n_1 \rfloor} \binom{n_1}{i} p_0^i (1 - p_0)^{n_1 - i}$ where p_0 is the true toxicity rate and η_1 is the toxicity threshold for Phase I.
 - Of particular interest is the transition from Phase II to Phase III which we model as:

$$P(s_3|s_2, n_2) = p_{23}(n_2) = \Phi\left(\frac{\sqrt{n_2}}{2}\delta - z_{1-\eta_2}\right)$$

where Φ is the cumulative distribution function (CDF) of the standard normal distribution:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2/2} dt$$

This is giving us the probability that we would observe a treatment effect this large or larger if the null hypothesis (no treatment effect) were true. A higher probability indicates stronger evidence of a treatment effect, making it more likely that the drug will progress to Phase III.

In this expression, δ is called the “normalized treatment effect”. In clinical trials, we’re often interested in the difference between the treatment and control groups. The “normalized” part means we’ve adjusted this difference for the variability in the data. Specifically $\delta = \frac{\mu_t - \mu_c}{\sigma}$ where μ_t is the mean outcome in the treatment group, μ_c is the mean outcome in the control group, and σ is the standard deviation of the outcome. A larger δ indicates a stronger treatment effect.

Furthermore, the term $z_{1-\eta_2}$ is the $(1-\eta_2)$ -quantile of the standard normal distribution. In other words, it’s the value where the probability of a standard normal random variable being greater than this value is η_2 . For example, if $\eta_2 = 0.05$, then $z_{1-\eta_2} \approx 1.645$. A smaller η_2 makes the trial more conservative, requiring stronger evidence to proceed to Phase III.

Finally, n_2 is the sample size for Phase II. The $\sqrt{n_2}$ term reflects that the precision of our estimate of the treatment effect improves with the square root of the sample size.

- $P(s_4|s_3, n_3) = p_{34}(n_3) = \Phi\left(\frac{\sqrt{n_3}}{2}\delta - z_{1-\eta_3}\right)$ where η_3 is the significance level for Phase III.

[resume]**Rewards (R):** The reward function captures the costs of running trials and the potential profit from a successful drug:

1. • $r(s_i, n_i) = -c_i(n_i)$ for $i = 1, 2, 3$, where $c_i(n_i)$ is the cost of running a trial with sample size n_i .
 - $r(s_4) = g_4$, where g_4 is the expected profit from a successful drug.
2. **Discount Factor (γ):** We use a discount factor $0 < \gamma \leq 1$ to account for the time value of money and risk preferences.

5.2.3 Infinite-Horizon MDPs

It often makes sense to model control problems over infinite horizons. We extend the previous setting and define the expected total reward of policy $\pi \in \Pi^{\text{HR}}$, v^π as:

$$v^\pi(s) = E \left[\sum_{t=1}^{\infty} r(S_t, A_t) \right] \quad (307)$$

One drawback of this model is that we could easily encounter values that are $+\infty$ or $-\infty$, even in a setting as simple as a single-state MDP which loops back into itself and where the accrued reward is nonzero.

Therefore, it is often more convenient to work with an alternative formulation which guarantees the existence of a limit: the expected total discounted reward of policy $\pi \in \Pi^{\text{HR}}$ is defined to be:

$$v_\gamma^\pi(s) \equiv \lim_{N \rightarrow \infty} E \left[\sum_{t=1}^N \gamma^{t-1} r(S_t, A_t) \right] \quad (308)$$

for $0 \leq \gamma < 1$ and when $\max_{s \in \mathcal{S}} \max_{a \in \mathcal{A}_s} |r(s, a)| = R_{\max} < \infty$, in which case, $|v_\gamma^\pi(s)| \leq (1 - \gamma)^{-1} R_{\max}$.

Finally, another possibility for the infinite-horizon setting is the so-called average reward or gain of policy $\pi \in \Pi^{\text{HR}}$ defined as:

$$g^\pi(s) \equiv \lim_{N \rightarrow \infty} \frac{1}{N} E \left[\sum_{t=1}^N r(S_t, A_t) \right] \quad (309)$$

We won't be working with this formulation in this course due to its inherent practical and theoretical complexities.

Extending the previous notion of optimality from finite-horizon models, a policy π^* is said to be discount optimal for a given γ if:

$$v_\gamma^{\pi^*}(s) \geq v_\gamma^\pi(s) \quad \text{for each } s \in \mathcal{S} \text{ and all } \pi \in \Pi^{\text{HR}} \quad (310)$$

Furthermore, the value of a discounted MDP $v_\gamma^*(s)$, is defined by:

$$v_\gamma^*(s) \equiv \max_{\pi \in \Pi^{\text{HR}}} v_\gamma^\pi(s) \quad (311)$$

More often, we refer to v_γ by simply calling it the optimal value function.

As for the finite-horizon setting, the infinite horizon discounted model does not require history-dependent policies, since for any $\pi \in \Pi^{\text{HR}}$ there exists a $\pi' \in \Pi^{\text{MR}}$ with identical total discounted reward:

$$v_\gamma^*(s) \equiv \max_{\pi \in \Pi^{\text{HR}}} v_\gamma^\pi(s) = \max_{\pi \in \Pi^{\text{MR}}} v_\gamma^\pi(s). \quad (312)$$

Random Horizon Interpretation of Discounting The use of discounting can be motivated both from a modeling perspective and as a means to ensure that the total reward remains bounded. From the modeling perspective, we can view discounting as a way to weight more or less importance on the immediate rewards vs. the long-term consequences. There is also another interpretation which stems from that of a finite horizon model but with an uncertain end time. More precisely:

Let $v_\nu^\pi(s)$ denote the expected total reward obtained by using policy π when the horizon length ν is random. We define it by:

$$v_\nu^\pi(s) \equiv E_s^\pi \left[E_\nu \left\{ \sum_{t=1}^\nu r(S_t, A_t) \right\} \right] \quad (313)$$

Theorem 5.5 (Random horizon interpretation of discounting). *Suppose that the horizon ν follows a geometric distribution with parameter γ , $0 \leq \gamma < 1$, independent of the policy such that $P(\nu = n) = (1 - \gamma)\gamma^{n-1}$, $n = 1, 2, \dots$, then $v_\nu^\pi(s) = v_\gamma^\pi(s)$ for all $s \in \mathcal{S}$.*

Proof. See proposition 5.3.1 in [Puterman \[1994\]](#).

By definition of the finite-horizon value function and the law of total expectation:

$$v_\nu^\pi(s) = \sum_{n=1}^{\infty} P(\nu = n) \cdot v_n^\pi(s) = \sum_{n=1}^{\infty} (1 - \gamma)\gamma^{n-1} \cdot E_s^\pi \left\{ \sum_{t=1}^n r(S_t, A_t) \right\}. \quad (314)$$

Combining the expectation with the sum over n :

$$v_\nu^\pi(s) = E_s^\pi \left\{ \sum_{n=1}^{\infty} (1 - \gamma)\gamma^{n-1} \sum_{t=1}^n r(S_t, A_t) \right\}. \quad (315)$$

Reordering the summations: Under the bounded reward assumption $|r(s, a)| \leq R_{\max}$ and $\gamma < 1$, we have

$$E_s^\pi \left\{ \sum_{n=1}^{\infty} \sum_{t=1}^n |r(S_t, A_t)| \cdot (1 - \gamma)\gamma^{n-1} \right\} \leq R_{\max} \sum_{n=1}^{\infty} n(1 - \gamma)\gamma^{n-1} = \frac{R_{\max}}{1 - \gamma} < \infty, \quad (316)$$

which justifies exchanging the order of summation by Fubini's theorem.

To reverse the order, note that the pair (n, t) with $1 \leq t \leq n$ can be reindexed by fixing t first and letting n range from t to ∞ :

$$\sum_{n=1}^{\infty} \sum_{t=1}^n = \sum_{t=1}^{\infty} \sum_{n=t}^{\infty}. \quad (317)$$

Therefore:

$$v_\nu^\pi(s) = E_s^\pi \left\{ \sum_{t=1}^{\infty} r(S_t, A_t) \sum_{n=t}^{\infty} (1 - \gamma)\gamma^{n-1} \right\}.$$

Evaluating the inner sum: Using the substitution $m = n - t + 1$ (so $n = m + t - 1$):

$$\begin{aligned} \sum_{n=t}^{\infty} (1 - \gamma)\gamma^{n-1} &= \sum_{m=1}^{\infty} (1 - \gamma)\gamma^{m+t-2} \\ &= \gamma^{t-1}(1 - \gamma) \sum_{m=1}^{\infty} \gamma^{m-1} \\ &= \gamma^{t-1}(1 - \gamma) \cdot \frac{1}{1 - \gamma} = \gamma^{t-1}. \end{aligned}$$

Substituting back:

$$v_\nu^\pi(s) = E_s^\pi \left\{ \sum_{t=1}^{\infty} \gamma^{t-1} r(S_t, A_t) \right\} = v_\gamma^\pi(s). \quad (318)$$

□

Vector Representation in Markov Decision Processes Let V be the set of bounded real-valued functions on a discrete state space S . This means any function $f \in V$ satisfies the condition:

$$\|f\| = \max_{s \in S} |f(s)| < \infty. \quad (319)$$

where notation $\|f\|$ represents the sup-norm (or ℓ_∞ -norm) of the function f .

When working with discrete state spaces, we can interpret elements of V as vectors and linear operators on V as matrices, allowing us to leverage tools from linear algebra. The sup-norm (ℓ_∞ norm) of matrix $\mathbf{H}$ is defined as:

$$\|\mathbf{H}\| \equiv \max_{s \in S} \sum_{j \in S} |\mathbf{H}_{s,j}| \quad (320)$$

where $\mathbf{H}_{s,j}$ represents the (s, j) -th component of the matrix $\mathbf{H}$. For a Markovian decision rule $\pi \in \Pi^{MD}$, we define:

$$\begin{aligned} \mathbf{r}_\pi(s) &\equiv r(s, \pi(s)), \quad \mathbf{r}_\pi \in R^{|S|}, \\ [\mathbf{P}_\pi]_{s,j} &\equiv p(j \mid s, \pi(s)), \quad \mathbf{P}_\pi \in R^{|S| \times |S|}. \end{aligned}$$

For a randomized decision rule $\pi \in \Pi^{MR}$, these definitions extend to:

$$\begin{aligned} \mathbf{r}_\pi(s) &\equiv \sum_{a \in A_s} \pi(a \mid s) r(s, a), \\ [\mathbf{P}_\pi]_{s,j} &\equiv \sum_{a \in A_s} \pi(a \mid s) p(j \mid s, a). \end{aligned}$$

In both cases, $\mathbf{r}_\pi$ denotes a reward vector in $R^{|S|}$, with each component $\mathbf{r}_\pi(s)$ representing the reward associated with state s . Similarly, $\mathbf{P}_\pi$ is a transition probability matrix in $R^{|S| \times |S|}$, capturing the transition probabilities under decision rule π .

For a nonstationary Markovian policy $\boldsymbol{\pi} = (\pi_1, \pi_2, \dots) \in \Pi^{MR}$, the expected total discounted reward is given by:

$$\mathbf{v}_\gamma^\pi(s) = E \left[\sum_{t=1}^{\infty} \gamma^{t-1} r(S_t, A_t) \mid S_1 = s \right]. \quad (321)$$

Using vector notation, this can be expressed as:

$$\begin{aligned} \mathbf{v}_\gamma^\pi &= \sum_{t=1}^{\infty} \gamma^{t-1} \mathbf{P}_{\boldsymbol{\pi}}^{t-1} \mathbf{r}_{\pi_1} \\ &= \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{r}_{\pi_2} + \gamma^2 \mathbf{P}_{\pi_1} \mathbf{P}_{\pi_2} \mathbf{r}_{\pi_3} + \dots \\ &= \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} (\mathbf{r}_{\pi_2} + \gamma \mathbf{P}_{\pi_2} \mathbf{r}_{\pi_3} + \gamma^2 \mathbf{P}_{\pi_2} \mathbf{P}_{\pi_3} \mathbf{r}_{\pi_4} + \dots). \end{aligned} \quad (322)$$

This formulation leads to a recursive relationship:

$$\begin{aligned}\mathbf{v}_\gamma^\pi &= \mathbf{r}_{\pi_1} + \gamma \mathbf{P}_{\pi_1} \mathbf{v}_\gamma^{\pi'} \\ &= \sum_{t=1}^{\infty} \gamma^{t-1} \mathbf{P}_\pi^{t-1} \mathbf{r}_{\pi_t}\end{aligned}$$

where $\pi' = (\pi_2, \pi_3, \dots)$.

For a stationary policy $\pi = \text{const}(\pi)$ with constant decision rule π , the total expected reward simplifies to:

$$\begin{aligned}\mathbf{v}_\gamma^\pi &= \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_\gamma^\pi \\ &= \sum_{t=1}^{\infty} \gamma^{t-1} \mathbf{P}_\pi^{t-1} \mathbf{r}_\pi\end{aligned}$$

This last expression is called a Neumann series expansion, and it's guaranteed to exist under the assumptions of bounded reward and discount factor strictly less than one.

Theorem 5.6 (Neumann Series and Invertibility). *The **spectral radius** of a matrix $\mathbf{H}$ is defined as:*

$$\rho(\mathbf{H}) \equiv \max_i |\lambda_i(\mathbf{H})| \quad (323)$$

where $\lambda_i(\mathbf{H})$ are the eigenvalues of $\mathbf{H}$.

Neumann Series Existence: For any matrix $\mathbf{H}$, the Neumann series

$$\sum_{t=0}^{\infty} \mathbf{H}^t = \mathbf{I} + \mathbf{H} + \mathbf{H}^2 + \dots \quad (324)$$

converges if and only if $\rho(\mathbf{H}) < 1$. When this condition holds, the matrix $(\mathbf{I} - \mathbf{H})$ is invertible and

$$(\mathbf{I} - \mathbf{H})^{-1} = \sum_{t=0}^{\infty} \mathbf{H}^t. \quad (325)$$

Note that for any induced matrix norm $\|\cdot\|$ (i.e., a norm satisfying $\|\mathbf{H}\mathbf{v}\| \leq \|\mathbf{H}\| \cdot \|\mathbf{v}\|$ for all vectors $\mathbf{v}$) and any matrix $\mathbf{H}$, the spectral radius is bounded by:

$$\rho(\mathbf{H}) \leq \|\mathbf{H}\|. \quad (326)$$

This inequality provides a practical way to verify the convergence condition $\rho(\mathbf{H}) < 1$ by checking the simpler condition $\|\mathbf{H}\| < 1$ rather than trying to compute the eigenvalues directly.

We can now verify that $(\mathbf{I} - \gamma \mathbf{P}_\pi)$ is invertible and the Neumann series converges.

1. **Norm of the transition matrix:** Since $\mathbf{P}_\pi$ is a stochastic matrix (each row sums to 1 and all entries are non-negative), its ℓ_∞ -norm is:

$$\|\mathbf{P}_\pi\| = \max_{s \in S} \sum_{j \in S} [\mathbf{P}_\pi]_{s,j} = \max_{s \in S} 1 = 1. \quad (327)$$

2. **Norm of the scaled matrix:** Using the homogeneity property of norms, we have:

$$\|\gamma \mathbf{P}_\pi\| = |\gamma| \cdot \|\mathbf{P}_\pi\| = |\gamma| \cdot 1 = |\gamma|. \quad (328)$$

3. **Bounding the spectral radius:** Since the spectral radius is bounded by the matrix norm:

$$\rho(\gamma \mathbf{P}_\pi) \leq \|\gamma \mathbf{P}_\pi\| = |\gamma|. \quad (329)$$

4. **Verifying convergence:** Since $0 \leq \gamma < 1$ by assumption, we have:

$$\rho(\gamma \mathbf{P}_\pi) \leq |\gamma| < 1. \quad (330)$$

This strict inequality guarantees that $(\mathbf{I} - \gamma \mathbf{P}_\pi)$ is invertible and the Neumann series converges.

Therefore, the Neumann series expansion converges and yields:

$$\mathbf{v}_\gamma^\pi = (\mathbf{I} - \gamma \mathbf{P}_\pi)^{-1} \mathbf{r}_\pi = \sum_{t=0}^{\infty} (\gamma \mathbf{P}_\pi)^t \mathbf{r}_\pi = \sum_{t=1}^{\infty} \gamma^{t-1} \mathbf{P}_\pi^{t-1} \mathbf{r}_\pi. \quad (331)$$

Consequently, for a stationary policy, $\mathbf{v}_\gamma^\pi$ can be determined as the solution to the linear equation:

$$\mathbf{v} = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}, \quad (332)$$

which can be rearranged to:

$$(\mathbf{I} - \gamma \mathbf{P}_\pi) \mathbf{v} = \mathbf{r}_\pi. \quad (333)$$

We can also characterize $\mathbf{v}_\gamma^\pi$ as the solution to an operator equation. More specifically, define the transformation L_π by

$$L_\pi \mathbf{v} \equiv \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v} \quad (334)$$

for any $\mathbf{v} \in V$. Intuitively, L_π takes a value function $\mathbf{v}$ as input and returns a new value function that combines immediate rewards ($\mathbf{r}_\pi$) with discounted future values ($\gamma \mathbf{P}_\pi \mathbf{v}$).

Note

While we often refer to L_π as a “linear operator” in the RL literature, it is technically an **affine operator** (or affine transformation), not a linear operator in the strict sense. To see why, recall that a linear operator $\mathcal{T}$ must satisfy:

1. **Additivity:** $\mathcal{T}(\mathbf{v}_1 + \mathbf{v}_2) = \mathcal{T}(\mathbf{v}_1) + \mathcal{T}(\mathbf{v}_2)$
2. **Homogeneity:** $\mathcal{T}(\alpha \mathbf{v}) = \alpha \mathcal{T}(\mathbf{v})$ for all scalars α

However, L_π fails the additivity test:

$$L_\pi(\mathbf{v}_1 + \mathbf{v}_2) = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi(\mathbf{v}_1 + \mathbf{v}_2) = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_1 + \gamma \mathbf{P}_\pi \mathbf{v}_2 \quad (335)$$

while

$$L_\pi(\mathbf{v}_1) + L_\pi(\mathbf{v}_2) = (\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_1) + (\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_2) = 2\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_1 + \gamma \mathbf{P}_\pi \mathbf{v}_2. \quad (336)$$

The presence of the constant term $\mathbf{r}_\pi$ makes L_π affine rather than linear. An affine operator has the form $\mathcal{A}(\mathbf{v}) = \mathbf{b} + \mathcal{T}(\mathbf{v})$, where $\mathbf{b}$ is a constant vector and $\mathcal{T}$ is a linear operator. In our case, $\mathbf{b} = \mathbf{r}_\pi$ and $\mathcal{T}(\mathbf{v}) = \gamma \mathbf{P}_\pi \mathbf{v}$.

Despite this technical distinction, the term “linear operator” is commonly used in the reinforcement learning literature when referring to L_π , following a slight abuse of terminology.

Therefore, we view L_π as an operator mapping elements of V to V : i.e., $L_\pi : V \rightarrow V$. The fact that the value function of a policy is the solution to a fixed-point equation can then be expressed with the statement:

$$\mathbf{v}_\gamma^\pi = L_\pi \mathbf{v}_\gamma^\pi. \quad (337)$$

This is a **fixed-point equation**: the value function $\mathbf{v}_\gamma^\pi$ is a fixed point of the operator L_π .

Solving Operator Equations The operator equation we encountered in MDPs, $\mathbf{v}_\gamma^\pi = L_\pi \mathbf{v}_\gamma^\pi$, is a specific instance of a more general class of problems known as operator equations. These equations appear in various fields of mathematics and applied sciences, ranging from differential equations to functional analysis.

Operator equations can take several forms, each with its own characteristics and solution methods:

1. **Fixed Point Form:** $x = T(x)$, where $T : X \rightarrow X$. Common in fixed-point problems, such as our MDP equation, we seek a fixed point x^* such that $x^* = T(x^*)$.

2. **General Operator Equation:** $T(x) = y$, where $T : X \rightarrow Y$. Here, X and Y can be different spaces. We seek an $x \in X$ that satisfies the equation for a given $y \in Y$.
3. **Nonlinear Equation:** $T(x) = 0$, where $T : X \rightarrow Y$. A special case of the general operator equation where we seek roots or zeros of the operator.
4. **Variational Inequality:** Find $x^* \in K$ such that $\langle T(x^*), x - x^* \rangle \geq 0$ for all $x \in K$. Here, K is a closed convex subset of X , and $T : K \rightarrow X^*$ (the dual space of X). These problems often arise in optimization, game theory, and partial differential equations.

Successive Approximation Method For equations in fixed point form, a common numerical solution method is successive approximation, also known as fixed-point iteration:

The convergence of successive approximation depends on the properties of the operator T . In the simplest and most common setting, we assume T is a contraction mapping. The Banach Fixed-Point Theorem then guarantees that T has a unique fixed point, and the successive approximation method will converge to this fixed point from any starting point. Specifically, T is a contraction if there exists a constant $q \in [0, 1)$ such that for all $x, y \in X$:

$$d(T(x), T(y)) \leq q \cdot d(x, y) \quad (338)$$

where d is the metric on X . In this case, the rate of convergence is linear, with error bound:

$$d(x_n, x^*) \leq \frac{q^n}{1 - q} d(x_1, x_0) \quad (339)$$

However, the contraction mapping condition is not the only one that can lead to convergence. For instance, if T is nonexpansive (i.e., Lipschitz continuous with Lipschitz constant 1) and X is a Banach space with certain geometrical properties (e.g., uniformly convex), then under additional conditions (e.g., T has at least one fixed point), the successive approximation method can still converge, albeit potentially more slowly than in the contraction case.

In practice, when dealing with specific problems like MDPs or differential equations, the properties of the operator often naturally align with one of these convergence conditions. For example, in discounted MDPs, the Bellman operator is a contraction in the supremum norm, which guarantees the convergence of value iteration.

Newton-Kantorovich Method The Newton-Kantorovich method is a generalization of Newton's method from finite dimensional vector spaces to infinite dimensional function spaces: rather than iterating in the space of vectors, we are iterating in the space of functions.

Newton's method is often written as the familiar update:

$$x_{k+1} = x_k - [DF(x_k)]^{-1}F(x_k), \quad (340)$$

which makes it look as though the essence of the method is “take a derivative and invert it.” But the real workhorse behind Newton’s method (both in finite and infinite dimensions) is **linearization**.

At each step, the idea is to replace the nonlinear operator $F : X \rightarrow Y$ by a local surrogate model of the form

$$F(x + h) \approx F(x) + Lh, \quad (341)$$

where L is a linear map capturing how small perturbations in the input propagate to changes in the output. This is a Taylor-like expansion in Banach spaces: the role of the derivative is precisely to provide the correct notion of such a linear operator.

To find a root of F , we impose the condition that the surrogate vanishes at the next iterate:

$$0 = F(x + h) \approx F(x) + Lh. \quad (342)$$

Solving this linear equation gives the increment h . In finite dimensions, L is the Jacobian matrix; in Banach spaces, it must be the **Fréchet derivative**.

But what exactly is a Fréchet derivative in infinite dimensions? To understand this, we need to generalize the concept of derivative from finite-dimensional calculus. In infinite-dimensional spaces, there are several notions of differentiability, each with different strengths and requirements:

1. Gâteaux (Directional) Derivative

We say that the Gâteaux derivative of F at x in a specific direction h is:

$$F'(x; h) = \lim_{t \rightarrow 0} \frac{F(x + th) - F(x)}{t} \quad (343)$$

This quantity measures how the function F changes along the ray $x + th$. While this limit may exist for each direction h separately, it doesn’t guarantee that the derivative is linear in h . This is a key limitation: the Gâteaux derivative can exist in all directions but still fail to provide a good linear approximation.

2. Hadamard Directional Derivative

Rather than considering a single direction of perturbation, we now consider a bundle of perturbations around h . We ask how the function changes as we approach the target direction from nearby directions. We say that F has a Hadamard directional derivative if:

$$F'(x; h) = \lim_{\substack{t \downarrow 0 \\ h' \rightarrow h}} \frac{F(x + th') - F(x)}{t} \quad (344)$$

This is a stronger condition than Gâteaux differentiability because it requires the limit to be uniform over nearby directions. However, it still doesn’t guarantee linearity in h .

3. Fréchet Derivative

The strongest and most natural notion: F is Fréchet differentiable at x if there exists a bounded linear operator L such that:

$$\lim_{h \rightarrow 0} \frac{\|F(x+h) - F(x) - Lh\|}{\|h\|} = 0 \quad (345)$$

This definition directly addresses the inadequacy of the previous notions. Unlike Gâteaux and Hadamard derivatives, the Fréchet derivative explicitly requires the existence of a linear operator L that provides a good approximation. Key properties:

- L must be **linear** in h (unlike the directional derivatives above)
- The approximation error is $o(\|h\|)$, uniform in all directions
- This is the “true” derivative: it generalizes the Jacobian matrix to infinite dimensions
- Notation: $L = F'(x)$ or $DF(x)$

Relationship:

$$\text{Fréchet differentiable} \Rightarrow \text{Hadamard directionally diff.} \Rightarrow \text{Gâteaux directionally diff.} \quad (346)$$

In the context of the Newton-Kantorovich method, we work with an operator $F : X \rightarrow Y$ where both X and Y are Banach spaces. The Fréchet derivative $F'(x)$ is the best linear approximation of F near x , and it's exactly this linear operator L that we use in our linearization $F(x+h) \approx F(x) + F'(x)h$.

Now apart from those mathematical technicalities, Newton-Kantorovich has in essence the same structure as that of the original Newton's method. That is, it applies the following sequence of steps:

1. **Linearize the Operator:** Given an approximation x_n , we consider the Fréchet derivative of F , denoted by $F'(x_n)$. This derivative is a linear operator that provides a local approximation of F near x_n .
2. **Set Up the Newton Step:** The method then solves the linearized equation for a correction h_n :

$$F'(x_n)h_n = -F(x_n). \quad (347)$$

This equation represents a linear system where h_n is chosen so that the linearized operator $F(x_n) + F'(x_n)h_n$ equals zero.

3. **Update the Solution:** The new approximation x_{n+1} is then given by:

$$x_{n+1} = x_n + h_n. \quad (348)$$

This correction step refines x_n , bringing it closer to the true solution.

4. **Repeat Until Convergence:** We repeat the linearization and update steps until the solution x_n converges to the desired tolerance, which can be verified by checking that $\|F(x_n)\|$ is sufficiently small, or by monitoring the norm $\|x_{n+1} - x_n\|$.

The convergence of Newton-Kantorovich does not hinge on F being a contraction over the entire domain (as it could be the case for successive approximation). The convergence properties of the Newton-Kantorovich method are as follows:

1. **Local Convergence:** Under mild conditions (e.g., F is Fréchet differentiable and $F'(x)$ is invertible near the solution), the method converges locally. This means that if the initial guess is sufficiently close to the true solution, the method will converge.
2. **Global Convergence:** Global convergence is not guaranteed in general. However, under stronger conditions (e.g., F is analytic and satisfies certain bounds), the method can converge globally.
3. **Rate of Convergence:** When the method converges, it typically exhibits quadratic convergence. This means that the error at each step is proportional to the square of the error at the previous step:

$$\|x_{n+1} - x^*\| \leq C\|x_n - x^*\|^2 \quad (349)$$

where x^* is the true solution and C is some constant. This quadratic convergence is significantly faster than the linear convergence typically seen in methods like successive approximation.

Optimality Equations for Infinite-Horizon MDPs Recall that in the finite-horizon setting, the optimality equations are:

$$v_n(s) = \max_{a \in A_s} \left\{ r(s, a) + \gamma \sum_{j \in S} p(j|s, a) v_{n+1}(j) \right\} \quad (350)$$

where $v_n(s)$ is the value function at time step n for state s , A_s is the set of actions available in state s , $r(s, a)$ is the reward function, γ is the discount factor, and $p(j|s, a)$ is the transition probability from state s to state j given action a .

Intuitively, we would expect that by taking the limit of n to infinity, we might get the nonlinear equations:

$$v(s) = \max_{a \in A_s} \left\{ r(s, a) + \gamma \sum_{j \in S} p(j|s, a) v(j) \right\} \quad (351)$$

which are called the optimality equations or Bellman equations for infinite-horizon MDPs.

We can adopt an operator-theoretic perspective by defining operators on the space V of bounded real-valued functions on the state space S . For a deterministic Markov rule $\pi \in \Pi^{MD}$, define the **policy-evaluation operator**:

$$(\mathbf{L}_\pi v)(s) = r(s, \pi(s)) + \gamma \sum_{j \in S} p(j|s, \pi(s))v(j) \quad (352)$$

The **Bellman optimality operator** is then:

$$\mathbf{L}v \equiv \max_{\pi \in \Pi^{MD}} \{\mathbf{r}_\pi + \gamma \mathbf{P}_\pi v\} \quad (353)$$

where Π^{MD} is the set of Markov deterministic decision rules, $\mathbf{r}_\pi$ is the reward vector under decision rule π , and $\mathbf{P}_\pi$ is the transition probability matrix under decision rule π .

Note that while we write $\max_{\pi \in \Pi^{MD}}$, we do not implement the above operator by enumerating all decision rules. Rather, the fact that we compare policies based on their value functions in a componentwise fashion means that maximizing over the space of Markovian deterministic rules reduces to the following update in component form:

$$(\mathbf{L}v)(s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_{j \in S} p(j|s, a)v(j) \right\} \quad (354)$$

For convenience, we define the **greedy selector** $\text{Greedy}(v) \in \Pi^{MD}$ that extracts an optimal decision rule from a value function:

$$\text{Greedy}(v)(s) \in \arg \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_{j \in S} p(j|s, a)v(j) \right\} \quad (355)$$

In Puterman's terminology, such a greedy selector is called ***v*-improving** (or **conserving** when it achieves the maximum). This operator will be useful for expressing algorithms succinctly:

- **Value iteration:** $v_{k+1} = \mathbf{L}v_k$, then extract $\pi = \text{Greedy}(v^*)$
- **Policy iteration:** $\pi_{k+1} = \text{Greedy}(v^{\pi_k})$ with v^{π_k} solving $v = \mathbf{L}_{\pi_k} v$

The equivalence between these two forms can be shown mathematically, as demonstrated in the following proposition and proof.

Proposition 5.2. *The operator $\mathbf{L}$ defined as a maximization over Markov deterministic decision rules:*

$$(\mathbf{L}v)(s) = \max_{\pi \in \Pi^{MD}} \left\{ r(s, \pi(s)) + \gamma \sum_{j \in S} p(j|s, \pi(s))v(j) \right\} \quad (356)$$

is equivalent to the componentwise maximization over actions:

$$(\mathbf{L}\mathbf{v})(s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right\} \quad (357)$$

Proof. Fix s . Let

$$Q_v(s, a) r(s, a) + \gamma \sum_j p(j | s, a) v(j). \quad (358)$$

For any rule $\pi \in \Pi^{MD}$, we have $(\mathbf{L}_\pi v)(s) = Q_v(s, \pi(s)) \leq \max_{a \in \mathcal{A}_s} Q_v(s, a)$. Taking the maximum over π gives

$$\max_{\pi \in \Pi^{MD}} (\mathbf{L}_\pi v)(s) \leq \max_{a \in \mathcal{A}_s} Q_v(s, a). \quad (359)$$

Conversely, choose a **greedy selector** $\pi^v \in \Pi^{MD}$ such that for each s ,

$$\pi^v(s) \in \arg \max_{a \in \mathcal{A}_s} Q_v(s, a) \quad (360)$$

(possible since $\mathcal{A}_s$ is finite; otherwise use a measurable ε -greedy selector). Then

$$(\mathbf{L}_{\pi^v} v)(s) = Q_v(s, \pi^v(s)) = \max_{a \in \mathcal{A}_s} Q_v(s, a), \quad (361)$$

so $\max_\pi (\mathbf{L}_\pi v)(s) \geq \max_a Q_v(s, a)$. Combining both inequalities yields equality. $\square$

Algorithms for Solving the Optimality Equations The optimality equations are operator equations. Therefore, we can apply general numerical methods to solve them. Applying the successive approximation method to the Bellman optimality equation yields a method known as “value iteration” in dynamic programming. A direct application of the blueprint for successive approximation yields the following algorithm:

The termination criterion in this algorithm is based on a specific bound that provides guarantees on the quality of the solution. This is in contrast to supervised learning, where we often use arbitrary termination criteria based on computational budget or early stopping when the learning curve flattens. This is because establishing implementable generalization bounds in supervised learning is challenging.

However, in the dynamic programming context, we can derive various bounds that can be implemented in practice. These bounds help us terminate our procedure with a guarantee on the precision of our value function and, correspondingly, on the optimality of the resulting policy.

Proposition 5.3 (Convergence of Value Iteration). *(Adapted from [Puterman \[1994\]](#) theorem 6.3.1)*

Let v_0 be any initial value function, $\varepsilon > 0$ a desired accuracy, and let $\{v_n\}$ be the sequence of value functions generated by value iteration, i.e., $v_{n+1} = \mathbf{L}v_n$ for $n \geq 0$, where $\mathbf{L}$ is the Bellman optimality operator. Then:

1. v_n converges to the optimal value function v_γ^* ,
2. The algorithm terminates in finite time,
3. The resulting policy π_ε is ε -optimal, and
4. When the algorithm terminates, v_{n+1} is within $\varepsilon/2$ of v_γ^* .

Proof. Parts 1 and 2 follow directly from the fact that $\mathbf{L}$ is a contraction mapping. Hence, by Banach's fixed-point theorem, it has a unique fixed point (which is v_γ^*), and repeated application of $\mathbf{L}$ will converge to this fixed point. Moreover, this convergence happens at a geometric rate, which ensures that we reach the termination condition in finite time.

To show that the Bellman optimality operator $\mathbf{L}$ is a contraction mapping, we need to prove that for any two value functions v and u :

$$\|\mathbf{L}v - \mathbf{L}u\|_\infty \leq \gamma \|v - u\|_\infty \quad (362)$$

where $\gamma \in [0, 1)$ is the discount factor and $\|\cdot\|_\infty$ is the supremum norm. Let's start by writing out the definition of $\mathbf{L}v$ and $\mathbf{L}u$:

$$\begin{aligned} (\mathbf{L}v)(s) &= \max_{a \in A} \left\{ r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right\} \\ (\mathbf{L}u)(s) &= \max_{a \in A} \left\{ r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) u(j) \right\} \end{aligned}$$

For any state s , let a_v be the action that achieves the maximum for $(\mathbf{L}v)(s)$, and a_u be the action that achieves the maximum for $(\mathbf{L}u)(s)$. By the definition of these maximizers:

$$\begin{aligned} (\mathbf{L}v)(s) &\geq r(s, a_u) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a_u) v(j) \\ (\mathbf{L}u)(s) &\geq r(s, a_v) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a_v) u(j) \end{aligned}$$

Subtracting these inequalities:

$$\begin{aligned}
(\mathbf{L}v)(s) - (\mathbf{L}u)(s) &\leq \gamma \sum_{j \in \mathcal{S}} p(j|s, a_v)(v(j) - u(j)) \\
(\mathbf{L}u)(s) - (\mathbf{L}v)(s) &\leq \gamma \sum_{j \in \mathcal{S}} p(j|s, a_u)(u(j) - v(j))
\end{aligned}$$

Taking the absolute value and using the fact that $\sum_{j \in \mathcal{S}} p(j|s, a) = 1$:

$$|(\mathbf{L}v)(s) - (\mathbf{L}u)(s)| \leq \gamma \max_{j \in \mathcal{S}} |v(j) - u(j)| = \gamma \|v - u\|_\infty \quad (363)$$

Since this holds for all $s \in \mathcal{S}$, taking the supremum over s gives:

$$\|\mathbf{L}v - \mathbf{L}u\|_\infty \leq \gamma \|v - u\|_\infty \quad (364)$$

Thus, $\mathbf{L}$ is a contraction mapping with contraction factor γ .

Now, let's prove parts 3 and 4. Suppose the algorithm has just terminated, i.e., $\|v_{n+1} - v_n\|_\infty < \frac{\varepsilon(1-\gamma)}{2\gamma}$ for some n . We want to show that our current value function v_{n+1} and the policy π_ε derived from it are close to optimal.

By the triangle inequality:

$$\|v_\gamma^{\pi_\varepsilon} - v_\gamma^*\|_\infty \leq \|v_\gamma^{\pi_\varepsilon} - v_{n+1}\|_\infty + \|v_{n+1} - v_\gamma^*\|_\infty \quad (365)$$

For the first term, since $v_\gamma^{\pi_\varepsilon}$ is the fixed point of $\mathbf{L}_{\pi_\varepsilon}$ and π_ε is greedy with respect to v_{n+1} (i.e., $\mathbf{L}_{\pi_\varepsilon} v_{n+1} = \mathbf{L}v_{n+1}$):

$$\begin{aligned}
\|v_\gamma^{\pi_\varepsilon} - v_{n+1}\|_\infty &= \|\mathbf{L}_{\pi_\varepsilon} v_\gamma^{\pi_\varepsilon} - v_{n+1}\|_\infty \\
&\leq \|\mathbf{L}_{\pi_\varepsilon} v_\gamma^{\pi_\varepsilon} - \mathbf{L}_{\pi_\varepsilon} v_{n+1}\|_\infty + \|\mathbf{L}_{\pi_\varepsilon} v_{n+1} - v_{n+1}\|_\infty \\
&= \|\mathbf{L}_{\pi_\varepsilon} v_\gamma^{\pi_\varepsilon} - \mathbf{L}_{\pi_\varepsilon} v_{n+1}\|_\infty + \|\mathbf{L}v_{n+1} - v_{n+1}\|_\infty \\
&\leq \gamma \|v_\gamma^{\pi_\varepsilon} - v_{n+1}\|_\infty + \gamma \|v_{n+1} - v_n\|_\infty
\end{aligned} \quad (366)$$

where we used that both $\mathbf{L}$ and $\mathbf{L}_{\pi_\varepsilon}$ are contractions with factor γ , and that $v_{n+1} = \mathbf{L}v_n$.

Rearranging:

$$\|v_\gamma^{\pi_\varepsilon} - v_{n+1}\|_\infty \leq \frac{\gamma}{1-\gamma} \|v_{n+1} - v_n\|_\infty \quad (367)$$

Similarly, since v_γ^* is the fixed point of $\mathbf{L}$:

$$\|v_{n+1} - v_\gamma^*\|_\infty = \|\mathbf{L}v_n - \mathbf{L}v_\gamma^*\|_\infty \leq \gamma \|v_n - v_\gamma^*\|_\infty \leq \frac{\gamma}{1-\gamma} \|v_{n+1} - v_n\|_\infty \quad (368)$$

Since $\|v_{n+1} - v_n\|_\infty < \frac{\varepsilon(1-\gamma)}{2\gamma}$:

$$\|v_\gamma^{\pi_\varepsilon} - v_{n+1}\|_\infty \leq \frac{\gamma}{1-\gamma} \cdot \frac{\varepsilon(1-\gamma)}{2\gamma} = \frac{\varepsilon}{2} \quad (369)$$

$$\|v_{n+1} - v_\gamma^*\|_\infty \leq \frac{\gamma}{1-\gamma} \cdot \frac{\varepsilon(1-\gamma)}{2\gamma} = \frac{\varepsilon}{2} \quad (370)$$

Combining these:

$$\|v_{\gamma^{\pi_\varepsilon}} - v_\gamma^*\|_\infty \leq \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon \quad (371)$$

This completes the proof, showing that v_{n+1} is within $\varepsilon/2$ of v_γ^* (part 4) and π_ε is ε -optimal (part 3). □

Bellman contraction laboratory Use the controls below to change the discount factor, transition persistence, reward asymmetry, and starting value. Before moving γ , predict how it will change the slope of the error envelope. The middle panel compares the observed error and Bellman residual with the contraction bound; the text below reports the final greedy policy.

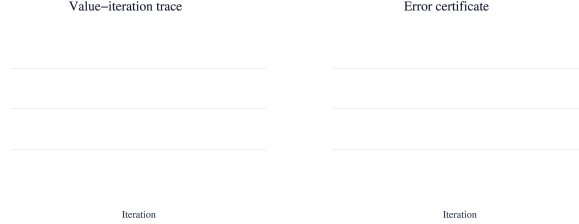


Figure 1: Static preview of value-iteration traces and a geometric contraction bound. The online book provides controls for γ , transitions, rewards, and the initial value.

Newton-Kantorovich Applied to Bellman Optimality We now apply the Newton-Kantorovich framework to the Bellman optimality equation. Let

$$(\mathbf{L}v)(s) = \max_{a \in A(s)} \left\{ r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v(s') \right\}. \quad (372)$$

The problem is to find v such that $\mathbf{L}v = v$, or equivalently $\mathbf{B}(v) := \mathbf{L}v - v = 0$. The operator $\mathbf{L}$ is piecewise affine, hence not globally differentiable, but it is directionally differentiable everywhere in the Hadamard sense and Fréchet differentiable at points where the maximizer is unique.

We consider three complementary perspectives for understanding and computing its derivative.

Perspective 1: Max of Affine Maps In tabular form, for finite state and action spaces, the Bellman operator can be written as a pointwise maximum of affine maps:

$$(\mathbf{L}v)(s) = \max_{a \in A(s)} \{r(s, a) + \gamma(P_a v)(s)\}, \quad (373)$$

where $P_a \in R^{|S| \times |S|}$ is the transition matrix associated with action a . Each $Q_a v := r^a + \gamma P_a v$ is affine in v . The operator $\mathbf{L}$ therefore computes the upper envelope of a finite set of affine functions at each state.

At any v , let the **active set** at state s be

$$\mathcal{A}^*(s; v) := \arg \max_{a \in A(s)} (Q_a v)(s). \quad (374)$$

Then the Hadamard directional derivative exists and is given by

$$(\mathbf{L}'(v; h))(s) = \max_{a \in \mathcal{A}^*(s; v)} \gamma(P_a h)(s). \quad (375)$$

If the active set is a singleton, this expression becomes linear in h , and $\mathbf{L}$ is Fréchet differentiable at v , with

$$\mathbf{L}'(v) = \gamma P_{\pi_v}, \quad (376)$$

where $\pi_v(s) := a^*(s)$ is the greedy policy at v .

Perspective 2: Envelope Theorem Consider now a value function approximated as a linear combination of basis functions:

$$v_c(s) = \sum_j c_j \phi_j(s). \quad (377)$$

At a node s_i , define the parametric maximization

$$v_i(c) := (\mathbf{L}v_c)(s_i) = \max_{a \in A(s_i)} \left\{ r(s_i, a) + \gamma \sum_j c_j E_{s'|s_i, a}[\phi_j(s')] \right\}. \quad (378)$$

Define

$$F_i(a, c) := r(s_i, a) + \gamma \sum_j c_j E_{s'|s_i, a}[\phi_j(s')], \quad (379)$$

so that $v_i(c) = \max_a F_i(a, c)$. Since F_i is linear in c , we can apply the **envelope theorem** (Danskin's theorem): if the optimizer $a_i^*(c)$ is unique or selected measurably, then

$$\frac{\partial v_i}{\partial c_j}(c) = \gamma E_{s'|s_i, a_i^*(c)}[\phi_j(s')]. \quad (380)$$

We do not need to differentiate the optimizer $a_i^*(c)$ itself. The result extends to the subdifferential case when ties occur, where the Jacobian becomes set-valued.

This result is useful when solving the collocation equation $\Phi c = v(c)$. Newton's method requires the Jacobian $v'(c)$, and this expression allows us to compute it without involving any derivatives of the optimal action.

Perspective 3: The Implicit Function Theorem The third perspective applies the implicit function theorem to understand when the Bellman operator is differentiable despite containing a max operator. The maximization problem defines an implicit relationship between the value function and the optimal action, and the implicit function theorem tells us when this relationship is smooth enough to differentiate through.

The Bellman operator is defined as

$$(\mathcal{L}v)(s) = \max_a \left\{ r(s, a) + \gamma \sum_j p(j \mid s, a) v(j) \right\}. \quad (381)$$

The difficulty is that the max operator encodes a discrete selection: which action achieves the maximum. To apply the implicit function theorem, we reformulate this as follows. For each action a , define the **action-value function**:

$$Q_a(v, s) := r(s, a) + \gamma \sum_j p(j \mid s, a) v(j). \quad (382)$$

The optimal action at v satisfies the **optimality condition**:

$$Q_{a^*(s)}(v, s) \geq Q_a(v, s) \quad \text{for all } a. \quad (383)$$

Now suppose that at a particular v , action $a^*(s)$ is a **strict local maximizer** in the sense that there exists $\delta > 0$ such that

$$Q_{a^*(s)}(v, s) > Q_a(v, s) + \delta \quad \text{for all } a \neq a^*(s). \quad (384)$$

This strict inequality is the regularity condition needed for the implicit function theorem. It ensures that the optimal action is unique at v and remains so in a neighborhood of v .

To see why, consider any perturbation $v + h$ with $\|h\|$ small. Since Q_a is linear in v , we have:

$$Q_a(v + h, s) = Q_a(v, s) + \gamma \sum_j p(j \mid s, a) h(j). \quad (385)$$

The perturbation term is bounded: $|\gamma \sum_j p(j \mid s, a) h(j)| \leq \gamma \|h\|$. Therefore, for $\|h\| < \delta/\gamma$, the strict gap ensures that

$$Q_{a^*(s)}(v + h, s) > Q_a(v + h, s) \quad \text{for all } a \neq a^*(s). \quad (386)$$

Thus $a^*(s)$ remains the unique maximizer throughout the neighborhood $\{v + h : \|h\| < \delta/\gamma\}$.

The implicit function theorem now applies: in this neighborhood, the mapping $v \mapsto a^*(s; v)$ is **constant** (and hence smooth), taking the value $a^*(s)$. This allows us to write

$$(\mathbf{L}v)(s) = Q_{a^*(s)}(v, s) = r(s, a^*(s)) + \gamma \sum_j p(j | s, a^*(s))v(j) \quad (387)$$

as an explicit formula that holds throughout the neighborhood. Since $Q_{a^*(s)}(\cdot, s)$ is an affine (hence smooth) function of v , we can differentiate it:

$$\frac{d}{dv}(\mathbf{L}v)(s) = \gamma P_{a^*(s)}. \quad (388)$$

More precisely, for any perturbation h :

$$(\mathbf{L}(v + h))(s) = (\mathbf{L}v)(s) + \gamma \sum_j p(j | s, a^*(s))h(j) + o(\|h\|). \quad (389)$$

This is the Fréchet derivative:

$$\mathbf{L}'(v) = \gamma P_{\pi_v}, \quad (390)$$

where $\pi_v(s) = a^*(s)$ is the greedy policy.

The role of the implicit function theorem: It guarantees that when the maximizer is unique with a strict gap (the regularity condition), the argmax function $v \mapsto a^*(s; v)$ is locally constant, which removes the non-differentiability of the max operator. Without this regularity condition (specifically, at points where multiple actions tie for optimality), the implicit function theorem does not apply, and the operator is not Fréchet differentiable. The active set perspective (Perspective 1) and the envelope theorem (Perspective 2) provide the tools to handle these non-smooth points.

Connection to Policy Iteration We return to the Newton-Kantorovich step:

$$(I - \mathbf{L}'(v_n))h_n = v_n - \mathbf{L}v_n, \quad v_{n+1} = v_n - h_n. \quad (391)$$

Suppose $\mathbf{L}'(v_n) = \gamma P_{\pi_{v_n}}$ for the greedy policy π_{v_n} . Then

$$(I - \gamma P_{\pi_{v_n}})v_{n+1} = r^{\pi_{v_n}}, \quad (392)$$

which is exactly policy evaluation for π_{v_n} . Recomputing the greedy policy from v_{n+1} yields the next iterate.

Thus, **policy iteration is Newton-Kantorovich** applied to the Bellman optimality equation. At points of nondifferentiability (when ties occur), the

operator is still semismooth, and policy iteration corresponds to a semismooth Newton method. The envelope theorem is what justifies the simplification of the Jacobian to γP_{π_v} , bypassing the need to differentiate through the optimizer. This completes the equivalence.

The Semismooth Newton Perspective The three perspectives we developed above (the active set view, the envelope theorem, and the implicit function theorem) all point toward a deeper framework for understanding Newton-type methods on non-smooth operators. This framework, known as semismooth Newton methods, was developed precisely to handle operators like the Bellman operator that are piecewise smooth but not globally differentiable. The connection between policy iteration and semismooth Newton methods has been rigorously developed in recent work [Gargiani et al. \[2022\]](#).

The classical Newton-Kantorovich method assumes the operator is Fréchet differentiable everywhere. The derivative exists, is unique, and varies continuously with the base point. But the Bellman operator L violates this assumption at any value function where multiple actions tie for optimality at some state. At such points, the implicit function theorem fails, and there is no unique Fréchet derivative.

Semismooth Newton methods address this by replacing the notion of a single Jacobian with a generalized derivative that captures the behavior of the operator near non-smooth points. The most commonly used generalized derivative is the Clarke subdifferential, which we can think of as the convex hull of all possible “candidate Jacobians” that arise from limits approaching the non-smooth point from different directions.

For the Bellman residual $B(v) = Lv - v$, the Clarke subdifferential at a point v can be characterized explicitly using our first perspective. Recall that at each state s , we defined the active set $\mathcal{A}^*(s; v) = \arg \max_a Q_a(v, s)$. When this set contains multiple actions, the operator is not Fréchet differentiable. However, it remains directionally differentiable in all directions, and the Clarke subdifferential consists of all matrices of the form

$$\partial B(v) = \{I - \gamma P_\pi : \pi(s) \in \mathcal{A}^*(s; v) \text{ for all } s\}. \quad (393)$$

In words, the generalized Jacobian is the set of all matrices $I - \gamma P_\pi$ where π is any policy that selects an action from the active set at each state. When the maximizer is unique everywhere, this set reduces to a singleton, and we recover the classical Fréchet derivative. When ties occur, the set has multiple elements: precisely the convex combinations mentioned in Perspective 1.

The semismooth Newton method for solving $B(v) = 0$ proceeds by selecting an element $J_k \in \partial B(v_k)$ at each iteration and solving

$$J_k h_k = -B(v_k), \quad v_{k+1} = v_k + h_k. \quad (394)$$

What this tells us is that any choice from the Clarke subdifferential yields a valid Newton-like update. In the context of the Bellman equation, choosing $J_k = I - \gamma P_{\pi_k}$ where π_k is any greedy policy corresponds exactly to the policy

evaluation step in policy iteration. The freedom in selecting which action to choose when ties occur translates to the freedom in selecting which element of the subdifferential to use.

Under appropriate regularity conditions (specifically, when the residual function is BD-regular or CD-regular), the semismooth Newton method converges locally at a quadratic rate [Gargiani et al. \[2022\]](#). This means that near the solution, the error decreases quadratically:

$$\|v_{k+1} - v^*\| \leq C\|v_k - v^*\|^2. \quad (395)$$

This theoretical result explains an empirical observation that has long been noted in practice: policy iteration typically converges in very few iterations, often just a handful, even when the state and action spaces are enormous and the space of possible policies is exponentially large.

The semismooth Newton framework also suggests a spectrum of methods interpolating between value iteration and policy iteration. Value iteration can be interpreted as a Newton-like method where we choose $J_k = I$ at every iteration, ignoring the dependence of $\mathbf{L}$ on v entirely. This choice guarantees global convergence through the contraction property but sacrifices the quadratic local convergence rate. Policy iteration, at the other extreme, uses the full generalized Jacobian $J_k = I - \gamma P_{\pi_k}$, achieving quadratic convergence but at the cost of solving a linear system at each iteration.

Between these extremes lie methods that use approximate Jacobians. One natural variant is to choose $J_k = \alpha I$ for some scalar $\alpha > 1$. This leads to the update

$$v_{k+1} = \frac{\alpha - 1}{\alpha} v_k + \frac{1}{\alpha} \mathbf{L} v_k. \quad (396)$$

This is known as α -value iteration or successive over-relaxation when $\alpha > 1$. For appropriate choices of α , this method retains global convergence while achieving better local rates than standard value iteration, and it requires only pointwise operations rather than solving a linear system. The Newton perspective thus unifies existing algorithms and generates new ones by systematically exploring different approximations to the generalized Jacobian.

The connection to semismooth Newton methods places policy iteration within a broader mathematical framework that extends far beyond dynamic programming. Semismooth Newton methods are used in optimization (for complementarity problems and variational inequalities), in PDE-constrained optimization (for problems with control constraints), and in economics (for equilibrium problems). The Bellman equation, viewed through this lens, is simply one instance of a piecewise smooth equation, and the tools developed for such equations apply directly.

Policy Iteration While we derived policy iteration-like steps from the Newton-Kantorovich method, it's worth examining policy iteration as a standalone algorithm, as it has been traditionally presented in the field of dynamic programming.

The policy iteration algorithm for discounted Markov decision problems is as follows:

As opposed to value iteration, this algorithm produces a sequence of both deterministic Markovian decision rules $\{\pi_n\}$ and value functions $\{\mathbf{v}^n\}$. We recognize in this algorithm the linearization step of the Newton-Kantorovich procedure, which takes place here in the policy evaluation step 3 where we solve the linear system $(\mathbf{I} - \gamma \mathbf{P}_{\pi_n})\mathbf{v} = \mathbf{r}_{\pi_n}$. In practice, this linear system could be solved either using direct methods (eg. Gaussian elimination), using simple iterative methods such as the successive approximation method for policy evaluation, or more sophisticated methods such as GMRES.

Self-checks

6 Approximate Dynamic Programming

Dynamic programming methods suffer from the curse of dimensionality and can quickly become difficult to apply in practice. We may also be dealing with large or continuous state or action spaces. We have seen so far that we could address this problem using discretization, or interpolation. These were already examples of approximate dynamic programming. In this chapter, we will see other forms of approximations meant to facilitate the optimization problem, either by approximating the optimality equations, the value function, or the policy itself. Approximation theory is at the heart of learning methods, and fundamentally, this chapter will be about the application of learning ideas to solve complex decision-making problems.

6.1 Smooth Bellman Optimality Equations

While the standard Bellman optimality equations use the max operator to determine the best action, an alternative formulation known as the smooth or soft Bellman optimality equations replaces this with a softmax operator. This approach originated from Rust [1987] and was later rediscovered in the context of maximum entropy inverse reinforcement learning Ziebart et al. [2008], which then led to soft Q-learning Haarnoja et al. [2017] and soft actor-critic Haarnoja et al. [2018a], a state-of-the-art deep reinforcement learning algorithm.

In the infinite-horizon setting, the smooth Bellman optimality equations take the form:

$$v_\gamma^*(s) = \frac{1}{\beta} \log \sum_{a \in \mathcal{A}_s} \exp \left(\beta \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v_\gamma^*(j) \right) \right) \quad (397)$$

Adopting an operator-theoretic perspective, we can define a nonlinear operator L_β such that the smooth value function of an MDP is then the solution to the following fixed-point equation:

$$(L_\beta \mathbf{v})(s) = \frac{1}{\beta} \log \sum_{a \in \mathcal{A}_s} \exp \left(\beta \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right) \right) \quad (398)$$

As $\beta \rightarrow \infty$, L_β converges to the standard Bellman operator L . Furthermore, it can be shown that the smooth Bellman operator is a contraction mapping in the supremum norm, and therefore has a unique fixed point. However, as opposed to the usual “hard” setting, the fixed point of L_β is associated with the value function of an optimal stochastic policy defined by the softmax distribution:

$$\pi(a|s) = \frac{\exp \left(\beta \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v_\gamma^*(j) \right) \right)}{\sum_{a' \in \mathcal{A}_s} \exp \left(\beta \left(r(s, a') + \gamma \sum_{j \in \mathcal{S}} p(j|s, a') v_\gamma^*(j) \right) \right)} \quad (399)$$

Despite the confusing terminology, the above “softmax” policy is simply the smooth counterpart to the argmax operator in the original optimality equation: it acts as a soft-argmax.

This formulation is interesting for several reasons. First, smoothness is a desirable property from an optimization standpoint. Unlike γ , we view β as a hyperparameter of our algorithm, which we can control to achieve the desired level of accuracy.

Second, while presented from an intuitive standpoint where we replace the max by the log-sum-exp (a smooth maximum) and the argmax by the softmax (a smooth argmax), this formulation can also be obtained from various other perspectives, offering theoretical tools and solution methods. For example, Rust [1987] derived this algorithm by considering a setting in which the rewards are stochastic and perturbed by a Gumbel noise variable. When considering the corresponding augmented state space and integrating the noise, we obtain smooth equations. This interpretation is leveraged by Rust for modeling purposes.

Smooth Value Iteration Algorithm The smooth value iteration algorithm replaces the max operator in standard value iteration with the logsumexp operator. Here’s the algorithm structure:

Differences from standard value iteration:

- Line 8 uses $\frac{1}{\beta} \log \sum_a \exp(\beta \cdot q(s, a))$ instead of $\max_a q(s, a)$
- Line 15 extracts a stochastic policy using softmax instead of a deterministic argmax policy
- As $\beta \rightarrow \infty$, the algorithm converges to standard value iteration
- Lower β values produce more stochastic policies with higher entropy

There is also a way to obtain this equation by starting from the energy-based formulation often used in supervised learning, in which we convert an unnormalized probability distribution into a distribution using the softmax transformation. This is essentially what Ziebart et al. [2008] did in their paper. Furthermore, this perspective bridges with the literature on probabilistic graphical models, in which we can now cast the problem of finding an optimal smooth policy into one of maximum likelihood estimation (an inference problem). This is the idea of control as inference, which also admits the converse - that of inference as control - used nowadays for deriving fast samples and amortized inference techniques using reinforcement learning Levine et al. [2018].

Finally, it’s worth noting that we can also derive this form by considering an entropy-regularized formulation in which we penalize for the entropy of our policy in the reward function term. This formulation admits a solution that coincides with the smooth Bellman equations Haarnoja et al. [2017].

6.1.1 Alternative Soft Maximum: Gaussian Uncertainty-Weighted Aggregation

The logsumexp operator provides one way to soften the hard maximum, but alternative approaches exist. When Q-value estimates have heterogeneous uncertainty (some actions estimated more precisely than others), we can weight actions by the probability they are optimal under a Gaussian uncertainty model. D’Eramo et al. [2016] proposed computing weights as:

$$w_{a'} = \int_{-\infty}^{+\infty} \phi\left(\frac{x - \hat{\mu}_{a'}}{\hat{\sigma}_{a'}/\sqrt{n}}\right) \prod_{b \neq a'} \Phi\left(\frac{x - \hat{\mu}_b}{\hat{\sigma}_b/\sqrt{n}}\right) dx \quad (400)$$

where $\hat{\mu}_{a'}$ and $\hat{\sigma}_{a'}$ are the sample mean and standard deviation of Q-value estimates, n is the sample size, and ϕ , Φ are the standard normal PDF and CDF. The soft Bellman target becomes $v(s') = \sum_{a'} w_{a'} q(s', a')$, a probability-weighted expectation rather than a hard maximum.

This differs from logsumexp in that it adapts to state-action-specific uncertainty (actions with tighter confidence intervals receive more weight), whereas logsumexp applies uniform smoothing via temperature β . The Gaussian-weighted approach requires maintaining variance estimates and computing integrals, making it more expensive than logsumexp. However, it provides a principled way to reduce overestimation bias in Q-learning while avoiding the pessimism of double Q-learning. We return to this estimator in the [simulation-based methods chapter](#) when discussing overestimation bias mitigation strategies.

6.1.2 Gumbel Noise on the Rewards

We can obtain the smooth Bellman equation by considering a setting in which we have Gumbel noise added to the reward function. This derivation provides both theoretical insight and connects to practical modeling scenarios where rewards have random perturbations.

Step 1: Define the Augmented MDP with Gumbel Noise At each time period and state s , we draw an **action-indexed shock vector**:

$$\epsilon_t(s) = (\epsilon_t(s, a))_{a \in \mathcal{A}_s}, \quad \text{where } \epsilon_t(s, a) \text{ i.i.d. } \sim \text{Gumbel}(\mu_\epsilon, 1/\beta) \quad (401)$$

These shocks are independent across time periods, states, and actions, and are independent of the MDP transition dynamics $p(\cdot|s, a)$.

The **Gumbel distribution** with location parameter μ and scale parameter $1/\beta$ has probability density function:

$$f(x; \mu, \beta) = \beta \exp(-\beta(x - \mu) - \exp(-\beta(x - \mu))) \quad (402)$$

To generate a Gumbel-distributed random variable, we can use inverse transform sampling: $X = \mu - \frac{1}{\beta} \ln(-\ln(U))$ where U is uniform on $(0, 1)$.

Zero-Mean Shocks

To ensure the shocks have zero mean, we set $\mu_\epsilon = -\gamma_E/\beta$ where $\gamma_E \approx 0.5772$ is the Euler-Mascheroni constant. This choice eliminates an additive constant that would otherwise appear in the smooth Bellman equation. For simplicity, we will adopt this convention throughout.

We now define an **augmented MDP** with:

- **Augmented state:** $\tilde{s} = (s, \epsilon)$ where $s \in \mathcal{S}$ and $\epsilon \in R^{|\mathcal{A}_s|}$
- **Augmented reward:** $\tilde{r}(\tilde{s}, a) = r(s, a) + \epsilon(a)$
- **Augmented transition:** $\tilde{p}(\tilde{s}' | \tilde{s}, a) = p(s' | s, a) \cdot p(\epsilon')$

The transition factorizes because the next shock vector ϵ' is drawn independently of the current state and action (conditional independence).

The Augmented State Space is Infinite-Dimensional

Even if the original state space $\mathcal{S}$ and action space $\mathcal{A}$ are finite, the augmented state space $\tilde{\mathcal{S}} = \mathcal{S} \times R^{|\mathcal{A}|}$ is **uncountably infinite** because each shock vector ϵ is a continuous random variable. Therefore:

- We cannot enumerate the augmented states
- Tabular dynamic programming methods do not apply directly
- The augmented value function $\tilde{v}(s, \epsilon)$ maps a continuous space to R

This motivates why we immediately marginalize over the shocks to obtain a finite-dimensional representation.

Step 2: The Hard Bellman Equation on the Augmented State Space

The Bellman optimality equation for the augmented MDP is:

$$\tilde{v}(s, \epsilon) = \max_{a \in \mathcal{A}_s} \{r(s, a) + \epsilon(a) + \gamma E_{s', \epsilon'} [\tilde{v}(s', \epsilon') | s, a]\} \quad (403)$$

Here the expectation is over the **next augmented state** (s', ϵ') , which includes both the next state $s' \sim p(\cdot | s, a)$ and the next shock vector $\epsilon' \sim p(\cdot)$.

This is a perfectly well-defined Bellman equation, and an optimal stationary policy exists:

$$\pi(s, \epsilon) \in \operatorname{argmax}_{a \in \mathcal{A}_s} \{r(s, a) + \epsilon(a) + \gamma E_{s', \epsilon'} [\tilde{v}(s', \epsilon') | s, a]\} \quad (404)$$

However, this equation is **computationally intractable** because:

- The state space is continuous and infinite-dimensional
- The shocks are fresh each period
- We would need to solve for $\tilde{v}$ over an uncountable domain

We never solve this equation directly. Instead, we use it as a mathematical device to derive the smooth Bellman equation.

Step 3: Define the Ex-Ante (Inclusive) Value Function The idea here is to consider the **expected value before observing the current shocks**. We define what some authors in econometrics call the **inclusive value** or **ex-ante value**:

$$v(s) := E_{\epsilon}[\tilde{v}(s, \epsilon)] \quad (405)$$

This is the value of being in state s **before** we observe the current-period shock vector ϵ .

Two Different Value Functions

It is crucial to distinguish:

- $\tilde{v}(s, \epsilon)$: the value **after** observing shocks (conditional on ϵ), defined on the augmented state space
- $v(s)$: the value **before** observing shocks (marginalizing over ϵ), defined on the original state space

The function $v(s)$ is what we actually compute and care about. The augmented value $\tilde{v}$ exists only as a proof device.

Step 4: Separate the Deterministic and Random Components Now we take the expectation of the augmented Bellman equation with respect to the **current shocks only** (everything that does not depend on the current ϵ can be pulled out).

First, note that by the law of iterated expectations and independence of shocks across time:

$$E_{\epsilon'}[\tilde{v}(s', \epsilon')] = v(s') \quad (406)$$

This follows from our definition of v and the fact that the next shock is independent of everything else.

Now define the **deterministic part** of the right-hand side:

$$x_a(s) := r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a)v(j) \quad (407)$$

This is the expected return from taking action a in state s **without the shock**. Using this notation, the augmented Bellman equation becomes:

$$\tilde{v}(s, \epsilon) = \max_{a \in \mathcal{A}_s} \{x_a(s) + \epsilon(a)\} \quad (408)$$

Taking the expectation over ϵ on both sides:

$$v(s) = E_\epsilon \left[\max_{a \in \mathcal{A}_s} \{x_a(s) + \epsilon(a)\} \right] \quad (409)$$

Expectation of a Max, Not Max of an Expectation

Notice carefully: we have $E[\max(\cdot)]$, **not** $\max E[\cdot]$. We are **not** swapping max and expectation.

The expression $E_\epsilon[\max_a \{x_a + \epsilon(a)\}]$ is the expected value of the maximum of Gumbel-perturbed utilities. The Gumbel random utility identity evaluates this quantity in closed form.

Step 5: Apply the Gumbel Random Utility Identity We now invoke a result from extreme value theory:

Lemma 6.1 (Gumbel Random Utility Identity). *Let $\epsilon_1, \dots, \epsilon_m$ be i.i.d. $\text{Gumbel}(\mu_\epsilon, 1/\beta)$ random variables. For any deterministic values $x_1, \dots, x_m \in R$:*

$$\max_{i=1, \dots, m} \{x_i + \epsilon_i\} \stackrel{d}{=} \frac{1}{\beta} \log \sum_{i=1}^m \exp(\beta x_i) + \zeta \quad (410)$$

where $\zeta \sim \text{Gumbel}(\mu_\epsilon, 1/\beta)$ (same distribution as the original shocks).

Taking expectations:

$$E \left[\max_{i=1, \dots, m} \{x_i + \epsilon_i\} \right] = \frac{1}{\beta} \log \sum_{i=1}^m \exp(\beta x_i) + \mu_\epsilon + \frac{\gamma_E}{\beta} \quad (411)$$

where $\gamma_E \approx 0.5772$ is the Euler-Mascheroni constant.

With mean-zero shocks ($\mu_\epsilon = -\gamma_E/\beta$), the constant term vanishes:

$$E \left[\max_{i=1, \dots, m} \{x_i + \epsilon_i\} \right] = \frac{1}{\beta} \log \sum_{i=1}^m \exp(\beta x_i) \quad (412)$$

Applying this identity to our problem (with mean-zero shocks):

$$v(s) = E_\epsilon \left[\max_{a \in \mathcal{A}_s} \{x_a(s) + \epsilon(a)\} \right] = \frac{1}{\beta} \log \sum_{a \in \mathcal{A}_s} \exp(\beta x_a(s)) \quad (413)$$

Substituting the definition of $x_a(s)$:

$$v(s) = \frac{1}{\beta} \log \sum_{a \in \mathcal{A}_s} \exp \left(\beta \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right) \right) \quad (414)$$

We have arrived at the **smooth Bellman equation**.

Step 6: Summary of the Derivation To recap the logical flow:

1. We constructed an augmented MDP with state (s, ϵ) where shocks perturb rewards
2. We wrote the standard Bellman equation for this augmented MDP (hard max, but over an infinite-dimensional state space)
3. We defined the ex-ante value $v(s) = E_\epsilon[\tilde{v}(s, \epsilon)]$ to eliminate the continuous shock component
4. We separated deterministic and random terms: $\tilde{v}(s, \epsilon) = \max_a \{x_a(s) + \epsilon(a)\}$
5. We applied the Gumbel identity to evaluate $E_\epsilon[\max_a \{\dots\}]$ in closed form as a log-sum-exp

The augmented MDP with shocks exists **only as a mathematical device**. We never approximate $\tilde{v}$, never discretize ϵ , and never enumerate the augmented state space. The only computational object we work with is $v(s)$ on the original (finite) state space, which satisfies the smooth Bellman equation.

Deriving the Optimal Smooth Policy Now that we have derived the smooth value function, we can also obtain the corresponding optimal policy. The question is: **what policy should we follow in the original MDP (without explicitly conditioning on shocks)?**

In the augmented MDP, the optimal policy is deterministic but depends on the shock realization:

$$\pi(s, \epsilon) \in \operatorname{argmax}_{a \in \mathcal{A}_s} \left\{ r(s, a) + \epsilon(a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right\} \quad (415)$$

However, we want a policy for the **original** state space s (not the augmented state). We obtain this by **marginalizing over the current shocks**, essentially asking: “what is the probability that action a is optimal when we average over all possible shock realizations?”

Define an indicator function:

$$I_a(\epsilon) = \begin{cases} 1 & \text{if } a \in \operatorname{argmax}_{a' \in \mathcal{A}_s} \{x_{a'}(s) + \epsilon(a')\} \\ 0 & \text{otherwise} \end{cases} \quad (416)$$

where $x_a(s) = r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a)v(j)$ as before.

The ex-ante probability that action a is optimal at state s is:

$$\pi(a|s) = E_{\epsilon}[I_a(\epsilon)] = P_{\epsilon}(a \in \operatorname{argmax}_{a'} \{x_{a'}(s) + \epsilon(a')\}) \quad (417)$$

This is the probability that action a achieves the maximum when utilities are perturbed by Gumbel noise.

Lemma 6.2 (Gumbel-Max Probability (Softmax)). *Let $\epsilon_1, \dots, \epsilon_m$ be i.i.d. Gumbel($\mu_{\epsilon}, 1/\beta$) random variables. For any deterministic values $x_1, \dots, x_m \in \mathbb{R}$, the probability that index i achieves the maximum is:*

$$P(i \in \operatorname{argmax}_j \{x_j + \epsilon_j\}) = \frac{\exp(\beta x_i)}{\sum_{j=1}^m \exp(\beta x_j)} \quad (418)$$

This holds regardless of the location parameter μ_{ϵ} .

Applying this result to our problem:

$$\pi(a|s) = \frac{\exp(\beta x_a(s))}{\sum_{a' \in \mathcal{A}_s} \exp(\beta x_{a'}(s))} = \frac{\exp\left(\beta \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a)v(j)\right)\right)}{\sum_{a' \in \mathcal{A}_s} \exp\left(\beta \left(r(s, a') + \gamma \sum_{j \in \mathcal{S}} p(j|s, a')v(j)\right)\right)} \quad (419)$$

This is the **softmax policy** or **Gibbs/Boltzmann policy** with inverse temperature β .

Properties:

- As $\beta \rightarrow \infty$: the policy becomes deterministic, concentrating on the action(s) with highest $x_a(s)$ (recovers standard greedy policy)
- As $\beta \rightarrow 0$: the policy becomes uniform over all actions (maximum entropy)
- For finite $\beta > 0$: the policy is stochastic, with probability mass proportional to exponentiated Q-values

This completes the derivation: the smooth Bellman equation yields a value function $v(s)$, and the corresponding optimal policy is the softmax over Q-values.

6.1.3 Regularized Markov Decision Processes

Regularized MDPs Geist et al. [2019] provide another perspective on how the smooth Bellman equations come to be. This framework offers a more general approach in which we seek to find optimal policies under the infinite horizon criterion while also accounting for a regularizer that influences the kind of policies we try to obtain.

Let's set up some necessary notation. First, recall that the policy evaluation operator for a stationary policy with decision rule π is defined as:

$$\mathbf{L}_{\pi} \mathbf{v} = \mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v} \quad (420)$$

where $\mathbf{r}_\pi$ is the expected reward vector under policy π , γ is the discount factor, and $\mathbf{P}_\pi$ is the state transition probability matrix under π . A complementary object to the value function is the q-function (or Q-factor) representation:

$$\begin{aligned} q_\gamma^\pi(s, a) &= r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v_\gamma^\pi(j) \\ v_\gamma^\pi(s) &= \sum_{a \in \mathcal{A}_s} \pi(a|s) q_\gamma^\pi(s, a) \end{aligned}$$

The policy evaluation operator can then be written in terms of the q-function as:

$$[\mathbf{L}_\pi v](s) = \langle \pi(\cdot|s), q(s, \cdot) \rangle \quad (421)$$

Legendre-Fenchel Transform The workhorse behind the theory of regularized MDPs is the Legendre-Fenchel transform, also known as the convex conjugate. For a strongly convex function $\Omega : \Delta_{\mathcal{A}} \rightarrow R$, its Legendre-Fenchel transform $\Omega^* : R^{\mathcal{A}} \rightarrow R$ is defined as:

$$\Omega^*(q(s, \cdot)) = \max_{\pi(\cdot|s) \in \Delta_{\mathcal{A}}} \langle \pi(\cdot|s), q(s, \cdot) \rangle - \Omega(\pi(\cdot|s)) \quad (422)$$

An important property of this transform is that it has a unique maximizing argument, given by the gradient of Ω^* . This gradient is Lipschitz and satisfies:

$$\nabla \Omega^*(q(s, \cdot)) = \arg \max_{\pi} \langle \pi(\cdot|s), q(s, \cdot) \rangle - \Omega(\pi(\cdot|s)) \quad (423)$$

An important example of a regularizer is the negative entropy, which gives rise to the smooth Bellman equations as we are about to see.

6.1.4 Regularized Bellman Operators

With these concepts in place, we can now define the regularized Bellman operators:

1. Regularized Policy Evaluation Operator ($\mathbf{L}_{\pi, \Omega}$):

$$[\mathbf{L}_{\pi, \Omega} v](s) = \langle q(s, \cdot), \pi(\cdot|s) \rangle - \Omega(\pi(\cdot|s)) \quad (424)$$

2. Regularized Bellman Optimality Operator ($\mathbf{L}_\Omega$):

$$[\mathbf{L}_\Omega v](s) = [\max_{\pi} \mathbf{L}_{\pi, \Omega} v](s) = \Omega^*(q(s, \cdot)) \quad (425)$$

It can be shown that the addition of a regularizer in these regularized operators still preserves the contraction properties, and therefore the existence of a

solution to the optimality equations and the convergence of successive approximation.

The regularized value function of a stationary policy with decision rule π , denoted by $v_{\pi,\Omega}$, is the unique fixed point of the operator equation:

$$\text{find } v \text{ such that } v = L_{\pi,\Omega}v \quad (426)$$

Under the usual assumptions on the discount factor and the boundedness of the reward, the value of a policy can also be found in closed form by solving for $\mathbf{v}$ in the linear system of equations:

$$(\mathbf{I} - \gamma \mathbf{P}_\pi) \mathbf{v} = \mathbf{r}_\pi - \mathbf{\Omega}_\pi \quad (427)$$

where $[\mathbf{\Omega}_\pi](s) = \Omega(\pi(\cdot|s))$ is the vector of regularization terms at each state. The associated state-action value function $q_{\pi,\Omega}$ is given by:

$$\begin{aligned} q_{\pi,\Omega}(s, a) &= r(s, a) + \sum_{j \in \mathcal{S}} \gamma p(j|s, a) v_{\pi,\Omega}(j) \\ v_{\pi,\Omega}(s) &= \sum_{a \in \mathcal{A}_s} \pi(a|s) q_{\pi,\Omega}(s, a) - \Omega(\pi(\cdot|s)) \end{aligned}$$

The regularized optimal value function v_Ω^* is then the unique fixed point of L_Ω in the fixed point equation:

$$\text{find } v \text{ such that } v = L_\Omega v \quad (428)$$

The associated state-action value function q_Ω^* is given by:

$$\begin{aligned} q_\Omega^*(s, a) &= r(s, a) + \sum_{j \in \mathcal{S}} \gamma p(j|s, a) v_\Omega^*(j) \\ v_\Omega^*(s) &= \Omega^*(q_\Omega^*(s, \cdot)) \end{aligned}$$

An important result in the theory of regularized MDPs is that there exists a unique optimal regularized policy. Specifically, if π_Ω^* is a conserving decision rule (i.e., $\pi_\Omega^* = \arg \max_\pi L_{\pi,\Omega} v_\Omega^*$), then the randomized stationary policy $\boldsymbol{\pi} = \text{const}(\pi_\Omega^*)$ is the unique optimal regularized policy.

In practice, once we have found v_Ω^* , we can derive the optimal decision rule by taking the gradient of the convex conjugate evaluated at the optimal action-value function:

$$\pi^*(\cdot|s) = \nabla \Omega^*(q_\Omega^*(s, \cdot)) \quad (429)$$

Recovering the Smooth Bellman Equations Under this framework, we can recover the smooth Bellman equations by choosing Ω to be the negative entropy, and obtain the softmax policy as the gradient of the convex conjugate. Let's show this explicitly:

1. Using the negative entropy regularizer:

$$\Omega(d(\cdot|s)) = \sum_{a \in \mathcal{A}_s} d(a|s) \ln d(a|s) \quad (430)$$

2. The convex conjugate:

$$\Omega^*(q(s, \cdot)) = \ln \sum_{a \in \mathcal{A}_s} \exp q(s, a) \quad (431)$$

3. Now, let's write out the regularized Bellman optimality equation:

$$v_\Omega^*(s) = \Omega^*(q_\Omega^*(s, \cdot)) \quad (432)$$

4. Substituting the expressions for Ω^* and q_Ω^* :

$$v_\Omega^*(s) = \ln \sum_{a \in \mathcal{A}_s} \exp \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v_\Omega^*(j) \right) \quad (433)$$

This matches the form of the smooth Bellman equation we derived earlier, with the log-sum-exp operation replacing the max operation of the standard Bellman equation.

Furthermore, the optimal policy is given by the gradient of Ω^* :

$$d^*(a|s) = \nabla \Omega^*(q_\Omega^*(s, \cdot)) = \frac{\exp(q_\Omega^*(s, a))}{\sum_{a' \in \mathcal{A}_s} \exp(q_\Omega^*(s, a'))} \quad (434)$$

This is the familiar softmax policy we encountered in the smooth MDP setting.

Smooth Policy Iteration Algorithm Now that we've seen how the regularized MDP framework leads to smooth Bellman equations, we present smooth policy iteration. Unlike value iteration which directly iterates the Bellman operator, policy iteration alternates between policy evaluation and policy improvement steps.

Key properties of smooth policy iteration:

1. **Entropy-regularized evaluation:** The policy evaluation step (line 12 of Algorithm Algorithm ??) accounts for the entropy bonus $\alpha H(\pi(\cdot|s))$ where $\alpha = 1/\beta$

2. **Stochastic policy improvement:** The policy improvement step (lines 12-14 of Algorithm Algorithm ??) uses softmax instead of deterministic argmax, producing a stochastic policy
3. **Temperature parameter:**
 - Higher $\beta \rightarrow$ policies close to deterministic (lower entropy) Lower $\beta \rightarrow$ more stochastic policies (higher entropy)
 - As $\beta \rightarrow \infty \rightarrow$ recovers standard policy iteration
4. **Convergence:** Like standard policy iteration, this algorithm converges to the unique optimal regularized value function and policy

Equivalence Between Smooth Bellman Equations and Entropy-Regularized MDPs

We have now seen two distinct ways to arrive at smooth Bellman equations. Earlier in this chapter, we introduced the logsumexp operator as a smooth approximation to the max operator, motivated by analytical tractability and the desire for differentiability. Just now, we derived the same equations through the lens of regularized MDPs, where we explicitly penalize the entropy of policies. These two perspectives are mathematically equivalent: solving the smooth Bellman equation with inverse temperature parameter β yields exactly the same optimal value function and optimal policy as solving the entropy-regularized MDP with regularization strength $\alpha = 1/\beta$. The two formulations are not merely similar. They describe identical optimization problems.

To see this equivalence clearly, consider the standard MDP problem with rewards $r(s, a)$ and transition probabilities $p(j|s, a)$. The regularized MDP framework tells us to solve:

$$\max_{\pi} E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] + \alpha E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t H(\pi(\cdot|s_t)) \right], \quad (435)$$

where $H(\pi(\cdot|s)) = -\sum_a \pi(a|s) \ln \pi(a|s)$ is the entropy of the policy at state s , and $\alpha > 0$ is the entropy regularization strength.

We can rewrite this objective by absorbing the entropy term into a modified reward function. Define the entropy-augmented reward:

$$\tilde{r}(s, a, \pi) = r(s, a) + \alpha H(\pi(\cdot|s)). \quad (436)$$

However, this formulation makes the reward depend on the entire policy at each state, which is awkward. We can reformulate this more cleanly by expanding the entropy term. Recall that the entropy is:

$$H(\pi(\cdot|s)) = -\sum_a \pi(a|s) \ln \pi(a|s). \quad (437)$$

When we take the expectation over actions drawn from π , we have:

$$E_{a \sim \pi(\cdot|s)}[H(\pi(\cdot|s))] = \sum_a \pi(a|s) \left[- \sum_{a'} \pi(a'|s) \ln \pi(a'|s) \right] = - \sum_{a'} \pi(a'|s) \ln \pi(a'|s), \quad (438)$$

since the entropy doesn't depend on which action is actually sampled. But we can also write this as:

$$H(\pi(\cdot|s)) = - \sum_a \pi(a|s) \ln \pi(a|s) = E_{a \sim \pi(\cdot|s)}[-\ln \pi(a|s)]. \quad (439)$$

This shows that adding $\alpha H(\pi(\cdot|s))$ to the expected reward at state s is equivalent to adding $-\alpha \ln \pi(a|s)$ to the reward of taking action a at state s . More formally:

$$\begin{aligned} & E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] + \alpha E_\pi \left[\sum_{t=0}^{\infty} \gamma^t H(\pi(\cdot|s_t)) \right] \\ &= E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] + \alpha E_\pi \left[\sum_{t=0}^{\infty} \gamma^t E_{a_t \sim \pi(\cdot|s_t)}[-\ln \pi(a_t|s_t)] \right] \\ &= E_\pi \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \ln \pi(a_t|s_t)) \right]. \end{aligned}$$

The entropy bonus at each state, when averaged over the policy, becomes a per-action penalty proportional to the negative log probability of the action taken. This reformulation is more useful because the modified reward now depends only on the state, the action taken, and the probability assigned to that specific action by the policy, not on the entire distribution over actions.

This expression shows that entropy regularization is equivalent to adding a state-action dependent penalty term $-\alpha \ln \pi(a|s)$ to the reward. Intuitively, this terms amounts to paying a cost for low-entropy (deterministic) policies.

Now, when we write down the Bellman equation for this entropy-regularized problem, at each state s we need to find the decision rule $d(\cdot|s) \in \Delta(\mathcal{A}_s)$ (a probability distribution over actions) that maximizes:

$$v(s) = \max_{d(\cdot|s) \in \Delta(\mathcal{A}_s)} \sum_a d(a|s) \left[r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) - \alpha \ln d(a|s) \right]. \quad (440)$$

Here $\Delta(\mathcal{A}_s) = \{d(\cdot|s) : d(a|s) \geq 0, \sum_a d(a|s) = 1\}$ denotes the probability simplex over actions available at state s . The optimization is over randomized decision rules at each state, constrained to be valid probability distributions.

This is a convex optimization problem with a linear constraint. We form the Lagrangian:

$$\mathcal{L}(d, \lambda) = \sum_a d(a|s) \left[r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) - \alpha \ln d(a|s) \right] - \lambda \left(\sum_a d(a|s) - 1 \right), \quad (441)$$

where λ is the Lagrange multiplier enforcing the normalization constraint. Taking the derivative with respect to $d(a|s)$ and setting it to zero:

$$r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) - \alpha(1 + \ln d^*(a|s)) - \lambda = 0. \quad (442)$$

Solving for $d^*(a|s)$:

$$d^*(a|s) = \exp \left(\frac{1}{\alpha} \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) - \lambda \right) \right). \quad (443)$$

Using the normalization constraint $\sum_a d^*(a|s) = 1$ to solve for λ :

$$\sum_a \exp \left(\frac{1}{\alpha} \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right) \right) = \exp \left(\frac{\lambda}{\alpha} \right). \quad (444)$$

Therefore:

$$\lambda = \alpha \ln \sum_a \exp \left(\frac{1}{\alpha} \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right) \right). \quad (445)$$

Substituting this back into the Bellman equation and simplifying:

$$v(s) = \alpha \ln \sum_a \exp \left(\frac{1}{\alpha} \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right) \right). \quad (446)$$

Setting $\beta = 1/\alpha$ (the inverse temperature), this becomes:

$$v(s) = \frac{1}{\beta} \ln \sum_a \exp \left(\beta \left(r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) v(j) \right) \right). \quad (447)$$

We recover the smooth Bellman equation we derived earlier using the logsumexp operator. The inverse temperature parameter β controls how closely the logsumexp approximates the max: as $\beta \rightarrow \infty$, we recover the standard Bellman equation, while for finite β , we have a smooth approximation that corresponds to optimizing with entropy regularization strength $\alpha = 1/\beta$.

The optimal policy is:

$$\pi^*(a|s) = \frac{\exp(\beta q^*(s, a))}{\sum_{a'} \exp(\beta q^*(s, a'))} = \text{softmax}_\beta(q^*(s, \cdot))(a), \quad (448)$$

which is exactly the softmax policy parametrized by inverse temperature.

The derivation establishes the complete equivalence: the value function v^* that solves the smooth Bellman equation is identical to the optimal value function v_Ω^* of the entropy-regularized MDP (with Ω being negative entropy and $\alpha = 1/\beta$), and the softmax policy that is greedy with respect to this value function achieves the maximum of the entropy-regularized objective. Both approaches yield the same numerical solution: the same values at every state and the same policy prescriptions. The only difference is how we conceptualize the problem: as smoothing the Bellman operator for computational tractability, or as explicitly trading off reward maximization against policy entropy.

This equivalence has important implications. When we use smooth Bellman equations with a logsumexp operator, we are implicitly solving an entropy-regularized MDP. Conversely, when we explicitly add entropy regularization to an MDP objective, we arrive at smooth Bellman equations as the natural description of optimality. This dual perspective will prove valuable in understanding various algorithms and theoretical results. For instance, in soft actor-critic methods and other maximum entropy reinforcement learning algorithms, the connection between smooth operators and entropy regularization provides both computational benefits (differentiability) and conceptual clarity (why we want stochastic policies).

Entropy-Regularized Dynamic Programming Algorithms While the smooth Bellman equations (using logsumexp) and entropy-regularized formulations are mathematically equivalent, it is instructive to present the algorithms explicitly in the entropy-regularized form, where the entropy bonus appears directly in the update equations.

Features:

- Line 11 updates the policy using the softmax of Q-values, with temperature α
- Line 14 explicitly computes the entropy-regularized value: expected Q-value plus entropy bonus
- The algorithm maintains and updates a stochastic policy throughout
- As $\alpha \rightarrow 0$ (or equivalently $\beta \rightarrow \infty$), this recovers standard value iteration

Features:

- **Policy Evaluation** (lines 3-15): Computes the value of the current policy including entropy bonus
 - Option 1: Iterative method (successive approximation)
 - Option 2: Direct solution via linear system
- **Policy Improvement** (lines 16-29): Updates policy to softmax over Q-values

- Line 14 shows the vector form: the linear system includes the entropy vector $\mathbf{H}_\pi$
- The algorithm alternates between evaluating the current stochastic policy and improving it
- Converges to the unique optimal entropy-regularized policy

6.1.5 Self-checks

6.2 Weighted Residual Methods for Functional Equations

The Bellman optimality equation $\mathbf{L}v = v$, whose contraction property underlies the convergence of value iteration, is a functional equation: an equation where the unknown is an entire function rather than a finite-dimensional vector. When the state space is continuous or very large, we cannot represent the value function exactly on a computer. We must instead work with finite-dimensional approximations. This motivates weighted residual methods (also called minimum residual methods), a general framework for transforming infinite-dimensional problems into tractable finite-dimensional ones [Chakraverty et al. \[2019\]](#), [Atkinson and Potra \[1987\]](#).

Learning Goals

After completing this chapter, you should be able to:

- Explain why functional equations require finite-dimensional approximation and how weighted residual methods provide a systematic framework
- Distinguish between Galerkin, collocation, and least-squares methods by their choice of test functions
- Apply function iteration and Newton’s method to solve projected Bellman equations
- State conditions under which projected value iteration converges (monotonicity for sup-norm, on-policy weighting for weighted L^2)
- Derive the LSTD equations as Galerkin projection with stationary distribution weighting

Prerequisites: Bellman operator and contraction property, basic numerical quadrature, linear algebra (matrix inverses, orthogonal projection).

6.2.1 A Motivating Example: Optimal Stopping with Continuous States

Before developing the general theory, consider a concrete example that illustrates the core challenge. An agent observes a state $s \in [0, 1]$ and must decide whether to **stop** (receive reward s and end the episode) or **continue** (receive nothing, and the state redraws uniformly on $[0, 1]$). With discount factor $\gamma = 0.9$, the Bellman optimality equation is:

$$v^*(s) = \max \left\{ s, \gamma \int_0^1 v^*(s') ds' \right\}. \quad (449)$$

The first term is the immediate payoff from stopping; the second is the discounted expected continuation value. Since the continuation value $\bar{v} =$

$\int_0^1 v^*(s') ds'$ is a constant (it doesn't depend on the current state s), the optimal policy has a **threshold structure**: stop if $s \geq s^*$ for some threshold s^* , continue otherwise.

At the threshold, the agent is indifferent: $s^* = \gamma \bar{v}$. Computing $\bar{v}$ by integrating v^* :

$$\bar{v} = \int_0^{s^*} \gamma \bar{v} ds' + \int_{s^*}^1 s' ds' = s^* \gamma \bar{v} + \frac{1 - (s^*)^2}{2}. \quad (450)$$

Substituting $s^* = \gamma \bar{v}$ and solving gives the exact threshold and value.

The exact value function is piecewise linear: constant at $\gamma \bar{v}$ below the threshold, equal to s above it. Now suppose we want to approximate v^* using a **polynomial basis** with n terms:

$$\hat{v}(s; \theta) = \sum_{j=0}^{n-1} \theta_j s^j = \theta_0 + \theta_1 s + \theta_2 s^2 + \dots \quad (451)$$

The **residual** at state s measures how far our approximation is from satisfying the Bellman equation:

$$R(s; \theta) = \max \left\{ s, \gamma \int_0^1 \hat{v}(s'; \theta) ds' \right\} - \hat{v}(s; \theta). \quad (452)$$

For a perfect solution, $R(s; \theta) = 0$ for all $s \in [0, 1]$. But a polynomial cannot exactly represent the kink at s^* . We must choose how to make the residual “small” across the state space.

Collocation picks n points $\{s_1, \dots, s_n\}$ and requires the residual to vanish exactly there:

$$R(s_i; \theta) = 0, \quad i = 1, \dots, n. \quad (453)$$

Galerkin requires the residual to be orthogonal to each basis function:

$$\int_0^1 R(s; \theta) s^j w(s) ds = 0, \quad j = 0, \dots, n-1. \quad (454)$$

This example illustrates the fundamental tension in weighted residual methods: with finite parameters, we cannot satisfy the Bellman equation everywhere. We must choose how to allocate our approximation capacity. The rest of this chapter develops the general theory behind these choices.

6.2.2 Testing Whether a Residual Vanishes

Consider a functional equation $\mathbf{N}(f) = 0$, where $\mathbf{N}$ is an operator and the unknown f is an entire function (in our case, the Bellman optimality equation $\mathbf{L}v = v$, which we can write as $\mathbf{N}(v) \equiv \mathbf{L}v - v = 0$). Suppose we have found a candidate approximate solution $\hat{f}$. To verify it satisfies $\mathbf{N}(\hat{f}) = 0$, we compute the **residual function** $R(s) = \mathbf{N}(\hat{f})(s)$. For a true solution, this residual should be the **zero function**: $R(s) = 0$ for every state s .

How might we test whether a function is zero? One approach: sample many input points $\{s_1, s_2, \dots, s_m\}$, check whether $R(s_i) = 0$ at each, and summarize the results into a single scalar test by computing a weighted sum $\sum_{i=1}^m w_i R(s_i)$ with weights $w_i > 0$. If R is zero everywhere, this sum is zero. If R is nonzero somewhere, we can choose points and weights to make the sum nonzero. For vectors in finite dimensions, the inner product $\langle \mathbf{r}, \mathbf{y} \rangle = \sum_{i=1}^n r_i y_i$ implements exactly this idea: it tests $\mathbf{r}$ by weighting and summing. Indeed, a vector $\mathbf{r} \in R^n$ equals zero if and only if $\langle \mathbf{r}, \mathbf{y} \rangle = 0$ for every vector $\mathbf{y} \in R^n$. To see why, suppose $\mathbf{r} \neq \mathbf{0}$. Choosing $\mathbf{y} = \mathbf{r}$ gives $\langle \mathbf{r}, \mathbf{r} \rangle = \|\mathbf{r}\|^2 > 0$, contradicting the claim that all inner products vanish.

The same principle extends to functions. A function R equals the zero function if and only if its “inner product” with every “test function” p vanishes:

$$R = 0 \quad \text{if and only if} \quad \langle R, p \rangle_w = \int_S R(s)p(s)w(s)ds = 0 \quad \text{for all test functions } p, \quad (455)$$

where $w(s) > 0$ is a weight function that is part of the inner product definition. Why does this work? For the same reason as in finite dimensions: if R is not the zero function, there must be some region where $R(s) \neq 0$. We can then choose a test function p that is nonzero in that same region (for instance, $p(s) = R(s)$ itself), which will produce $\langle R, p \rangle_w = \int R(s)p(s)w(s)ds > 0$, witnessing that R is nonzero. Conversely, if R is the zero function, then $\langle R, p \rangle_w = 0$ for any test function p .

This ability to **distinguish between different functions using inner products** is a fundamental principle from functional analysis. Just as we can test a vector by taking inner products with other vectors, we can test a function by taking inner products with other functions.

Connection to Functional Analysis

The principle that “a function equals zero if and only if it has zero inner product with all test functions” is a consequence of the **Hahn-Banach theorem**, one of the cornerstones of functional analysis. The theorem guarantees that for any nonzero function R in a suitable function space, there exists a continuous linear functional (which can be represented as an inner product with some test function p) that produces a nonzero value when applied to R . This is often phrased as “the dual space separates points.”

While you don’t need to know the Hahn-Banach theorem to use weighted residual methods, it provides the rigorous mathematical foundation ensuring that our inner product tests are theoretically sound. The constructive argument we gave above (choosing $p = R$) works in simple cases with well-behaved functions, but the Hahn-Banach theorem extends this guarantee to much more general settings.

This transforms the pointwise condition “ $R(s) = 0$ for all s ” (infinitely many

conditions, one per state) into an equivalent condition about inner products. We still cannot test against *all* possible test functions, since there are infinitely many of those too. But the inner product perspective suggests a natural computational strategy: choose a finite collection of test functions $\{p_1, \dots, p_n\}$ and use them to construct n conditions that we can actually compute.

From Variational Conditions to Optimization Making a residual “small” is an optimization problem. We want to find θ that minimizes $\|R(\cdot; \theta)\|$ for some norm. Different methods correspond to different choices of norm:

- Minimize the weighted L^2 norm $\|R\|_w^2 = \int R(s)^2 w(s) ds$
- Minimize a discrete norm $\sum_j \omega_j R(s_j)^2$ at selected points
- Minimize $\|R\|$ in a dual norm induced by the approximation space

The first-order conditions for these optimization problems take the form $\langle R, p_j \rangle_w = 0$ for appropriate “test functions” p_j . The variational formulation is useful for analysis, but we are simply minimizing the residual in a chosen norm.

The rest of this chapter develops the computational framework: how to parameterize the unknown function, define the residual, choose a norm, and solve the resulting finite-dimensional problem.

6.2.3 The General Framework

Consider an operator equation of the form

$$\mathbf{N}(f) = 0, \tag{456}$$

where $\mathbf{N} : B_1 \rightarrow B_2$ is a continuous operator between complete normed vector spaces B_1 and B_2 . For the Bellman equation, we have $\mathbf{N}(v) = \mathbf{L}v - v$, so that solving $\mathbf{N}(v) = 0$ is equivalent to finding the fixed point $v = \mathbf{L}v$.

Just as we transcribed infinite-dimensional continuous optimal control problems into finite-dimensional discrete optimal control problems in earlier chapters, we seek a finite-dimensional approximation to this infinite-dimensional functional equation. Recall that for continuous optimal control, we adopted control parameterization: we represented the control trajectory using a finite set of basis functions (piecewise constants, polynomials, splines) and searched over the finite-dimensional coefficient space instead of the infinite-dimensional function space. For integrals in the objective and constraints, we used numerical quadrature to approximate them with finite sums.

We follow the same strategy here. We parameterize the value function using a finite set of basis functions $\{\varphi_1, \dots, \varphi_n\}$, commonly polynomials (Chebyshev, Legendre), though other function classes (splines, radial basis functions, neural networks) are possible, and search for coefficients $\theta = (\theta_1, \dots, \theta_n)$ in R^n . When integrals appear in the Bellman operator or projection conditions, we approximate them using numerical quadrature. The projection method approach consists of several conceptual steps that accomplish this transcription.

Step 1: Choose a Finite-Dimensional Approximation Space We begin by selecting a basis $\Phi = \{\varphi_1, \varphi_2, \dots, \varphi_n\}$ and approximating the unknown function as a linear combination:

$$\hat{f}(x) = \sum_{i=1}^n \theta_i \varphi_i(x). \quad (457)$$

The choice of basis functions φ_i is problem-dependent. Common choices include:

- **Polynomials:** For smooth problems, we might use Chebyshev polynomials or other orthogonal polynomial families
- **Splines:** For problems where we expect the solution to have regions of different smoothness
- **Radial basis functions:** For high-dimensional problems where tensor product methods become intractable

The number of basis functions n determines the flexibility of our approximation. In practice, we start with small n and increase it until the approximation quality is satisfactory. The only unknowns now are the coefficients $\theta = (\theta_1, \dots, \theta_n)$.

While the classical presentation of projection methods focuses on polynomial bases, the framework applies equally well to other function classes. Neural networks, for instance, can be viewed through this lens: a neural network $\hat{f}(x; \theta)$ with parameters θ defines a flexible function class, and many training procedures can be interpreted as projection methods with specific choices of test functions or residual norms. The distinction is that classical methods typically use predetermined basis functions with linear coefficients, while neural networks use adaptive nonlinear features. Throughout this chapter, we focus on the classical setting to develop the core concepts, but the principles extend naturally to modern function approximators.

Step 2: Define the Residual Function Since we are approximating f with $\hat{f}$, the operator $\mathbf{N}$ will generally not vanish exactly. Instead, we obtain a **residual function**:

$$R(x; \theta) = \mathbf{N}(\hat{f}(\cdot; \theta))(x). \quad (458)$$

This residual measures how far our candidate solution is from satisfying the equation at each point x . As we discussed in the introduction, we want to make this residual small—an optimization problem whose formulation depends on how we measure “small.”

Step 3: Minimize the Residual Having chosen our basis and defined the residual, we must find θ that makes the residual small. This is an optimization problem: we minimize $\|R(\cdot; \theta)\|$ for some norm. The choice of norm determines the method:

Method	Norm being mini- mized	Conditions (n equa- tions)
Least squares	$\ R\ _w^2 = \int R(x; \theta)^2 w(x) dx$	$\int R \cdot \frac{\partial R}{\partial \theta_j} w dx = 0, j = 1, \dots, n$
Galerkin	$\ R\ _{V^*}$ (dual norm of approx. space)	$\int R(x; \theta) \varphi_j(x) w(x) dx = 0, j = 1, \dots, n$
Collocation	$\sum_i \omega_i R(x_i; \theta)^2$ (discrete)	$R(x_i; \theta) = 0, i = 1, \dots, n$

The first-order conditions for each optimization problem yield n equations in the n unknowns $\theta_1, \dots, \theta_n$. We now describe each method, starting with the most computationally attractive.

Collocation: Make the Residual Zero at Selected Points The simplest approach is to choose n points $\{x_1, \dots, x_n\}$ and require the residual to vanish exactly at each:

$$R(x_i; \theta) = 0, \quad i = 1, \dots, n. \quad (459)$$

This gives n equations for n unknowns. Collocation is computationally attractive because it avoids integration entirely—we only evaluate R at discrete points. The resulting system is:

$$N(\hat{f}(\cdot; \theta))(x_i) = 0, \quad i = 1, \dots, n. \quad (460)$$

For a linear operator, this is a linear system; for the Bellman equation, it is nonlinear due to the max.

Verify for yourself: with $n = 2$ collocation points and $n = 2$ basis functions, the system $\Phi\theta = t$ is a 2×2 linear system. What must be true about the collocation matrix Φ for this system to have a unique solution?

The choice of collocation points matters. **Orthogonal collocation** (or **spectral collocation**) places points at the zeros of the n -th orthogonal polynomial in a family (Chebyshev, Legendre, etc.). For Chebyshev polynomials $T_0, T_1, \dots, T_{n-1}$, we place collocation points at the zeros of $T_n(x)$. These points are also optimal nodes for Gauss quadrature, so:

- We get the computational simplicity of pointwise evaluation $R(x_i) = 0$

- When we need integrals (inside the Bellman operator), the collocation points double as quadrature nodes with exactness for polynomials up to degree $2n - 1$
- For smooth problems, spectral approximations achieve **exponential convergence**: the error decreases like $O(e^{-cn})$ as we add basis functions, compared to $O(h^{p+1})$ for piecewise polynomials

The Chebyshev interpolation theorem guarantees that forcing $R(x_i; \theta) = 0$ at these carefully chosen points makes $R(x; \theta)$ small everywhere, with well-conditioned systems and near-optimal interpolation error.

Galerkin: Make the Residual Orthogonal to the Approximation Space The Galerkin method requires the residual to be orthogonal to each basis function:

$$\int_S R(x; \theta) \varphi_i(x) w(x) dx = 0, \quad i = 1, \dots, n. \quad (461)$$

To understand why this is optimal, consider the approximation space $\mathcal{V} = \text{span}\{\varphi_1, \dots, \varphi_n\}$ as an n -dimensional subspace. If the residual R is orthogonal to all basis functions, then by linearity, R is orthogonal to every function in $\mathcal{V}$:

$$\langle R, g \rangle_w = \left\langle R, \sum_{i=1}^n c_i \varphi_i \right\rangle_w = \sum_{i=1}^n c_i \langle R, \varphi_i \rangle_w = 0 \quad \text{for all } g \in \mathcal{V}. \quad (462)$$

The residual has “zero overlap” with our approximation space—it is as “invisible” to our basis as possible. This is the defining property of orthogonal projection.

In what sense is Galerkin minimizing a norm? The **dual norm** of R with respect to $\mathcal{V}$ measures R by its largest inner product with functions in $\mathcal{V}$:

$$\|R\|_{\mathcal{V}^*} = \sup_{\substack{g \in \mathcal{V} \\ \|g\|_w = 1}} |\langle R, g \rangle_w|. \quad (463)$$

The Galerkin conditions $\langle R, \varphi_j \rangle_w = 0$ for all j imply $\langle R, g \rangle_w = 0$ for all $g \in \mathcal{V}$, so $\|R\|_{\mathcal{V}^*} = 0$. Galerkin makes the residual “invisible” when measured against the approximation space—it minimizes the dual norm to zero.

A finite-dimensional analogy: to approximate a vector $\mathbf{v} \in \mathbb{R}^3$ using only the xy -plane, the best approximation is $\hat{\mathbf{v}} = (v_1, v_2, 0)$. The error $\mathbf{r} = \mathbf{v} - \hat{\mathbf{v}} = (0, 0, v_3)$ points purely in the z -direction, orthogonal to the plane. The Galerkin condition generalizes this: the residual is orthogonal to the approximation space.

Galerkin requires integration to compute the conditions, making it more expensive per iteration than collocation. However, when using orthogonal polynomial bases with matching weight functions, the integrals simplify and the resulting systems are well-conditioned.

Least Squares: Minimize the L^2 Norm of the Residual The most direct approach is to minimize the weighted L^2 norm of the residual:

$$\min_{\theta} \int_S R(x; \theta)^2 w(x) dx. \quad (464)$$

The first-order conditions are:

$$\int_S R(x; \theta) \frac{\partial R(x; \theta)}{\partial \theta_j} w(x) dx = 0, \quad j = 1, \dots, n. \quad (465)$$

This directly minimizes how far our approximation is from satisfying the equation. For the Bellman equation $R = \mathbf{L}\hat{v} - \hat{v}$, this is **Bellman residual minimization**: we minimize $\|\mathbf{L}\hat{v} - \hat{v}\|_w^2$.

The gradient $\frac{\partial R}{\partial \theta_j}$ involves differentiating the operator $\mathbf{N}$. For the Bellman operator with its max, this requires the Envelope Theorem (discussed in Step 4). The need to differentiate through the operator distinguishes least squares from Galerkin and collocation.

Fitted Q-Iteration: Project, Then Iterate For iterative methods, there is a computationally simpler alternative to minimizing the residual directly. **Fitted Q-Iteration (FQI)** uses a two-step iteration:

1. Apply the Bellman operator to get a target: $f_k = \mathbf{L}\hat{v}_k$
2. Project the target back onto the approximation space: $\hat{v}_{k+1} = \arg \min_{\theta} \|\hat{v}(\cdot; \theta) - f_k\|_w^2$

The projection step solves $\min_{\theta} \|\hat{v} - f_k\|_w^2$, whose first-order conditions are $\langle \hat{v} - f_k, \varphi_j \rangle_w = 0$. This is a standard least-squares fit of the basis to the target values. Combining these steps gives:

$$\hat{v}_{k+1} = \Pi_w \mathbf{L}\hat{v}_k, \quad (466)$$

where Π_w denotes orthogonal projection onto $\text{span}\{\varphi_j\}$ with respect to the weighted inner product.

FQI does **not** minimize the Bellman residual $\|\mathbf{L}\hat{v} - \hat{v}\|^2$ directly. It projects, then iterates. FQI's projection step uses only the gradient of $\hat{v}$ with respect to θ (the “semi-gradient”), while Bellman residual minimization requires differentiating through $\mathbf{L}$ (the “full gradient”). We return to this distinction when discussing temporal difference learning.

Step 4: Solve the Finite-Dimensional Problem The conditions from Step 3 give us a finite-dimensional problem to solve:

- **Collocation:** n equations $R(x_i; \theta) = 0$
- **Galerkin:** n equations $\int R(x; \theta) \varphi_i(x) w(x) dx = 0$

- **Least squares:** minimize $\int R(x; \theta)^2 w(x) dx$

In each case, we have n equations (or first-order conditions) in n unknowns $\theta_1, \dots, \theta_n$. For the Bellman equation, these systems are nonlinear due to the max operator.

Computational Cost and Conditioning The **computational cost per iteration** varies significantly across methods:

- **Collocation:** Cheapest to evaluate since $P_i(\theta) = R(x_i; \theta)$ requires only pointwise evaluation (no integration). The Jacobian is also cheap: $J_{ij} = \frac{\partial R(x_i; \theta)}{\partial \theta_j}$.
- **Galerkin and moments:** More expensive due to integration. Computing $P_i(\theta) = \int R(x; \theta) p_i(x) w(x) dx$ requires numerical quadrature. Each Jacobian entry requires integrating $\frac{\partial R}{\partial \theta_j} p_i w$.
- **Least squares:** Most expensive when done via the objective function, which requires integrating $R^2 w$. However, the first-order conditions reduce it to a system like Galerkin, with test functions $p_i = \partial R / \partial \theta_i$.

For methods requiring integration, the choice of quadrature rule should match the basis. Gaussian quadrature with nodes at orthogonal polynomial zeros is efficient. When combined with collocation at those same points, the quadrature is exact for polynomials up to a certain degree. This coordination between quadrature and collocation makes **orthogonal collocation** effective.

The **conditioning** of the system depends on the choice of test functions. The Jacobian matrix has entries:

$$J_{ij} = \frac{\partial P_i}{\partial \theta_j} = \left\langle \frac{\partial R(\cdot; \theta)}{\partial \theta_j}, p_i \right\rangle_w. \quad (467)$$

When test functions are orthogonal (or nearly so), the Jacobian tends to be well-conditioned. This is why orthogonal polynomial bases are preferred in Galerkin methods: they produce Jacobians with controlled condition numbers. Poorly chosen basis functions or collocation points can lead to nearly singular Jacobians, causing numerical instability. Orthogonal bases and carefully chosen collocation points (like Chebyshev nodes) help maintain good conditioning.

Two Main Solution Approaches We have two fundamentally different ways to solve the projection equations: **function iteration** (exploiting fixed-point structure) and **Newton's method** (exploiting smoothness). The choice depends on whether the original operator equation has contraction properties and how well those properties are preserved by the finite-dimensional approximation.

Method 1: Function Iteration (Successive Approximation) When the operator equation has the form $f = \mathsf{T}f$ where T is a contraction, the most natural approach is to iterate the operator directly:

$$\hat{f}^{(k+1)} = \mathsf{T}\hat{f}^{(k)}. \quad (468)$$

The infinite-dimensional iteration becomes a finite-dimensional iteration in coefficient space once we choose our weighted residual method. Given a current approximation $\hat{f}^{(k)}(x; \theta^{(k)})$, how do we find the coefficients $\theta^{(k+1)}$ for the next iterate $\hat{f}^{(k+1)}$?

Different weighted residual methods answer this differently. For **collocation**, we proceed in two steps:

1. **Evaluate the operator:** At each collocation point x_i , compute what the next iterate should be: $t_i^{(k)} = (\mathsf{T}\hat{f}^{(k)})(x_i)$. These n target values tell us what $\hat{f}^{(k+1)}$ should equal at the collocation points.
2. **Find matching coefficients:** Determine $\theta^{(k+1)}$ so that $\hat{f}^{(k+1)}(x_i; \theta^{(k+1)}) = t_i^{(k)}$ for all i . This is a linear system: $\sum_j \theta_j^{(k+1)} \varphi_j(x_i) = t_i^{(k)}$.

In matrix form: $\Phi\theta^{(k+1)} = t^{(k)}$, where Φ is the collocation matrix with entries $\Phi_{ij} = \varphi_j(x_i)$. Solving this system gives $\theta^{(k+1)} = \Phi^{-1}t^{(k)}$.

For **Galerkin**, the projection condition $\langle \hat{f}^{(k+1)} - \mathsf{T}\hat{f}^{(k+1)}, \varphi_i \rangle_w = 0$ directly gives a system for $\theta^{(k+1)}$. When T is linear in its argument (as in many integral equations), this is a linear system. When T is nonlinear (as in the Bellman equation), we must solve a nonlinear system at each iteration, though each solution still only involves n unknowns rather than an infinite-dimensional function.

When T is a contraction in the infinite-dimensional space with constant $\gamma < 1$, iterating it pulls any starting function toward the unique fixed point. The hope is that the finite-dimensional operator, evaluating T and projecting back onto the span of the basis functions, inherits this contraction property. When it does, function iteration converges globally from any initial guess, with each iteration reducing the error by a factor of roughly γ . This is computationally attractive: we only evaluate the operator and solve a linear system (for collocation) or a relatively simple system (for other methods).

However, the finite-dimensional approximation doesn't always preserve contraction. High-order polynomial bases, in particular, can create oscillations between basis functions that amplify rather than contract errors. Even when contraction is preserved, convergence can be painfully slow when γ is close to 1, the "weak contraction" regime common in economic problems with patient agents ($\gamma \approx 0.95$ or higher). Finally, not all operator equations naturally present themselves as contractions; some require reformulation (like $f = f - \alpha \mathsf{N}(f)$), and finding a good α can be problem-specific.

Method 2: Newton's Method Alternatively, we can treat the projection equations as a rootfinding problem $G(\theta) = 0$ where $G_i(\theta) = P_i(\theta)$ for test

function methods, or solve the first-order conditions for least squares. **Newton's method** uses the update:

$$\theta^{(k+1)} = \theta^{(k)} - J_G(\theta^{(k)})^{-1} G(\theta^{(k)}), \quad (469)$$

where $J_G(\theta)$ is the Jacobian of G at θ .

To apply this update, we must compute the Jacobian entries $J_{ij} = \frac{\partial G_i}{\partial \theta_j}$. For collocation, $G_i(\theta) = \hat{f}(x_i; \theta) - (\mathbb{T}\hat{f}(\cdot; \theta))(x_i)$, so:

$$\frac{\partial G_i}{\partial \theta_j} = \frac{\partial \hat{f}(x_i; \theta)}{\partial \theta_j} - \frac{\partial (\mathbb{T}\hat{f}(\cdot; \theta))(x_i)}{\partial \theta_j}. \quad (470)$$

The first term is straightforward (it's just $\varphi_j(x_i)$ for a linear approximation). The second term requires differentiating the operator $\mathbb{T}$ with respect to the parameters.

When $\mathbb{T}$ involves optimization (as in the Bellman operator $\mathbb{L}v = \max_a \{r(s, a) + \gamma E[v(s')]\}$), computing this derivative appears problematic because the max operator is not differentiable. However, the **Envelope Theorem** resolves this difficulty.

Before reading the box below, try differentiating $v(\theta) = \max_x f(x, \theta)$ using the chain rule. What term involving $\partial x^/\partial \theta$ appears? Why might this term vanish at an optimum?*

The Envelope Theorem

Setup: Consider a smooth objective function $f(\mathbf{x}, \boldsymbol{\theta})$ and define the optimal value:

$$v(\boldsymbol{\theta}) = \max_{\mathbf{x}} f(\mathbf{x}, \boldsymbol{\theta}). \quad (471)$$

Let $\mathbf{x}^*(\boldsymbol{\theta})$ denote the maximizer, satisfying the first-order condition $\nabla_{\mathbf{x}} f(\mathbf{x}^*(\boldsymbol{\theta}), \boldsymbol{\theta}) = \mathbf{0}$.

The Result: To find how the optimal value changes with $\boldsymbol{\theta}$, we can compute:

$$\nabla_{\boldsymbol{\theta}} v(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} f(\mathbf{x}^*(\boldsymbol{\theta}), \boldsymbol{\theta}). \quad (472)$$

That is, differentiate the objective with respect to $\boldsymbol{\theta}$ while treating the maximizer $\mathbf{x}^*$ as constant. We don't need to compute $\frac{\partial \mathbf{x}^*}{\partial \boldsymbol{\theta}}$ because at the optimum, small changes in $\mathbf{x}$ don't affect the value (first-order condition), so the direct effect dominates.

Why it works: By the chain rule, $\nabla_{\boldsymbol{\theta}} v = \nabla_{\boldsymbol{\theta}} f + \underbrace{(\nabla_{\mathbf{x}} f)^\top}_{\mathbf{0} \text{ at optimum}} \frac{\partial \mathbf{x}^*}{\partial \boldsymbol{\theta}}$.

Application to Bellman equations: For $[\mathbb{L}v](s) = \max_a \{r(s, a) + \gamma E[v(s')]\}$, the derivative with respect to parameters in v can be computed by treating the optimal action as constant. For example, if $v(s; \theta) = \sum_j \theta_j \varphi_j(s)$:

$$\frac{\partial[\mathbb{L}v](s)}{\partial\theta_j} = \gamma E[\varphi_j(s') \mid s, a^*(s; \theta)], \quad (473)$$

where $a^*(s; \theta)$ is the optimal action given parameters θ .

Important assumptions: The objective f is smooth, the maximizer is unique and in the interior (or constraints are smooth with stable active sets), and the first-order condition holds.

With the Envelope Theorem providing a tractable way to compute Jacobians for problems involving optimization, Newton’s method becomes practical for weighted residual methods applied to Bellman equations and similar problems. The method offers **quadratic convergence** near the solution. Once in the neighborhood of the true fixed point, Newton’s method typically converges in just a few iterations. Unlike function iteration, it doesn’t rely on the finite-dimensional approximation preserving any contraction property, making it applicable to a broader range of problems, particularly those with high-order polynomial bases or large discount factors where function iteration struggles.

However, Newton’s method demands more from both the algorithm and the user. Each iteration requires computing and solving a full Jacobian system, making the per-iteration cost significantly higher than function iteration. The method is also sensitive to initialization: started far from the solution, Newton’s method may diverge or converge to spurious fixed points that the finite-dimensional problem introduces but the original infinite-dimensional problem lacks. When applying the Envelope Theorem, implementation becomes more complex. We must track the optimal action at each evaluation point and compute the Jacobian entries using the formula above (expected basis function values at next states under optimal actions), though the economic interpretation (tracking how value propagates through optimal decisions) often makes the computation conceptually clearer than explicit derivative calculations would be.

Comparison and Practical Recommendations	Method	Convergence	Per-iteration cost
	Function iteration	Linear (when contraction holds)	Low
	Newton’s method	Quadratic (near solution)	Moderate (Jacobian + solve)

Which method to use? When the problem has strong contraction (small γ , well-conditioned bases, shape-preserving approximations like linear interpolation or splines), function iteration is simple and robust. For weak contraction (large γ , high-order polynomials), a hybrid approach works well: run function iteration for several iterations to enter the basin of attraction, then switch to

Newton's method for rapid final convergence. When the finite-dimensional approximation destroys contraction entirely (common with non-monotone bases), Newton's method may be necessary from the start, though careful initialization (from a coarser approximation or perturbation methods) is required.

Quasi-Newton methods like BFGS or Broyden offer a middle ground. They approximate the Jacobian using function evaluations only, avoiding explicit derivative computations while maintaining superlinear convergence. This can be useful when computing the exact Jacobian via the Envelope Theorem is expensive or when the approximation quality is acceptable.

Step 5: Verify the Solution Once we have computed a candidate solution $\hat{f}$, we must verify its quality. Projection methods optimize $\hat{f}$ with respect to specific criteria (specific test functions or collocation points), but we should check that the residual is small everywhere, including directions or points we did not optimize over.

Typical diagnostic checks include:

- Computing $\|R(\cdot; \theta)\|$ using a more accurate quadrature rule than was used in the optimization
- Evaluating $R(x; \theta)$ at many points not used in the fitting process
- If using Galerkin with the first n basis functions, checking orthogonality against higher-order basis functions

In summary, we have established a template: parameterize the unknown function using basis functions, define a residual measuring how far from a solution we are, and impose conditions via inner products with test functions. Different test functions yield different methods: Galerkin uses the basis itself, collocation uses delta functions at chosen points, and least squares uses residual gradients. We now apply this framework to the Bellman equation.

6.2.4 Application to the Bellman Equation

Consider the Bellman optimality equation $v(s) = \mathcal{L}v(s) = \max_{a \in \mathcal{A}_s} \{r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a)v(j)\}$. For a candidate approximation $\hat{v}(s) = \sum_{i=1}^n \theta_i \varphi_i(s)$, the residual is:

$$R(s; \theta) = \mathcal{L}\hat{v}(s) - \hat{v}(s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a)\hat{v}(j) \right\} - \sum_{i=1}^n \theta_i \varphi_i(s). \quad (474)$$

We examine how collocation and Galerkin, the two most common weighted residual methods for Bellman equations, specialize the general solution approaches from Step 4.

Collocation For collocation, we choose n states $\{s_1, \dots, s_n\}$ and require the Bellman equation to hold exactly at these points:

$$\sum_{j=1}^n \theta_j \varphi_j(s_i) = \max_{a \in \mathcal{A}_{s_i}} \left\{ r(s_i, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s_i, a) \sum_{\ell=1}^n \theta_\ell \varphi_\ell(j) \right\}, \quad i = 1, \dots, n. \quad (475)$$

It helps to define the **parametric Bellman operator** $L_\varphi : R^n \rightarrow R^n$ by $[L_\varphi(\theta)]_i = [L\hat{v}(\cdot; \theta)](s_i)$, the Bellman operator evaluated at collocation point s_i . Let Φ be the $n \times n$ matrix with entries $\Phi_{ij} = \varphi_j(s_i)$. Then the collocation equations become $\Phi\theta = L_\varphi(\theta)$.

Function iteration for collocation proceeds as follows. Given current coefficients $\theta^{(k)}$, we evaluate the Bellman operator at each collocation point to get target values $t_i^{(k)} = [L_\varphi(\theta^{(k)})]_i$. We then find new coefficients by solving the linear system $\Phi\theta^{(k+1)} = t^{(k)}$. This is parametric value iteration: apply the Bellman operator, fit the result.

When the state space is continuous, we approximate expectations using numerical quadrature (Gauss-Hermite for normal shocks, etc.). The method is simple and robust when the finite-dimensional approximation preserves contraction, but can be slow for large discount factors.

Newton's method for collocation treats the problem as rootfinding: $G(\theta) = \Phi\theta - L_\varphi(\theta) = 0$. The Jacobian is $J_G = \Phi - J_{L_\varphi}$, where the Envelope Theorem (Step 4) gives us $[J_{L_\varphi}]_{ij} = \gamma \sum_{s'} p(s'|s_i, a_i^*(\theta)) \varphi_j(s')$. Here $a_i^*(\theta)$ is the optimal action at collocation point s_i given the current coefficients.

This converges rapidly near the solution but requires good initialization and more computation per iteration than function iteration. The method is equivalent to policy iteration: each step evaluates the value of the current greedy policy, then improves it.

Why is collocation popular for Bellman equations? Because it avoids integration when testing the residual. We only evaluate the Bellman operator at n discrete points. In contrast, Galerkin requires integrating the residual against each basis function.

Worked Example: Collocation on the Optimal Stopping Problem

Returning to our motivating example, let us trace through the collocation algorithm with $n = 4$ polynomial basis functions at Chebyshev nodes:

Galerkin For Galerkin, we use the basis functions themselves as test functions. The conditions are:

$$\int_{\mathcal{S}} [L\hat{v}(s; \theta) - \hat{v}(s; \theta)] \varphi_i(s) w(s) ds = 0, \quad i = 1, \dots, n. \quad (476)$$

where $w(s)$ is a weight function (often the stationary distribution $d^\pi(s)$ in RL applications, or simply $w(s) = 1$). Expanding this:

$$\int_S \left[\max_a \{r(s, a) + \gamma E[v(s')]\} - \sum_j \theta_j \varphi_j(s) \right] \varphi_i(s) w(s) ds = 0. \quad (477)$$

Function iteration for Galerkin works differently than for collocation. Given $\theta^{(k)}$, we cannot simply evaluate the Bellman operator and fit. Instead, we must solve an integral equation. At each iteration, we seek $\theta^{(k+1)}$ satisfying:

$$\int_S \sum_j \theta_j^{(k+1)} \varphi_j(s) \varphi_i(s) w(s) ds = \int_S [\mathcal{L}\hat{v}(s; \theta^{(k)})] \varphi_i(s) w(s) ds. \quad (478)$$

The left side is a linear system (the “mass matrix” $M_{ij} = \int \varphi_i \varphi_j w$), and the right side requires integrating the Bellman operator output against each test function. When the basis functions are orthogonal polynomials with matching weight w , the mass matrix is diagonal, simplifying the solve. But we still need numerical integration to evaluate the right side. This makes Galerkin substantially more expensive than collocation per iteration.

Newton’s method for Galerkin similarly requires integration. The residual is $R(s; \theta) = \mathcal{L}\hat{v}(s; \theta) - \hat{v}(s; \theta)$, and we need $G_i(\theta) = \int R(s; \theta) \varphi_i(s) w(s) ds = 0$. The Jacobian entry is:

$$J_{ij} = \int \left[\frac{\partial \mathcal{L}\hat{v}(s; \theta)}{\partial \theta_j} - \varphi_j(s) \right] \varphi_i(s) w(s) ds. \quad (479)$$

The Envelope Theorem gives $\frac{\partial \mathcal{L}\hat{v}(s; \theta)}{\partial \theta_j} = \gamma E[\varphi_j(s') \mid s, a^*(s; \theta)]$, so we must integrate expected basis function values (under optimal actions) against test functions and weight. This requires both numerical integration and careful tracking of optimal actions across the state space, making it substantially more complex than collocation’s pointwise evaluation.

The advantage of Galerkin over collocation lies in its theoretical properties: when using orthogonal polynomials, Galerkin provides optimal approximation in the weighted L^2 norm. For smooth problems, this can yield better accuracy per degree of freedom than collocation. In practice, collocation’s computational simplicity usually outweighs Galerkin’s theoretical optimality for Bellman equations, especially in high-dimensional problems where integration becomes prohibitively expensive.

The algorithms above reduce the infinite-dimensional Bellman fixed-point problem to finite-dimensional coefficient computation. Collocation avoids integration entirely by requiring exact satisfaction at discrete points, while Galerkin imposes weighted orthogonality conditions requiring numerical quadrature. Both can be solved via function iteration (when contraction is preserved) or Newton’s method (for faster convergence near the solution). The discrete MDP specialization below reveals connections to algorithms widely used in reinforcement learning.

Exercises: Collocation and Galerkin

1. **Effect of collocation points.** For the optimal stopping problem, compare collocation with $n = 4$ Chebyshev nodes versus $n = 4$ equally-spaced nodes. Which choice gives smaller maximum error?
2. **Threshold location.** The optimal stopping policy has a threshold structure. Using your polynomial approximation $\hat{v}(s)$, estimate the threshold by finding where $\hat{v}(s) = s$. Compare to the exact threshold.
3. **Orthogonal polynomials.** For $\mathcal{S} = [-1, 1]$, write out the Galerkin conditions using Chebyshev polynomials T_i and weight $w(x) = 1/\sqrt{1-x^2}$. What makes this choice computationally convenient?
4. **Newton vs. function iteration.** How does the iteration count depend on γ ? Try $\gamma \in \{0.5, 0.9, 0.99\}$.

Galerkin for Discrete MDPs: LSTD and LSPI When the state space is discrete and finite, the Galerkin conditions simplify dramatically. The integrals become sums, and we can write everything in matrix form. This specialization shows the connection to algorithms widely used in reinforcement learning.

For a discrete state space $\mathcal{S} = \{s_1, \dots, s_m\}$, the Galerkin orthogonality conditions

$$\int_{\mathcal{S}} [\mathbb{L}\hat{v}(s; \theta) - \hat{v}(s; \theta)] \varphi_i(s) w(s) ds = 0 \quad (480)$$

become weighted sums over states:

$$\sum_{s \in \mathcal{S}} \xi(s) [\mathbb{L}\hat{v}(s; \theta) - \hat{v}(s; \theta)] \varphi_i(s) = 0, \quad i = 1, \dots, n, \quad (481)$$

where $\xi(s) \geq 0$ with $\sum_s \xi(s) = 1$ is a probability distribution over states. Define the feature matrix $\Phi \in R^{m \times n}$ with entries $\Phi_{si} = \varphi_i(s)$ (each row contains the features for one state), and let $\Xi = \text{diag}(\xi)$ be the diagonal matrix with the state distribution on the diagonal.

Policy Evaluation: LSTD For **policy evaluation** with a fixed policy π , the Bellman operator is linear:

$$[\mathbb{L}_{\pi}\hat{v}](s) = r(s, \pi(s)) + \gamma \sum_{j \in \mathcal{S}} p(j|s, \pi(s)) \hat{v}(j). \quad (482)$$

With linear function approximation $\hat{v}(s) = \varphi(s)^{\top} \theta = \sum_i \theta_i \varphi_i(s)$, this becomes:

$$[\mathbf{L}_\pi \hat{v}](s) = r(s, \pi(s)) + \gamma \sum_{j \in \mathcal{S}} p(j|s, \pi(s)) \sum_i \theta_i \varphi_i(j). \quad (483)$$

Let $\mathbf{r}_\pi \in R^m$ be the vector of rewards $[\mathbf{r}_\pi]_s = r(s, \pi(s))$, and $\mathbf{P}_\pi \in R^{m \times m}$ be the transition matrix with $[\mathbf{P}_\pi]_{sj} = p(j|s, \pi(s))$. Then $\mathbf{L}_\pi \hat{v} = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \Phi \theta$ in vector form.

The Galerkin conditions require $\langle \mathbf{L}_\pi \hat{v} - \hat{v}, \varphi_i \rangle_\xi = 0$ for all basis functions, which in matrix form is:

$$\Phi^\top \Xi (\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \Phi \theta - \Phi \theta) = \mathbf{0}. \quad (484)$$

Rearranging:

$$\Phi^\top \Xi (\Phi - \gamma \mathbf{P}_\pi \Phi) \theta = \Phi^\top \Xi \mathbf{r}_\pi. \quad (485)$$

This is the **LSTD (Least Squares Temporal Difference)** solution. The matrix $\mathbf{A} = \Phi^\top \Xi (\Phi - \gamma \mathbf{P}_\pi \Phi)$ and vector $\mathbf{b} = \Phi^\top \Xi \mathbf{r}_\pi$ give the linear system $\mathbf{A} \theta = \mathbf{b}$.

When ξ is the stationary distribution of policy π (so $\xi^\top \mathbf{P}_\pi = \xi^\top$), this system has a unique solution, and the projected Bellman operator $\mathbf{P} \mathbf{L}_\pi$ is a contraction in the weighted L^2 norm $\|\cdot\|_\xi$. This is the theoretical foundation for TD learning with linear function approximation. The fixed point computed here is the same one that TD(0) converges to stochastically; we derive the incremental algorithm in the Monte Carlo chapter.

Check that the dimensions work out: if we have m states and n basis functions, what are the dimensions of Φ , Ξ , $\mathbf{P}_\pi$, and the matrix $\mathbf{A} = \Phi^\top \Xi (\Phi - \gamma \mathbf{P}_\pi \Phi)$?

Worked Example: LSTD for Policy Evaluation To illustrate LSTD concretely, consider a 3-state Markov chain under a fixed policy:

$$\mathbf{P}_\pi = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.3 & 0.4 & 0.3 \\ 0.1 & 0.3 & 0.6 \end{pmatrix}, \quad \mathbf{r}_\pi = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix}. \quad (486)$$

The Bellman Optimality Equation: Function Iteration and Newton's Method For the **Bellman optimality equation**, the max operator introduces nonlinearity:

$$[\mathbf{L} \hat{v}](s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) \hat{v}(j) \right\}. \quad (487)$$

The Galerkin conditions become:

$$F(\theta) \equiv \Phi^\top \Xi (\mathbf{L} \hat{v}(\cdot; \theta) - \Phi \theta) = \mathbf{0}, \quad (488)$$

where the Bellman operator must be evaluated at each state s to find the optimal action and compute the target value. This is a system of n nonlinear equations in n unknowns.

Function iteration applies the Bellman operator and projects back. Given $\theta^{(k)}$, compute the greedy policy $\pi^{(k)}(s) = \arg \max_a \{r(s, a) + \gamma \sum_{j \in \mathcal{S}} p(j|s, a) \varphi(j)^\top \theta^{(k)}\}$ at each state, then solve:

$$\Phi^\top \Xi(\Phi - \gamma \mathbf{P}_{\pi^{(k)}} \Phi) \theta^{(k+1)} = \Phi^\top \Xi \mathbf{r}_{\pi^{(k)}}. \quad (489)$$

This evaluates the current greedy policy using LSTD, then implicitly improves by computing a new greedy policy at the next iteration. However, convergence can be slow when the finite-dimensional approximation poorly preserves contraction.

Newton's method treats $G(\theta) = 0$ as a rootfinding problem and uses the Jacobian to accelerate convergence. The Jacobian of G is:

$$J_G(\theta) = \frac{\partial G}{\partial \theta} = \Phi^\top \Xi \left(\frac{\partial \hat{v}(\cdot; \theta)}{\partial \theta} - \Phi \right). \quad (490)$$

To compute $\frac{\partial \hat{v}(s; \theta)}{\partial \theta_j}$, we use the Envelope Theorem from Step 4. At the current $\theta^{(k)}$, let $a^*(s; \theta^{(k)})$ be the optimal action at state s . Then:

$$\frac{\partial [\mathbf{L}\hat{v}](s; \theta^{(k)})}{\partial \theta_j} = \gamma \sum_{j \in \mathcal{S}} p(j|s, a^*(s; \theta^{(k)})) \varphi_j(j). \quad (491)$$

Define the policy $\pi^{(k)}(s) = a^*(s; \theta^{(k)})$. The Jacobian becomes:

$$J_G(\theta^{(k)}) = \Phi^\top \Xi(\gamma \mathbf{P}_{\pi^{(k)}} \Phi - \Phi) = -\Phi^\top \Xi(\Phi - \gamma \mathbf{P}_{\pi^{(k)}} \Phi). \quad (492)$$

The Newton update $\theta^{(k+1)} = \theta^{(k)} - J_G(\theta^{(k)})^{-1} G(\theta^{(k)})$ simplifies. We have:

$$G(\theta^{(k)}) = \Phi^\top \Xi(\mathbf{L}\hat{v}(\cdot; \theta^{(k)}) - \Phi \theta^{(k)}). \quad (493)$$

At each state s , the greedy value is $[\mathbf{L}\hat{v}(\cdot; \theta^{(k)})](s) = r(s, \pi^{(k)}(s)) + \gamma \sum_j p(j|s, \pi^{(k)}(s)) \varphi(j)^\top \theta^{(k)}$, which equals $[\mathbf{L}_{\pi^{(k)}} \hat{v}(\cdot; \theta^{(k)})](s)$. Thus:

$$G(\theta^{(k)}) = \Phi^\top \Xi(\mathbf{r}_{\pi^{(k)}} + \gamma \mathbf{P}_{\pi^{(k)}} \Phi \theta^{(k)} - \Phi \theta^{(k)}). \quad (494)$$

The Newton step becomes:

$$\theta^{(k+1)} = \theta^{(k)} + [\Phi^\top \Xi(\Phi - \gamma \mathbf{P}_{\pi^{(k)}} \Phi)]^{-1} \Phi^\top \Xi(\mathbf{r}_{\pi^{(k)}} + \gamma \mathbf{P}_{\pi^{(k)}} \Phi \theta^{(k)} - \Phi \theta^{(k)}). \quad (495)$$

Multiplying through and simplifying:

$$\Phi^\top \Xi(\Phi - \gamma \mathbf{P}_{\pi^{(k)}} \Phi) \theta^{(k+1)} = \Phi^\top \Xi \mathbf{r}_{\pi^{(k)}}. \quad (496)$$

This is **LSPI (Least Squares Policy Iteration)**. Each Newton step:

1. Computes the greedy policy $\pi^{(k)}(s) = \arg \max_a \{r(s, a) + \gamma \sum_j p(j|s, a) \varphi(j)^\top \theta^{(k)}\}$
2. Solves the LSTD equation for this policy to get $\theta^{(k+1)}$

Newton's method for the Galerkin-projected Bellman optimality equation is equivalent to policy iteration in the function approximation setting. Just as Newton's method for collocation corresponded to policy iteration (Step 4), Newton's method for discrete Galerkin gives LSPI.

Galerkin projection with linear function approximation reduces policy iteration to a sequence of linear systems, each solvable in closed form. For discrete MDPs, we can compute the matrices $\Phi^\top \Xi \Phi$ and $\Phi^\top \Xi P_\pi \Phi$ exactly.

6.2.5 Extension to Nonlinear Approximators

The weighted residual methods developed so far have focused on linear function classes: polynomial bases, piecewise linear interpolants, and linear combinations of fixed basis functions. Neural networks, kernel methods, and decision trees do not fit this template. How does the framework extend to nonlinear approximators?

Recall the Galerkin approach for linear approximation $v_\theta = \sum_{i=1}^d \theta_i \varphi_i$. The orthogonality conditions $\langle v - Lv, \varphi_i \rangle_w = 0$ for all i define a linear system with a closed-form solution. These equations arise from minimizing $\|v - Lv\|_w^2$ over the subspace, since at the minimum, the gradient with respect to each coefficient must vanish. The connection between norm minimization and orthogonality holds generally. For any norm $\|\cdot\|_w$ induced by an inner product $\langle \cdot, \cdot \rangle_w$, minimizing $\|f(\theta)\|_w^2$ with respect to parameters requires $\frac{\partial}{\partial \theta_i} \|f(\theta)\|_w^2 = 0$. Since $\|f\|_w^2 = \langle f, f \rangle_w$, the chain rule gives $2\langle f, \frac{\partial f}{\partial \theta_i} \rangle_w = 0$. Minimizing the residual norm is thus equivalent to requiring orthogonality $\langle f, \frac{\partial f}{\partial \theta_i} \rangle_w = 0$ for all i . The equivalence holds for any choice of inner product: weighted L^2 integrals for Galerkin, sums over collocation points for collocation, or sampled expectations for neural networks.

For nonlinear function classes parameterized by $\theta \in R^p$ (neural networks, kernel expansions), the same minimization principle applies:

$$\theta^* = \arg \min_{\theta} \|v_\theta - Lv_\theta\|_w^2. \quad (497)$$

The first-order stationarity condition yields orthogonality:

$$\left\langle v_\theta - Lv_\theta, \frac{\partial v_\theta}{\partial \theta_i} \right\rangle_w = 0 \quad \text{for all } i. \quad (498)$$

The test functions are now the partial derivatives $\frac{\partial v_\theta}{\partial \theta_i}$, which span the tangent space to the manifold $\{v_\theta : \theta \in R^p\}$ at the current parameters. In the linear case $v_\theta = \sum_i \theta_i \varphi_i$, the partial derivative $\frac{\partial v_\theta}{\partial \theta_i} = \varphi_i$ recovers the fixed basis functions of Galerkin. For nonlinear parameterizations, the test functions change with θ , and the orthogonality conditions define a nonlinear system solved by iterative gradient descent.

The **dual pairing** formulation [Legrand and Junca \[2025\]](#) extends this framework to settings where test objects need not be regular functions. We have been informal about this distinction in our treatment of collocation, but the Dirac deltas $\delta(x - x_i)$ used there are not classical functions. They are distributions, defined rigorously only through their action on test functions via $\langle \mathbf{N}(v), \delta(x - x_i) \rangle = (\mathbf{N}v)(x_i)$. The simple calculus argument for orthogonality does not apply directly to such objects; the dual pairing framework provides the proper mathematical foundation. The induced dual norm $\|\mathbf{N}(v)\|_* = \sup_{\|w\|=1} |\langle \mathbf{N}(v), w \rangle|$ measures residuals by their worst-case effect on test functions, a perspective that has inspired adversarial formulations [Zang et al. \[2020\]](#) where both trial and test functions are learned.

The minimum residual framework thus connects classical projection methods to modern function approximation. The unifying principle is orthogonality of residuals to test functions. Linear methods use fixed test functions and admit closed-form solutions. Nonlinear methods use parameter-dependent test functions and require iterative optimization.

We now turn to the question of convergence: when does the iteration $v_{k+1} = \text{PL}v_k$ converge?

6.2.6 Monotone Projection and the Preservation of Contraction

The informal discussion of shape preservation hints at a deeper theoretical question: **when does the function iteration method converge?** Recall from our discussion of collocation that function iteration proceeds in two steps:

1. Apply the Bellman operator at collocation points: $t^{(k)} = v(\theta^{(k)})$ where $t_i^{(k)} = \mathbf{L}\hat{v}^{(k)}(s_i)$
2. Fit new coefficients to match these targets: $\Phi\theta^{(k+1)} = t^{(k)}$, giving $\theta^{(k+1)} = \Phi^{-1}v(\theta^{(k)})$

We can reinterpret this iteration in **function space** rather than coefficient space. Let $\mathbf{P}$ be the **projection operator** that takes any function f and returns its approximation in $\text{span}\{\varphi_1, \dots, \varphi_n\}$. For collocation, $\mathbf{P}$ is the interpolation operator: $(\mathbf{P}f)(s)$ is the unique linear combination of basis functions that matches f at the collocation points. Then Step 2 can be written as: fit $\hat{v}^{(k+1)}$ so that $\hat{v}^{(k+1)}(s_i) = \mathbf{L}\hat{v}^{(k)}(s_i)$ for all collocation points, which means $\hat{v}^{(k+1)} = \mathbf{P}(\mathbf{L}\hat{v}^{(k)})$.

In other words, function iteration is equivalent to **projected value iteration in function space**:

$$\hat{v}^{(k+1)} = \text{PL}\hat{v}^{(k)}. \quad (499)$$

We know that standard value iteration $v_{k+1} = \mathbf{L}v_k$ converges because $\mathbf{L}$ is a γ -contraction in the sup norm. But now we're iterating with the **composed operator** PL instead of $\mathbf{L}$ alone.

This PL structure is not specific to collocation. It is inherent in all projection methods. The general pattern is always the same: apply the Bellman operator to get a target function $L\hat{v}^{(k)}$, then project it back onto our approximation space to get $\hat{v}^{(k+1)}$. The projection step defines an operator P that depends on our choice of test functions:

- For **collocation**, P interpolates values at collocation points
- For **Galerkin**, P is orthogonal projection with respect to $\langle \cdot, \cdot \rangle_w$
- For **least squares**, P minimizes the weighted residual norm

But regardless of which projection method we use, iteration takes the form $\hat{v}^{(k+1)} = PL\hat{v}^{(k)}$.

The central question is whether the composition PL inherits the contraction property of L . If not, the iteration may diverge, oscillate, or converge to a spurious fixed point even though the original problem is well-posed.

Monotone Approximators and Stability The answer turns out to depend on specific properties of the approximation operator P . This theory was developed independently across multiple research communities: computational economics [Judd \[1992, 1996\]](#), [McGrattan \[1997\]](#), [Santos and Vigo-Aguiar \[1998\]](#), economic dynamics [Stachurski \[2009\]](#), and reinforcement learning [Gordon \[1995, 1999\]](#). These communities arrived at essentially the same mathematical conditions.

Monotonicity Implies Nonexpansiveness It turns out that approximation operators satisfying simple structural properties automatically preserve contraction.

Proposition 6.1 (Monotone operators are nonexpansive (Stachurski)). *Let $P : \mathcal{V} \rightarrow \mathcal{V}$ be a linear operator on the space $\mathcal{V}$ of bounded real-valued functions on S . If P satisfies:*

1. **Monotonicity:** $f \leq g$ pointwise implies $Pf \leq Pg$
2. **Constant preservation:** $P\mathbf{1} = \mathbf{1}$ where $\mathbf{1}$ is the constant function equal to 1

Then P is nonexpansive in the sup norm: $\|Pf - Pg\|_\infty \leq \|f - g\|_\infty$ for all $f, g \in \mathcal{V}$.

Proof. Let $M = \|f - g\|_\infty$. Then $-M \leq f(s) - g(s) \leq M$ for all s , which can be written as $g - M\mathbf{1} \leq f \leq g + M\mathbf{1}$. By monotonicity, $P(g - M\mathbf{1}) \leq Pf \leq P(g + M\mathbf{1})$. By linearity and constant preservation, $Pg - M\mathbf{1} \leq Pf \leq Pg + M\mathbf{1}$, which means $|Pf(s) - Pg(s)| \leq M$ for all s . Therefore $\|Pf - Pg\|_\infty \leq \|f - g\|_\infty$. $\square$

This proposition shows that monotonicity and constant preservation automatically imply nonexpansiveness. There is no need to verify this separately. The intuition is that a monotone, constant-preserving operator acts like a weighted average that respects order structure and cannot amplify differences between functions.

Preservation of Contraction Combining nonexpansiveness with the contraction property of the Bellman operator yields the main stability result.

Theorem 6.3 (Stability of projected value iteration (Santos-Vigo-Aguiar)). *Let $L : \mathcal{V} \rightarrow \mathcal{V}$ be a γ -contraction on the space $\mathcal{V}$ of bounded functions with respect to the sup norm. Let $P : \mathcal{V} \rightarrow \mathcal{V}$ be a linear approximation operator satisfying monotonicity and constant preservation.*

Then the composed operator PL is a γ -contraction, and projected value iteration $v_{k+1} = PLv_k$ converges globally to a unique fixed point $v_P \in \text{Range}(P)$ with approximation error:

$$\|v_P - v^*\|_\infty \leq \frac{1}{1-\gamma} \|Pv^* - v^*\|_\infty, \quad (500)$$

where v^* is the true value function.

Proof. By Proposition 6.1, P is nonexpansive since it satisfies monotonicity and constant preservation. Since L is a γ -contraction, we have $\|Lf - Lg\|_\infty \leq \gamma\|f - g\|_\infty$. Therefore:

$$\|PLf - PLg\|_\infty \leq \|Lf - Lg\|_\infty \leq \gamma\|f - g\|_\infty, \quad (501)$$

showing that PL is a γ -contraction. The error bound follows from fixed-point analysis: $v^* - v_P = (I - PL)^{-1}(v^* - Pv^*)$, and since PL is a γ -contraction, $\|(I - PL)^{-1}\| \leq (1 - \gamma)^{-1}$. $\square$

This error bound tells us that the fixed-point error is controlled by how well P can represent v^* . If $v^* \in \text{Range}(P)$, then $Pv^* = v^*$ and the error vanishes. Otherwise, the error is proportional to the approximation error $\|Pv^* - v^*\|_\infty$, amplified by the factor $(1 - \gamma)^{-1}$.

Averagers in Discrete-State Problems For discrete-state problems, the monotonicity conditions have a natural interpretation as **averaging with nonnegative weights**. This characterization was developed by Gordon in the context of reinforcement learning.

Definition 6.1 (Averager (Gordon)). *An operator $P : R^{|\mathcal{S}|} \rightarrow R^{|\mathcal{S}|}$ is an **averager** if $Pv = Wv$ where W is a $|\mathcal{S}| \times |\mathcal{S}|$ stochastic matrix: $w_{ij} \geq 0$ and $\sum_j w_{ij} = 1$ for all i .*

Averagers automatically satisfy the monotonicity conditions: linearity follows from matrix multiplication, monotonicity follows from nonnegativity of entries, and constant preservation follows from row sums equaling one.

Theorem 6.4 (Stability with averagers (Gordon)). *If $\mathbf{P}$ is an averager and $\mathbf{L}$ is the Bellman operator (a γ -contraction), then $\mathbf{PL}$ is a γ -contraction, and value iteration $v_{k+1} = \mathbf{PL}v_k$ converges to a unique fixed point.*

This specializes the Santos-Vigo-Aguiar theorem to discrete states, expressed in the probabilistic language of stochastic matrices. The stochastic matrix characterization connects to Markov chain theory: $\mathbf{P}v$ represents expected values after one transition, and the monotonicity property reflects the fact that expectations preserve order.

Examples of averagers include state aggregation (averaging values within groups), K-nearest neighbors (averaging over nearest states), kernel smoothing with positive kernels, and multilinear interpolation on grids (barycentric weights are nonnegative and sum to one). **Counterexamples** include linear least squares regression (projection matrix may have negative entries) and high-order polynomial interpolation (Runge phenomenon produces negative weights).

The following table summarizes which common approximation operators satisfy the monotonicity conditions:

Method	Monotone?	Notes
Piecewise linear interpolation	Yes	Always an averager; guaranteed stability
Multilinear interpolation (grid)	Yes	Barycentric weights are nonnegative and sum to one
Shape-preserving splines (Schumaker)	Yes	Designed to maintain monotonicity
State aggregation	Yes	Exact averaging within groups
Kernel smoothing (positive kernels)	Yes	If kernel integrates to one
High-order polynomial interpolation	No	Oscillations violate monotonicity (Runge phenomenon)
Least squares projection (arbitrary basis)	No	Projection matrix may have negative entries
Fourier/spectral methods	No	Not monotone-preserving in general
Neural networks	No	Highly flexible but no monotonicity guarantees

The distinction between “safe” (monotone) and “potentially unstable” (non-monotone) approximators provides rigorous foundation for the folk wisdom that linear interpolation is reliable while high-order polynomials can be dangerous for value iteration. But notice that the table’s verdict on “least squares projection”

is somewhat abstract. It doesn't specifically address the three weighted residual methods we introduced at the start of this chapter.

The choice of solution method determines which approximation operators are safe to use. Successive approximation (fixed-point iteration) requires monotone approximators to guarantee convergence. Rootfinding methods like Newton's method do not require monotonicity. Stability depends on numerical properties of the Jacobian rather than contraction preservation. These considerations suggest hybrid strategies. One approach runs a few iterations with a monotone method to generate an initial guess, then switches to Newton's method with a smooth approximation for rapid final convergence.

Connecting Back to Collocation, Galerkin, and Least Squares We have now developed a general stability theory for projected value iteration and surveyed which approximation operators are monotone. But what does this mean for the three specific weighted residual methods we introduced at the start of this chapter: **collocation**, **Galerkin**, and **least squares**? Each method defines a different projection operator P , and we now need to determine which satisfy the monotonicity conditions that guarantee convergence.

Collocation with piecewise linear interpolation is monotone. When we use collocation with piecewise linear basis functions on a grid, the projection operator performs linear interpolation between grid points. At any state s between grid points s_i and s_{i+1} , the interpolated value is:

$$(Pv)(s) = \frac{s_{i+1} - s}{s_{i+1} - s_i} v(s_i) + \frac{s - s_i}{s_{i+1} - s_i} v(s_{i+1}). \quad (502)$$

The interpolation weights (barycentric coordinates) are nonnegative and sum to one, making this an averager in Gordon's sense. Therefore collocation with piecewise linear bases satisfies the monotonicity conditions and the Santos-Vigo-Aguilar stability theorem applies. The folk wisdom that "linear interpolation is safe for value iteration" has rigorous theoretical foundation.

Galerkin projection is generally not monotone. The Galerkin projection operator for a general basis $\{\varphi_1, \dots, \varphi_n\}$ has the form:

$$P = \Phi(\Phi^\top \mathbf{W} \Phi)^{-1} \Phi^\top \mathbf{W}, \quad (503)$$

where $\mathbf{W}$ is a diagonal weight matrix and Φ contains the basis function evaluations. This projection matrix typically has **negative entries**. To see why, consider a simple example with polynomial basis functions $\{1, x, x^2\}$ on $[-1, 1]$. The projection of a function onto this space involves computing $(\Phi^\top \mathbf{W} \Phi)^{-1}$, and the resulting operator can map nonnegative functions to functions with negative values. This is the same phenomenon underlying the Runge phenomenon in high-order polynomial interpolation: the projection weights oscillate in sign.

Since Galerkin projection is not monotone, the sup norm contraction theory does not guarantee convergence of projected value iteration $v_{k+1} = PLv_k$ with Galerkin.

Least squares methods share the non-monotonicity issue. The least squares projection operator minimizes $\|\mathbf{N}(\hat{f})\|_w^2$ and has the same mathematical form as Galerkin projection. It is a linear projection onto $\text{span}\{\varphi_1, \dots, \varphi_n\}$ with respect to a weighted inner product. Like Galerkin, the projection matrix typically contains negative entries and violates monotonicity.

The monotone approximator framework successfully covers collocation with simple bases, but leaves two important methods, Galerkin and least squares, without convergence guarantees. These methods are used in least-squares temporal difference learning (LSTD) and modern reinforcement learning with linear function approximation. We need a different analytical framework to understand when these non-monotone projections lead to convergent algorithms.

Monotone projections (piecewise linear interpolation, state aggregation) automatically preserve the Bellman operator's contraction property, guaranteeing convergence of projected value iteration. Non-monotone projections (Galerkin, high-order polynomials) may destroy contraction in the sup norm, requiring either different solution methods (Newton) or analysis in different norms. The next section develops the latter approach for policy evaluation.

Exercises: Monotonicity and Convergence

1. **Verifying monotonicity.** Consider piecewise linear interpolation on a 5-point grid. Write out the interpolation weights for a point between grid points 2 and 3. Verify that all weights are nonnegative and sum to one.
2. **A non-monotone example.** Using Lagrange interpolation with 4 equally spaced nodes on $[0, 1]$, compute the interpolation weights for the point $x = 0.1$. Show that some weights are negative.
3. **State aggregation.** Consider a discrete MDP with states $\{1, 2, 3, 4\}$ aggregated into two groups: $\{1, 2\}$ and $\{3, 4\}$. Write out the aggregation operator as a matrix.
4. **Contraction constant.** For the composed operator $\mathbf{PL}$ with a monotone $\mathbf{P}$, prove that the contraction constant is exactly γ .

6.2.7 Beyond Monotone Approximators

The monotone approximator theory gives us a clean sufficient condition for convergence: if $\mathbf{P}$ is monotone (and constant-preserving), then $\mathbf{P}$ is non-expansive in the sup norm $\|\cdot\|_\infty$. Since $\mathbf{L}$ is a γ -contraction in the sup norm, their composition $\mathbf{PL}$ is also a γ -contraction in the sup norm, guaranteeing convergence of projected value iteration.

But what if $\mathbf{P}$ is not monotone? Can we still guarantee convergence? Galerkin

and least squares projections typically violate monotonicity, yet they are widely used in practice, particularly in reinforcement learning through least-squares temporal difference learning (LSTD). In general, proving convergence for non-monotone projections is difficult. However, for the special case of **policy evaluation**, computing the value function v_π of a fixed policy π , we can establish convergence by working in a different norm.

The Policy Evaluation Problem and LSTD Consider the policy evaluation problem: given policy π , we want to solve the policy Bellman equation $v_\pi = r_\pi + \gamma \mathbf{P}_\pi v_\pi$, where r_π and $\mathbf{P}_\pi$ are the reward vector and transition matrix under π . This is the core computational task in policy iteration, actor-critic algorithms, and temporal difference learning. In reinforcement learning, we typically learn from sampled experience: trajectories $(s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots)$ generated by following π . If the Markov chain induced by π is ergodic, the state distribution converges to a stationary distribution ξ satisfying $\xi^\top \mathbf{P}_\pi = \xi^\top$.

This distribution determines which states appear frequently in our data. States visited often contribute more samples and have more influence on any learned approximation. States visited rarely contribute little. For a linear approximation $v_\theta(s) = \sum_j \theta_j \varphi_j(s)$, the **least-squares temporal difference (LSTD)** algorithm computes coefficients by solving:

$$\Phi^\top \Xi (\Phi - \gamma \mathbf{P}_\pi \Phi) \theta = \Phi^\top \Xi \mathbf{r}_\pi, \quad (504)$$

where Φ is the matrix of basis function evaluations and $\Xi = \text{diag}(\xi)$. We write this matrix equation for analysis purposes, but the actual algorithm does not compute it this way. For large state spaces, we cannot enumerate all states to form Φ or explicitly represent the transition matrix $\mathbf{P}_\pi$. Instead, the practical algorithm accumulates sums from sampled transitions (s, r, s') , incrementally building the matrices $\Phi^\top \Xi \Phi$ and $\Phi^\top \Xi \mathbf{P}_\pi \Phi$ without ever forming the full objects. The algorithm is derived from first principles through temporal difference learning, and the Galerkin perspective provides an interpretation of what it computes.

LSTD as Projected Bellman Equation To see what this equation means, let $\hat{v} = \Phi \theta$ be the solution. Expanding the parentheses:

$$\Phi^\top \Xi \Phi \theta - \gamma \Phi^\top \Xi \mathbf{P}_\pi \Phi \theta = \Phi^\top \Xi \mathbf{r}_\pi. \quad (505)$$

Moving all terms to the left side and factoring out $\Phi^\top \Xi$:

$$\Phi^\top \Xi (\Phi \theta - \gamma \mathbf{P}_\pi \Phi \theta - \mathbf{r}_\pi) = 0. \quad (506)$$

Since $\hat{v} = \Phi \theta$ and the policy Bellman operator is $\mathbf{L}_\pi \hat{v} = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \hat{v}$, we can write:

$$\Phi^\top \Xi (\hat{v} - \mathbf{L}_\pi \hat{v}) = 0. \quad (507)$$

Let φ_j denote the j -th column of Φ , which contains the evaluations of the j -th basis function at all states. The equation above says that for each j :

$$\varphi_j^\top \Xi(\hat{v} - \mathbf{L}_\pi \hat{v}) = 0. \quad (508)$$

But $\varphi_j^\top \Xi(\hat{v} - \mathbf{L}_\pi \hat{v})$ is exactly the ξ -weighted inner product $\langle \varphi_j, \hat{v} - \mathbf{L}_\pi \hat{v} \rangle_\xi$. So the residual $\hat{v} - \mathbf{L}_\pi \hat{v}$ is orthogonal to every basis function, and therefore orthogonal to the entire subspace $\text{span}(\Phi)$.

By definition, the orthogonal projection $\mathbf{P}y$ of a vector y onto a subspace is the unique vector in that subspace such that $y - \mathbf{P}y$ is orthogonal to the subspace. Here, $\hat{v}$ lies in $\text{span}(\Phi)$ (since $\hat{v} = \Phi\theta$), and we have just shown that $\mathbf{L}_\pi \hat{v} - \hat{v}$ is orthogonal to $\text{span}(\Phi)$. Therefore, $\hat{v} = \mathbf{P}\mathbf{L}_\pi \hat{v}$, where $\mathbf{P}$ is orthogonal projection onto $\text{span}(\Phi)$ with respect to the ξ -weighted inner product:

$$\langle u, v \rangle_\xi = u^\top \Xi v, \quad \|v\|_\xi = \sqrt{v^\top \Xi v}, \quad \mathbf{P} = \Phi(\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi. \quad (509)$$

The weighting by ξ is not arbitrary. Temporal difference learning performs stochastic updates using individual transitions: $\theta_{k+1} = \theta_k + \alpha_k(r + \gamma v_{\theta_k}(s') - v_{\theta_k}(s)) \nabla v_{\theta_k}(s)$, with states sampled from ξ . The ODE analysis of this stochastic process (Borkar-Meyn theory) shows convergence to a fixed point, which can be expressed in closed form as the ξ -weighted projected Bellman operator. LSTD is an algorithm that computes this analytical fixed point.

Orthogonal Projection is Non-Expansive Suppose ξ is the steady-state distribution: $\xi^\top \mathbf{P}_\pi = \xi^\top$. Our goal is to establish that $\mathbf{P}\mathbf{L}_\pi$ is a contraction in $\|\cdot\|_\xi$. If we can establish that $\mathbf{P}$ is non-expansive in this norm and that $\mathbf{L}_\pi$ is a γ -contraction in $\|\cdot\|_\xi$, then their composition will be a γ -contraction:

$$\|\mathbf{P}\mathbf{L}_\pi v - \mathbf{P}\mathbf{L}_\pi w\|_\xi \leq \|\mathbf{L}_\pi v - \mathbf{L}_\pi w\|_\xi \leq \gamma \|v - w\|_\xi. \quad (510)$$

First, we establish that orthogonal projection is non-expansive. For any vector v , we can decompose $v = \mathbf{P}v + (v - \mathbf{P}v)$, where $(v - \mathbf{P}v)$ is orthogonal to the subspace $\text{span}(\Phi)$. By the Pythagorean theorem in the $\|\cdot\|_\xi$ inner product:

$$\|v\|_\xi^2 = \|\mathbf{P}v\|_\xi^2 + \|v - \mathbf{P}v\|_\xi^2. \quad (511)$$

Since $\|v - \mathbf{P}v\|_\xi^2 \geq 0$, we have:

$$\|v\|_\xi^2 \geq \|\mathbf{P}v\|_\xi^2. \quad (512)$$

Taking square roots of both sides (which preserves the inequality since both norms are non-negative):

$$\|\mathbf{P}v\|_\xi \leq \|v\|_\xi. \quad (513)$$

This holds for all v , so $\mathbf{P}$ is non-expansive in $\|\cdot\|_\xi$.

Contraction of $\mathbf{L}_\pi$ in $\|\cdot\|_\xi$ To show $\mathbf{L}_\pi = r_\pi + \gamma\mathbf{P}_\pi$ is a γ -contraction, we need to verify:

$$\|\mathbf{L}_\pi v - \mathbf{L}_\pi w\|_\xi = \|\gamma\mathbf{P}_\pi(v - w)\|_\xi = \gamma\|\mathbf{P}_\pi(v - w)\|_\xi. \quad (514)$$

This will be at most $\gamma\|v - w\|_\xi$ if $\mathbf{P}_\pi$ is non-expansive, meaning $\|\mathbf{P}_\pi z\|_\xi \leq \|z\|_\xi$ for any vector z . We therefore need to establish that $\mathbf{P}_\pi$ is non-expansive in $\|\cdot\|_\xi$.

Before reading the proof below, try to show that $\mathbf{P}_\pi$ is non-expansive in $\|\cdot\|_\xi$. Hint: what property of ξ relates it to $\mathbf{P}_\pi$?

Consider the squared norm of $\mathbf{P}_\pi z$. By definition of the weighted norm:

$$\|\mathbf{P}_\pi z\|_\xi^2 = \sum_s \xi(s) [(\mathbf{P}_\pi z)(s)]^2. \quad (515)$$

The s -th component of $\mathbf{P}_\pi z$ is $(\mathbf{P}_\pi z)(s) = \sum_{s'} p(s'|s, \pi(s)) z(s')$. This is a weighted average of the values $z(s')$ with weights $p(s'|s, \pi(s))$ that sum to one. Therefore:

$$\|\mathbf{P}_\pi z\|_\xi^2 = \sum_s \xi(s) \left[\sum_{s'} p(s'|s, \pi(s)) z(s') \right]^2. \quad (516)$$

Since the function $x \mapsto x^2$ is convex, Jensen's inequality applied to the probability distribution $p(\cdot|s, \pi(s))$ gives:

$$\left[\sum_{s'} p(s'|s, \pi(s)) z(s') \right]^2 \leq \sum_{s'} p(s'|s, \pi(s)) z(s')^2. \quad (517)$$

Substituting this into the norm expression:

$$\|\mathbf{P}_\pi z\|_\xi^2 \leq \sum_s \xi(s) \sum_{s'} p(s'|s, \pi(s)) z(s')^2 = \sum_{s'} z(s')^2 \sum_s \xi(s) p(s'|s, \pi(s)). \quad (518)$$

The stationarity condition $\xi^\top \mathbf{P}_\pi = \xi^\top$ means $\sum_s \xi(s) p(s'|s, \pi(s)) = \xi(s')$ for all s' . Therefore:

$$\|\mathbf{P}_\pi z\|_\xi^2 \leq \sum_{s'} z(s')^2 \xi(s') = \|z\|_\xi^2. \quad (519)$$

Taking square roots, $\|\mathbf{P}_\pi z\|_\xi \leq \|z\|_\xi$, so $\mathbf{P}_\pi$ is non-expansive in $\|\cdot\|_\xi$. This makes $\mathbf{L}_\pi = r_\pi + \gamma\mathbf{P}_\pi$ a γ -contraction in $\|\cdot\|_\xi$. Composing with the non-expansive projection:

$$\|\mathbf{P}\mathbf{L}_\pi v - \mathbf{P}\mathbf{L}_\pi w\|_\xi \leq \|\mathbf{L}_\pi v - \mathbf{L}_\pi w\|_\xi \leq \gamma\|v - w\|_\xi. \quad (520)$$

By Banach's fixed-point theorem, $\mathbf{P}\mathbf{L}_\pi$ has a unique fixed point and iterates converge from any initialization.

Interpretation: The On-Policy Condition The result shows that convergence depends on matching the weighting to the operator. We cannot choose an arbitrary weighted L^2 norm and expect $\mathbf{P}\mathbf{L}_\pi$ to be a contraction. Instead, the weighting ξ must have a specific relationship with the transition matrix $\mathbf{P}_\pi$ in the operator $\mathbf{L}_\pi$: namely, ξ must be the stationary distribution of $\mathbf{P}_\pi$. This is what makes the weighted geometry compatible with the operator’s structure. When this match holds, Jensen’s inequality gives us non-expansiveness of $\mathbf{P}_\pi$ in the $\|\cdot\|_\xi$ norm, and the composition $\mathbf{P}\mathbf{L}_\pi$ inherits the contraction property.

In reinforcement learning, this has a practical interpretation. When we learn by following policy π and collecting transitions (s, a, r, s') , the states we visit are distributed according to the stationary distribution of π . This is **on-policy learning**. The LSTD algorithm uses data sampled from this distribution, which means the empirical weighting naturally matches the operator structure. Our analysis shows that the iterative algorithm $v_{k+1} = \mathbf{P}\mathbf{L}_\pi v_k$ converges to the same fixed point that LSTD computes in closed form.

This is fundamentally different from the monotone approximator theory. There, we required structural properties of $\mathbf{P}$ itself (monotonicity, constant preservation) to guarantee that $\mathbf{P}$ preserves the sup-norm contraction property of $\mathbf{L}$. Here, we place no such restriction on $\mathbf{P}$. Galerkin projection is not monotone. Instead, convergence depends on matching the norm to the operator. When ξ does not match the stationary distribution, as in off-policy learning where data comes from a different behavior policy, the Jensen inequality argument breaks down. The operator $\mathbf{P}_\pi$ need not be non-expansive in $\|\cdot\|_\xi$, and $\mathbf{P}\mathbf{L}_\pi$ may fail to contract. This explains divergence phenomena such as Baird’s counterexample Baird [1995].

Exercises: LSTD and the On-Policy Condition

1. **Computing the stationary distribution.** For the 3-state Markov chain in the LSTD example, compute the stationary distribution ξ satisfying $\xi^\top \mathbf{P}_\pi = \xi^\top$.
2. **LSTD with stationary weighting.** Recompute the LSTD solution using the stationary distribution instead of uniform weighting. Compare the approximation error.
3. **Off-policy divergence.** Consider a weighting $\xi' = (0.9, 0.05, 0.05)$ that does not match the stationary distribution. Implement projected value iteration and observe whether it converges.
4. **Proving non-expansiveness fails off-policy.** For $\xi' = (0.9, 0.05, 0.05)$, find a vector z such that $\|\mathbf{P}z\|_{\xi'} > \|z\|_{\xi'}$.

The Bellman Optimality Case Can we extend this weighted L^2 analysis to the Bellman optimality operator $\mathbf{L}v = \max_a[r_a + \gamma\mathbf{P}_a v]$? The answer is no, at least not with this approach. The obstacle appears at the Jensen inequality step. For policy evaluation, we had:

$$\|\mathbf{P}_\pi z\|_\xi^2 = \sum_s \xi(s) \left[\sum_{s'} p(s'|s, \pi(s)) z(s') \right]^2. \quad (521)$$

The inner term is a convex combination of the values $z(s')$, which allowed us to apply Jensen's inequality to the convex function $x \mapsto x^2$. For the optimal Bellman operator, we would need to bound:

$$\left[\max_a \sum_{s'} p(s'|s, a) z(s') \right]^2. \quad (522)$$

But the maximum of convex combinations is not itself a convex combination. It is a pointwise maximum. Jensen's inequality does not apply. We cannot conclude that $\max_a[\mathbf{P}_a z]$ is non-expansive in any weighted L^2 norm.

Is convergence of PL with Galerkin projection impossible, or merely difficult to prove? The situation is subtle. In practice, fitted Q-iteration and approximate value iteration with neural networks often work well, suggesting that some form of stability exists. But there are also well-documented divergence examples (e.g., Q-learning with linear function approximation can diverge). The theoretical picture remains incomplete. Some results exist for restricted function classes or under strong assumptions on the MDP structure, but no general convergence guarantee like the policy evaluation result is available. The interplay between the max operator, the projection, and the norm geometry is not well understood. This is an active area of research in reinforcement learning theory.

Despite these theoretical gaps, the practical algorithm template is straightforward. We now present fitted-value iteration as a meta-algorithm that combines any supervised learning method with the Bellman operator.

6.2.8 Fitted-Value/Q Iteration (FVI/FQI)

We have developed weighted residual methods through abstract functional equations: choose test functions, impose orthogonality conditions $\langle R, p_i \rangle_w = 0$, solve for coefficients. What are we actually computing when we solve these equations by successive approximation? The answer is simpler than the formalism suggests: **function iteration with a fitting step**.

Recall that the weighted residual conditions $\langle v - \mathbf{L}v, p_i \rangle_w = 0$ define a fixed-point problem $v = \mathbf{P}\mathbf{L}v$, where $\mathbf{P}$ is a projection operator onto $\text{span}(\Phi)$. We can solve this by iteration: $v_{k+1} = \mathbf{P}\mathbf{L}v_k$. Under appropriate conditions (monotonicity of $\mathbf{P}$, or matching the weight to the operator for policy evaluation), this converges to a solution.

In parameter space, this iteration becomes a fitting procedure. Consider Galerkin projection with a finite state space of n states. Let Φ be the $n \times d$

matrix of basis evaluations, $\mathbf{W}$ the diagonal weight matrix, and $\mathbf{y}$ the vector of Bellman operator evaluations: $y_i = (\mathbf{L}v_k)(s_i)$. The projection is:

$$\boldsymbol{\theta}_{k+1} = (\boldsymbol{\Phi}^\top \mathbf{W} \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{W} \mathbf{y}. \quad (523)$$

This is weighted least-squares regression: fit $\boldsymbol{\Phi}\boldsymbol{\theta}$ to targets $\mathbf{y}$. For collocation, we require exact interpolation $\boldsymbol{\Phi}\boldsymbol{\theta}_{k+1} = \mathbf{y}$ at chosen collocation points. For continuous state spaces, we approximate the Galerkin integrals using sampled states, reducing to the same finite-dimensional fitting problem. The abstraction remains consistent: function iteration in the abstract becomes **generate targets, fit to targets, repeat** in the implementation.

This extends beyond linear basis functions. Neural networks, decision trees, and kernel methods all implement variants of this procedure. Given data $\{(s_i, y_i)\}$ where $y_i = (\mathbf{L}v_k)(s_i)$, each method produces a function $v_{k+1} : \mathcal{S} \rightarrow R$ by fitting to the targets. The projection operator $\mathbf{P}$ is simply one instantiation of a fitting procedure. Galerkin and collocation correspond to specific choices of approximation class and loss function.

The abstraction **fit** encapsulates all the complexity of function approximation, whether that involves solving a linear system, running gradient descent, or training an ensemble. Any regression model with a **fit(X, y)** interface works: **LinearRegression**, **RandomForestRegressor**, **GradientBoostingRegressor**, **MLPRegressor**, or custom neural networks. The projection operator $\mathbf{P}$ is one instantiation: when $\mathcal{F}$ is a linear subspace and we minimize weighted squared error, we recover Galerkin or collocation. The broader view is that FVI reduces dynamic programming to repeated calls to a supervised learning subroutine.

The following code demonstrates fitted-value iteration on the optimal stopping problem:

A limitation of FVI/FQI is that it assumes we can evaluate the Bellman operator exactly. Computing $y_i = (\mathbf{L}v_k)(s_i)$ requires knowing transition probabilities and summing over all next states. In practice, we often have only a simulator or observed data. The next chapter shows how to approximate these expectations from samples, connecting the fitted-value iteration framework to simulation-based methods.

6.2.9 Summary

This chapter developed weighted residual methods for solving functional equations like the Bellman equation. We approximate the value function using a finite basis, then impose conditions that make the residual orthogonal to chosen test functions. Different choices of test functions yield different methods: Galerkin tests against the basis itself, collocation tests at specific points, and least squares minimizes the residual norm. All reduce to the same computational pattern: generate Bellman targets, fit a function approximator, repeat.

Convergence depends on how the projection interacts with the Bellman operator. The contraction property of the Bellman operator reappears here: whether projected iteration converges depends on whether the projection preserves this

contraction or whether we work in a norm compatible with the operator. For monotone projections (piecewise linear interpolation, state aggregation), the composition $\mathbf{P}\mathbf{L}$ inherits the contraction property in the sup norm and iteration converges. For non-monotone projections like Galerkin, convergence requires matching the weighting to the stationary distribution, which holds in on-policy settings. The Bellman optimality case remains theoretically incomplete.

Throughout this chapter, we assumed access to the transition model: computing $\mathbf{L}v(s)$ requires summing over all next states weighted by transition probabilities. In practice, we often have only a simulator or observed trajectories, not an explicit model. The next chapter addresses this gap. Monte Carlo methods estimate expectations from sampled transitions, replacing exact Bellman operator evaluations with sample averages. This connects the projection framework developed here to the simulation-based algorithms used in reinforcement learning.

6.2.10 Self-checks

6.3 Simulation-Based Methods

6.3.1 Monte Carlo Integration in Approximate Dynamic Programming

The projection methods from the previous chapter showed how to transform the infinite-dimensional fixed-point problem $Lv = v$ into a finite-dimensional one by choosing basis functions $\{\varphi_i\}$ and imposing conditions that make the residual $R(s) = Lv(s) - v(s)$ small. Different projection conditions (Galerkin orthogonality, collocation at points, least squares minimization) yield different finite-dimensional systems to solve.

However, we left unresolved the question of how to evaluate the Bellman operator itself. Applying L at any state requires computing an expectation:

$$(Lv)(s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \int v(s') p(ds'|s, a) \right\} \quad (524)$$

For discrete state spaces with a manageable number of states, this expectation is a finite sum we can compute exactly. For continuous or high-dimensional state spaces, we need numerical integration. The projection methods framework is compatible with any quadrature scheme, but leaves the choice of integration method unspecified.

This chapter addresses the **integration subproblem** that arises at two levels in approximate dynamic programming. First, we must evaluate the transition expectation $\int v(s') p(ds'|s, a)$ within the Bellman operator itself. Second, when using projection methods like Galerkin or least squares, we encounter outer integrations for enforcing orthogonality conditions or minimizing residuals over distributions of states. Both require numerical approximation in continuous or high-dimensional spaces.

We begin by examining deterministic numerical integration methods: quadrature rules that approximate integrals by evaluating integrands at carefully chosen points with associated weights. We discuss how to coordinate the choice of quadrature with the choice of basis functions to balance approximation accuracy and computational cost. Then we turn to **Monte Carlo integration**, which approximates expectations using random samples rather than deterministic quadrature points. This shift from deterministic to stochastic integration is what brings us into machine learning territory. When we replace exact transition probabilities with samples drawn from simulations or real interactions, projection methods combined with Monte Carlo integration become what the operations research community calls **simulation-based approximate dynamic programming** and what the machine learning community calls **reinforcement learning**. By relying on samples rather than explicit probability functions, we move from model-based planning to data-driven learning.

Evaluating the Bellman Operator with Numerical Quadrature Before turning to Monte Carlo methods, we examine the structure of the numerical integration problem. When we apply the Bellman operator to an approximate value function $\hat{v}(s; \theta) = \sum_i \theta_i \varphi_i(s)$, we must evaluate integrals of the form:

$$\int \hat{v}(s'; \theta) p(s'|s, a) ds' = \int \left(\sum_i \theta_i \varphi_i(s') \right) p(s'|s, a) ds' = \sum_i \theta_i \int \varphi_i(s') p(s'|s, a) ds' \quad (525)$$

This shows two independent approximations:

1. **Value function approximation:** We represent v using basis functions $\{\varphi_i\}$ and coefficients θ
2. **Quadrature approximation:** We approximate each integral $\int \varphi_i(s') p(s'|s, a) ds'$ numerically

These choices are independent but should be coordinated. To see why, consider what happens in projection methods when we iterate $\hat{v}^{(k+1)} = \mathbf{P}\mathbf{L}\hat{v}^{(k)}$. In practice, we cannot evaluate $\mathbf{L}$ exactly due to the integrals, so we compute $\hat{v}^{(k+1)} = \mathbf{P}\tilde{\mathbf{L}}\hat{v}^{(k)}$ instead, where $\tilde{\mathbf{L}}$ denotes the Bellman operator with numerical quadrature.

The error in a single iteration can be bounded by the triangle inequality. Let v denote the true fixed point $v = \mathbf{L}v$. Then:

$$\begin{aligned} \|v - \hat{v}^{(k+1)}\| &= \|v - \mathbf{P}\tilde{\mathbf{L}}\hat{v}^{(k)}\| \\ &\leq \|v - \mathbf{P}\mathbf{L}v\| + \|\mathbf{P}\mathbf{L}v - \mathbf{P}\mathbf{L}\hat{v}^{(k)}\| + \|\mathbf{P}\mathbf{L}\hat{v}^{(k)} - \mathbf{P}\tilde{\mathbf{L}}\hat{v}^{(k)}\| \\ &= \underbrace{\|v - \mathbf{P}\mathbf{L}v\|}_{\text{best approximation error}} + \underbrace{\|\mathbf{P}\mathbf{L}v - \mathbf{P}\mathbf{L}\hat{v}^{(k)}\|}_{\text{contraction of iterate error}} + \underbrace{\|\mathbf{P}\mathbf{L}\hat{v}^{(k)} - \mathbf{P}\tilde{\mathbf{L}}\hat{v}^{(k)}\|}_{\text{quadrature error}} \end{aligned} \quad (526)$$

The first term is the best we can do with our basis (how well $\mathbf{P}$ approximates the true solution). The second term decreases with iterations when $\mathbf{P}\mathbf{L}$ is a contraction. The third term is the error from replacing exact integrals with quadrature, and it does not vanish as iterations proceed.

To make this concrete, consider evaluating $(\mathbf{L}\hat{v})(s)$ for our current approximation $\hat{v}(s; \theta) = \sum_i \theta_i \varphi_i(s)$. We want:

$$(\mathbf{L}\hat{v})(s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_i \theta_i \int \varphi_i(s') p(s'|s, a) ds' \right\} \quad (527)$$

But we compute instead:

$$(\tilde{\mathbf{L}}\hat{v})(s) = \max_{a \in \mathcal{A}_s} \left\{ r(s, a) + \gamma \sum_i \theta_i \sum_j w_j \varphi_i(s'_j) \right\} \quad (528)$$

where $\{(s'_j, w_j)\}$ are quadrature nodes and weights. If the quadrature error $\|(\mathbf{L}\hat{v})(s) - (\tilde{\mathbf{L}}\hat{v})(s)\|$ is large relative to the basis approximation quality, we cannot exploit the expressive power of the basis. For instance, degree-10

Chebyshev polynomials represent smooth functions to $O(10^{-8})$ accuracy, but combined with rectangle-rule quadrature (error $O(h^2) \approx 10^{-2}$), the quadrature term dominates the error bound. We pay the cost of storing and manipulating 10 coefficients but achieve only $O(h^2)$ convergence in the quadrature mesh size.

This echoes the coordination principle from continuous optimal control (Chapter on trajectory optimization): when transcribing a continuous-time problem, we use the same quadrature nodes for both the running cost integral and the dynamics integral. There, coordination ensures that “where we pay” aligns with “where we enforce” the dynamics. Here, coordination ensures that integration accuracy matches approximation accuracy. Both are instances of balancing multiple sources of error in numerical methods.

Standard basis-quadrature pairings achieve this balance:

- Piecewise constant or linear elements with midpoint or trapezoidal rules
- Chebyshev polynomials with Gauss-Chebyshev quadrature
- Legendre polynomials with Gauss-Legendre quadrature
- Hermite polynomials with Gauss-Hermite quadrature (for Gaussian shocks)

To make this concrete, we examine what these pairings look like in practice for collocation and Galerkin projection.

Orthogonal Collocation with Chebyshev Polynomials Consider approximating the value function using Chebyshev polynomials of degree $n - 1$:

$$\hat{v}(s; \theta) = \sum_{i=0}^{n-1} \theta_i T_i(s) \quad (529)$$

For orthogonal collocation, we place collocation points at the zeros of $T_n(s)$, denoted $\{s_j\}_{j=1}^n$. At each collocation point, we require the Bellman equation to hold exactly:

$$\hat{v}(s_j; \theta) = \max_{a \in \mathcal{A}} \left\{ r(s_j, a) + \gamma \int \hat{v}(s'; \theta) p(ds' | s_j, a) \right\} \quad (530)$$

The integral on the right must be approximated using quadrature. With Chebyshev-Gauss quadrature using the same nodes $\{s_k\}_{k=1}^n$ and weights $\{w_k\}_{k=1}^n$, this becomes:

$$\hat{v}(s_j; \theta) = \max_{a \in \mathcal{A}} \left\{ r(s_j, a) + \gamma \sum_{k=1}^n w_k \hat{v}(s_k; \theta) p(s_k | s_j, a) \right\} \quad (531)$$

Substituting the basis representation $\hat{v}(s_k; \theta) = \sum_{i=0}^{n-1} \theta_i T_i(s_k)$:

$$\sum_{i=0}^{n-1} \theta_i T_i(s_j) = \max_{a \in \mathcal{A}} \left\{ r(s_j, a) + \gamma \sum_{k=1}^n w_k p(s_k | s_j, a) \sum_{i=0}^{n-1} \theta_i T_i(s_k) \right\} \quad (532)$$

Rearranging:

$$\sum_{i=0}^{n-1} \theta_i T_i(s_j) = \max_{a \in \mathcal{A}} \left\{ r(s_j, a) + \gamma \sum_{i=0}^{n-1} \theta_i \underbrace{\sum_{k=1}^n w_k T_i(s_k) p(s_k | s_j, a)}_{B_{ji}^a} \right\} \quad (533)$$

This yields a system of n nonlinear equations (one per collocation point):

$$\sum_{i=0}^{n-1} T_i(s_j) \theta_i = \max_{a \in \mathcal{A}} \left\{ r(s_j, a) + \gamma \sum_{i=0}^{n-1} B_{ji}^a \theta_i \right\}, \quad j = 1, \dots, n \quad (534)$$

The matrix elements B_{ji}^a can be precomputed once the quadrature nodes, weights, and transition probabilities are known. Solving this system gives the coefficient vector θ .

Galerkin Projection with Hermite Polynomials For a problem with Gaussian shocks, we might use Hermite polynomials $\{H_i(s)\}_{i=0}^{n-1}$ weighted by the Gaussian density $\phi(s)$. The Galerkin condition requires:

$$\int (\mathbf{L}\hat{v}(s; \theta) - \hat{v}(s; \theta)) H_j(s) \phi(s) ds = 0, \quad j = 0, \dots, n-1 \quad (535)$$

Expanding the Bellman operator:

$$\int \left[\max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \int \hat{v}(s'; \theta) p(ds' | s, a) \right\} - \hat{v}(s; \theta) \right] H_j(s) \phi(s) ds = 0 \quad (536)$$

We approximate this outer integral using Gauss-Hermite quadrature with nodes $\{s_\ell\}_{\ell=1}^m$ and weights $\{w_\ell\}_{\ell=1}^m$:

$$\sum_{\ell=1}^m w_\ell \left[\max_{a \in \mathcal{A}} \left\{ r(s_\ell, a) + \gamma \int \hat{v}(s'; \theta) p(ds' | s_\ell, a) \right\} - \hat{v}(s_\ell; \theta) \right] H_j(s_\ell) = 0 \quad (537)$$

The inner integral (transition expectation) is also approximated using Gauss-Hermite quadrature:

$$\int \hat{v}(s'; \theta) p(ds' | s_\ell, a) \approx \sum_{k=1}^m w_k \hat{v}(s_k; \theta) p(s_k | s_\ell, a) \quad (538)$$

Substituting the basis representation and collecting terms:

$$\sum_{\ell=1}^m w_{\ell} H_j(s_{\ell}) \max_{a \in \mathcal{A}} \left\{ r(s_{\ell}, a) + \gamma \sum_{i=0}^{n-1} \theta_i \sum_{k=1}^m w_k H_i(s_k) p(s_k | s_{\ell}, a) \right\} = \sum_{\ell=1}^m w_{\ell} H_j(s_{\ell}) \sum_{i=0}^{n-1} \theta_i H_i(s_{\ell}) \quad (539)$$

This gives n nonlinear equations in n unknowns. The right-hand side simplifies using orthogonality: when using the same quadrature nodes for projection and integration, $\sum_{\ell=1}^m w_{\ell} H_j(s_{\ell}) H_i(s_{\ell}) = \delta_{ij} \|H_j\|^2$.

In both cases, the basis-quadrature pairing ensures that:

1. The quadrature nodes appear in both the transition expectation and the outer projection
2. The quadrature accuracy matches the polynomial approximation order
3. Precomputed matrices capture the dynamics, making iterations efficient

Monte Carlo Integration Deterministic quadrature rules work well when the state space has low dimension, the transition density is smooth, and we can evaluate it cheaply at arbitrary points. In many stochastic control problems none of these conditions truly hold. The state may be high dimensional, the dynamics may be given by a simulator rather than an explicit density, and the cost of each call to the model may be large. In that regime, deterministic quadrature becomes brittle. Monte Carlo methods offer a different way to approximate expectations, one that relies only on the ability to **sample** from the relevant distributions.

Monte Carlo as randomized quadrature Consider a single expectation of the form

$$J = \int f(x) p(dx) = E[f(X)], \quad (540)$$

where $X \sim p$ and f is some integrable function. Monte Carlo integration approximates J by drawing independent samples $X^{(1)}, \dots, X^{(N)} \sim p$ and forming the sample average

$$\hat{J}_N \equiv \frac{1}{N} \sum_{n=1}^N f(X^{(n)}). \quad (541)$$

This estimator has two basic properties that will matter throughout this chapter.

First, it is **unbiased**:

$$E[\hat{J}_N] = E \left[\frac{1}{N} \sum_{n=1}^N f(X^{(n)}) \right] = \frac{1}{N} \sum_{n=1}^N E[f(X^{(n)})] = E[f(X)] = J. \quad (542)$$

Second, its variance scales as $1/N$. If we write

$$\sigma^2 \equiv \text{Var}[f(X)], \quad (543)$$

then independence of the samples gives

$$\text{Var}(\hat{J}_N) = \frac{1}{N^2} \sum_{n=1}^N \text{Var}(f(X^{(n)})) = \frac{\sigma^2}{N}. \quad (544)$$

The central limit theorem then says that for large N ,

$$\sqrt{N}(\hat{J}_N - J) \implies \mathcal{N}(0, \sigma^2), \quad (545)$$

so the integration error decays at rate $O(N^{-1/2})$. This rate is slow compared to high-order quadrature in low dimension, but it has one crucial advantage: it does not explicitly depend on the dimension of x . Monte Carlo integration pays in variance, not in an exponential growth in the number of nodes.

It is often helpful to view Monte Carlo as **randomized quadrature**. A deterministic quadrature rule selects nodes x_j and weights w_j in advance and computes

$$\sum_j w_j f(x_j). \quad (546)$$

Monte Carlo can be written in the same form: if we draw $X^{(n)}$ from density p , the sample average

$$\hat{J}_N = \frac{1}{N} \sum_{n=1}^N f(X^{(n)}) \quad (547)$$

is just a quadrature rule with random nodes and equal weights. More advanced Monte Carlo schemes, such as importance sampling, change both the sampling distribution and the weights, but the basic idea remains the same.

Monte Carlo evaluation of the Bellman operator We now apply this to the Bellman operator. For a fixed value function v and a given state-action pair (s, a) , the transition part of the Bellman operator is

$$\int v(s') p(ds' | s, a) = E[v(S') | S = s, A = a]. \quad (548)$$

If we can simulate next states $S'^{(1)}, \dots, S'^{(N)}$ from the transition kernel $p(\cdot | s, a)$, either by calling a simulator or by interacting with the environment, we can approximate this expectation by

$$\hat{E}_N[v(S') | s, a] \equiv \frac{1}{N} \sum_{n=1}^N v(S'^{(n)}). \quad (549)$$

If the immediate reward is also random, say

$$r = r(S, A, S') \quad \text{with} \quad (S', r) \sim p(\cdot \mid s, a), \quad (550)$$

we can approximate the full one-step return

$$E[r + \gamma v(S') \mid S = s, A = a] \quad (551)$$

by

$$\hat{G}_N(s, a) \equiv \frac{1}{N} \sum_{n=1}^N [r^{(n)} + \gamma v(S'^{(n)})], \quad (552)$$

where $(r^{(n)}, S'^{(n)})$ are independent samples given (s, a) . Again, this is an unbiased estimator of the Bellman expectation for fixed v .

Plugging this into the Bellman operator gives a **Monte Carlo Bellman operator**:

$$(\hat{\mathbf{L}}_N v)(s) \equiv \max_{a \in \mathcal{A}_s} \left\{ \hat{G}_N(s, a) \right\}. \quad (553)$$

The expectation inside the braces is now replaced by a random sample average. In model-based settings, we implement this by simulating many next states for each candidate action a at state s . In model-free settings, we obtain samples from real interactions and re-use them to estimate the expectation.

At this stage nothing about approximation or projection has entered yet. For a fixed value function v , Monte Carlo provides unbiased, noisy evaluations of $(\mathbf{L}v)(s)$. The approximation question arises once we couple this stochastic evaluation with basis functions and projections.

Sampling the outer expectations Projection methods introduce a second layer of integration. In Galerkin and least squares schemes, we choose a distribution μ over states (and sometimes actions) and enforce conditions of the form

$$\int R(s; \theta) p_i(s) \mu(ds) = 0 \quad \text{or} \quad \int R(s; \theta)^2 \mu(ds) \text{ is minimized.} \quad (554)$$

Here $R(s; \theta)$ is the residual function, such as

$$R(s; \theta) = (\mathbf{L}\hat{v})(s; \theta) - \hat{v}(s; \theta), \quad (555)$$

and p_i are test functions or derivatives of the residual with respect to parameters.

Note a subtle but important shift in perspective from the previous chapter. There, the weight function $w(s)$ in the inner product $\langle f, g \rangle_w = \int f(s)g(s)w(s)ds$ could be any positive weight function (not necessarily normalized). For Monte Carlo integration, however, we need a probability distribution we can sample

from. We write μ for this sampling distribution and express projection conditions as integrals with respect to μ : $\int R(s; \theta) p_i(s) \mu(ds)$. If a problem was originally formulated with an unnormalized weight $w(s)$, we must either (i) normalize it to define μ , or (ii) use importance sampling with a different μ and reweight samples by $w(s)/\mu(s)$. In reinforcement learning, μ is typically the empirical state visitation distribution from collected trajectories.

These outer integrals over s are generally not easier to compute than the inner transition expectations. Monte Carlo gives a way to approximate them as well. If we can draw states $S^{(1)}, \dots, S^{(M)} \sim \mu$, we can approximate, for example, the Galerkin orthogonality conditions by

$$\int R(s; \theta) p_i(s) \mu(ds) \approx \frac{1}{M} \sum_{m=1}^M R(S^{(m)}; \theta) p_i(S^{(m)}). \quad (556)$$

Similarly, a least squares objective

$$\int R(s; \theta)^2 \mu(ds) \quad (557)$$

is approximated by the empirical risk

$$\frac{1}{M} \sum_{m=1}^M R(S^{(m)}; \theta)^2. \quad (558)$$

If we now substitute Monte Carlo estimates for both the inner transition expectations and the outer projection conditions, we obtain fully simulation-based schemes. We no longer need explicit access to the transition kernel $p(\cdot | s, a)$ or the state distribution μ . It is enough to be able to **sample** from them, either through a simulator or by interacting with the real system.

Simulation-Based Projection Methods Monte Carlo integration replaces both levels of integration in approximate dynamic programming. The transition expectation in the Bellman operator becomes $\frac{1}{N} \sum_{n=1}^N v(S'^{(n)})$, and the outer integrals for projection become empirical averages over sampled states. Writing $\mathbf{P}_M$ for projection using M state samples and $\hat{\mathbf{L}}_N$ for the Monte Carlo Bellman operator with N transition samples, the iteration becomes

$$\hat{v}^{(k+1)} = \mathbf{P}_M \hat{\mathbf{L}}_N \hat{v}^{(k)}. \quad (559)$$

In the typical reinforcement learning setting, $N = 1$: each observed transition (s_i, a_i, r_i, s'_i) provides exactly one sample of the next state. This means we work with high-variance single-sample estimates $r_i + \gamma v(s'_i)$ rather than averaged returns. Variance reduction comes from aggregating information across different transitions through the function approximator and from collecting long trajectories with many transitions. We examine this single-sample constraint in detail after introducing Q-functions below.

Unlike deterministic quadrature, which introduces a fixed bias at each iteration, Monte Carlo introduces random error with zero mean but nonzero variance.

However, combining Monte Carlo with the maximization in the Bellman operator creates a systematic problem: while the estimate of the expected return for any individual action is unbiased, taking the maximum over these noisy estimates introduces upward bias. This overestimation compounds through value iteration and degrades the resulting policies. We address this challenge in the [next chapter on fitted Q-iteration](#).

Interactive sample-size diagnostic Before moving across the figure, predict what multiplying N by four should do to the root mean squared error. Hover over a point to compare the empirical result with the $1/\sqrt{N}$ prediction.

Amortizing Action Selection via Q-Functions Monte Carlo integration enables model-free approximate dynamic programming: we no longer need explicit transition probabilities $p(s'|s, a)$, only the ability to sample next states. However, one computational challenge remains. The standard formulation of an optimal decision rule is

$$\pi(s) = \arg \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \int v(s') p(ds'|s, a) \right\}. \quad (560)$$

Even with an optimal value function v^* in hand, extracting an action at state s requires evaluating the transition expectation for each candidate action. In the model-free setting, this means we must draw Monte Carlo samples from each action's transition distribution every time we select an action. This repeated sampling “at inference time” wastes computation, especially when the same state is visited multiple times.

We can amortize this computation by working at a different level of representation. Define the **state-action value function** (or **Q-function**)

$$q(s, a) = r(s, a) + \gamma \int v(s') p(ds'|s, a). \quad (561)$$

The Q-function caches the result of evaluating each action at each state. Once we have q , action selection reduces to a finite maximization:

$$\pi(s) = \arg \max_{a \in \mathcal{A}(s)} q(s, a). \quad (562)$$

No integration appears in this expression. The transition expectation has been precomputed and stored in q itself.

The optimal Q-function q^* satisfies its own Bellman equation. Substituting the definition of $v^*(s) = \max_a q^*(s, a)$ into the expression for q :

$$q^*(s, a) = r(s, a) + \gamma \int p(ds'|s, a) \max_{a' \in \mathcal{A}(s')} q^*(s', a'). \quad (563)$$

This defines a Bellman operator on Q-functions:

$$(\mathcal{L}q)(s, a) = r(s, a) + \gamma \int p(ds'|s, a) \max_{a' \in \mathcal{A}(s')} q(s', a'). \quad (564)$$

Like the Bellman operator on value functions, $\mathbf{L}$ is a γ -contraction in the sup-norm, guaranteeing a unique fixed point q^* . We can thus apply the same projection and Monte Carlo techniques developed for value functions to Q-functions. The computational cost shifts from action selection (repeated sampling at decision time) to training (evaluating expectations during the iterations of approximate value iteration). Once q is learned, acting is cheap.

The algorithm above evaluates the expectation $\int p(ds'|s, a) \max_{a'} q(s', a'; \theta_n)$ at each state-action pair. In principle, we could approximate this integral using many Monte Carlo samples: draw N next states from $p(\cdot|s, a)$ and average $\frac{1}{N} \sum_{i=1}^N \max_{a'} q(s'_i, a')$ to reduce variance. This is standard Monte Carlo integration applied to the Bellman operator.

The Single-Sample Paradigm Most reinforcement learning algorithms do not follow this pattern. Instead, they work with datasets of **transition tuples** $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^M$, where each tuple records a single observed next state s'_i from executing action a_i in state s_i . Each transition provides exactly one sample of the dynamics: the target becomes $y_i = r_i + \gamma \max_{a'} q(s'_i, a')$ using only the single observed s'_i .

This design choice has two origins. First, it reflects genuine constraints in some problem settings. A robot executing action a in state s observes one outcome s' and cannot rewind to the exact same state to sample alternative continuations. Real physical systems, biological experiments, and costly interactions (clinical trials, manufacturing processes) provide one sample per executed action. However, many domains where RL is applied do not face this constraint. Game emulators support state saving and restoration, physics simulators can reset to arbitrary configurations, and software environments can be cloned cheaply. The single-sample pattern persists in these settings not from necessity but from algorithmic convention.

Second, and perhaps more fundamentally, the single-sample paradigm emerged from reinforcement learning’s roots in computational neuroscience and models of biological intelligence. Early work viewed RL as a model of animal learning, where organisms experience one consequence per action and must learn from streaming sensory data. This perspective shaped the field’s algorithmic agenda: develop methods that learn from single sequential experiences, as biological agents do. Temporal difference learning, Q-learning, and actor-critic methods all follow this template.

These origins matter because they established design choices that persist even when the original constraints do not apply. In many practical applications, we have fast simulators that support arbitrary state initialization (e.g., physics engines for robotics, game emulators for Atari). We could collect multiple next-state samples per (s, a) pair to reduce variance in target computation. However, most algorithms do not exploit this capability. The transition tuple (s_i, a_i, r_i, s'_i) with $N = 1$ remains the standard data structure, and methods compensate for high variance through other means: larger replay buffers, function approximation that pools information across samples, multi-step returns,

and careful optimization strategies.

This creates an asymmetry in how algorithms use Monte Carlo integration. For the outer expectation (sampling which states to evaluate at), algorithms freely adjust sample size: offline methods reuse entire datasets, replay buffers store thousands of transitions, online methods process one sample per step. For the inner expectation (sampling next states to evaluate the Bellman operator at a given (s, a)), the overwhelming majority of practical methods use $N = 1$ regardless of whether the environment permits larger samples.

The single-sample paradigm shapes three aspects of algorithm design:

1. **Data structure:** Algorithms store and manipulate tuples $\{(s_i, a_i, r_i, s'_i)\}$, not state-action pairs with multiple next-state samples
2. **Variance reduction:** Comes from aggregation across different samples through function approximation and from trajectory length, not from repeated sampling at fixed (s, a)
3. **Target computation:** Each transition yields exactly one target $y_i = r_i + \gamma \max_{a'} q(s'_i, a')$ evaluated at the single observed s'_i

The algorithms that follow in the next chapters all adopt this single-sample structure. Understanding its origins clarifies when the design choice might be revisited: in domains with cheap, resettable simulators and high target variance, averaging multiple next-state samples per (s, a) pair offers a direct variance reduction mechanism that is rarely exploited in current practice.

Overestimation Bias and Mitigation Strategies Monte Carlo integration provides unbiased estimates of individual expectations, but when we apply the Bellman operator’s maximization to these noisy estimates, a systematic problem arises: taking the maximum over noisy values creates upward bias. This overestimation compounds through iterative algorithms and can severely degrade the quality of learned policies. This section examines the sources of this bias and presents two approaches for mitigating it.

Sources of Overestimation Bias When we apply Monte Carlo integration to the Bellman operator, each individual expectation is unbiased. The problem arises when we maximize over these noisy estimates. To make this precise, consider a given state-action pair (s, a) and value function v . Define the true continuation value

$$\mu(s, a) \equiv \int v(s') p(ds' | s, a), \quad (565)$$

and its Monte Carlo approximation with N samples:

$$\hat{\mu}_N(s, a) \equiv \frac{1}{N} \sum_{i=1}^N v(s'_i), \quad s'_i \sim p(\cdot | s, a). \quad (566)$$

Each estimator is unbiased: $E[\hat{\mu}_N(s, a)] = \mu(s, a)$. The Monte Carlo Bellman operator is

$$(\hat{\mathbf{L}}_N v)(s) = \max_{a \in \mathcal{A}_s} \{r(s, a) + \gamma \hat{\mu}_N(s, a)\}. \quad (567)$$

While each $\hat{\mu}_N(s, a)$ is unbiased, the maximization introduces systematic overestimation. To see why, let a^* be the truly optimal action. The maximum of any collection is at least as large as any particular element:

$$E[\max_a \{r(s, a) + \gamma \hat{\mu}_N(s, a)\}] \geq E[r(s, a^*) + \gamma \hat{\mu}_N(s, a^*)] = r(s, a^*) + \gamma \mu(s, a^*) = (\mathbf{L}v)(s). \quad (568)$$

The inequality is strict whenever multiple actions have nonzero variance. The maximization selects whichever action happens to receive a positive noise realization, and that inflated estimate contributes to the target value. This is Jensen's inequality for the max operator: $E[\max_a Y_a] \geq \max_a E[Y_a]$ for random variables $\{Y_a\}$, since the max is convex. Even though we start with unbiased estimators, taking the maximum breaks unbiasedness.

The next diagnostic holds every true action value at zero. Use the legend to isolate an estimator and hover over the points. The maximum estimator becomes increasingly optimistic as the number of actions grows, while independent evaluation removes that effect in expectation.

Operationally, this means that repeatedly sampling fresh states and taking the max will not, on average, converge to the true value but systematically land above it. Unlike noise that averages out, this bias persists no matter how many samples we draw. Worse, it compounds across iterations as overestimates feed into future target computations.

Remark 6.1 (Connection to deterministic policies). *This inequality also underlies why deterministic policies are optimal in MDPs (Theorem Theorem 5.4): $\max_a w(a) \geq \sum_a q(a)w(a)$ shows randomization cannot improve expected returns. The same mathematical principle appears in two contexts: (1) deterministic policies suffice, (2) maximization over noisy estimates creates upward bias.*

Monte Carlo value iteration applies $v_{k+1} = \hat{\mathbf{L}}_N v_k$ repeatedly. The overestimation bias does not stay confined to a single iteration: it accumulates through the recursion. At iteration 2, we compute Monte Carlo estimates $\hat{\mu}_N(s, a) = \frac{1}{N} \sum_{i=1}^N v_1(s'_i)$, but v_1 is already biased upward from iteration 1. Averaging an overestimated function and then maximizing over noisy estimates compounds the bias: $E[v_k] \geq E[v_{k-1}] \geq \dots \geq v^*$. Without correction, this feedback loop can produce severely inflated value estimates that yield poor policies.

Learning the Bias Correction One approach, developed by Keane and Wolpin [Keane and Wolpin \[1994\]](#) in the context of dynamic discrete choice models, treats the bias itself as a quantity to be learned and subtracted. For a

given value function v and Monte Carlo sample size N , define the bias at state s as

$$\delta(s) = E[\widehat{\mathbb{L}}_N v(s)] - (\mathbb{L}v)(s) \geq 0. \quad (569)$$

While we cannot compute $\delta(s)$ directly (we lack both the expectation and the exact Bellman application), the bias has structure. It depends on observable quantities: the number of actions $|\mathcal{A}_s|$, the sample size N , and the spread of action values. When one action dominates, $\delta(s)$ is small. When several actions have similar values, noise is more likely to flip the maximizer, increasing $\delta(s)$.

Rather than deriving $\delta(s)$ analytically, Keane and Wolpin proposed learning it empirically. The strategy follows a “simulate on a subset, interpolate everywhere” template common in econometric dynamic programming. At a carefully chosen set of states, we run both high-fidelity simulation (many samples or exact integration) and the low-fidelity N -sample estimate used in value iteration. The gap between these estimates provides training data for the bias. We then fit a regression model g_η that predicts $\delta(s)$ from features of the state and action-value distribution. During value iteration, we subtract the predicted bias from the raw Monte Carlo maximum.

Useful features for predicting the bias include the spread of action values (from a separate high-fidelity simulation), the number of actions, and gaps to the best action. These are cheap to compute and track regimes where maximization bias is large. The procedure can be summarized as:

The regression model g_η is trained offline by comparing high and low-fidelity simulations at a grid of states, then applied during each value iteration step. This approach has been influential in econometrics for structural estimation problems. However, it has seen limited adoption in reinforcement learning. The computational overhead is substantial: it requires high-fidelity simulation at training states, careful feature engineering, and maintaining the regression model throughout learning. More critically, the circular dependency between the bias estimate and the value function can amplify errors if g_η is misspecified.

Analytical Bias Correction Lee et al. [2013], Lee and Powell [2019] developed an alternative that derives the bias analytically using extreme value theory rather than learning it empirically. Their analysis considers the distributional structure of the noise in Monte Carlo estimates. When rewards are Gaussian with variance σ^2 , the bias in $\max_a \widehat{R}(s, a)$ can be computed from extreme value theory.

For a single-state MDP where all randomness comes from stochastic rewards, suppose that for each action a , the observed reward is $\widehat{R}(s, a) = \mu_a + \varepsilon_a$ where $\varepsilon_a \sim \mathcal{N}(0, \sigma^2)$. When we form the target $\max_a \widehat{R}(s, a)$, we are taking the maximum of $M = |\mathcal{A}(s)|$ independent Gaussians. After appropriate centering and scaling, this maximum converges in distribution to a Gumbel random variable. The expected bias can be computed as:

$$E \left[\max_a \varepsilon_a \right] \approx \sigma \left(\frac{\xi}{b_M} + b_M \right) \quad (570)$$

where $b_M = \sqrt{2 \log M - \log \log M - \log 4\pi}$ and $\xi \approx 0.5772$ is the Euler-Mascheroni constant. This quantity is positive and grows with the number of actions M . The bias scales with the noise level σ and with the action space size.

The corrected target becomes:

$$\tilde{y}_i = r_i + \gamma \max_{a'} q(s'_i, a'; \theta^{(n)}) - B(s_i, a_i) \quad (571)$$

where the bias correction term is:

$$B(s, a) = \left(\frac{\xi}{b_{|A(s)|}} + b_{|A(s)|} \right) \hat{\sigma}(s, a) \quad (572)$$

Here $\hat{\sigma}(s, a) = \sqrt{\widehat{\text{Var}}[R(s, a)]}$ is the estimated standard deviation of the reward at state-action pair (s, a) . This can be computed from multiple observed reward samples at the same (s, a) pair if available.

This approach is exact for single-state MDPs where Q-values equal expected rewards and all bias comes from reward noise. For general multi-state MDPs, the situation is more complex. The bias arises from the max operator over Q-values at the next state s' , not from reward stochasticity at the current state. Lee and Powell's multistate extension addresses this by introducing a separate correction term B^T for transition stochasticity, estimated empirically by averaging $\max_{a'} q(s', a')$ over multiple sampled next states from the same (s, a) pair, rather than using an analytical formula. The single-state analytical correction remains valuable for bandit problems and as a first-order approximation when reward variance dominates, but does not directly address max-operator bias from function approximation error in the Q-values themselves. The correction is optimal specifically for Gaussian errors. If the true errors have heavier tails, the correction may be insufficient.

Lee-Powell's analytical formula can be viewed as deriving what Keane-Wolpin learned empirically, replacing the regression model with a closed-form expression from extreme value theory. The trade-off is generality versus computational cost: Keane-Wolpin adapts to any noise structure but requires expensive high-fidelity simulations at training states; Lee-Powell assumes Gaussian noise but evaluates instantly once $\hat{\sigma}$ is estimated. Both subtract a correction term from targets while preserving squared error loss.

An alternative to explicit bias correction is to replace the hard max operator with a soft maximum. The [regularized MDP chapter](#) discusses smooth Bellman operators that use logsumexp or Gaussian uncertainty-weighted aggregation to avoid the discontinuity and overestimation inherent in taking a hard maximum. These approaches modify the Bellman operator itself rather than correcting its bias after the fact, preserving squared error loss while constructing smoother targets. Another approach, examined in the [next chapter](#), changes the loss function itself to match the noise structure in TD targets.

Decoupling Selection and Evaluation An alternative approach modifies the estimator itself to break the coupling that creates bias. In the standard Monte Carlo update $\max_a \{r(s, a) + \gamma \hat{\mu}_N(s, a)\}$, the same noisy estimate both selects which action looks best and provides the value assigned to that action. To see the problem, decompose the estimator into its mean and noise:

$$\hat{\mu}_N(s, a) = \mu(s, a) + \varepsilon_a, \quad (573)$$

where ε_a is zero-mean noise. Whichever action happens to have the largest ε_a gets selected, and that same positive noise inflates the target value:

$$Y = r(s, a^*) + \gamma \mu(s, a^*) + \gamma \varepsilon_{a^*}, \quad (574)$$

where $a^* = \arg \max_a \{r(s, a) + \gamma \mu(s, a) + \gamma \varepsilon_a\}$. This coupling (using the same random variable ε_{a^*} for both selection and evaluation) produces $E[Y] \geq \max_a \{r(s, a) + \gamma \mu(s, a)\}$.

Double Q-learning [Van Hasselt et al. \[2016\]](#) breaks this coupling by maintaining two independent Monte Carlo estimates. Conceptually, for each action a at state s , we would draw two separate sets of samples from $p(\cdot|s, a)$:

$$\hat{\mu}_N^{(1)}(s, a) = \mu(s, a) + \varepsilon_a^{(1)}, \quad \hat{\mu}_N^{(2)}(s, a) = \mu(s, a) + \varepsilon_a^{(2)}, \quad (575)$$

where $\varepsilon_a^{(1)}$ and $\varepsilon_a^{(2)}$ are **independent zero-mean noise terms**. We use the first estimate to select the action but the second to evaluate it:

$$a^* = \arg \max_a \left\{ r(s, a) + \gamma \hat{\mu}_N^{(1)}(s, a) \right\}, \quad Y = r(s, a^*) + \gamma \hat{\mu}_N^{(2)}(s, a^*). \quad (576)$$

Note that a^* is a random variable. It is the result of maximizing over noisy estimates $\hat{\mu}_N^{(1)}(s, a)$, and therefore depends on the noise $\varepsilon^{(1)}$. The target value Y uses the second estimator $\hat{\mu}_N^{(2)}(s, a^*)$, evaluating it at the randomly selected action a^* . Expanding the target value to make the dependencies explicit:

$$Y = r(s, a^*) + \gamma \hat{\mu}_N^{(2)}(s, a^*) = r(s, a^*) + \gamma \mu(s, a^*) + \gamma \varepsilon_{a^*}^{(2)}, \quad (577)$$

where a^* itself depends on $\varepsilon^{(1)}$. To see why this eliminates evaluation bias, write the expectation as a nested expectation:

$$E[Y] = E_{\varepsilon^{(1)}} \left[E_{\varepsilon^{(2)}} \left[r(s, a^*) + \gamma \mu(s, a^*) + \gamma \varepsilon_{a^*}^{(2)} \mid a^* \right] \right]. \quad (578)$$

The inner expectation, conditioned on the selected action a^* , equals:

$$E_{\varepsilon^{(2)}} \left[r(s, a^*) + \gamma \mu(s, a^*) + \gamma \varepsilon_{a^*}^{(2)} \mid a^* \right] = r(s, a^*) + \gamma \mu(s, a^*), \quad (579)$$

because $E[\varepsilon_{a^*}^{(2)} \mid a^*] = 0$. Why does this conditional expectation vanish? Because a^* is a random variable determined entirely by $\varepsilon^{(1)}$ (through the argmax operation), while $\varepsilon^{(2)}$ is independent of $\varepsilon^{(1)}$. Therefore, $\varepsilon^{(2)}$ is independent of

a^* . By the fundamental property of independence, for any random variables X and Y where $X \perp Y$, we have $E[X | Y] = E[X]$. Applying this:

$$E[\varepsilon_{a^*}^{(2)} | a^*] = E[\varepsilon_{a^*}^{(2)}] = 0, \quad (580)$$

where the final equality holds because each $\varepsilon_a^{(2)}$ has zero mean. Taking the outer expectation:

$$E[Y] = E_{\varepsilon^{(1)}} [r(s, a^*) + \gamma \mu(s, a^*)]. \quad (581)$$

The evaluation noise contributes zero on average because it is independent of the selection. However, double Q-learning does not eliminate all bias. Two distinct sources of bias arise when maximizing over noisy estimates:

1. **Selection bias:** Using $\arg \max_a \{r(s, a) + \gamma \mu(s, a) + \gamma \varepsilon_a^{(1)}\}$ tends to select actions that received positive noise. The noise $\varepsilon^{(1)}$ causes us to favor actions that got lucky, so a^* is not uniformly distributed but skewed toward actions with positive $\varepsilon_a^{(1)}$. This means $E_{\varepsilon^{(1)}} [\mu(s, a^*)] \geq \max_a \mu(s, a)$.
2. **Evaluation bias:** In the standard estimator, we use the *same* noisy estimate $\hat{\mu}_N(s, a^*)$ that selected a^* to also evaluate it. If a^* was selected because ε_{a^*} happened to be large and positive, that same inflated estimate provides the target value. This creates additional bias beyond the selection effect.

Double Q-learning eliminates evaluation bias by using independent noise for evaluation: even though a^* is biased toward actions with positive $\varepsilon^{(1)}$, the evaluation uses $\varepsilon^{(2)}$ which is independent of the selection, so $E[\varepsilon_{a^*}^{(2)} | a^*] = 0$. Selection bias remains, but removing evaluation bias substantially reduces total overestimation.

Remark 6.2 (Implementation via different Q-functions). *The conceptual framework above assumes we can draw multiple independent samples from $p(\cdot | s, a)$ for each state-action pair. In practice, this would require resetting a simulator to the same state and sampling multiple times, which is often infeasible. The practical implementation achieves the same effect differently: maintain two Q-functions that are trained on different data or updated at different times (e.g., one is a slowly-updating target network). Since the two Q-functions experience different noise realizations during training, their errors remain less correlated than if we used a single Q-function for both selection and evaluation. This is how Double DQN works, as we'll see in the [next chapter on fitted Q-iteration](#).*

The following algorithm implements this principle with Monte Carlo integration. We maintain two Q-functions and alternate which one selects actions and which one evaluates them. The bias reduction mechanism applies whether we store Q-values in a table or use function approximation.

Note that this algorithm is written in the theoretical form that assumes we can draw N samples from each (s, a) pair. In practice with single-trajectory

data ($N = 1$, one quadruple (s, a, r, s') per transition), the double Q-learning principle still applies but we work with the single observed next state s' and maintain two Q-functions that are trained on different subsets of data or updated at different frequencies to achieve the independence needed for bias reduction.

At lines 9 and 12, $q^{(1)}$ selects the action but $q^{(2)}$ evaluates it. The noise in $q^{(1)}$ influences which action is chosen, but the independent noise in $q^{(2)}$ does not inflate the evaluation. Lines 10 and 13 apply the same principle symmetrically for updating $q^{(2)}$.

Weighted Estimators: A Third Approach The Keane-Wolpin and double Q-learning approaches represent two strategies for addressing overestimation bias: explicit correction and decoupled estimation. A third approach, developed by D'Eramo et al. [D'Eramo et al. \[2016\]](#), replaces the hard maximum with a probability-weighted average that accounts for uncertainty in value estimates.

The **maximum estimator (ME)** used in standard Q-learning takes the maximum of sample mean Q-values:

$$\hat{\mu}_*^{ME} = \max_{a \in \mathcal{A}} \hat{\mu}_a \quad (582)$$

where $\hat{\mu}_a$ is the sample mean Q-value for action a . While each individual $\hat{\mu}_a$ may be unbiased, the maximum is systematically upward biased. The bias can be bounded by:

$$\text{Bias}(\hat{\mu}_*^{ME}) \leq \sqrt{\frac{M-1}{M} \sum_{a=1}^M \frac{\sigma_a^2}{n_a}} \quad (583)$$

where $M = |\mathcal{A}|$ is the number of actions, σ_a^2 is the variance of Q-values for action a , and n_a is the number of samples used to estimate it.

The **double estimator (DE)** from double Q-learning reduces this bias by decoupling selection and evaluation, but introduces negative bias:

$$E[\hat{\mu}_*^{DE}] \leq \max_a \mu_a \quad (584)$$

This pessimism can be problematic when one action is clearly superior to others, as the estimator may systematically undervalue the optimal action.

The **weighted estimator (WE)** provides a middle ground. Instead of taking a hard maximum, compute a weighted average where weights reflect the probability that each action is optimal. Under a Gaussian approximation for the sampling distribution of Q-value estimates, the weight for action a is:

$$w_a = P\left(a \in \arg \max_{a' \in \mathcal{A}} \hat{\mu}_{a'}\right) = \int_{-\infty}^{+\infty} \phi\left(\frac{x - \hat{\mu}_a}{\hat{\sigma}_a / \sqrt{n_a}}\right) \prod_{b \neq a} \Phi\left(\frac{x - \hat{\mu}_b}{\hat{\sigma}_b / \sqrt{n_b}}\right) dx \quad (585)$$

where ϕ and Φ are the standard normal PDF and CDF. This integral computes the probability that action a achieves the maximum by integrating over

all possible true values x . For each x , the ϕ term gives the probability density that action a has true value x , while the product of Φ terms gives the probability that all other actions have true values below x .

The weighted estimator then becomes:

$$\hat{\mu}_*^{WE} = \sum_{a \in \mathcal{A}} w_a \hat{\mu}_a \quad (586)$$

D'Eramo et al. established that this estimator satisfies $\text{Bias}(\hat{\mu}_*^{WE}) \leq \text{Bias}(\hat{\mu}_*^{ME})$ and $\text{Bias}(\hat{\mu}_*^{WE}) \geq \text{Bias}(\hat{\mu}_*^{DE})$. The weighted estimator thus provides a middle ground between the optimism of ME and the pessimism of DE.

The relative performance depends on the problem structure. When one action is clearly superior ($\mu_1 - \mu_2 \gg \sigma$), the signal dominates the noise and ME performs well with minimal overestimation. When multiple actions have similar values ($\mu_1 \approx \mu_2$), eliminating overestimation becomes critical and DE excels despite its pessimism. In intermediate cases, which represent most practical scenarios, WE adapts to the actual uncertainty and avoids both extremes.

The weighted estimator can be incorporated into Q-learning by maintaining variance estimates alongside Q-values and computing the weighted target at each step. The weights w_a themselves can serve as an exploration policy that naturally adapts to estimation uncertainty. This approach adapts automatically to heterogeneous uncertainty across actions and provides balanced bias, but requires maintaining variance estimates (adding memory overhead) and computing integrals (expensive for large action spaces). The method assumes Gaussian sampling distributions, though it proves robust in practice even when this assumption is violated.

The weighted estimator relates to the smooth Bellman operators discussed in the [regularized MDP chapter](#). Both replace the discontinuous hard maximum with smooth aggregation, but weighted estimation adapts to state-action-specific uncertainty while logsumexp applies uniform smoothing via temperature. The choice among ME, DE, and WE depends on computational constraints and problem characteristics. When variance estimates are available and action spaces are small, WE offers a principled approach that balances the extremes of overestimation and underestimation. When maintaining variance is expensive or action spaces are large, double Q-learning provides a simpler alternative that eliminates evaluation bias without explicit probability weighting.

Summary and Forward Look This chapter developed the simulation-based approach to approximate dynamic programming by replacing analytical integration with Monte Carlo sampling. We showed how deterministic quadrature rules give way to randomized estimation when transition densities are expensive to evaluate or the state space is high-dimensional. We need only the ability to **sample** from transition distributions, not to compute their densities explicitly.

Three foundational components emerged:

1. **Monte Carlo Bellman operator:** Replace exact expectations $\int v(s')p(ds'|s, a)$ with sample averages $\frac{1}{N} \sum v(s'_i)$, typically with $N = 1$ in practice

2. **Q-functions:** Cache state-action values to amortize the cost of action selection, eliminating the need for repeated sampling at decision time
3. **Bias mitigation:** Address the systematic overestimation that arises from maximizing over noisy estimates through four approaches: learning empirical corrections (Keane-Wolpin), analytical adjustment from extreme value theory (Lee-Powell), decoupling selection from evaluation (double Q-learning), and probability-weighted aggregation (weighted estimators)

The algorithms presented (Parametric Q-Value Iteration, Keane-Wolpin, Double Q) remain in the theoretical setting where we can draw multiple samples from each state-action pair and choose optimization parameters freely. The [next chapter](#) addresses the practical constraints of reinforcement learning: working with fixed offline datasets of single-sample transitions, choosing function approximators and optimization strategies, and understanding the algorithmic design space that yields methods like FQI, NFQI, and DQN.

Self-checks

6.3.2 Fitted Q-Iteration Methods

The [previous chapter](#) established the theoretical foundations of simulation-based approximate dynamic programming: Monte Carlo integration for evaluating expectations, Q-functions for efficient action selection, and techniques for mitigating overestimation bias. Those developments assumed we could sample freely from transition distributions and choose optimization parameters without constraint. This chapter develops a unified framework for fitted Q-iteration algorithms that spans both offline and online settings. We begin with batch algorithms that learn from fixed datasets, then show how the same template generates online methods like DQN through systematic variations in data collection, optimization strategy, and function approximation.

Design Choices in FQI Methods All FQI methods share the same two-level structure built on three core ingredients: a buffer $\mathcal{B}_t$ of transitions inducing an empirical distribution $\hat{P}_{\mathcal{B}_t}$, a target function g that computes regression targets from the current Q-function, and an optimization procedure to fit the Q-function to the resulting targets. At iteration n , the outer loop constructs targets from individual transitions using the target function: for each transition (s_i, a_i, r_i, s'_i) , we compute $y_i^{(n)} = g(s_i, a_i, r_i, s'_i; \theta_n)$ where typically $g(s, a, r, s'; \theta) = r + \gamma \max_{a'} q(s', a'; \theta)$. The inner loop solves the regression problem $\min_{\theta} E_{((s,a),y) \sim \hat{P}_n^{\text{fit}}} [\ell(q(s, a; \theta), y)]$ to find parameters that match these targets. We can write this abstractly as:

```

repeat  $n = 0, 1, 2, \dots$ 
    Sample transitions from  $\hat{P}_{\mathcal{B}_n}$  and construct targets via  $g(\cdot; \theta_n)$ 
     $\theta^{(n+1)} \leftarrow \text{fit}(\hat{P}_n^{\text{fit}}, \theta_{\text{init}}, K)$ 
until convergence

```

(587)

The **fit** operation minimizes the regression loss using K optimization steps (gradient descent for neural networks, tree construction for ensembles, matrix inversion for linear models) starting from initialization θ_{init} . Standard supervised learning uses random initialization ($\theta_{\text{init}} = \theta_0$) and runs to convergence ($K = \infty$). Reinforcement learning algorithms vary these choices: warm starting from the previous iteration ($\theta_{\text{init}} = \theta^{(n)}$), partial optimization ($K \in \{10, \dots, 100\}$), or single-step updates ($K = 1$).

The buffer $\mathcal{B}_t$ may stay fixed (batch setting) or change (online setting), but the targets always change because they depend on the evolving target function $g(\cdot; \theta_n)$. In practice, we typically have one observed next state per transition, giving $g(s_i, a_i, r_i, s'_i; \theta) = r_i + \gamma \max_{a'} q(s'_i, a'; \theta)$ for transition tuples (s_i, a_i, r_i, s'_i) .

Remark 6.3 (Notation: Buffer vs Regression Distribution). *We maintain a careful distinction between two empirical distributions throughout:*

- **Buffer distribution** $\hat{P}_{\mathcal{B}_t}$ over transitions $\tau = (s, a, r, s')$: The empirical distribution induced by the replay buffer $\mathcal{B}_t$, which contains raw experience tuples. This is fixed (offline) or evolves via online collection (adding new transitions, dropping old ones).
- **Regression distribution** $\hat{P}_t^{\text{fit}}$ over pairs $((s, a), y)$: The empirical distribution over supervised learning targets. This changes every outer iteration n as we recompute targets using the current target function $g(\cdot; \theta_n)$.

The relationship: at iteration n , we construct $\hat{P}_n^{\text{fit}}$ from $\hat{P}_{\mathcal{B}_n}$ by applying the target function:

$$\hat{P}_n^{\text{fit}} = (\text{id}, g(\cdot; \theta_n))_{\#} \hat{P}_{\mathcal{B}_n} \quad (588)$$

where $g(s, a, r, s'; \theta_n) = r + \gamma \max_{a'} q(s', a'; \theta_n)$ or uses the smooth logsumexp operator.

This distinction matters pedagogically: the **buffer distribution** $\hat{P}_{\mathcal{B}_t}$ is fixed (offline) or evolves via online collection, while the **regression distribution** $\hat{P}_t^{\text{fit}}$ evolves via target recomputation. Fitted Q -iteration is the outer loop over target recomputation, not the inner loop over gradient steps.

This template provides a blueprint for instantiating concrete algorithms. Six design axes generate algorithmic diversity: the function approximator (trees, neural networks, linear models), the Bellman operator (hard max vs smooth logsumexp, discussed in the [regularized MDP chapter](#)), the inner optimization strategy (full convergence, K steps, or single step), the initialization scheme (cold vs warm start), the data collection mechanism (offline, online, replay buffer), and bias mitigation approaches (none, double Q -learning, learned correction). While individual algorithms include additional refinements, these axes capture the primary sources of variation. The table below shows how several well-known methods instantiate this template:

Algorithm	Approximation	Bellman	Inner Loop	Initialization	Data	Bias Fix
FQI Ernst et al. [2005]	Extra Trees	Hard	Full	Cold	Offline	None
NFQI Riedmiller [2005]	Neural Net	Hard	Full	Warm	Offline	None
Q-learning Watkins [1989]	Any	Hard	K=1	Warm	Online	None
DQN Mnih et al. [2013]	Deep NN	Hard	K=1	Warm	Replay	None
Double DQN Van Hasselt et al. [2016]	Deep NN	Hard	K=1	Warm	Replay	Double Q
Soft Q Haarnoja et al. [2017]	Neural Net	Smooth	K steps	Warm	Replay	None

This table omits continuous action methods (NFQCA, DDPG, SAC), which introduce an additional design dimension. We address those in the [continuous action chapter](#). The initialization choice becomes particularly important when moving from batch to online algorithms.

Plug-In Approximation with Empirical Distributions The exact Bellman operator involves expectations under the true transition law (combined with the behavior policy), which we denote abstractly by P over transitions $\tau = (s, a, r, s')$:

$$(\mathbf{L}q)(s, a) = r(s, a) + \gamma \int \max_{a'} q(s', a') P(ds' | s, a) \quad (589)$$

In fitted Q-iteration we never see P directly. Instead, we collect a finite set of transitions in a buffer $\mathcal{B}_t = \{\tau_1, \dots, \tau_{|\mathcal{B}_t|}\}$ and work with the **empirical distribution**

$$\hat{P}_{\mathcal{B}_t} = \frac{1}{|\mathcal{B}_t|} \sum_{\tau \in \mathcal{B}_t} \delta_\tau \quad (590)$$

where δ_τ denotes a Dirac delta (point mass) centered at transition τ . Each δ_τ is a probability distribution that places mass 1 at the single point τ and mass 0 everywhere else. The sum creates a mixture distribution: a uniform distribution over the finite set of observed transitions. Sampling from $\hat{P}_{\mathcal{B}_t}$ means picking one transition uniformly at random from the buffer.

For any integrand $g(\tau)$ (loss, TD error, gradient term), expectations under P are approximated by expectations under $\hat{P}_{\mathcal{B}_t}$ using the **sample average estimator**:

$$E_{\tau \sim P}[g(\tau)] \approx E_{\tau \sim \hat{P}_{\mathcal{B}_t}}[g(\tau)] = \frac{1}{|\mathcal{B}_t|} \sum_{\tau \in \mathcal{B}_t} g(\tau) \quad (591)$$

This is exactly the sample average estimator from the Monte Carlo chapter, applied now to transitions. Conceptually, fitted Q-iteration performs **plug-in approximate dynamic programming**. The plug-in principle is a general approach from statistics: when an algorithm requires an unknown population quantity, substitute its sample-based estimator. Here, we replace the unknown transition law P with the empirical distribution $\hat{P}_{\mathcal{B}_t}$ and run value iteration using this empirical Bellman operator:

$$(\hat{\mathbb{L}}_{\mathcal{B}_t} q)(s, a) r(s, a) + \gamma E_{(r', s') | (s, a) \sim \hat{P}_{\mathcal{B}_t}} \left[\max_{a'} q(s', a') \right] \quad (592)$$

From a computational viewpoint, we could describe all of this using sample averages and mini-batch gradients. The empirical distribution notation provides three benefits. First, it unifies offline and online algorithms: both perform value iteration under an empirical law $\hat{P}_{\mathcal{B}_t}$, differing only in whether $\mathcal{B}_t$ remains fixed or evolves. Second, it shows that methods like DQN perform stochastic optimization of a sample average approximation objective $E_{\tau \sim \hat{P}_{\mathcal{B}_t}}[\ell(\cdot)]$, not some ad hoc non-stationary procedure. Third, it cleanly separates two sources of approximation that we will examine shortly: statistical bootstrap (resampling from $\hat{P}_{\mathcal{B}_t}$) versus temporal-difference bootstrap (using estimated values in targets).

Remark 6.4 (Mathematical Formulation of Empirical Distributions). *The empirical distribution $\hat{P}_{\mathcal{B}_t} = \frac{1}{|\mathcal{B}_t|} \sum_{\tau \in \mathcal{B}_t} \delta_\tau$ is a discrete probability measure over $|\mathcal{B}_t|$ points regardless of whether state and action spaces are continuous or discrete. For any measurable set $A \subseteq \mathcal{S} \times \mathcal{A} \times \mathcal{R} \times \mathcal{S}$:*

$$\hat{P}_{\mathcal{B}_t}(A) = \frac{1}{|\mathcal{B}_t|} \sum_{\tau \in \mathcal{B}_t} 1[\tau \in A] = \frac{|\{\tau \in \mathcal{B}_t : \tau \in A\}|}{|\mathcal{B}_t|} \quad (593)$$

The empirical distribution assigns probability $1/|\mathcal{B}_t|$ to each observed tuple and zero elsewhere.

Data, Buffers, and the Unified Template Fitted Q-iteration is built around three ingredients at any time t :

1. A **replay buffer** $\mathcal{B}_t$ containing transitions, inducing an empirical distribution $\hat{P}_{\mathcal{B}_t}$
2. A **target function** $g(s, a, r, s'; \boldsymbol{\theta}) : \mathcal{S} \times \mathcal{A} \times \mathcal{R} \times \mathcal{S} \times \Theta \mapsto \mathcal{R}$ that computes regression targets for individual transitions. Unlike the Bellman operator $\mathcal{T}$ which acts on function spaces, g operates on transition tuples with parameters $\boldsymbol{\theta}$ (after the semicolon) specifying the current Q-function. Typically $g(s, a, r, s'; \boldsymbol{\theta}) = r + \gamma \max_{a'} q(s', a'; \boldsymbol{\theta})$ (hard max) or uses the smooth logsumexp operator.
3. A **loss function** ℓ and **optimization budget** (replay ratio, number of gradient steps)

Pushing transitions through the target function transforms $\hat{P}_{\mathcal{B}_t}$ into a regression distribution $\hat{P}_t^{\text{fit}}$ over pairs $((s, a), y)$, as described in the notation remark above. The inner loop minimizes the empirical risk $E_{((s, a), y) \sim \hat{P}_t^{\text{fit}}}[\ell(q(s, a; \boldsymbol{\theta}), y)]$ via stochastic gradient descent on mini-batches.

Different algorithms correspond to different ways of evolving the buffer $\mathcal{B}_t$ and different replay ratios:

- **Offline FQI.** We start from a fixed dataset $\mathcal{D}$ and never collect new data. The buffer is constant, $\mathcal{B}_t \equiv \mathcal{D}$ and $\hat{P}_{\mathcal{B}_t} \equiv \hat{P}_{\mathcal{D}}$, and only the target function $g(\cdot; \boldsymbol{\theta}_t)$ changes as the Q-function evolves.
- **Replay (DQN-style).** The buffer is a **circular buffer** of fixed capacity B . At each interaction step we append the new transition and, if the buffer is full, drop the oldest one: $\mathcal{B}_t = \{\tau_{t-B+1}, \dots, \tau_t\}$ and $\hat{P}_{\mathcal{B}_t} = \frac{1}{|\mathcal{B}_t|} \sum_{\tau \in \mathcal{B}_t} \delta_\tau$. The empirical distribution slides forward through time, but at each update we still sample uniformly from the current buffer.
- **Fully online Q-learning.** Tabular Q-learning is the degenerate case with buffer size $B = 1$: we only keep the most recent transition. Then $\hat{P}_{\mathcal{B}_t}$ is supported on a single point and each update uses that one sample once.

The **replay ratio** (optimization steps per environment transition) quantifies data reuse:

$$\text{Replay ratio} = \begin{cases} K \cdot N_{\text{epochs}} \cdot N/b & \text{Offline (batch size } b \text{ from } N \text{ transitions)} \\ b & \text{DQN (one step per transition, batch size } b) \\ 1 & \text{Online Q-learning} \end{cases} \quad (594)$$

where K is the number of gradient steps per outer iteration. Large replay ratios reuse the same empirical distribution $\hat{P}_{\mathcal{B}_t}$ many times, implementing

sample average approximation (offline FQI). Small replay ratios use each sample once, implementing stochastic approximation (online Q-learning). Higher replay ratios reduce variance of estimates under $\hat{P}_{\mathcal{B}_t}$ but risk overfitting to the idiosyncrasies of the current empirical law, especially when it reflects outdated policies or narrow coverage.

Remark 6.5 (Two Notions of Bootstrap). *This perspective separates two different “bootstraps” that appear in FQI:*

1. **Statistical bootstrap over data.** *By sampling with replacement from $\hat{P}_{\mathcal{B}_t}$, we approximate expectations under the (unknown) transition distribution P with expectations under the empirical distribution $\hat{P}_{\mathcal{B}_t}$. This is identical to bootstrap resampling in statistics. Mini-batch training is exactly this: we treat the observed transitions as if they were the entire population and approximate expectations by repeatedly resampling from the empirical law. The replay ratio controls how many such bootstrap samples we take per environment interaction.*
2. **Temporal-difference bootstrap over values.** *When we compute $y = r + \gamma \max_{a'} q(s', a'; \theta)$, we replace the unknown continuation value by our current estimate. This is the TD notion of bootstrapping and the source of maximization bias studied in the previous chapter.*

FQI, DQN, and their variants combine both: the empirical distribution $\hat{P}_{\mathcal{B}_t}$ encodes how we reuse data (statistical bootstrap), and the target function g encodes how we bootstrap values (TD bootstrap). Most bias-correction techniques (Keane–Wolpin, Double Q-learning, Gumbel losses) modify the second kind of bootstrapping while leaving the statistical bootstrap unchanged.

Every algorithm in this chapter minimizes an empirical risk of the form $E_{((s,a),y) \sim \hat{P}_t^{\text{fit}}}[\ell(q(s,a;\theta), y)]$, where expectations are computed via the sample average estimator over the buffer. Algorithmic diversity arises from choices of buffer evolution, target function, loss, and optimization schedule.

Batch Algorithms: Ernst’s FQI and NFQI We begin with the simplest case from the buffer perspective. We are given a fixed transition dataset $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ and never collect new data. The replay buffer is frozen:

$$\mathcal{B}_t \equiv \mathcal{D}, \quad \hat{P}_{\mathcal{B}_t} \equiv \hat{P}_{\mathcal{D}} \quad (595)$$

so the only thing that changes across iterations is the target function $g(\cdot; \theta_n)$ induced by the current Q-function. Every outer iteration of FQI samples from the same empirical distribution $\hat{P}_{\mathcal{D}}$ but pushes it through a new target function, producing a new regression distribution $\hat{P}_n^{\text{fit}}$.

At each outer iteration n , we construct the input set $X^{(n)} = \{(s_i, a_i)\}_{i=1}^N$ from the same transitions (the state-action pairs remain fixed), compute targets $y_i^{(n)} = g(s_i, a_i, r_i, s'_i; \theta^{(n)}) = r_i + \gamma \max_{a'} q(s'_i, a'; \theta^{(n)})$ using the current Q-function, and solve the regression problem $\theta^{(n+1)} \leftarrow \text{fit}(X^{(n)}, y^{(n)}, \theta_{\text{init}}, K)$.

The buffer $\mathcal{B}_t = \mathcal{D}$ never changes, but the targets change at every iteration because they depend on the evolving target function $g(\cdot; \theta_n)$. Each transition (s_i, a_i, r_i, s'_i) provides a single Monte Carlo sample s'_i for evaluating the Bellman operator at (s_i, a_i) , giving us $\hat{\mathbf{L}}_1 q$ with $N = 1$.

The following algorithm is simply approximate value iteration where expectations under the transition kernel P are replaced by expectations under the fixed empirical distribution $\hat{P}_{\mathcal{D}}$:

The `fit` operation in line 10 abstracts the inner optimization loop that minimizes $\sum_{i=1}^N \ell(q(s_i, a_i; \theta), y_i^{(n)})$. This line encodes the algorithmic choice of which function approximator to use and how to optimize it. The initialization θ_{init} and number of optimization steps K control whether we use cold or warm starting and whether we optimize to convergence or perform partial updates.

Fitted Q-Iteration (FQI): Ernst et al. [Ernst et al. \[2005\]](#) instantiate this template with extremely randomized trees (extra trees), an ensemble method that partitions the state-action space into regions with piecewise constant Q-values. The `fit` operation trains the ensemble until completion using the tree construction algorithm. Trees handle high-dimensional inputs naturally and the ensemble reduces overfitting. FQI uses cold start initialization: $\theta_{\text{init}} = \theta_0$ (randomly initialized) at every iteration, since trees don't naturally support incremental refinement. The loss ℓ is squared error. This method demonstrated that batch reinforcement learning could work with complex function approximators on continuous-state problems.

Neural Fitted Q-Iteration (NFQI): Riedmiller [Riedmiller \[2005\]](#) replaces the tree ensemble with a neural network $q(s, a; \theta)$, providing smooth interpolation across the state-action space. The `fit` operation runs gradient-based optimization (RProp, chosen for its insensitivity to hyperparameter choices) to convergence: train the network until the loss stops decreasing (multiple epochs through the full dataset $\mathcal{D}$), corresponding to $K = \infty$ in our framework. NFQI uses warm start initialization: $\theta_{\text{init}} = \theta_n$ at iteration n , meaning the network continues learning from the previous iteration's weights rather than resetting. This ensures the network accurately represents the projected Bellman operator before moving to the next outer iteration. For episodic tasks with goal and forbidden regions, Riedmiller uses modified target computations (detailed below).

Remark 6.6 (Goal State Heuristics in NFQI). *For episodic tasks with goal states S^+ and forbidden states S^- , Riedmiller modifies the target structure:*

$$y_i^{(n)} = \begin{cases} c(s_i, a_i, s'_i) & \text{if } s'_i \in S^+ \text{ (goal reached)} \\ C^- & \text{if } s'_i \in S^- \text{ (forbidden state, typically } C^- = 1.0) \\ c(s_i, a_i, s'_i) + \gamma \max_{a'} q(s'_i, a'; \theta_n) & \text{otherwise} \end{cases} \quad (596)$$

where $c(s, a, s')$ is the immediate cost. Goal states have zero future cost (no bootstrapping), forbidden states have high penalty, and regular states use the standard Bellman backup. Additionally, the **hint-to-goal heuristic** adds synthetic transitions (s, a, s') where $s \in S^+$ with target value $c(s, a, s') = 0$ to

explicitly clamp the Q -function to zero in the goal region. This stabilizes learning by encoding the boundary condition without requiring additional prior knowledge.

From Nested to Flattened Q-Iteration Fitted Q-iteration has an inherently nested structure: an outer loop performs approximate value iteration by computing Bellman targets, and an inner loop performs regression by fitting the function approximator to those targets. This nested structure shows that FQI is approximate dynamic programming with function approximation, distinct from supervised learning with changing targets.

When the inner loop uses gradient descent for K steps, we have:

$$\text{fit}(\mathcal{D}_n^{\text{fit}}, \theta_n, K) = \theta_n - \alpha \sum_{k=0}^{K-1} \nabla_{\theta} \mathcal{L}(\theta_n^{(k)}; \mathcal{D}_n^{\text{fit}}) \quad (597)$$

This is a sequence of updates $\theta_n^{(k+1)} = \theta_n^{(k)} - \alpha \nabla_{\theta} \mathcal{L}(\theta_n^{(k)}; \mathcal{D}_n^{\text{fit}})$ for $k = 0, \dots, K-1$ starting from $\theta_n^{(0)} = \theta_n$. Since these inner updates are themselves a loop, we can algebraically rewrite the nested loops as a single flattened loop. This flattening is purely representational. The algorithm remains approximate value iteration, but the presentation obscures the conceptual structure.

In the flattened form, the parameters used for computing targets are called the **target network** θ_{target} , which corresponds to θ_n in the nested form. The target network gets updated every K steps, marking the boundaries between outer iterations.

In contrast, the parameters being actively trained via gradient descent at each step are called the **online network** θ_t . In the nested view, these correspond to $\theta_n^{(k)}$ within the inner loop. The term “online” here refers to the fact that these parameters are continuously updated at each gradient step, not that data must be collected online. The distinction is between the actively-training parameters (θ_t) and the frozen parameters used for target computation (θ_{target}).

The online/target terminology is standard in the deep RL community. Many modern algorithms, especially those that collect data online like DQN, are presented in flattened form. This can make them appear different from batch methods when they are the same template with different design choices.

Tree ensemble methods like random forests or extra trees have no continuous parameter space and no gradient-based optimization. The **fit** operation builds the entire tree structure in one pass. There’s no sequence of incremental updates to unfold into a single loop. Ernst’s FQI [Ernst et al. \[2005\]](#) retains the explicit nested structure with cold start initialization at each outer iteration, while neural methods can be flattened.

Making the Nested Structure Explicit To see how flattening works, we first make the nested structure completely explicit by expanding the **fit** operation to show the inner gradient descent loop. In terms of the buffer notation, the inner loop approximately minimizes the empirical risk:

$$E_{((s,a),y) \sim \hat{P}_n^{\text{fit}}}[\ell(q(s,a;\boldsymbol{\theta}),y)] \quad (598)$$

induced by the fixed buffer $\mathcal{B}_n = \mathcal{D}$ and target function $g(\cdot; \boldsymbol{\theta}_n)$. Starting from the generic batch FQI template (Algorithm Algorithm ??), we replace the abstract `fit` call with explicit gradient updates:

This makes the two-level structure completely transparent. The outer loop (indexed by n) computes targets $y_{s,a} = r + \gamma \max_{a'} q(s', a'; \boldsymbol{\theta}_n)$ using the parameters $\boldsymbol{\theta}_n$ from the end of the previous outer iteration. These targets remain **fixed** throughout the entire inner loop. The inner loop (indexed by k) performs K gradient steps to fit $q(s, a; \boldsymbol{\theta})$ to the regression dataset $\mathcal{D}_n^{\text{fit}} = \{((s_i, a_i), y_i)\}$, warm starting from $\boldsymbol{\theta}_n$ (the parameters that computed the targets). After K steps, the inner loop produces $\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n^{(K)}$, which becomes the starting point for the next outer iteration.

The notation $\boldsymbol{\theta}_n^{(k)}$ indicates that we are at inner step k within outer iteration n . The targets depend only on $\boldsymbol{\theta}_n = \boldsymbol{\theta}_n^{(0)}$, not on the intermediate inner iterates $\boldsymbol{\theta}_n^{(k)}$ for $k > 0$.

Flattening into a Single Loop We can now flatten the nested structure by treating all gradient steps uniformly and using a global step counter t instead of separate outer/inner counters. We introduce a **target network** $\boldsymbol{\theta}_{\text{target}}$ that holds the parameters used for computing targets. This target network gets updated every K steps, which marks what would have been the boundary between outer iterations. The transformation works as follows: we replace the outer counter n and inner counter k with a single counter t , where $t = nK + k$. When n advances from n to $n+1$, this corresponds to K steps of t : from $t = nK$ to $t = (n+1)K$. The target network $\boldsymbol{\theta}_{\text{target}}$ equals $\boldsymbol{\theta}_n$ throughout outer iteration n and gets updated via $\boldsymbol{\theta}_{\text{target}} \leftarrow \boldsymbol{\theta}_t$ every K steps (when $t \bmod K = 0$). Parameters that were $\boldsymbol{\theta}_n^{(k)}$ in nested form become $\boldsymbol{\theta}_t$ in flattened form.

This transformation is purely algebraic. No algorithmic behavior changes, only the presentation:

At step t , we have $n = \lfloor t/K \rfloor$ (outer iteration) and $k = t \bmod K$ (position within inner loop). The target network $\boldsymbol{\theta}_{\text{target}}$ equals $\boldsymbol{\theta}_n$ throughout the K steps from $t = nK$ to $t = (n+1)K - 1$, then gets updated to $\boldsymbol{\theta}_{n+1}$ at $t = (n+1)K$. The parameters $\boldsymbol{\theta}_t$ correspond to $\boldsymbol{\theta}_n^{(k)}$ in the nested form. The flattening reindexes the iteration structure: with $K = 10$, outer iteration $n = 3$, inner step $k = 7$ becomes global step $t = 3 \cdot 10 + 7 = 37$.

Flattening replaces the pair (n, k) by a single global step index t , but the underlying empirical distribution $\hat{P}_{\mathcal{D}}$ remains the same. We still sample from the fixed offline dataset throughout.

The target network arises directly from flattening the nested FQI structure. When DQN is presented with a target network that updates every K steps, this is approximate value iteration in flattened form. The algorithm still performs outer loop (value iteration) and inner loop (regression), but the presentation obscures this structure. The periodic target updates mark the boundaries be-

tween outer iterations. DQN is batch approximate DP in flattened form, using online data collection with a replay buffer.

Smooth Target Updates via Exponential Moving Average An alternative to periodic hard updates is **exponential moving average (EMA)** (also called Polyak averaging), which updates the target network smoothly at every step rather than synchronizing it every K steps:

With EMA updates, the target network slowly tracks the online network instead of making discrete jumps. For small τ (typically $\tau \in [0.001, 0.01]$), the target lags behind the online network by roughly $1/\tau$ steps. This provides smoother learning dynamics and avoids the discontinuous changes in targets that occur with periodic hard updates. The EMA approach became popular with DDPG Lillicrap et al. [2015] for continuous control and is now standard in algorithms like TD3 Fujimoto et al. [2018] and SAC Haarnoja et al. [2018a].

Online Algorithms: DQN and Extensions We now keep the same fitted Q-iteration template but allow the buffer $\mathcal{B}_t$ and its empirical distribution $\hat{P}_{\mathcal{B}_t}$ to evolve while we learn. Instead of repeatedly sampling from a fixed $\hat{P}_{\mathcal{D}}$, we collect new transitions during learning and store them in a circular replay buffer.

Note that “online” here takes on a second meaning: we collect data online (interacting with the environment) while also maintaining the online network (actively-updated parameters θ_t) and target network (frozen parameters θ_{target}) distinction from the flattened FQI structure. The online network now plays a dual role: it is both the parameters being trained and the policy used to collect new data for the buffer.

Deep Q-Network (DQN) instantiates the online template with moderate choices along the design axes: buffer capacity $B \approx 10^6$, mini-batch size $b \approx 32$, and target network update frequency $K \approx 10^4$. DQN is not an ad hoc collection of tricks. It is fitted Q-iteration in flattened form (as developed in the nested-to-flattened transformation earlier) with an evolving buffer $\mathcal{B}_t$ and low replay ratio.

Deep Q-Network (DQN) Deep Q-Network (DQN) maintains a circular replay buffer of capacity B (typically $B \approx 10^6$). At each environment step, we store the new transition and sample a mini-batch of size b from the buffer for training. This increases the replay ratio, reducing gradient variance at the cost of older, potentially off-policy data.

DQN uses two copies of the Q-network parameters:

- The **online network** θ_t : actively updated at each gradient step (corresponds to $\theta_n^{(k)}$ in the nested view). It serves a dual role: (1) the policy for collecting new data via ϵ -greedy action selection, and (2) the parameters being trained.
- The **target network** θ_{target} : frozen parameters updated every K steps (corresponds to θ_n in the nested view). Used only for computing Bellman

targets.

As shown in the nested-to-flattened section, the target network is not a stabilization trick but the natural consequence of periodic outer-iteration boundaries in flattened FQL. The target network keeps targets fixed for K gradient steps, which corresponds to the inner loop in the nested view:

Double Deep Q-Network (Double DQN) Double DQN addresses overestimation bias by decoupling action selection from evaluation. Instead of using the target network for both purposes (as DQN does), Double DQN uses the online network (currently-training parameters θ_t) to select which action looks best, then uses the target network (frozen parameters θ_{target}) to evaluate that action:

Double DQN leaves $\mathcal{B}_t$ and $\hat{P}_{\mathcal{B}_t}$ unchanged and modifies only the target function g by decoupling action selection (online network) from evaluation (target network). Compare the two approaches:

- **DQN (blue)**: Uses the target network θ_{target} for BOTH selecting which action is best AND evaluating it: $\max_{a'} q(s', a'; \theta_{\text{target}})$ implicitly finds $a^* = \arg \max_{a'} q(s', a'; \theta_{\text{target}})$ and then evaluates $q(s', a^*; \theta_{\text{target}})$
- **Double DQN (red + green)**: Uses the online network θ_t to select $a^* = \arg \max_{a'} q(s', a'; \theta_t)$, then uses the target network θ_{target} to evaluate $q(s', a^*; \theta_{\text{target}})$

This decoupling prevents overestimation bias because the noise in action selection (from the online network) is independent of the noise in action evaluation (from the target network). Recall that the online network represents the currently-training parameters being updated at each gradient step, while the target network represents the frozen parameters used for computing targets. By using different parameters for selection versus evaluation, we break the correlation that leads to systematic overestimation in the standard max operator.

Q-Learning: The Limiting Case DQN and Double DQN use moderate settings: $B \approx 10^6$, $K \approx 10^4$, mini-batch size $b \approx 32$. We can ask: what happens at the extreme where we minimize buffer capacity, eliminate replay, and update targets every step? This limiting case gives classical Q-learning [Watkins \[1989\]](#): buffer capacity $B = 1$, target network frequency $K = 1$, and mini-batch size $b = 1$.

In the buffer perspective, Q-learning has $\mathcal{B}_t = \{(s_t, a_t, r_t, s'_t)\}$ with $|\mathcal{B}_t| = 1$. The empirical distribution $\hat{P}_{\mathcal{B}_t}$ collapses to a Dirac mass at the current transition. We use each transition exactly once then discard it. There is no separate target network: the parameters used for computing targets are immediately updated after each step ($K = 1$, or equivalently $\tau = 1$ in EMA). This makes Q-learning a **stochastic approximation** method with replay ratio 1.

Stochastic approximation is a general framework for solving equations of the form $E[h(X; \theta)] = 0$ using noisy samples, without computing expectations explicitly. The classic example is root-finding: given function $f(\theta; Z)$ whose expectation $E[f(\theta; Z)] = F(\theta)$ we want to solve $F(\theta) = 0$, the Robbins-Monro procedure updates:

$$\theta_{t+1} = \theta_t - \alpha_t f(\theta_t; Z_t) \quad (599)$$

using noisy samples Z_t without ever computing $F(\theta)$ or its Jacobian. This is analogous to Newton's method in the deterministic case, but replaces exact gradients with stochastic estimates and avoids computing or inverting the Jacobian. Under diminishing step sizes ($\alpha_t \rightarrow 0$, $\sum_t \alpha_t = \infty$), the iterates converge to solutions of $F(\theta) = 0$.

Q-learning fits this framework by solving the Bellman residual equation. Recall from the [projection methods chapter](#) that the Bellman equation $q^* = \mathbf{L}q^*$ can be written as a residual equation $\mathbf{N}(q) \equiv \mathbf{L}q - q = 0$. For a parameterized Q-function $q(s, a; \theta)$, the residual at observed transition (s, a, r, s') is:

$$R(s, a, r, s'; \theta) = r + \gamma \max_{a'} q(s', a'; \theta) - q(s, a; \theta) \quad (600)$$

This is the TD error. Q-learning is a stochastic approximation method for solving $E_{\tau \sim P}[R(\tau; \theta)] = 0$ where P is the distribution of transitions under the behavior policy. Each observed transition provides a noisy sample of the residual, and the algorithm updates parameters using the gradient of the squared residual without computing expectations explicitly.

The algorithm works with any function approximator that supports incremental updates. The general form applies a gradient descent step to minimize the squared TD error. The only difference between linear and nonlinear cases is how the gradient $\nabla_{\theta} q(s, a; \theta)$ is computed.

The gradient in line 5 depends on the choice of function approximator:

- **Tabular:** Uses one-hot features $\phi(s, a) = \mathbf{e}_{(s,a)}$, giving table-lookup updates $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$. Converges under standard stochastic approximation conditions (detailed below).
- **Linear:** $q(s, a; \theta) = \theta^\top \phi(s, a)$ where $\phi : \mathcal{S} \times \mathcal{A} \rightarrow R^d$ is a feature map. The gradient is $\nabla_{\theta} q(s, a; \theta) = \phi(s, a)$, giving the update $\theta_{t+1} = \theta_t + \alpha(y - \theta_t^\top \phi(s, a)) \phi(s, a)$. Convergence is not guaranteed in general; the max operator combined with function approximation can cause instability.
- **Nonlinear:** $q(s, a; \theta)$ is a neural network with weights θ . The gradient $\nabla_{\theta} q(s, a; \theta)$ is computed via backpropagation. Convergence guarantees do not exist.

Convergence Analysis via the ODE Method Tabular Q-learning converges under diminishing step sizes ($\alpha_t \rightarrow 0$, $\sum_t \alpha_t = \infty$) and sufficient exploration [Watkins and Dayan \[1992\]](#), [Jaakkola et al. \[1994\]](#), [Tsitsiklis \[1994\]](#).

The standard proof technique is the **ordinary differential equation (ODE) method** [Kushner and Yin \[2003\]](#), [Borkar and Meyn \[2000\]](#), which analyzes stochastic approximation algorithms by studying an associated deterministic dynamical system.

The ODE method connects stochastic approximation to deterministic dynamics through a two-step argument. First, we show that as step sizes shrink, the discrete stochastic recursion $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha_t h(\boldsymbol{\theta}_t, Z_t)$ tracks the continuous deterministic ODE $\frac{d\boldsymbol{\theta}}{dt} = \bar{h}(\boldsymbol{\theta})$ where $\bar{h}(\boldsymbol{\theta}) = E[h(\boldsymbol{\theta}, Z)]$ is the expected update direction. Second, we prove convergence of the ODE using Lyapunov functions or other dynamical systems tools. If the ODE converges to an equilibrium, the stochastic algorithm converges to the same point.

For tabular Q-learning, the update at state-action pair (s, a) is:

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t [r + \gamma \max_{a'} Q_t(s', a') - Q_t(s, a)] \quad (601)$$

where (s, a, r, s') is sampled from the transition distribution P induced by the behavior policy. The expected update direction is:

$$\bar{h}(Q)(s, a) = E_{(s, a, r, s') \sim P} [r + \gamma \max_{a'} Q(s', a') - Q(s, a) \mid (s, a) \text{ visited}] \quad (602)$$

Under sufficient exploration (all state-action pairs visited infinitely often), this expectation is well-defined. Let $\xi(s, a)$ denote the stationary distribution over state-action pairs under the behavior policy. The ODE becomes:

$$\frac{dQ(s, a)}{dt} = \sum_{s'} p(s' \mid s, a) [r(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (603)$$

This is a deterministic dynamical system. The fixed points satisfy $Q(s, a) = E_{s'} [r(s, a, s') + \gamma \max_{a'} Q(s', a')]$, which is the Bellman optimality equation. Convergence is established by showing this ODE has a global Lyapunov function: the distance $\|Q - Q^*\|$ to the optimal Q-function decreases along trajectories. The Bellman operator is a contraction, which ensures the ODE trajectories converge to Q^* .

The interaction between the stationary distribution ξ and the empirical distribution $\hat{P}_{\mathcal{B}_t}$ is subtle. In pure stochastic approximation (Q-learning), we have $\mathcal{B}_t = \{(s_t, a_t, r_t, s'_t)\}$ with a single transition. Over time, as we explore, the sequence of visited state-action pairs (s_t, a_t) follows the behavior policy, and under ergodicity, the empirical frequency with which we update each (s, a) converges to the stationary distribution $\xi(s, a)$. The ODE method formalizes this: the expected update $\bar{h}(Q)$ averages over P , which implicitly weights by how often we visit each (s, a) under the stationary distribution.

For linear Q-learning with function approximation, the ODE analysis becomes more complex. The [projection methods chapter](#) shows that non-monotone projections (like linear least squares) can fail to preserve contraction properties.

The max operator in Q-learning creates additional complications that prevent general convergence guarantees, even though the algorithm may work in practice for well-chosen features.

Regression Losses and Noise Models Fix a particular time t and buffer contents $\mathcal{B}_t$. Sampling from $\mathcal{B}_t$ and pushing transitions through the target function $g(\cdot; \theta_t)$ gives a regression distribution $\hat{P}_t^{\text{fit}}$ over pairs $((s, a), y)$. The **fit** operation in the inner loop is then a standard statistical estimation problem: given empirical samples from $\hat{P}_t^{\text{fit}}$, choose parameters θ to minimize a loss:

$$\mathcal{L}(\theta; \mathcal{B}_t) \approx E_{((s,a),y) \sim \hat{P}_t^{\text{fit}}} [\ell(q(s, a; \theta), y)] \quad (604)$$

The choice of loss ℓ implicitly specifies a noise model for the targets y under $\hat{P}_t^{\text{fit}}$. Squared error corresponds to Gaussian noise, absolute error to Laplace noise, Gumbel regression to extreme-value noise, and classification losses to non-parametric noise models over value bins. Because Bellman targets are noisy, biased, and bootstrapped, this choice has a direct impact on how the algorithm interprets the empirical distribution $\hat{P}_t^{\text{fit}}$.

This section examines alternative loss functions for the regression step. The standard approach uses squared error, but the noise in Bellman targets has special structure due to the max operator and bootstrapping. Two strategies have shown empirical success: Gumbel regression, which uses the proper likelihood for extreme-value noise, and classification-based methods, which avoid parametric noise assumptions by working with distributions over value bins.

Gumbel Regression Extreme value theory tells us that the maximum of Gaussian errors has Gumbel-distributed tails. If we take this distribution seriously, maximum likelihood estimation should use a Gumbel likelihood rather than a Gaussian one. Garg, Tang, Kahn, and Levine [Garg et al. \[2023\]](#) developed this idea in Extreme Q-Learning (XQL). Instead of modeling the Bellman error as additive Gaussian noise:

$$y_i = q^*(s_i, a_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \quad (605)$$

they model it as Gumbel noise:

$$y_i = q^*(s_i, a_i) + \varepsilon_i, \quad \varepsilon_i \sim -\text{Gumbel}(0, \beta) \quad (606)$$

The negative Gumbel distribution arises because we are modeling errors in targets that overestimate the true value. The corresponding maximum likelihood loss is Gumbel regression:

$$\mathcal{L}_{\text{Gumbel}}(\theta) = \sum_i \left[\frac{q(s_i, a_i; \theta) - y_i}{\beta} + \exp \left(\frac{q(s_i, a_i; \theta) - y_i}{\beta} \right) \right] \quad (607)$$

The temperature parameter β controls the heaviness of the tail. The score function (gradient with respect to q) is:

$$\frac{\partial \mathcal{L}_{\text{Gumbel}}}{\partial q} = \frac{1}{\beta} \left[1 + \exp\left(\frac{q-y}{\beta}\right) \right] \quad (608)$$

When $q < y$ (underestimation), the exponential term is small and the gradient is mild. When $q > y$ (overestimation), the gradient grows exponentially with the error. This asymmetry deliberately penalizes overestimation more heavily than underestimation. When targets are systematically biased upward due to the max operator, this loss geometry pushes the estimates toward conservative Q-values.

The Gumbel loss can be understood as the natural likelihood for problems involving max operators, just as the Gaussian is the natural likelihood for problems involving averages. The central limit theorem tells us that sums converge to Gaussians; extreme value theory tells us that maxima converge to Gumbel (for light-tailed base distributions). Squared error is optimal for Gaussian noise; Gumbel regression is optimal for Gumbel noise.

The left panel shows the Gumbel loss as a function of the error $q - y$. Notice the asymmetry: the loss grows exponentially for positive errors (overestimation) but increases only linearly for negative errors (underestimation). The parameter β controls the degree of this asymmetry—smaller β values create more aggressive penalization of overestimation.

The right panel shows the gradient (score function), which determines how strongly the optimizer pushes back against errors. For overestimation ($q > y$), the gradient grows exponentially, creating strong corrective pressure. For underestimation ($q < y$), the gradient remains relatively flat (approaching $1/\beta$). This is in stark contrast to squared error (dashed line), which treats over- and underestimation symmetrically. By tuning β , we can calibrate how aggressively the loss combats the overestimation bias induced by the max operator in the Bellman target.

Remark 6.7 (XQL vs Soft Q-Learning: Target Function vs Loss). *XQL (Gumbel loss) and Soft Q-learning both involve Gumbel distributions, but they operate at different levels of the FQI template. Recall our unified framework: buffer $\mathcal{B}_t$, target function g , loss ℓ , optimization budget K .*

Soft Q-learning changes the target function by using the smooth Bellman operator from [regularized MDPs](#):

$$g^{\text{soft}}(s, a, r, s'; \boldsymbol{\theta}) = r + \gamma \frac{1}{\beta} \log \sum_{a'} \exp(\beta q(s', a'; \boldsymbol{\theta})) \quad (609)$$

This replaces the hard max with logsumexp for entropy regularization. The loss remains standard: $\ell(q, y) = (q - y)^2$. The Gumbel distribution appears through the Gumbel-max trick ($\text{logsumexp} = \text{soft max}$), but this is about making the optimal policy stochastic, not about the noise structure in TD errors.

XQL keeps the hard max target function:

$$g^{\text{hard}}(s, a, r, s'; \boldsymbol{\theta}) = r + \gamma \max_{a'} q(s', a'; \boldsymbol{\theta}) \quad (610)$$

and instead changes the loss to Gumbel regression: $\ell_{\text{Gumbel}}(q, y) = \frac{q-y}{\beta} + \exp(\frac{q-y}{\beta})$. The Gumbel distribution here models the noise in the TD errors themselves, not the policy.

The asymmetric penalization in XQL (exponential penalty for overestimation) comes from the score function of the Gumbel likelihood, which is designed to handle extreme-value noise. Soft Q-learning uses symmetric L2 loss, so it does not preferentially penalize overestimation. The logsumexp smoothing in Soft Q reduces maximization bias by averaging over actions, but this is a property of the target function, not the loss geometry.

Both can be combined: Soft Q-learning with Gumbel loss would change both the target function (logsumexp) and the loss (asymmetric penalization).

The practical advantage of XQL is that the loss function itself handles the asymmetric error structure through its score function. We do not need to estimate variances or compute weighted averages. XQL has shown improvements in both value-based and actor-critic methods, particularly in offline reinforcement learning where the max-operator bias compounds across iterations without corrective exploration.

Classification-Based Q-Learning From the buffer viewpoint, nothing changes upstream: we still sample transitions from $\hat{P}_{\mathcal{B}_t}$ and apply the same target function $g(\cdot; \theta_t)$. Classification-based Q-learning changes only the loss ℓ and target representation. Instead of regressing on a scalar $y \in \mathcal{R}$ with L2, we represent values as categorical distributions over bins and use cross-entropy loss.

Choose a finite grid $z_1 < z_2 < \dots < z_K$ spanning plausible return values. The network outputs logits $\ell_{\theta}(s, a) \in \mathcal{R}^K$ converted to probabilities:

$$p_{\theta}(k | s, a) = \frac{\exp(\ell_{\theta}(s, a)_k)}{\sum_{j=1}^K \exp(\ell_{\theta}(s, a)_j)}, \quad q(s, a; \theta) = \sum_{k=1}^K z_k p_{\theta}(k | s, a) \quad (611)$$

Each scalar TD target y_i is converted to a target distribution via the two-hot encoding. If y_i falls between bins z_j and z_{j+1} :

$$q_j(y_i) = \frac{z_{j+1} - y_i}{z_{j+1} - z_j}, \quad q_{j+1}(y_i) = \frac{y_i - z_j}{z_{j+1} - z_j}, \quad q_k(y_i) = 0 \text{ for } k \notin \{j, j+1\} \quad (612)$$

This is barycentric interpolation: $\sum_k z_k q_k(y_i) = y_i$ recovers the scalar exactly, placing the two-hot encoding within the same framework as linear interpolation in the [dynamic programming chapter](#) (Algorithm Algorithm ??).

The left panel shows two-hot encoding for $y = 0.7$ on a grid of 11 bins. The target value is distributed over exactly two adjacent bins z_j and z_{j+1} with weights that are barycentric coordinates: the weight assigned to each bin is inversely proportional to the distance from the target to that bin. The inset

verifies that $\sum_k z_k q_k(y)$ exactly recovers the original target—this is the same linear interpolation formula used throughout the book. The right panel shows that different targets produce different two-hot patterns, each concentrating mass on the two bins surrounding the target value.

The loss minimizes cross-entropy:

$$\mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) = -E_{((s,a),y) \sim \hat{P}_t^{\text{fit}}} \left[\sum_{k=1}^K q_k(y) \log p_{\boldsymbol{\theta}}(k \mid s, a) \right] \quad (613)$$

which projects the target distribution onto the predicted distribution in KL geometry on the simplex Δ^{K-1} rather than L2 on R . The gradient is $\nabla_{\ell_{\theta}} \mathcal{L}_{\text{CE}} = p_{\theta} - q$, bounded in magnitude regardless of target size.

This provides three sources of implicit robustness. First, gradient influence is bounded: each sample contributes $O(1)$ gradient magnitude per bin, unlike L2 where error magnitude E contributes gradient proportional to E . Second, the finite grid $[z_1, z_K]$ clips extreme targets to boundary bins, preventing outliers from dominating the regression scale. Third, the two-hot encoding spreads mass across neighboring bins, providing label smoothing that averages noisy targets at the same (s, a) .

The two-hot weights $q_j(y_i), q_{j+1}(y_i)$ are barycentric coordinates, identical to linear interpolation in the [dynamic programming chapter](#) (Algorithm Algorithm ??). This places the encoding within Gordon’s monotone approximator framework (Definition Definition 6.1): targets are convex combinations preserving order and boundedness. The neural network predicting $p_{\theta}(\cdot \mid s, a)$ is non-monotone, making classification-based Q-learning a hybrid: monotone target structure paired with flexible function approximation.

Empirically, cross-entropy loss scales better with network capacity. Farebrother et al. [Farebrother et al. \[2024\]](#) found that L2-based DQN and CQL degrade when Q-networks scale to large ResNets, while classification loss (specifically HL-Gauss, which uses Gaussian smoothing instead of two-hot) maintains performance. The combination of KL geometry, quantization, and smoothing prevents overfitting to noisy targets that plagues L2 with high-capacity networks.

Summary Fitted Q-iteration has a two-level structure: an outer loop applies the Bellman operator to construct targets, an inner loop fits a function approximator to those targets. All algorithms in this chapter instantiate this template with different choices of buffer $\mathcal{B}_t$, target function g , loss ℓ , and optimization budget K .

The empirical distribution $\hat{P}_{\mathcal{B}_t}$ unifies offline and online methods through plug-in approximation: replace unknown transition law P with $\hat{P}_{\mathcal{B}_t}$ and minimize empirical risk $E_{((s,a),y) \sim \hat{P}_t^{\text{fit}}}[\ell(q, y)]$. Offline uses fixed $\mathcal{B}_t \equiv \mathcal{D}$ (sample average approximation), online uses circular buffer (Q-learning with $B = 1$ is stochastic approximation, DQN with large B is hybrid).

Target networks and online networks arise from flattening the nested loops. Merging inner gradient steps with outer value iteration creates a single loop

where two sets of parameters coexist: the **online network** θ_t (actively updated at each gradient step, corresponds to $\theta_n^{(k)}$) and the **target network** θ_{target} (frozen for computing targets, updated every K steps to mark outer-iteration boundaries, corresponds to θ_n). In online algorithms like DQN, the online network additionally serves as the behavior policy for data collection.

The [next chapter](#) directly parameterizes and optimizes policies instead of searching over value functions.

Self-checks

6.3.3 Fitted Q-Iteration for Continuous Action Spaces

The previous chapter showed how fitted Q-iteration handles large state spaces through function approximation. FQI maintains a unified structure across batch and online settings: a replay buffer $\mathcal{B}_t$ inducing empirical distribution $\hat{P}_{\mathcal{B}_t}$, a target map T_q derived from the Bellman operator, a loss function ℓ , and an optimization budget. Algorithms differ in how they instantiate these components (buffer evolution, hard vs soft Bellman, update frequency), but all follow the same template.

However, this framework breaks down when the action space becomes large or continuous. Computing Bellman targets requires evaluating $\max_{a' \in \mathcal{A}} q(s', a'; \theta)$ for each next state s' . When actions are continuous ($\mathcal{A} \subset \mathbb{R}^m$), this maximization requires solving a nonlinear program at every target computation. For a replay buffer with millions of transitions, this becomes computationally prohibitive.

This chapter addresses the continuous action problem while maintaining the FQI framework. We develop several approaches, unified by a common theme: **amortization**. Rather than solving the optimization problem $\max_a q(s, a; \theta)$ repeatedly at inference time, we invest computational effort during training to learn a mapping that directly produces good actions. This trades training-time cost for inference-time speed.

The strategies we examine are:

1. **Explicit optimization** (Section 2): Solve the maximization numerically for a subset of states, accepting the computational cost for exact solutions.
2. **Policy network amortization** (Sections 3-5): Learn a deterministic or stochastic policy network π_w that approximates $\arg \max_a q(s, a; \theta)$ or the optimal stochastic policy, enabling fast action selection via a single forward pass. This includes both hard-max methods (DDPG, TD3) and soft-max methods (SAC, PCL).

Each approach represents a different point in the computation-accuracy trade-off, and all fit within the FQI template by modifying how targets are computed.

Explicit Optimization Recall that in fitted Q methods, the main idea is to compute the Bellman operator only at a subset of all states, relying on function approximation to generalize to the remaining states. At each step of the successive approximation loop, we build a dataset of input state-action pairs mapped to their corresponding optimality operator evaluations:

$$\mathcal{D}_n = \{((s, a), (\mathcal{L}q)(s, a; \theta_n)) \mid (s, a) \in \mathcal{B}\} \quad (614)$$

This dataset is then fed to our function approximator (neural network, random forest, linear model) to obtain the next set of parameters:

$$\theta_{n+1} \leftarrow \text{fit}(\mathcal{D}_n) \quad (615)$$

While this strategy allows us to handle very large or even infinite (continuous) state spaces, it still requires maximizing over actions ($\max_{a \in \mathcal{A}}$) during the dataset creation when computing the operator $\mathbf{L}$ for each basepoint. This maximization becomes computationally expensive for large action spaces. We can address this by adding another level of optimization: for each sample added to our regression dataset, we employ numerical optimization methods to find actions that maximize the Bellman operator for the given state.

The above pseudocode introduces a generic `maximize` routine which represents any numerical optimization method that searches for an action maximizing the given function. This approach is versatile and can be adapted to different types of action spaces. For continuous action spaces, we can employ standard nonlinear optimization methods like gradient descent or L-BFGS (e.g., using `scipy.optimize.minimize`). For large discrete action spaces, we can use integer programming solvers - linear integer programming if the Q-function approximator is linear in actions, or mixed-integer nonlinear programming (MINLP) solvers for nonlinear Q-functions. The choice of solver depends on the structure of our Q-function approximator and the constraints on our action space.

While explicit optimization provides exact solutions, it becomes computationally expensive when we need to compute targets for millions of transitions in a replay buffer. Can we avoid solving an optimization problem at every decision? The answer is amortization.

Amortized Optimization Approach This process is computationally intensive. We can “amortize” some of this computation by replacing the explicit optimization for each sample with a direct mapping that gives us an approximate maximizer directly. For Q-functions, recall that the operator is given by:

$$(\mathbf{L}q)(s, a) = r(s, a) + \gamma \int p(ds'|s, a) \max_{a' \in \mathcal{A}(s')} q(s', a') \quad (616)$$

If q^* is the optimal state-action value function, then $v^*(s) = \max_a q^*(s, a)$, and we can derive the optimal policy directly by computing the decision rule:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}(s)} q^*(s, a) \quad (617)$$

Since q^* is a fixed point of $\mathbf{L}$, we can write:

$$\begin{aligned} q^*(s, a) &= (\mathbf{L}q^*)(s, a) \\ &= r(s, a) + \gamma \int p(ds'|s, a) \max_{a' \in \mathcal{A}(s')} q^*(s', a') \\ &= r(s, a) + \gamma \int p(ds'|s, a) q^*(s', \pi^*(s')) \end{aligned}$$

Note that π^* is implemented by our `maximize` numerical solver in the procedure above. A practical strategy would be to collect these maximizer values at

each step and use them to train a function approximator that directly predicts these solutions. Due to computational constraints, we might want to compute these exact maximizer values only for a subset of states, based on some computational budget, and use the fitted decision rule to generalize to the remaining states. This leads to the following amortized version:

Note that the policy $\pi_{\mathbf{w}}$ is being trained on a dataset $\mathcal{D}_\pi$ containing optimal actions computed with respect to an evolving Q-function. Specifically, at iteration n , we collect pairs $(s', a_{s'}^*)$ where $a_{s'}^* = \arg \max_a q(s', a; \boldsymbol{\theta}_n)$. However, after updating to $\boldsymbol{\theta}_{n+1}$, these actions may no longer be optimal with respect to the new Q-function.

A natural approach to handle this staleness would be to maintain only the most recent optimization data. We could modify our procedure to keep a sliding window of K iterations, where at iteration n , we only use data from iterations $\max(0, n - K)$ to n . This would be implemented by augmenting each entry in $\mathcal{D}_\pi$ with a timestamp:

$$\mathcal{D}_\pi^{(n)} = \{(s', a_{s'}^*, t) \mid t \in \{n - K, \dots, n\}\} \quad (618)$$

where t indicates the iteration at which the optimal action was computed. When fitting the policy network, we would then only use data points that are at most K iterations old:

$$\mathbf{w}_{n+1} \leftarrow \text{fit}(\{(s', a_{s'}^*) \mid (s', a_{s'}^*, t) \in \mathcal{D}_\pi^{(n)}, n - K \leq t \leq n\}) \quad (619)$$

This introduces a trade-off between using more data (larger K) versus using more recent, accurate data (smaller K). The choice of K would depend on how quickly the Q-function evolves and the computational budget available for computing exact optimal actions.

The main limitation of this approach, beyond the out-of-distribution drift, is that it requires computing exact optimal actions via the solver for states in $\mathcal{B}_{\text{opt}}$. Can we reduce or eliminate this computational expense? As the policy improves at selecting actions, we can bootstrap from these increasingly better choices. Continuously amortizing these improving actions over time creates a virtuous cycle of self-improvement toward the optimal policy. However, this bootstrapping process requires careful management: moving too quickly can destabilize training.

Deterministic Parametrized Policies In this section, we consider deterministic parametrized policies of the form $\pi_{\mathbf{w}}(s)$ which directly output an action given a state. This approach differs from stochastic policies that output probability distributions over actions, making it particularly suitable for continuous control problems where the optimal policy is often deterministic. Fitted Q-value methods can be naturally extended to simultaneously learn both the Q-function and such a deterministic policy.

The Amortization Problem for Continuous Actions When actions are continuous, $a \in R^d$, extracting a greedy policy from a Q-function becomes computationally expensive. Consider a robot arm control task where the action is a d -dimensional torque vector. To act greedily given Q-function $q(s, a; \theta)$, we must solve:

$$\pi(s) = \arg \max_{a \in \mathcal{A}} q(s, a; \theta), \quad (620)$$

where $\mathcal{A} \subset R^d$ is a continuous set (often a box or polytope). This requires running an optimization algorithm at every time step. For neural network Q-functions, this means solving a nonlinear program whose objective involves forward passes through the network.

After training converges, the agent must select actions in real-time during deployment. Running interior-point methods or gradient-based optimizers at every decision creates unacceptable latency, especially in high-frequency control where decisions occur at 100Hz or faster.

The solution is to **amortize** the optimization cost by learning a separate policy network $\pi_w(s)$ that directly outputs actions. During training, we optimize w so that $\pi_w(s) \approx \arg \max_a q(s, a; \theta)$ for states we encounter. At deployment, action selection reduces to a single forward pass through the policy network: $a = \pi_w(s)$. The computational cost of optimization is paid during training (where time is less constrained) rather than at inference.

This introduces a second approximation beyond the Q-function. We now have two function approximators: a **critic** $q(s, a; \theta)$ that estimates values, and an **actor** $\pi_w(s)$ that selects actions. The critic is trained using Bellman targets as in standard fitted Q-iteration. The actor is trained to maximize the critic:

$$w \leftarrow w + \alpha E_s [\nabla_w q(s, \pi_w(s); \theta)], \quad (621)$$

where the expectation is over states in the dataset or replay buffer. This gradient ascent pushes the actor toward actions that the critic considers valuable. By the chain rule, this equals $(\nabla_a q(s, a; \theta)|_{a=\pi_w(s)}) \cdot (\nabla_w \pi_w(s))$, which can be efficiently computed via backpropagation through the composition of the two networks.

Neural Fitted Q-Iteration for Continuous Actions (NFQCA) NFQCA [Hafner and Riedmiller, 2011] extends the NFQI template from the [previous chapter](#) to handle continuous action spaces by replacing the $\max_{a'} q(s', a'; \theta)$ operator in the Bellman target with a parameterized policy $\pi_w(s')$. This transforms fitted Q-iteration into an actor-critic method: the critic $q(s, a; \theta)$ evaluates state-action pairs via the standard regression step, while the actor $\pi_w(s)$ provides actions by directly maximizing the learned Q-function.

The algorithm retains the two-level structure of NFQI: an outer loop performs approximate value iteration by computing Bellman targets, and an inner loop fits the Q-function to those targets. NFQCA adds a third component

(policy improvement) that updates $\mathbf{w}$ to maximize the Q-function over states sampled from the dataset.

From Discrete to Continuous Actions Recall from the [FQI chapter](#) that NFQI computes Bellman targets using the hard max:

$$y_{s,a} = r + \gamma \max_{a' \in \mathcal{A}} q(s', a'; \boldsymbol{\theta}_n) \quad (622)$$

When $\mathcal{A}$ is finite and small, this max is computed by enumeration. When $\mathcal{A}$ is continuous or high-dimensional, enumeration is intractable. NFQCA replaces the max with a parameterized policy that approximately solves the maximization:

$$y_{s,a} = r + \gamma q(s', \pi_{\mathbf{w}_n}(s'); \boldsymbol{\theta}_n) \quad (623)$$

The policy $\pi_{\mathbf{w}}(s)$ acts as an **amortized optimizer**: instead of solving $\arg \max_{a'} q(s', a')$ from scratch at each state s' during target computation, we train a neural network to output near-optimal actions directly. The term “amortized” refers to spreading the cost of optimization across training: we pay once to learn $\pi_{\mathbf{w}}$, then reuse it for all future target computations.

To train the policy, we maximize the expected Q-value under the distribution of states in the dataset. If we had access to the optimal Q-function q^* , we would solve:

$$\max_{\mathbf{w}} E_{s \sim \hat{P}_{\mathcal{D}}} [q^*(s, \pi_{\mathbf{w}}(s))] \quad (624)$$

where $\hat{P}_{\mathcal{D}}$ is the empirical distribution over states induced by the offline dataset $\mathcal{D}$. In practice, we use the current Q-function approximation $q(s, a; \boldsymbol{\theta}_{n+1})$ after it has been fitted to the latest targets. The expectation is approximated by the sample average over states appearing in $\mathcal{D}$:

$$\max_{\mathbf{w}} \frac{1}{|\mathcal{D}|} \sum_{(s,a,r,s') \in \mathcal{D}} q(s, \pi_{\mathbf{w}}(s); \boldsymbol{\theta}_{n+1}) \quad (625)$$

This policy improvement step runs after the Q-function has been updated, using the newly-fitted critic to guide the actor toward higher-value actions. Both the Q-function fitting and policy improvement use gradient-based optimization on the respective objectives.

The algorithm structure mirrors NFQI (Algorithm Algorithm ?? in the [FQI chapter](#)) with two extensions. First, target computation (line 7-8) replaces the discrete max with a policy network call $\pi_{\mathbf{w}_n}(s')$, making the Bellman operator tractable for continuous actions. Second, after fitting the Q-function (line 11), we add a policy improvement step (line 13) that updates $\mathbf{w}$ to maximize the Q-function evaluated at policy-generated actions over states in the dataset.

Both **fit** operations use gradient descent with warm starting, consistent with the NFQI template. The Q-function minimizes squared Bellman error using targets computed with the current policy. The policy maximizes the

Q-function via gradient ascent on the composition $q(s, \pi_{\mathbf{w}}(s); \boldsymbol{\theta}_{n+1})$, which is differentiable end-to-end when both networks are differentiable. The gradient with respect to $\mathbf{w}$ is:

$$\nabla_{\mathbf{w}} q(s, \pi_{\mathbf{w}}(s); \boldsymbol{\theta}) = \nabla_a q(s, a; \boldsymbol{\theta}) \Big|_{a=\pi_{\mathbf{w}}(s)} \cdot \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(s) \quad (626)$$

computed via the chain rule (backpropagation through the actor into the critic). Modern automatic differentiation libraries handle this composition automatically.

Deep Deterministic Policy Gradient (DDPG) We now extend NFQCA to the online setting with evolving replay buffers, mirroring how DQN extended NFQI in the [FQI chapter](#). Just as DQN allowed $\mathcal{B}_t$ and $\hat{P}_{\mathcal{B}_t}$ to evolve during learning instead of using a fixed offline dataset, DDPG [Lillicrap et al., 2015] collects new transitions during training and stores them in a circular replay buffer.

Like DQN, DDPG uses the flattened FQI structure with target networks. But where DQN maintains a single target network $\boldsymbol{\theta}_{\text{target}}$ for the Q-function, DDPG maintains **two** target networks: one for the critic $\boldsymbol{\theta}_{\text{target}}$ and one for the actor $\mathbf{w}_{\text{target}}$. Both are updated periodically (every K steps) to mark outer-iteration boundaries, following the same nested-to-flattened transformation shown for DQN.

The online network now plays a triple role in DDPG: (1) the parameters being actively trained ($\boldsymbol{\theta}_t$ for critic, $\mathbf{w}_t$ for actor), (2) the policy used to collect new data, and (3) the gradient source for policy improvement. The target networks serve only one purpose: computing stable Bellman targets.

Exploration via Action Noise Since the policy $\pi_{\mathbf{w}}(s)$ is deterministic, exploration requires adding noise to actions during data collection:

$$a = \pi_{\mathbf{w}_t}(s) + \eta_t \quad (627)$$

where η_t is exploration noise. The original DDPG paper used an Ornstein-Uhlenbeck (OU) process, which generates temporally correlated noise through the discretized stochastic differential equation:

$$\eta_{t+1} = \eta_t + \theta(\mu - \eta_t)\Delta t + \sigma\sqrt{\Delta t}\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, 1) \quad (628)$$

where μ is the long-term mean (typically 0), θ controls the strength of mean reversion, σ scales the random fluctuations, and Δt is the time step. The term $\theta(\mu - \eta_t)\Delta t$ acts like damped motion through a viscous fluid: when η_t deviates from μ , this force pulls it back smoothly without oscillation. The random term $\sigma\sqrt{\Delta t}\epsilon_t$ adds perturbations, creating noise that wanders but is gently pulled back toward μ . This temporal correlation produces smoother exploration trajectories than independent Gaussian noise.

However, later work (including TD3, discussed below) found that simple uncorrelated Gaussian noise $\eta_t \sim \mathcal{N}(0, \sigma^2)$ works equally well and is easier to tune. The exploration mechanism is orthogonal to the core algorithmic structure.

The algorithm structure parallels DQN (Algorithm ?? in the [FQI chapter](#)) with the continuous-action extensions from NFQCA. Lines 1-5 initialize both networks and their targets, following the same pattern as DQN but with an additional actor network. Line 3 uses the online actor with exploration noise for data collection, replacing DQN’s ε -greedy selection. Line 7 computes targets using both target networks: the actor target $\pi_{\mathbf{w}_{\text{target}}}(s'_i)$ selects the next action, the critic target $q(\cdot; \boldsymbol{\theta}_{\text{target}})$ evaluates it. This replaces the $\max_{a'}$ operator in DQN. Lines 8-9 update both networks: critic via TD error minimization, actor via policy gradient through the updated critic. Line 10 performs periodic hard updates every K steps, marking outer-iteration boundaries.

The policy gradient in line 9 uses the chain rule to backpropagate through the actor-critic composition:

$$\nabla_{\mathbf{w}} q(s, \pi_{\mathbf{w}}(s); \boldsymbol{\theta}) = \nabla_a q(s, a; \boldsymbol{\theta}) \Big|_{a=\pi_{\mathbf{w}}(s)} \cdot \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(s) \quad (629)$$

This is identical to the NFQCA gradient, but now computed on mini-batches sampled from an evolving replay buffer rather than a fixed offline dataset. The critic gradient $\nabla_a q(s, a; \boldsymbol{\theta})$ at the policy-generated action provides the direction of steepest ascent in Q-value space, weighted by how sensitive the policy output is to its parameters via $\nabla_{\mathbf{w}} \pi_{\mathbf{w}}(s)$.

Twin Delayed Deep Deterministic Policy Gradient (TD3) DDPG inherits the overestimation bias from DQN’s use of the max operator in Bellman targets. TD3 [\[Fujimoto et al., 2018\]](#) addresses this through three modifications to the DDPG template, following similar principles to Double DQN but adapted for continuous actions and taking a more conservative approach.

Twin Q-Networks and the Minimum Operator Recall from the [Monte Carlo chapter](#) that overestimation arises when we use the same noisy estimate both to select which action looks best and to evaluate that action. Double Q-learning breaks this coupling by maintaining two independent estimators with noise terms $\varepsilon_a^{(1)}$ and $\varepsilon_a^{(2)}$:

$$a^* = \arg \max_a \left\{ r(s, a) + \gamma \hat{\mu}_N^{(1)}(s, a) \right\}, \quad Y = r(s, a^*) + \gamma \hat{\mu}_N^{(2)}(s, a^*). \quad (630)$$

When $\varepsilon^{(1)}$ and $\varepsilon^{(2)}$ are independent, the tower property of conditional expectation gives $E[\varepsilon_{a^*}^{(2)} \mid a^*] = E[\varepsilon_{a^*}^{(2)}] = 0$ because a^* (determined by $\varepsilon^{(1)}$) is independent of $\varepsilon^{(2)}$. This eliminates **evaluation bias**: we no longer use the same positive noise that selected an action to also inflate its value. By conditioning on the selected action and then taking expectations over the independent evaluation noise, the bias in the evaluation term vanishes.

Double DQN (Algorithm Algorithm ??) implements this principle in the discrete action setting by using the online network θ_t for selection ($a_i^* \leftarrow \arg \max_{a'} q(s'_i, a'; \theta_t)$) and the target network θ_{target} for evaluation ($y_i \leftarrow r_i + \gamma q(s'_i, a_i^*; \theta_{\text{target}})$). Since these networks experience different training noise, their errors are approximately independent, achieving the independence condition needed to eliminate evaluation bias. However, **selection bias** remains: the argmax still picks actions that received positive noise in the selection network, so $E_{\varepsilon^{(1)}}[\mu(s, a^*)] \geq \max_a \mu(s, a)$.

TD3 takes a more conservative approach. Instead of decoupling selection from evaluation, TD3 maintains **twin Q-networks** $q^A(s, a; \theta^A)$ and $q^B(s, a; \theta^B)$ trained on the same data with different random initializations. When computing targets, TD3 uses the target policy $\pi_{w_{\text{target}}}(s')$ to select actions (no maximization over a discrete set), then takes the minimum of the two Q-networks' evaluations:

$$y_i = r_i + \gamma \min \left(q^A(s'_i, \tilde{a}_i; \theta_{\text{target}}^A), q^B(s'_i, \tilde{a}_i; \theta_{\text{target}}^B) \right) \quad (631)$$

where $\tilde{a}_i = \pi_{w_{\text{target}}}(s'_i)$. This minimum operation provides a pessimistic estimate: if the two Q-networks have independent errors $q^A(s', a) = q^*(s', a) + \varepsilon^A$ and $q^B(s', a) = q^*(s', a) + \varepsilon^B$, then $E[\min(q^A, q^B)] \leq q^*(s', a)$, producing systematic underestimation rather than overestimation.

The connection to the conditional independence framework is subtle but important. While Double DQN uses independence to eliminate bias in expectation (one network selects, another evaluates), TD3 uses independence to construct a deliberate lower bound. Both approaches rely on maintaining two Q-functions with partially decorrelated errors, achieved through different initializations and stochastic gradients during training, but they aggregate these functions differently. Double DQN's decoupling targets unbiased estimation by breaking the correlation between selection and evaluation noise. TD3's minimum operation targets robust estimation by taking the most pessimistic view when the two networks disagree.

This trade-off between bias and robustness is deliberate. In actor-critic methods, the policy gradient pushes toward actions with high Q-values. Overestimation is particularly harmful because it can lead the policy to exploit erroneous high-value regions. Underestimation is generally safer: the policy may ignore some good actions, but it will not be misled into pursuing actions that only appear valuable due to approximation error. The minimum operation implements a “trust the pessimist” principle that complements the policy optimization objective.

TD3 also introduces two additional modifications beyond the clipped double Q-learning. First, target policy smoothing adds clipped noise to the target policy's actions when computing targets: $\tilde{a} = \pi_{w_{\text{target}}}(s') + \text{clip}(\varepsilon, -c, c)$. This regularization prevents the policy from exploiting narrow peaks in the Q-function approximation error by averaging over nearby actions. Second, delayed policy updates change the actor update frequency: the actor updates every d steps instead of every step. This reduces per-update error by letting the critics converge

before the actor adapts to them.

TD3 also replaces DDPG’s hard target updates with **exponential moving average (EMA)** updates, following the smooth update scheme from Algorithm Algorithm ?? in the FQI chapter. Instead of copying $\theta_{\text{target}} \leftarrow \theta_t$ every K steps, EMA smoothly tracks the online network: $\theta_{\text{target}} \leftarrow \tau\theta_t + (1-\tau)\theta_{\text{target}}$ at every update. For small $\tau \in [0.001, 0.01]$, the target lags behind the online network by roughly $1/\tau$ steps, providing smoother learning dynamics.

The algorithm structure parallels Double DQN but with continuous actions. Lines 8.1-8.2 implement clipped double Q-learning: smoothing adds noise to target actions (preventing exploitation of Q-function artifacts), and the min operation (highlighted in blue) provides pessimistic value estimates. Both critics update toward the same shared target (lines 10-11), but their different initializations and stochastic gradient noise keep their errors partially decorrelated, following the same principle underlying Double DQN’s independence assumption. Line 13 gates policy updates to every d steps (typically $d = 2$), and lines 13.2-13.4 use EMA updates following Algorithm Algorithm ??.

TD3 simplifies exploration by replacing DDPG’s Ornstein-Uhlenbeck process with uncorrelated Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma_{\text{explore}}^2)$ (line 5.3). This eliminates the need to tune multiple OU parameters while providing equally effective exploration.

Soft Actor-Critic DDPG and TD3 address continuous actions by learning a deterministic policy that amortizes the $\arg \max_a q(s, a)$ operation. But deterministic policies have a fundamental limitation: they require external exploration noise (Gaussian perturbations in TD3) and can converge to suboptimal deterministic behaviors without adequate coverage of the state-action space.

The smoothing chapter presents an alternative: entropy-regularized MDPs, where the agent maximizes expected return plus a bonus for policy randomness. This yields stochastic policies with exploration built into the objective itself. The smooth Bellman operator replaces the hard max with a soft-max:

$$v^*(s) = \frac{1}{\beta} \log \sum_{a \in \mathcal{A}} \exp(\beta \cdot q^*(s, a)) \quad (632)$$

where $\beta = 1/\alpha$ is the inverse temperature and α is the entropy regularization weight. For finite action spaces, this log-sum-exp is easy to compute. But for continuous actions $\mathcal{A} \subset \mathbb{R}^m$, the sum becomes an integral:

$$v^*(s) = \frac{1}{\beta} \log \int_{\mathcal{A}} \exp(\beta \cdot q^*(s, a)) da \quad (633)$$

This integral is intractable. We face an infinite-dimensional sum over the continuous action space. The very smoothness that gives us stochastic policies creates a new computational barrier, distinct from but analogous to the arg max problem in standard FQI.

From Intractable Integral to Tractable Expectation Soft actor-critic (SAC) [Haarnoja et al., 2018a,b] exploits an equivalence between the intractable integral and an expectation. The optimal policy under entropy regularization is the Boltzmann distribution $\pi^*(a|s) \propto \exp(\beta \cdot q^*(s, a))$. Under this policy, the soft value function becomes:

$$v^*(s) = E_{a \sim \pi^*(\cdot|s)} [q^*(s, a) - \alpha \log \pi^*(a|s)] \quad (634)$$

This converts the intractable integral into an expectation we can estimate by sampling. SAC learns a parametric policy π_ϕ that approximates the Boltzmann distribution, enabling fast action selection via a single forward pass. For bootstrap targets, SAC samples $\tilde{a}' \sim \pi_\phi(\cdot|s')$ and computes:

$$y = r + \gamma \left[\min_{j=1,2} q_{\theta_{\text{target}}}^j(s', \tilde{a}') - \alpha \log \pi_\phi(\tilde{a}'|s') \right] \quad (635)$$

The minimum over twin Q-networks applies the clipped double-Q trick from TD3. Exploration comes from the policy’s stochasticity rather than external noise.

Learning the Policy: Matching the Boltzmann Distribution The Q-network update assumes a policy π_ϕ that approximates the Boltzmann distribution $\pi^*(a|s) \propto \exp(\beta \cdot q^*(s, a))$. Training such a policy presents a problem: the Boltzmann distribution requires the partition function $Z(s) = \int_{\mathcal{A}} \exp(\beta \cdot q(s, a)) da$, the very integral we are trying to avoid. SAC sidesteps this by minimizing the KL divergence from the policy to the (unnormalized) Boltzmann distribution:

$$\min_{\phi} E_{s \sim \mathcal{D}} \left[D_{KL} \left(\pi_\phi(\cdot|s) \parallel \frac{\exp(\beta \cdot q_\theta(s, \cdot))}{Z_\theta(s)} \right) \right] \quad (636)$$

Since $\log Z_\theta(s)$ does not depend on ϕ , this reduces to:

$$\min_{\phi} E_{s \sim \mathcal{D}} E_{a \sim \pi_\phi(\cdot|s)} [\alpha \log \pi_\phi(a|s) - q_\theta(s, a)] \quad (637)$$

This pushes probability toward high Q-value actions while the $\log \pi_\phi$ term penalizes concentrating probability mass, maintaining entropy. The entropy bonus comes from the KL divergence structure rather than from an explicit regularization term.

To estimate gradients of this objective, we face a technical problem: the policy parameters ϕ appear in the sampling distribution π_ϕ , making $\nabla_\phi E_{a \sim \pi_\phi}[\cdot]$ difficult to compute. SAC uses a Gaussian policy $\pi_\phi(a|s) = \mathcal{N}(\mu_\phi(s), \sigma_\phi(s)^2)$ with the reparameterization trick. Express samples as a deterministic function of parameters and independent noise:

$$a = f_\phi(s, \epsilon) = \mu_\phi(s) + \sigma_\phi(s) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (638)$$

This moves ϕ out of the sampling distribution and into the integrand:

$$\min_{\phi} E_{s \sim \mathcal{D}} E_{\epsilon \sim \mathcal{N}(0, I)} [\alpha \log \pi_{\phi}(f_{\phi}(s, \epsilon) | s) - q_{\theta}(s, f_{\phi}(s, \epsilon))] \quad (639)$$

We can now differentiate through f_{ϕ} and the Q-network, as DDPG differentiates through a deterministic policy. SAC extends this by sampling noise ϵ at each gradient step rather than outputting a single deterministic action.

The algorithm interleaves three updates. The Q-networks (lines 7-10) follow fitted Q-iteration with the soft Bellman target: sample a next action $\tilde{a}'_i$ from the current policy, compute the entropy-adjusted target $y_i = r_i + \gamma[\min_j q_{\text{target}}^j(s'_i, \tilde{a}'_i) - \alpha \log \pi_{\phi}(\tilde{a}'_i | s'_i)]$, and minimize squared error. The minimum over twin Q-networks mitigates overestimation as in TD3. The policy (lines 12-13) updates to match the Boltzmann distribution induced by the current Q-function, using the reparameterization trick for gradient estimation. Target networks update via EMA (lines 15-16) to stabilize training.

The stochastic policy serves the same amortization purpose as in DDPG and TD3: it replaces the intractable $\arg \max$ operation with a fast network forward pass. SAC's entropy regularization produces exploration through the policy's inherent stochasticity rather than external noise. This makes SAC more robust to hyperparameters and eliminates the need to tune exploration schedules.

Path Consistency Learning (PCL) DDPG, TD3, and SAC all follow the same solution template from fitted Q-iteration: compute Bellman targets using the current Q-function, fit the Q-function to those targets, repeat. This is **successive approximation**, the function iteration approach $v_{k+1} = \text{PL} v_k$ from the [projection methods chapter](#).

Path Consistency Learning (PCL) [Nachum et al., 2017] solves the Bellman equation differently. Instead of iterating the operator, it directly minimizes a **residual**. This is the least-squares approach from projection methods: solve $\mathbf{N}(v) = 0$ by minimizing $\|\mathbf{N}(v)\|^2$. The method exploits special structure (smooth Bellman operators under deterministic dynamics) that conventional methods cannot leverage.

The Path Consistency Property Consider the entropy-regularized Q-function Bellman equation from the [smoothing chapter](#). Under general stochastic dynamics, it involves an expectation over next states:

$$q^*(s, a) = r(s, a) + \gamma E_{s'}[v^*(s')] \quad (640)$$

Suppose the dynamics are deterministic: $s' = f(s, a)$. The next state is uniquely determined, so the expectation disappears:

$$q^*(s, a) = r(s, a) + \gamma v^*(f(s, a)) \quad (641)$$

The value function relates to Q-functions through the soft-max:

$$v^*(s) = \alpha \log \int_{\mathcal{A}} \exp(q^*(s, a) / \alpha) da \quad (642)$$

Contrast two cases: general policies versus the optimal Boltzmann policy.
For general policies, the value equals an expectation:

$$v^\pi(s) = E_{a \sim \pi(\cdot|s)}[q^\pi(s, a) - \alpha \log \pi(a|s)] \quad (643)$$

This is an average. For a single observed action a , we have:

$$q^\pi(s, a) - \alpha \log \pi(a|s) = v^\pi(s) + \varepsilon(s, a) \quad (644)$$

where $\varepsilon(s, a)$ is sampling error with $E_{a \sim \pi}[\varepsilon(s, a)] = 0$. Individual actions give noisy estimates that fluctuate around the mean.

For the optimal policy under entropy regularization, the Boltzmann structure produces an exact pointwise identity. The optimal policy is:

$$\pi^*(a|s) = \frac{\exp(q^*(s, a)/\alpha)}{\exp(v^*(s)/\alpha)} \quad (645)$$

Taking logarithms and rearranging:

$$v^*(s) = q^*(s, a) - \alpha \log \pi^*(a|s) \quad \text{for all } a \quad (646)$$

This holds exactly for every action a , not just in expectation. There is no sampling error. The advantage $q^*(s, a) - v^*(s)$ is encoded in the log-probability: suboptimal actions have low $q^*(s, a)$ but also large $-\alpha \log \pi^*(a|s)$ (low probability means large negative log-probability), and these terms balance exactly to give $v^*(s)$.

Now take a trajectory segment $(s_0, a_0, s_1, a_1, \dots, s_d)$ where each transition follows the deterministic dynamics $s_{t+1} = f(s_t, a_t)$. Start with $q^*(s_0, a_0) = r_0 + \gamma v^*(s_1)$ and use equation (646) to substitute $v^*(s_1) = q^*(s_1, a_1) - \alpha \log \pi^*(a_1|s_1)$ exactly:

$$q^*(s_0, a_0) = r_0 + \gamma[q^*(s_1, a_1) - \alpha \log \pi^*(a_1|s_1)] \quad (647)$$

Substitute $q^*(s_1, a_1) = r_1 + \gamma v^*(s_2)$:

$$q^*(s_0, a_0) = r_0 + \gamma r_1 - \gamma \alpha \log \pi^*(a_1|s_1) + \gamma^2 v^*(s_2) \quad (648)$$

Continue this telescoping for d steps. Each substitution is exact:

$$q^*(s_0, a_0) = \sum_{t=0}^{d-1} \gamma^t r_t - \alpha \sum_{t=1}^{d-1} \gamma^t \log \pi^*(a_t|s_t) + \gamma^d v^*(s_d) \quad (649)$$

Apply equation (646) once more to get $v^*(s_0) = q^*(s_0, a_0) - \alpha \log \pi^*(a_0|s_0)$:

$$v^*(s_0) = \sum_{t=0}^{d-1} \gamma^t [r_t - \alpha \log \pi^*(a_t|s_t)] + \gamma^d v^*(s_d) \quad (650)$$

Rearranging gives the **path consistency residual**:

$$R(s_0 : s_d; \pi^*, v^*) = v^*(s_0) - \gamma^d v^*(s_d) - \sum_{t=0}^{d-1} \gamma^t [r_t - \alpha \log \pi^*(a_t | s_t)] = 0 \quad (651)$$

The telescoping produces an exact identity: $R = 0$ for every action sequence, not just in expectation. The behavior policy never appears because the constraint holds as a deterministic identity for any observed $(s_0, a_0, \dots, s_d)$. This enables off-policy learning without importance sampling.

Remark 6.8 (Contrasting General Policies and Optimal Boltzmann Policies). *The distinction between equations (643) and (646) is subtle but crucial.*

For general policies (equation (643)), the value is an average over actions sampled from the policy. Individual actions give noisy estimates: if we draw $a \sim \pi(\cdot | s)$, then $q^\pi(s, a) - \alpha \log \pi(a | s) = v^\pi(s) + \varepsilon$ where ε is a zero-mean random variable. We need to average many samples to estimate $v^\pi(s)$ accurately. Multi-step telescoping would accumulate these sampling errors $\varepsilon_0, \varepsilon_1, \dots, \varepsilon_{d-1}$, producing noisy residuals even at the true solution. Off-policy learning would require importance weights to correct for using actions from a different behavior policy.

For the optimal entropy-regularized policy (equation (646)), the Boltzmann structure collapses the expectation to a pointwise identity. The relationship $v^*(s) = q^*(s, a) - \alpha \log \pi^*(a | s)$ holds exactly for every action a , optimal or not. A suboptimal action has low $q^*(s, a)$ (low expected return) and low $\pi^*(a | s)$ (low probability), making $-\alpha \log \pi^*(a | s)$ large. These terms balance precisely to give $v^*(s)$. No sampling error exists. The telescoping is exact, producing a residual that equals zero for every action sequence, not just in expectation. Off-policy learning works because the constraint holds as a deterministic identity for any observed path.

This property is unique to soft-max operators. For hard-max, $v^*(s) = \max_a q^*(s, a)$ holds only when a is optimal. Suboptimal actions satisfy $v^*(s) > q^*(s, a)$, an inequality that cannot be used to construct a residual.

Structural Requirements: Deterministic Dynamics and Entropy Regularization PCL's two structural requirements (deterministic dynamics and entropy regularization) are not arbitrary design choices. Each addresses a fundamental theoretical issue.

Deterministic Dynamics: Avoiding the Double Sampling Problem Under stochastic dynamics, the Q-function Bellman equation has an expectation over next states:

$$q^*(s, a) = r(s, a) + \gamma E_{s' \sim p(\cdot | s, a)}[v^*(s')] \quad (652)$$

The exact relationship (646) still holds, so we can write the path consistency constraint. But now consider what PCL minimizes: the **squared** residual $E[R^2]$ where

$$R = v_\phi(s_0) - \gamma^d v_\phi(s_d) - \sum_{t=0}^{d-1} \gamma^t [r_t - \alpha \log \pi_\theta(a_t | s_t)] \quad (653)$$

At the true optimum (v^*, π^*) , the constraint is $E[R] = 0$, which implies $(E[R])^2 = 0$. But PCL minimizes $E[R^2]$, and by Jensen's inequality:

$$E[R^2] \geq (E[R])^2 \quad (654)$$

with equality only when R has zero variance. Under stochastic dynamics, even at optimality, individual trajectory residuals are random variables with mean zero but positive variance (due to transition noise). Minimizing $E[R^2]$ to zero would require driving $\text{Var}(R) \rightarrow 0$, which is impossible and pushes the solution away from the true optimum.

This is Baird's **double sampling problem** [Baird, 1995]. To get an unbiased gradient of $(E[R])^2$, we need:

$$\nabla(E[R])^2 = 2E[R] \cdot \nabla E[R] = 2E[R] \cdot E[\nabla R] \quad (655)$$

This requires two independent samples of the next state from the same (s, a) pair: one for estimating $E[R]$ and one for $E[\nabla R]$. With a simulator, this is possible. With real trajectories, it is not.

Under deterministic dynamics, R is deterministic (no transition noise), so $E[R^2] = (E[R])^2$ and Jensen's inequality holds with equality. Minimizing the squared residual is equivalent to solving $E[R] = 0$.

Entropy Regularization: Enabling All-Action Consistency Attempt the same path consistency derivation with the hard-max Bellman operator. Under deterministic dynamics, the Q-function satisfies:

$$q^*(s, a) = r(s, a) + \gamma v^*(f(s, a)) \quad (656)$$

where $v^*(s) = \max_{a'} q^*(s, a')$ and the optimal policy is $\pi^*(s) = \arg \max_a q^*(s, a)$ (deterministic).

Now try to relate $v^*(s)$ to an arbitrary observed action a . For the optimal action $a^* \in \arg \max_{a'} q^*(s, a')$, we have:

$$v^*(s) = q^*(s, a^*) \quad (657)$$

But for a suboptimal action $a \neq a^*$:

$$v^*(s) = \max_{a'} q^*(s, a') > q^*(s, a) \quad (658)$$

This is an inequality, not an equation. There is no formula expressing $v^*(s)$ in terms of $q^*(s, a)$ for suboptimal actions.

Attempt the multi-step telescoping. Start with $q^*(s_0, a_0) = r_0 + \gamma v^*(s_1)$. To continue, we need to express $v^*(s_1)$ using the observed action a_1 . But we only have:

$$v^*(s_1) \geq q^*(s_1, a_1) \quad (659)$$

with equality only if a_1 happens to be optimal at s_1 . We cannot substitute this into the Q-function equation to get an exact telescoping. The derivation breaks at the first step.

Compare this to the soft-max case. The Boltzmann structure gives equation (646): $v^*(s) = q^*(s, a) - \alpha \log \pi^*(a|s)$ for all actions a . The log-probability term compensates exactly for suboptimality: low-probability actions have large $-\alpha \log \pi^*(a|s)$, which adds to the low $q^*(s, a)$ to recover $v^*(s)$. This enables exact substitution at every step:

$$v^*(s_1) = q^*(s_1, a_1) - \alpha \log \pi^*(a_1|s_1) \quad (\text{exact for any } a_1) \quad (660)$$

The telescoping proceeds without inequalities or restrictions on which actions were chosen. Multi-step hard-max Q-learning lacks theoretical justification for off-policy data because when we observe a trajectory with suboptimal actions, we cannot write an exact path consistency constraint.

Both requirements are structural:

Requirement	Addresses
Deterministic dynamics	Double sampling bias: ensures $E[R^2] = (E[R])^2$
Entropy regularization	All-action consistency (equation (646))

Without deterministic dynamics, residual minimization is biased. Without entropy regularization, the constraint holds only for optimal actions.

The Learning Objective Equation (651) provides a constraint that the optimal (v^*, π^*) must satisfy: the residual equals zero for every observed path. For parametric approximations (v_ϕ, π_θ) that are not yet optimal, the residual is nonzero:

$$R(s_0 : s_d; \theta, \phi) = v_\phi(s_0) - \gamma^d v_\phi(s_d) - \sum_{t=0}^{d-1} \gamma^t [r_t - \alpha \log \pi_\theta(a_t|s_t)] \quad (661)$$

PCL minimizes the squared residual over observed path segments:

$$\min_{\theta, \phi} \sum_{\text{segments}} \frac{1}{2} R(s_i : s_{i+d}; \theta, \phi)^2 \quad (662)$$

This is the least-squares residual approach from the [projection methods chapter](#). SAC computes targets y_i and fits to them (successive approximation). PCL directly minimizes the residual without computing targets or performing a separate fitting step.

Gradient descent gives:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_\pi \sum_i R_i \cdot \alpha \sum_{t=0}^{d-1} \gamma^t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}_k}(a_{i+t}|s_{i+t}) \quad (663)$$

$$\phi_{k+1} = \phi_k - \eta_v \sum_i R_i [\nabla_{\phi} v_{\phi_k}(s_i) - \gamma^d \nabla_{\phi} v_{\phi_k}(s_{i+d})] \quad (664)$$

where $R_i = R(s_i : s_{i+d}; \boldsymbol{\theta}_k, \phi_k)$. Large residuals drive larger updates.

The algorithm collects trajectories from the current policy and stores them in a replay buffer. At each iteration, it samples a trajectory (possibly old) and performs gradient descent on the path residual for all d -step segments. The replay buffer enables off-policy learning: trajectories from old policies, expert demonstrations, or exploratory behavior all provide valid training signals.

Unified Parameterization: Single Q-Network Algorithm Algorithm ?? uses separate networks for policy and value. But we can use a single Q-network $q_{\boldsymbol{\theta}}(s, a)$ and derive both:

$$v_{\boldsymbol{\theta}}(s) = \alpha \log \sum_a \exp(q_{\boldsymbol{\theta}}(s, a)/\alpha), \quad \pi_{\boldsymbol{\theta}}(a|s) = \frac{\exp(q_{\boldsymbol{\theta}}(s, a)/\alpha)}{\sum_{a'} \exp(q_{\boldsymbol{\theta}}(s, a')/\alpha)} \quad (665)$$

The path residual becomes:

$$R(s_i : s_{i+d}; \boldsymbol{\theta}) = v_{\boldsymbol{\theta}}(s_i) - \gamma^d v_{\boldsymbol{\theta}}(s_{i+d}) - \sum_{t=0}^{d-1} \gamma^t [r_{i+t} - \alpha \log \pi_{\boldsymbol{\theta}}(a_{i+t}|s_{i+t})] \quad (666)$$

and the gradient combines both value and policy contributions through the same parameters. This unified architecture eliminates the actor-critic separation: one Q-network serves both roles.

Connection to Existing Methods Single-step case ($d = 1$): The path residual becomes $R(s : s'; \boldsymbol{\theta}, \phi) = v_{\phi}(s) - \gamma v_{\phi}(s') - r + \alpha \log \pi_{\boldsymbol{\theta}}(a|s)$. For unified parameterization where $v_{\boldsymbol{\theta}}(s) = q_{\boldsymbol{\theta}}(s, a) - \alpha \log \pi_{\boldsymbol{\theta}}(a|s)$ exactly, this becomes $R = q_{\boldsymbol{\theta}}(s, a) - r - \gamma v_{\boldsymbol{\theta}}(s')$, the soft Bellman residual. Minimizing $\sum_i R_i^2$ is equivalent to soft Q-learning, though SAC solves this via successive approximation (compute targets, fit) rather than direct residual minimization.

No entropy ($\alpha \rightarrow 0$): The residual becomes $R = v(s_i) - \gamma^d v(s_{i+d}) - \sum_t \gamma^t r_t$, the negative d -step advantage. But unlike A2C/A3C where v tracks the current policy's value, PCL's value converges to v^* because the residual couples policy and value through the optimality condition.

Multi-step with hard-max: No analog exists. The hard-max Bellman operator $\max_a q(s, a)$ does not have an exact pointwise relationship like equation (646). Multi-step telescoping would accumulate errors from the max operator, making the constraint valid only in expectation under the optimal policy. The soft-max structure enables exact off-policy path consistency.

PCL vs SAC: Residual Minimization vs Successive Approximation

Both methods solve entropy-regularized MDPs but use fundamentally different solution strategies:

Aspect	SAC	PCL
Solution method	Successive approximation: compute targets y_i , fit q to targets	Residual minimization: minimize $\sum_i R_i^2$ directly
Update structure	Target computation + regression step	Single gradient step on squared residual
Target networks	Required (mark outer-iteration boundaries)	None (residual constraint, not target fitting)
Temporal horizon	Single-step TD: $y = r + \gamma V(s')$	Multi-step paths: accumulate over d steps
Off-policy handling	Replay buffer with single-sample bias	No importance sampling (works for any trajectory)
Dynamics requirement	General stochastic transitions	Deterministic transitions $s' = f(s, a)$
Architecture	Twin Q-networks + policy network	Single Q-network (unified parameterization)

PCL requires deterministic dynamics. It gains multi-step telescoping and off-policy learning without importance weights, but only for deterministic systems (robotic manipulation, many control tasks). SAC works for general stochastic MDPs.

PCL as Amortization PCL amortizes at a different level than DDPG/TD3/SAC.

Those methods amortize the action maximization: learn a policy network that outputs $\arg \max_a q(s, a)$ directly. PCL amortizes the solution of the Bellman equation itself. Instead of repeatedly applying the Bellman operator (which requires $\int_{\mathcal{A}} \exp(q/\alpha) da$ at every iteration), PCL samples path segments and minimizes their residual. The computational cost of verifying optimality across all states and path lengths is distributed across training through sampled gradient updates.

Model Predictive Path Integral Control SAC and PCL both learn policies that approximate the Boltzmann distribution $\pi^*(a|s) \propto \exp(\beta \cdot q^*(s, a))$ induced by entropy regularization. This amortization allows fast action selection at deployment: a single forward pass through the policy network. An alternative approach forgoes learning entirely and instead performs optimization at every decision.

Model Predictive Path Integral control (MPPI) [Williams et al., 2017] uses the Boltzmann weighting directly for action sequence selection. Given a dynamics model $s_{t+1} = f(s_t, a_t)$ and current state s_0 , MPPI samples K action

sequences $\{\mathbf{a}^{(i)}\}_{i=1}^K$, rolls them out to get costs $C^{(i)} = \sum_{t=0}^{H-1} c(s_t^{(i)}, a_t^{(i)})$, and computes the optimal action as a weighted average:

$$a_0^* = \sum_{i=1}^K w^{(i)} a_0^{(i)}, \quad w^{(i)} = \frac{\exp(-C^{(i)}/\lambda)}{\sum_j \exp(-C^{(j)}/\lambda)} \quad (667)$$

where $\lambda > 0$ is a temperature parameter. The weighting $w^{(i)} \propto \exp(-C^{(i)}/\lambda)$ is exactly the Boltzmann distribution. MPPI solves the entropy-regularized objective:

$$\min_{\mathbf{a}} E_{\xi} \left[\sum_{t=0}^{H-1} c(s_t, a_t) + \lambda H(\pi) \right] \quad (668)$$

where π is the distribution over action sequences and $H(\pi) = -E[\log \pi(\mathbf{a})]$ is entropy. The importance sampling estimate approximates the optimal action under this objective. The temperature λ controls the trade-off between exploitation (focus on low-cost sequences) and exploration (maintain entropy).

The algorithm samples perturbed action sequences around a nominal trajectory $\{\bar{a}_t\}$ (often the previous optimal sequence, shifted forward). The Boltzmann weights assign high probability to low-cost sequences. After executing a_0^* , the agent observes the next state and replans.

MPPI as Non-Amortized Optimization The contrast between MPPI and the methods in this chapter illuminates what amortization provides. SAC learns a policy π_{ϕ} that approximates the Boltzmann distribution over actions at each state. PCL learns a Q-function from which the Boltzmann policy can be derived. Both invest computational effort during training to enable fast action selection at deployment: a single forward pass.

MPPI performs full optimization at every decision. At each state, it samples action sequences, weights them by exponentiated costs, and returns the weighted average. No learning occurs. The policy is implicitly defined by the optimization procedure itself.

This trade-off has practical consequences:

Aspect	Amortized (SAC, PCL)	Non-Amortized (MPPI)
Action selection	Single forward pass	$O(KH)$ model evaluations
Generalization	Policy generalizes across states	Optimization from scratch at each state
Model requirement	None (SAC) or deterministic (PCL)	Accurate dynamics model
Approximation error	Policy network approximation	None (exact optimization)
Adaptability	Requires retraining for new tasks	Adapts immediately to new cost functions

MPPI excels at real-time control for systems with fast, accurate models (robotics, autonomous vehicles). The replanning handles model errors and disturbances without retraining. However, the per-step computation ($K \approx 100$ -1000 rollouts) makes it expensive for complex dynamics or long horizons.

The entropy regularization that connects SAC, PCL, and MPPI is not coincidental. All three methods solve variants of the soft Bellman equation. SAC and PCL amortize the solution by learning value functions and policies. MPPI solves it directly through sampling. The Boltzmann weighting emerges in all cases as the optimal policy structure under entropy regularization.

An Alternative: Euler Equation Methods The methods developed in this chapter all parameterize policies, but they remain rooted in the Bellman equation. NFQCA, DDPG, TD3, and SAC learn Q-functions through successive approximation, then derive policies by maximizing these Q-functions. PCL minimizes a path residual derived from the soft Bellman equation. The policy serves as an amortized optimizer for a value-based objective.

There is a different approach, developed in computational economics [Judd, 1992, Rust, 1996, Judd, 1998], that also parameterizes policies but solves an entirely different functional equation. Consider a control problem with continuous states and actions, deterministic dynamics $s' = f(s, a)$, and differentiable reward $r(s, a)$. The optimal action $\pi^*(s)$ satisfies the first-order condition:

$$\left. \frac{\partial r(s, a)}{\partial a} \right|_{a=\pi^*(s)} + \gamma \left. \frac{\partial v^*(s')}{\partial s'} \right|_{s'=f(s, \pi^*(s))} \left. \frac{\partial f(s, a)}{\partial a} \right|_{a=\pi^*(s)} = 0. \quad (669)$$

This Euler equation expresses optimality through derivatives rather than through the max operator. For problems with special structure (the Euler class, where dynamics are affine in the controlled state), envelope theorems eliminate v^* entirely, yielding a closed functional equation $\mathcal{E}(\pi)(s) = 0$ in the policy alone.

With a parameterized policy $\pi_\theta(s)$, we can discretize via collocation or Galerkin projection:

$$G(\theta) := \begin{bmatrix} \mathcal{E}(\pi_\theta)(s_1) \\ \vdots \\ \mathcal{E}(\pi_\theta)(s_N) \end{bmatrix} = 0. \quad (670)$$

This is root-finding, not fixed-point iteration. Newton-type methods replace the successive approximation of fitted Q-iteration. The Euler operator is not a contraction, so convergence guarantees are problem-dependent.

What does this mean for reinforcement learning? The Euler approach shares the amortization idea: learn a policy network that directly outputs actions. But the training objective comes from first-order optimality conditions rather than from Bellman residuals or Q-function maximization. This raises questions worth considering. Could Euler-style objectives provide useful training signals

for actor-critic methods? When dynamics are known or learned, could first-order conditions offer advantages over value-based objectives? The connection between these traditions remains underexplored.

Summary When actions are continuous, computing $\max_a q(s, a; \theta)$ at each Bellman target requires solving a nonlinear program. Amortization replaces this runtime optimization with a learned policy network: a single forward pass instead of an optimization loop. The FQI structure from the [previous chapter](#) remains intact, with the policy network providing actions for target computation.

NFQCA, DDPG, TD3, and SAC use successive approximation: compute Bellman targets, fit Q-function to targets, update policy to maximize Q-function. Deterministic policies (DDPG, TD3) require external exploration noise; stochastic policies (SAC) explore through their inherent randomness. TD3 and SAC use twin Q-networks with $\min(q^1, q^2)$ to mitigate overestimation. PCL takes a different approach, minimizing the path consistency residual directly rather than iterating the Bellman operator. This requires deterministic dynamics but enables multi-step constraints without importance sampling.

MPPI forgoes learning entirely, performing Boltzmann-weighted optimization at every decision. This avoids policy approximation error but requires $O(KH)$ model rollouts per action. SAC, PCL, and MPPI all solve entropy-regularized objectives; SAC and PCL amortize the solution while MPPI computes it directly.

The [next chapter](#) takes a different approach: parameterize the policy directly and optimize via gradient ascent on expected return. The starting point is derivative estimation for stochastic optimization rather than Bellman equations, though value functions return as variance reduction tools in actor-critic methods.

Self-checks

6.3.4 Policy Gradient Methods

The [previous chapter](#) showed how to handle continuous action spaces in fitted Q-iteration by amortizing action selection with policy networks. Methods like NFQCA, DDPG, TD3, and SAC all learn both a Q-function and a policy, using the Q-function to guide policy improvement. This chapter explores a different approach: optimizing policies directly without maintaining explicit value functions.

Direct policy optimization offers several advantages. First, it naturally handles stochastic policies, which can be essential for partially observable environments or problems requiring explicit exploration. Second, it avoids the detour through value function approximation, which may introduce errors that compound during policy extraction. Third, for problems with simple policy classes but complex value landscapes, directly searching in policy space can be more efficient than searching in value space.

The foundation of policy gradient methods rests on computing gradients of expected returns with respect to policy parameters. This chapter develops the mathematical machinery needed for this computation, starting with general derivative estimation techniques from stochastic optimization, then specializing to reinforcement learning settings, and finally examining variance reduction methods that make these estimators practical.

Derivative Estimation for Stochastic Optimization Consider optimizing an objective that involves an expectation:

$$J(\theta) = E_{x \sim p(x; \theta)}[f(x, \theta)] \quad (671)$$

For concreteness, consider a simple example where $x \sim \mathcal{N}(\theta, 1)$ and $f(x, \theta) = x^2\theta$. The derivative we seek is:

$$\frac{d}{d\theta} J(\theta) = \frac{d}{d\theta} \int x^2 \theta p(x; \theta) dx \quad (672)$$

While we can compute this exactly for the Gaussian example, this is often impossible for more general problems. We might then be tempted to approximate our objective using samples:

$$J(\theta) \approx \frac{1}{N} \sum_{i=1}^N f(x_i, \theta), \quad x_i \sim p(x; \theta) \quad (673)$$

Then differentiate this approximation:

$$\frac{d}{d\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial \theta} f(x_i, \theta) \quad (674)$$

However, this naive approach ignores that the samples themselves depend on θ . The correct derivative requires the product rule:

$$\frac{d}{d\theta} J(\theta) = \int \frac{\partial}{\partial \theta} [f(x, \theta) p(x; \theta)] dx = \int \left[\frac{\partial f}{\partial \theta} p(x; \theta) + f(x, \theta) \frac{\partial p(x; \theta)}{\partial \theta} \right] dx \quad (675)$$

While the first term could be numerically integrated using Monte Carlo, the second one cannot as it is not in the form of an expectation.

To transform our objective so that the Monte Carlo estimator for the objective could be differentiated directly while ensuring that the resulting derivative is unbiased, there are two main solutions: a change of measure, or a change of variables.

The Likelihood Ratio Method One solution comes from rewriting our objective using a proposal distribution $q(x)$ that does not depend on θ :

$$J(\theta) = \int f(x, \theta) \frac{p(x; \theta)}{q(x)} q(x) dx = E_{x \sim q(x)} \left[f(x, \theta) \frac{p(x; \theta)}{q(x)} \right] \quad (676)$$

Define the likelihood ratio $\rho(x, q, \theta) \equiv \frac{p(x; \theta)}{q(x)}$, where we treat q as a separate argument. The objective becomes:

$$J(\theta) = E_{x \sim q(x)} [f(x, \theta) \rho(x, q, \theta)] \quad (677)$$

When we differentiate J , we take the partial derivative with respect to θ while holding q fixed (since q does not depend on θ):

$$\frac{d}{d\theta} J(\theta) = E_{x \sim q(x)} \left[f(x, \theta) \frac{\partial \rho}{\partial \theta}(x, q, \theta) + \rho(x, q, \theta) \frac{\partial f}{\partial \theta}(x, \theta) \right] \quad (678)$$

The partial derivative of ρ with respect to θ (treating q as fixed) is:

$$\frac{\partial \rho}{\partial \theta}(x, q, \theta) = \frac{1}{q(x)} \frac{\partial p(x; \theta)}{\partial \theta} = \rho(x, q, \theta) \frac{\partial \log p(x; \theta)}{\partial \theta} \quad (679)$$

Now fix any reference parameter θ_0 and choose the proposal distribution $q(x) = p(x; \theta_0)$. This is a *fixed* distribution that does not change as θ varies. We simply evaluate the family $p(x; \cdot)$ at the specific point θ_0 . With this choice, evaluating the gradient at $\theta = \theta_0$ gives $\rho(x, q, \theta_0) = p(x; \theta_0)/p(x; \theta_0) = 1$. The gradient formula becomes:

$$\left. \frac{d}{d\theta} J(\theta) \right|_{\theta=\theta_0} = E_{x \sim p(x; \theta_0)} \left[f(x, \theta_0) \left. \frac{\partial \log p(x; \theta)}{\partial \theta} \right|_{\theta_0} + \left. \frac{\partial f(x, \theta)}{\partial \theta} \right|_{\theta_0} \right] \quad (680)$$

Since θ_0 is arbitrary, we can drop the subscript and write the **score function estimator** as:

$$\frac{d}{d\theta} J(\theta) = E_{x \sim p(x; \theta)} \left[f(x, \theta) \frac{\partial \log p(x; \theta)}{\partial \theta} + \frac{\partial f(x, \theta)}{\partial \theta} \right] \quad (681)$$

The Reparameterization Trick An alternative approach eliminates the θ -dependence in the sampling distribution by expressing x through a deterministic transformation of the noise:

$$x = g(\epsilon, \theta), \quad \epsilon \sim q(\epsilon) \quad (682)$$

Therefore if we want to sample from some target distribution $p(x; \theta)$, we can do so by first sampling from a simple base distribution $q(\epsilon)$ (like a standard normal) and then transforming those samples through a carefully chosen function g . If $g(\cdot, \theta)$ is invertible, the change of variables formula tells us how these distributions relate:

$$p(x; \theta) = q(g^{-1}(x, \theta)) \left| \det \frac{\partial g^{-1}(x, \theta)}{\partial x} \right| = q(\epsilon) \left| \det \frac{\partial g(\epsilon, \theta)}{\partial \epsilon} \right|^{-1} \quad (683)$$

For example, if we want to sample from any multivariate Gaussian distributions with covariance matrix Σ and mean μ , it suffices to be able to sample from a standard normal noise and compute the linear transformation:

$$x = \mu + \Sigma^{1/2} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (684)$$

where $\Sigma^{1/2}$ is the matrix square root obtained via Cholesky decomposition. In the univariate case, this transformation is simply:

$$x = \mu + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1) \quad (685)$$

where $\sigma = \sqrt{\sigma^2}$ is the standard deviation (square root of the variance).

Common Examples of Reparameterization

The Truncated Normal Distribution When we need samples constrained to an interval $[a, b]$, we can use the truncated normal distribution. To sample from it, we transform uniform noise through the inverse cumulative distribution function (CDF) of the standard normal:

$$x = \Phi^{-1}(u\Phi(b) + (1 - u)\Phi(a)), \quad u \sim \text{Uniform}(0, 1) \quad (686)$$

Here:

- $\Phi(z) = \frac{1}{2} \left[1 + \text{erf} \left(\frac{z}{\sqrt{2}} \right) \right]$ is the CDF of the standard normal distribution
- Φ^{-1} is its inverse (the quantile function)
- $\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$ is the error function

The resulting samples follow a normal distribution restricted to $[a, b]$, with the density properly normalized over this interval.

The Kumaraswamy Distribution When we need samples in the unit interval $[0,1]$, a natural choice might be the Beta distribution. However, its inverse CDF doesn't have a closed form. Instead, we can use the Kumaraswamy distribution as a convenient approximation, which allows for a simple reparameterization:

$$x = (1 - (1 - u^\alpha)^{1/\beta}), \quad u \sim \text{Uniform}(0, 1) \quad (687)$$

where:

- $\alpha, \beta > 0$ are shape parameters that control the distribution
- α determines the concentration around 0
- β determines the concentration around 1
- The distribution is similar to $\text{Beta}(\alpha, \beta)$ but with analytically tractable CDF and inverse CDF

The Kumaraswamy distribution has density:

$$f(x; \alpha, \beta) = \alpha\beta x^{\alpha-1}(1-x^\alpha)^{\beta-1}, \quad x \in [0, 1] \quad (688)$$

The Gumbel-Softmax Distribution When sampling from a categorical distribution with probabilities $\{\pi_i\}$, one approach uses $\text{Gumbel}(0, 1)$ noise combined with the argmax of log-perturbed probabilities:

$$\text{argmax}_i(\log \pi_i + g_i), \quad g_i \sim \text{Gumbel}(0, 1) \quad (689)$$

This approach, known in machine learning as the Gumbel-Max trick, relies on sampling Gumbel noise from uniform random variables through the transformation $g_i = -\log(-\log(u_i))$ where $u_i \sim \text{Uniform}(0, 1)$. To see why this gives us samples from the categorical distribution, consider the probability of selecting category i :

$$\begin{aligned} P(\text{argmax}_j(\log \pi_j + g_j) = i) &= P(\log \pi_i + g_i > \log \pi_j + g_j \text{ for all } j \neq i) \\ &= P(g_i - g_j > \log \pi_j - \log \pi_i \text{ for all } j \neq i) \end{aligned}$$

Since the difference of two Gumbel random variables follows a logistic distribution, $g_i - g_j \sim \text{Logistic}(0, 1)$, and these differences are independent for different j (due to the independence of the original Gumbel variables), we can write:

$$\begin{aligned} P(\text{argmax}_j(\log \pi_j + g_j) = i) &= \prod_{j \neq i} P(g_i - g_j > \log \pi_j - \log \pi_i) \\ &= \prod_{j \neq i} \frac{\pi_i}{\pi_i + \pi_j} = \pi_i \end{aligned}$$

The last equality requires some additional algebra to show, but follows from the fact that these probabilities must sum to 1 over all i .

While we have shown that the Gumbel-Max trick gives us exact samples from a categorical distribution, the argmax operation isn't differentiable. For stochastic optimization problems of the form:

$$E_{x \sim p(x; \theta)}[f(x)] = E_{\epsilon \sim \text{Gumbel}(0,1)}[f(g(\epsilon, \theta))] \quad (690)$$

we need g to be differentiable with respect to θ . This leads us to consider a continuous relaxation where we replace the hard argmax with a temperature-controlled softmax:

$$z_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum_j \exp((\log \pi_j + g_j)/\tau)} \quad (691)$$

As $\tau \rightarrow 0$, this approximation approaches the argmax:

$$\lim_{\tau \rightarrow 0} \frac{\exp(x_i/\tau)}{\sum_j \exp(x_j/\tau)} = \begin{cases} 1 & \text{if } x_i = \max_j x_j \\ 0 & \text{otherwise} \end{cases} \quad (692)$$

The resulting distribution over the probability simplex is called the Gumbel-Softmax (or Concrete) distribution. The temperature parameter τ controls the discreteness of our samples: smaller values give samples closer to one-hot vectors but with less stable gradients, while larger values give smoother gradients but more diffuse samples.

Numerical Analysis of Gradient Estimators Let us examine the behavior of our three gradient estimators for the stochastic optimization objective:

$$J(\theta) = E_{x \sim \mathcal{N}(\theta, 1)}[x^2 \theta] \quad (693)$$

To get an analytical expression for the derivative, first note that we can factor out θ to obtain $J(\theta) = \theta E[x^2]$ where $x \sim \mathcal{N}(\theta, 1)$. By definition of the variance, we know that $\text{Var}(x) = E[x^2] - (E[x])^2$, which we can rearrange to $E[x^2] = \text{Var}(x) + (E[x])^2$. Since $x \sim \mathcal{N}(\theta, 1)$, we have $\text{Var}(x) = 1$ and $E[x] = \theta$, therefore $E[x^2] = 1 + \theta^2$. This gives us:

$$J(\theta) = \theta(1 + \theta^2) \quad (694)$$

Now differentiating with respect to θ using the product rule yields:

$$\frac{d}{d\theta} J(\theta) = 1 + 3\theta^2 \quad (695)$$

For concreteness, we fix $\theta = 1.0$ and analyze samples drawn using Monte Carlo estimation with batch size 1000 and 1000 independent trials. Evaluating at $\theta = 1$ gives us $\frac{d}{d\theta} J(\theta)|_{\theta=1} = 1 + 3(1)^2 = 4$, which serves as our ground truth against which we compare our estimators:

1. First, we consider the naive estimator that incorrectly differentiates the Monte Carlo approximation:

$$\hat{g}_{\text{naive}}(\theta) = \frac{1}{N} \sum_{i=1}^N x_i^2 \quad (696)$$

For $x \sim \mathcal{N}(1, 1)$, we have $E[x^2] = \theta^2 + 1 = 2.0$ and $E[\hat{g}_{\text{naive}}] = 2.0$. We should therefore expect a bias of about -2 in our experiment.

2. Then we compute the score function estimator:

$$\hat{g}_{\text{SF}}(\theta) = \frac{1}{N} \sum_{i=1}^N [x_i^2 \theta (x_i - \theta) + x_i^2] \quad (697)$$

This estimator is unbiased with $E[\hat{g}_{\text{SF}}] = 4$

3. Finally, through the reparameterization $x = \theta + \epsilon$ where $\epsilon \sim \mathcal{N}(0, 1)$, we obtain:

$$\hat{g}_{\text{RT}}(\theta) = \frac{1}{N} \sum_{i=1}^N [2\theta(\theta + \epsilon_i) + (\theta + \epsilon_i)^2] \quad (698)$$

This estimator is also unbiased with $E[\hat{g}_{\text{RT}}] = 4$.

The numerical experiments corroborate our theory. The naive estimator consistently underestimates the true gradient by 2.0, though it maintains a relatively small variance. This systematic bias would make it unsuitable for optimization despite its low variance. The score function estimator corrects this bias but introduces substantial variance. While unbiased, this estimator would require many samples to achieve reliable gradient estimates. Finally, the reparameterization trick achieves a much lower variance while remaining unbiased. While this experiment is for didactic purposes only, it reproduces what is commonly found in practice: that when applicable, the reparameterization estimator tends to perform better than the score function counterpart.

Score Function Methods in Reinforcement Learning The score function estimator from the previous section applies directly to reinforcement learning. Since it requires only the ability to evaluate and differentiate $\log \pi_{\mathbf{w}}(a|s)$, it works with any differentiable policy, including discrete action spaces where reparameterization is unavailable. It requires no model of the environment dynamics.

Let $G(\tau) \equiv \sum_{t=0}^T r(s_t, a_t)$ be the sum of undiscounted rewards in a trajectory τ . The stochastic optimization problem we face is to maximize:

$$J(\mathbf{w}) = E_{\tau \sim p(\tau; \mathbf{w})}[G(\tau)] \quad (699)$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory and $G(\tau)$ is the total return. Applying the score function estimator, we get:

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= \nabla_{\mathbf{w}} E_{\tau}[G(\tau)] \\ &= E_{\tau}[G(\tau) \nabla_{\mathbf{w}} \log p(\tau; \mathbf{w})] \\ &= E_{\tau} \left[G(\tau) \nabla_{\mathbf{w}} \sum_{t=0}^T \log \pi_{\mathbf{w}}(a_t | s_t) \right] \\ &= E_{\tau} \left[G(\tau) \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \right]\end{aligned}$$

We have eliminated the need to know the transition probabilities in this estimator since the probability of a trajectory factorizes as:

$$p(\tau; \mathbf{w}) = p(s_0) \prod_{t=0}^T \pi_{\mathbf{w}}(a_t | s_t) p(s_{t+1} | s_t, a_t) \quad (700)$$

Therefore, only the policy depends on $\mathbf{w}$. When taking the logarithm of this product, we get a sum where all the $\mathbf{w}$ -independent terms vanish. The final estimator samples trajectories under the distribution $p(\tau; \mathbf{w})$ and computes:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \approx \frac{1}{N} \sum_{i=1}^N \left[G(\tau^{(i)}) \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t^{(i)} | s_t^{(i)}) \right] \quad (701)$$

This is a direct application of the score function estimator. However, we rarely use this form in practice and instead make several improvements to further reduce the variance.

Leveraging Conditional Independence Given the Markov property of the MDP, rewards r_k for $k < t$ are conditionally independent of action a_t given the history $h_t = (s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t)$. This allows us to only need to consider future rewards when computing policy gradients.

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= E_{\tau} \left[\sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \sum_{k=0}^T r_k \right] \\ &= E_{\tau} \left[\sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \left(\sum_{k=0}^{t-1} r_k + \sum_{k=t}^T r_k \right) \right] \\ &= E_{\tau} \left[\sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \sum_{k=t}^T r_k \right]\end{aligned}$$

The conditional independence assumption means that the term $E_{\tau} \left[\sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \sum_{k=0}^{t-1} r_k \right]$ vanishes. To see this, factor the trajectory distribution as:

$$p(\tau) = p(s_0, \dots, s_t, a_0, \dots, a_{t-1}) \pi_{\mathbf{w}}(a_t | s_t) p(s_{t+1}, \dots, s_T, a_{t+1}, \dots, a_T | s_t, a_t) \quad (702)$$

We can now re-write a single term of this summation as:

$$E_{\tau} \left[\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \sum_{k=0}^{t-1} r_k \right] = E_{s_{0:t}, a_{0:t-1}} \left[\sum_{k=0}^{t-1} r_k E_{a_t} [\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)] \right] \quad (703)$$

The inner expectation is zero because

$$\begin{aligned} E_{a_t} [\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)] &= \int \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \pi_{\mathbf{w}}(a_t | s_t) da_t \\ &= \int \frac{\nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a_t | s_t)}{\pi_{\mathbf{w}}(a_t | s_t)} \pi_{\mathbf{w}}(a_t | s_t) da_t \\ &= \int \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a_t | s_t) da_t \\ &= \nabla_{\mathbf{w}} \int \pi_{\mathbf{w}}(a_t | s_t) da_t \\ &= \nabla_{\mathbf{w}} 1 = 0 \end{aligned}$$

The Monte Carlo estimator becomes:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \approx \frac{1}{N} \sum_{i=1}^N \left[\sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t^{(i)} | s_t^{(i)}) \sum_{k=t}^T r_k^{(i)} \right] \quad (704)$$

This gives us the REINFORCE algorithm:

The benefit of this estimator compared to the naive one (which would weight each score function by the full trajectory return $G(\tau)$) is that it generally has less variance. This variance reduction arises from the conditional independence structure we exploited: past rewards do not depend on future actions. More formally, this estimator is an instance of a variance reduction technique known as the Extended Conditional Monte Carlo Method.

The Surrogate Loss Perspective The algorithm above computes a gradient estimate $\hat{g}$ explicitly. In practice, implementations using automatic differentiation frameworks take a different approach: they define a **surrogate loss** whose gradient matches the REINFORCE estimator. For a single trajectory, consider:

$$L_{\text{surrogate}}(\mathbf{w}) = - \sum_{t=0}^T \log \pi_{\mathbf{w}}(a_t | s_t) G_t \quad (705)$$

where the returns G_t and actions a_t are treated as fixed constants (detached from the computation graph). Taking the gradient with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}} L_{\text{surrogate}} = - \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) G_t \quad (706)$$

Minimizing this surrogate loss via gradient descent yields the same update as maximizing expected return via REINFORCE. The negative sign converts our maximization problem into a minimization suitable for standard optimizers.

This surrogate loss is **not** the expected return $J(\mathbf{w})$ we are trying to maximize. It is a computational device that produces the correct gradient at the current parameter values. Several properties distinguish it from a true loss function:

1. **It changes each iteration.** The returns G_t come from trajectories sampled under the current policy. After updating $\mathbf{w}$, we must collect new trajectories and construct a new surrogate loss.
2. **Its value is not meaningful.** Unlike supervised learning where the loss measures prediction error, the numerical value of $L_{\text{surrogate}}$ has no direct interpretation. Only its gradient matters.
3. **It is valid only locally.** The surrogate loss provides the correct gradient only at the parameters used to collect the data. Moving far from those parameters invalidates the gradient estimate.

This perspective explains why policy gradient code often looks different from the pseudocode above. Instead of computing $\hat{g}$ explicitly, implementations define the surrogate loss and call `loss.backward()`:

```
# Surrogate loss implementation (single trajectory)
log_probs = [policy.log_prob(a_t, s_t) for s_t, a_t in trajectory]
returns = compute_returns(rewards)
surrogate_loss = -sum(lp * G for lp, G in zip(log_probs, returns))
surrogate_loss.backward() # computes REINFORCE gradient
optimizer.step()
```

Variance Reduction via Control Variates Recall that the REINFORCE gradient estimator, after leveraging conditional independence, takes the form:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \approx \frac{1}{N} \sum_{i=1}^N \left[\sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t^{(i)} | s_t^{(i)}) \sum_{k=t}^T r_k^{(i)} \right] \quad (707)$$

This is a sum over trajectories and timesteps. The gradient contribution at timestep t of trajectory i is:

$$\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t^{(i)} | s_t^{(i)}) \sum_{k=t}^T r_k^{(i)} \quad (708)$$

While unbiased, this estimator suffers from high variance because the return $\sum_{k=t}^T r_k$ can vary significantly across trajectories even for the same state-action pair. The control variate method provides a principled way to reduce this variance.

General Control Variate Theory For a general estimator Z of some quantity $\mu = E[Z]$, and a control variate C with known expectation $E[C] = 0$, we can construct:

$$Z_{cv} = Z - \alpha C \quad (709)$$

This remains unbiased since $E[Z_{cv}] = E[Z] - \alpha E[C] = E[Z]$. The variance is:

$$\text{Var}(Z_{cv}) = \text{Var}(Z) + \alpha^2 \text{Var}(C) - 2\alpha \text{Cov}(Z, C) \quad (710)$$

The $-2\alpha \text{Cov}(Z, C)$ term is what enables variance reduction. If Z and C are positively correlated, we can choose $\alpha > 0$ to make this term negative and large in magnitude, reducing the overall variance. However, the $\alpha^2 \text{Var}(C)$ term grows quadratically with α , so if we make α too large, this quadratic term will eventually dominate and the variance will increase rather than decrease. The variance as a function of α is a parabola opening upward, with a unique minimum. Setting $\frac{d}{d\alpha} \text{Var}(Z_{cv}) = 0$ gives:

$$\alpha^* = \frac{\text{Cov}(Z, C)}{\text{Var}(C)} \quad (711)$$

This is the coefficient from ordinary least squares regression: we predict the estimator Z using the control variate C as the predictor. Since $E[C] = 0$, the linear model is $Z \approx E[Z] + \alpha^* C$, where α^* is the OLS slope coefficient. The control variate estimator $Z_{cv} = Z - \alpha^* C$ computes the residual: the part of Z that cannot be explained by C .

Substituting α^* into the variance formula yields:

$$\text{Var}(Z_{cv}) = \text{Var}(Z) - \frac{[\text{Cov}(Z, C)]^2}{\text{Var}(C)} = (1 - R^2) \text{Var}(Z) \quad (712)$$

where $R^2 = \frac{[\text{Cov}(Z, C)]^2}{\text{Var}(Z) \text{Var}(C)}$ is the coefficient of determination from regressing Z on C . The variance reduction is $R^2 \text{Var}(Z)$: the better C predicts Z , the more variance we eliminate.

Application to REINFORCE In the reinforcement learning setting, our REINFORCE gradient estimator is a sum over timesteps: $\sum_{t=0}^T Z_t$ where each Z_t represents the gradient contribution at timestep t . We apply control variates separately to each term. Since $\text{Var}(\sum_t Z_t) = \sum_t \text{Var}(Z_t) + \sum_{t \neq s} \text{Cov}(Z_t, Z_s)$, reducing the variance of each Z_t reduces the total variance, though we do not explicitly address the cross-timestep covariance terms.

For a given trajectory at state s_t , the gradient contribution at time t is:

$$Z_t = \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \sum_{k=t}^T r_k \quad (713)$$

This is the product of the score function $\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)$ and the return-to-go $\sum_{k=t}^T r_k$. We can subtract any state-dependent function $b(s_t)$ from the return without introducing bias, as long as $b(s_t)$ does not depend on a_t . This is because:

$$E_{a_t \sim \pi_{\mathbf{w}}(\cdot | s_t)} [\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) b(s_t)] = b(s_t) E_{a_t} [\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)] = 0 \quad (714)$$

where the last equality follows from the score function identity (??).

We can now define our control variate as:

$$C_t = \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \cdot b(s_t) \quad (715)$$

where $b(s_t)$ is a **baseline function** that depends only on the state. This satisfies $E[C_t | s_t] = 0$. Our control variate estimator becomes:

$$Z_{t,cv} = Z_t - C_t = \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \left(\sum_{k=t}^T r_k - b(s_t) \right) \quad (716)$$

The optimal baseline $b^*(s_t)$ minimizes the variance. To find it, consider the scalar parameter case for simplicity. Write $g(a_t) \equiv \nabla_w \log \pi_w(a_t | s_t)$ and $G_t = \sum_{k=t}^T r_k$. We want to minimize:

$$b^*(s_t) = \arg \min_b \text{Var}_{a_t \sim \pi_{\mathbf{w}}(\cdot | s_t)} [g(a_t)(G_t - b)] \quad (717)$$

Since the mean does not depend on b , minimizing the variance is equivalent to minimizing the second moment $E[g(a_t)^2(G_t - b)^2 | s_t]$. Expanding and taking the derivative with respect to b gives:

$$b^*(s_t) = \frac{E_{a_t | s_t} [g(a_t)^2 G_t]}{E_{a_t | s_t} [g(a_t)^2]} \quad (718)$$

For vector-valued parameters $\mathbf{w}$, we minimize a scalar proxy such as the trace of the covariance matrix, which yields the same formula with $\|\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)\|^2$ in place of $g(a_t)^2$:

$$b^*(s_t) = \frac{E_{a_t | s_t} [\|\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)\|^2 G_t]}{E_{a_t | s_t} [\|\nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t)\|^2]} \quad (719)$$

This is the exact optimal baseline: a weighted average of returns where the weights are the squared norms of the score function. In practice, we treat the squared norm as roughly constant across actions at a given state, which leads to the simpler and widely used choice:

$$b(s_t) \approx E[G_t|s_t] = v^{\pi_w}(s_t) \quad (720)$$

With this approximation, the variance-reduced gradient contribution at timestep t becomes:

$$Z_{cv,t} = \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t|s_t) \left(\sum_{k=t}^T r_k - v^{\pi_w}(s_t) \right) \quad (721)$$

The term in parentheses is exactly the **advantage function**: $A^{\pi_w}(s_t, a_t) = q^{\pi_w}(s_t, a_t) - v^{\pi_w}(s_t)$, where the Q-function is approximated by the Monte Carlo return $\sum_{k=t}^T r_k$. The full gradient estimate for a trajectory is then the sum over all timesteps:

$$\hat{g} = \sum_{t=0}^T Z_{cv,t} = \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t|s_t) (G_t - v^{\pi_w}(s_t)) \quad (722)$$

In practice, we do not have access to the true value function and must learn it. Unlike the methods in the [amortization chapter](#), where we learned value functions to approximate the *optimal* Q-function, here our goal is **policy evaluation**: estimating the value of the *current* policy π_w . The same function approximation techniques apply, but we target v^{π_w} rather than v^* . The simplest approach is to regress from states to Monte Carlo returns, learning what [Williams \[1992\]](#) called a “baseline”:

When implementing this algorithm nowadays, we always use mini-batching to make full use of our GPUs. Therefore, a more representative variant for this algorithm would be:

The value function is trained by regressing states directly to their sampled Monte Carlo returns G . Advantage normalization (step 2.5) is not part of the optimal baseline derivation but improves optimization in practice and is standard in modern implementations.

Generalized Advantage Estimation The baseline construction gave us a gradient estimator of the form:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t^{(i)}|s_t^{(i)}) \left(G_t^{(i)} - v(s_t^{(i)}) \right) \quad (723)$$

where $G_t = \sum_{k=t}^T r_k$ is the Monte Carlo return from time t . For each visited state-action pair (s_t, a_t) , the term in parentheses

$$\hat{A}_t^{\text{MC}} = G_t - v(s_t) \quad (724)$$

is a Monte Carlo estimate of the advantage $A^{\pi}(s_t, a_t) = q^{\pi}(s_t, a_t) - v^{\pi}(s_t)$. If the baseline equals the true value function, $v = v^{\pi}$, then $E[\hat{A}_t^{\text{MC}}|s_t, a_t] = A^{\pi}(s_t, a_t)$, so this estimator is unbiased.

However, as an estimator it has two limitations. First, it has high variance because G_t depends on all future rewards. Second, it uses the value function only as a baseline, not as a predictor of long-term returns. We essentially discard the information in $v(s_{t+1}), v(s_{t+2}), \dots$.

GAE addresses these issues by constructing a family of estimators that interpolate between pure Monte Carlo and pure bootstrapping. A parameter λ controls the bias-variance tradeoff.

Decomposing the Monte Carlo Advantage Fix a value function $v(s)$ (not necessarily equal to v^π) and define the one-step **residual**:

$$\delta_t = r_t + \gamma v(s_{t+1}) - v(s_t) \quad (725)$$

Start from the Monte Carlo advantage and add and subtract $\gamma v(s_{t+1})$:

$$\begin{aligned} G_t - v(s_t) &= r_t + \gamma G_{t+1} - v(s_t) \\ &= r_t + \gamma v(s_{t+1}) - v(s_t) + \gamma(G_{t+1} - v(s_{t+1})) \\ &= \delta_t + \gamma(G_{t+1} - v(s_{t+1})) \end{aligned}$$

Applying this decomposition recursively yields:

$$G_t - v(s_t) = \sum_{l=0}^{T-t} \gamma^l \delta_{t+l} \quad (726)$$

The Monte Carlo advantage is exactly the discounted sum of future residuals. This is an algebraic identity, not an approximation.

The sequence $\{\delta_{t+l}\}_{l \geq 0}$ provides incremental corrections to the value function as we move forward in time. The term δ_t depends only on (s_t, a_t, s_{t+1}) ; δ_{t+1} depends on $(s_{t+1}, a_{t+1}, s_{t+2})$, and so on. As l increases, the corrections become more noisy (they depend on more random outcomes) and more sensitive to errors in the value function at later states. Although the full sum is unbiased when $v = v^\pi$, it can have high variance and can be badly affected by approximation error in v .

GAE as a Shrinkage Estimator The decomposition above suggests a family of estimators that downweight residuals farther in the future. Let $\lambda \in [0, 1]$ and define:

$$A_t^\lambda = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l} \quad (727)$$

This is the **generalized advantage estimator** $A_t^{\text{GAE}(\gamma, \lambda)}$.

Two special cases illustrate the extremes. When $\lambda = 1$, we recover the Monte Carlo advantage:

$$A_t^{\lambda=1} = \sum_{l=0}^{T-t} \gamma^l \delta_{t+l} = G_t - v(s_t) \quad (728)$$

When $\lambda = 0$, we keep only the immediate residual:

$$A_t^{\lambda=0} = \delta_t = r_t + \gamma v(s_{t+1}) - v(s_t) \quad (729)$$

Intermediate values $0 < \lambda < 1$ interpolate between these extremes. The influence of δ_{t+l} decays geometrically as $(\gamma\lambda)^l$. The parameter λ acts as a shrinkage parameter: small λ shrinks the estimator toward the one-step residual; large λ allows the estimator to behave more like the Monte Carlo advantage.

If $v = v^\pi$ is the true value function, then $E[\delta_t | s_t, a_t] = A^\pi(s_t, a_t)$ and $E[\delta_{t+l} | s_t, a_t] = 0$ for $l \geq 1$. In this case:

$$E[A_t^\lambda | s_t, a_t] = \sum_{l=0}^{T-t} (\gamma\lambda)^l E[\delta_{t+l} | s_t, a_t] = A^\pi(s_t, a_t) \quad (730)$$

for all $\lambda \in [0, 1]$. When the value function is exact, GAE is unbiased regardless of λ ; changing λ only affects variance.

In practice, we approximate v^π with a function approximator, and the residuals δ_{t+l} inherit approximation error. Distant residuals involve multiple applications of the approximate value function and are more contaminated by modeling error. Downweighting them (choosing $\lambda < 1$) introduces bias but can reduce variance and limit the impact of those errors.

Mixture of Multi-Step Estimators Another perspective on GAE comes from multi-step returns. Define the k -step return from time t :

$$G_t^{(k)} = \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k v(s_{t+k}) \quad (731)$$

and the corresponding k -step advantage estimator $A_t^{(k)} = G_t^{(k)} - v(s_t)$. Each $A_t^{(k)}$ uses k rewards before bootstrapping; larger k means more variance but less bootstrapping error.

The GAE estimator can be written as a geometric mixture:

$$A_t^\lambda = (1 - \lambda) \sum_{k=1}^{T-t} \lambda^{k-1} A_t^{(k)} \quad (732)$$

GAE is a weighted average of the k -step advantage estimators, with shorter horizons weighted more heavily when λ is small.

Using GAE in the Policy Gradient Once we choose λ , we plug A_t^λ in place of $G_t - v(s_t)$ in the policy gradient estimator:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t^{(i)} | s_t^{(i)}) A_t^{\lambda, (i)} \quad (733)$$

We still use a control variate to reduce variance (the baseline v), but now we construct the advantage target by smoothing the sequence of residuals $\{\delta_t\}$ with a geometrically decaying kernel.

For the value function, it is convenient to define the **λ -return**:

$$G_t^\lambda = A_t^\lambda + v(s_t) \quad (734)$$

When $\lambda = 1$, G_t^λ reduces to the Monte Carlo return; when $\lambda = 0$, it becomes the one-step bootstrapped target $r_t + \gamma v(s_{t+1})$.

When $\lambda = 1$, this reduces (up to advantage normalization) to the Monte Carlo baseline algorithm earlier in the chapter. When $\lambda = 0$, advantages become the one-step residuals δ_t , and the λ -returns reduce to standard one-step bootstrapped targets.

Actor-Critic as the $\lambda = 0$ Limit The case $\lambda = 0$ is particularly simple. The advantage becomes:

$$A_t^{\lambda=0} = \delta_t = r_t + \gamma v(s_{t+1}) - v(s_t) \quad (735)$$

and the policy update reduces to:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_w \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \delta_t \quad (736)$$

while the value update becomes a standard one-step regression toward $r_t + \gamma v(s_{t+1})$. This gives the online actor-critic algorithm:

This algorithm was derived by Sutton in his 1984 thesis as an “adaptive heuristic” for temporal credit assignment. In the language of this chapter, it is the $\lambda = 0$ member of the GAE family: it uses the most local residual δ_t as both the target for the value function and the advantage estimate for the policy gradient.

Likelihood Ratio Methods in Reinforcement Learning The score function estimator from the previous section is a special case of the likelihood ratio method where the proposal distribution equals the target distribution. We now consider the general case where they differ.

Recall the likelihood ratio gradient estimator from the beginning of this chapter. For objective $J(\theta) = E_{x \sim p(x; \theta)}[f(x)]$ and any proposal distribution $q(x)$:

$$\nabla_{\theta} J(\theta) = E_{x \sim q(x)} [f(x) \rho(x; q, \theta) \nabla_{\theta} \log p(x; \theta)] \quad (737)$$

where $\rho(x; q, \theta) = \frac{p(x; \theta)}{q(x)}$ is the likelihood ratio. The partial derivative $\frac{\partial \rho}{\partial \theta} = \rho \nabla_{\theta} \log p$ holds because x is treated as fixed, having been sampled from q , which does not depend on θ .

In reinforcement learning, let $x = \tau$ be a trajectory, $f(\tau) = G(\tau)$ the return, $p(\tau; \mathbf{w})$ the trajectory distribution under policy $\pi_{\mathbf{w}}$, and $q(\tau)$ the trajectory distribution under some other policy π_q . The gradient becomes:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = E_{\tau \sim \pi_q} \left[G(\tau) \rho(\tau) \sum_{t=0}^T \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \right] \quad (738)$$

where the trajectory likelihood ratio simplifies because transition probabilities cancel:

$$\rho(\tau) = \frac{p(\tau; \mathbf{w})}{q(\tau)} = \prod_{t=0}^T \frac{\pi_{\mathbf{w}}(a_t | s_t)}{\pi_q(a_t | s_t)} = \prod_{t=0}^T \rho_t \quad (739)$$

This product of $T + 1$ ratios can become extremely large or small as T grows, leading to high variance. The temporal structure provides some relief: since $E_{a \sim \pi_q}[\rho] = 1$, future ratios ρ_k for $k > t$ that do not affect the reward r_t can be marginalized out. However, past ratios $\rho_{0:t-1}$ are still needed to correctly weight the probability of reaching state s_t .

In practice, algorithms like PPO and TRPO make an additional approximation: they use only the per-step ratio ρ_t rather than the cumulative product $\rho_{0:t}$. This ignores the mismatch between the state distributions induced by the two policies. Combined with a baseline $b(s_t)$, the approximate estimator is:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) \approx E_{\tau \sim \pi_q} \left[\sum_{t=0}^T \rho_t \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) (G_t - b(s_t)) \right] \quad (740)$$

This approximation corresponds to maximizing the **importance-weighted surrogate objective**:

$$L^{\text{IS}}(\mathbf{w}) = E_{\tau \sim \pi_q} \left[\sum_{t=0}^T \rho_t A_t \right] \quad (741)$$

where $A_t = G_t - b(s_t)$. Taking the gradient with respect to $\mathbf{w}$, only ρ_t depends on $\mathbf{w}$ (since trajectories are sampled from π_q):

$$\nabla_{\mathbf{w}} L^{\text{IS}}(\mathbf{w}) = E_{\tau \sim \pi_q} \left[\sum_{t=0}^T A_t \nabla_{\mathbf{w}} \rho_t \right] \quad (742)$$

The gradient of the ratio is:

$$\nabla_{\mathbf{w}} \rho_t = \nabla_{\mathbf{w}} \frac{\pi_{\mathbf{w}}(a_t | s_t)}{\pi_q(a_t | s_t)} = \frac{\nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a_t | s_t)}{\pi_q(a_t | s_t)} = \rho_t \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) \quad (743)$$

Substituting back:

$$\nabla_{\mathbf{w}} L^{\text{IS}}(\mathbf{w}) = E_{\tau \sim \pi_q} \left[\sum_{t=0}^T \rho_t \nabla_{\mathbf{w}} \log \pi_{\mathbf{w}}(a_t | s_t) A_t \right] \quad (744)$$

This matches equation (740). When $\pi_q = \pi_{\mathbf{w}}$, the ratios $\rho_t = 1$ and we recover the score function estimator. The approximation error grows as the policies diverge, which motivates the trust region and clipping mechanisms discussed below.

Variance and the Dominance Condition The ratio $\rho_t = \pi_{\mathbf{w}}(a_t | s_t) / \pi_q(a_t | s_t)$ is well-behaved only when the two policies are similar. If $\pi_{\mathbf{w}}$ assigns high probability to an action where π_q assigns low probability, the ratio explodes. For example, if $\pi_q(a|s) = 0.01$ and $\pi_{\mathbf{w}}(a|s) = 0.5$, then $\rho = 50$, amplifying any noise in the advantage estimate.

Importance sampling also requires the **dominance condition**: the support of $\pi_{\mathbf{w}}$ must be contained in the support of π_q . If $\pi_{\mathbf{w}}(a|s) > 0$ but $\pi_q(a|s) = 0$, the ratio is undefined. Stochastic policies typically have full support, but the ratio can still become arbitrarily large as $\pi_q(a|s) \rightarrow 0$.

A common use case is to set $\pi_q = \pi_{\mathbf{w}_{\text{old}}}$, a previous version of the policy. This allows reusing data across multiple gradient steps: collect trajectories once, then update $\mathbf{w}$ several times. But each update moves $\mathbf{w}$ further from $\mathbf{w}_{\text{old}}$, making the ratios more extreme. Eventually, the gradient signal is dominated by a few samples with large weights.

Proximal Policy Optimization The variance issues suggest a natural solution: keep the ratio ρ_t close to 1 by ensuring the new policy stays close to the behavior policy. This keeps the importance-weighted surrogate $L^{\text{IS}}(\mathbf{w})$ from (741) well-behaved.

Trust Region Policy Optimization (TRPO) formalizes this by adding a constraint on the KL divergence between the old and new policies:

$$\max_{\mathbf{w}} L^{\text{IS}}(\mathbf{w}) \quad \text{subject to} \quad E_s [D_{\text{KL}}(\pi_{\mathbf{w}_{\text{old}}}(\cdot | s) \| \pi_{\mathbf{w}}(\cdot | s))] \leq \delta \quad (745)$$

The KL constraint ensures that the two distributions remain similar, which bounds how extreme the importance weights can become. This is a constrained optimization problem, and one could in principle apply standard methods such as projected gradient descent or augmented Lagrangian approaches (as discussed in the [trajectory optimization chapter](#)). TRPO takes a different approach: it uses a second-order Taylor approximation of the KL constraint around the current parameters and solves the resulting trust region subproblem using conjugate gradient methods. This involves computing the Fisher information matrix (the Hessian of the KL divergence), which adds computational overhead.

Proximal Policy Optimization (PPO) achieves similar behavior through a simpler mechanism: rather than constraining the distributions to be similar, it

directly clips the ratio ρ_t to prevent it from moving too far from 1. This is a construction-level guarantee rather than an optimization-level constraint.

From Trajectory Expectations to State-Action Averages Before defining the PPO objective, we need to clarify the relationship between the trajectory-level surrogate (741) and the state-action level objective that PPO actually optimizes. The importance-weighted surrogate is defined as an expectation over trajectories:

$$L^{\text{IS}}(\mathbf{w}) = E_{\tau \sim \pi_q} \left[\sum_{t=0}^T \rho_t A_t \right] \quad (746)$$

We can rewrite this as an expectation over state-action pairs by introducing a sampling distribution. For a finite horizon T , define the **averaged time-marginal distribution**:

$$\xi_{\pi_q}(s, a) = \frac{1}{T+1} \sum_{t=0}^T d_t^{\pi_q}(s) \pi_q(a|s) \quad (747)$$

where $d_t^{\pi_q}(s)$ is the probability of being in state s at time t when following policy π_q from the initial distribution. This is the uniform mixture over the time-indexed state-action distributions: we pick a timestep t uniformly at random from $\{0, 1, \dots, T\}$, then sample (s, a) from the joint distribution at that timestep.

With this definition, the trajectory expectation becomes:

$$E_{\tau \sim \pi_q} \left[\sum_{t=0}^T \rho_t A_t \right] = (T+1) \cdot E_{(s,a) \sim \xi_{\pi_q}} [\rho(s, a) A(s, a)] \quad (748)$$

The factor $(T+1)$ is just a constant that does not affect the optimization. This reformulation shows that the importance-weighted surrogate is equivalent to an expectation over state-action pairs drawn from the averaged time-marginal distribution. This is not a stationary distribution or a discounted visitation distribution, but the empirical mixture induced by the finite-horizon rollout procedure.

The Clipped Surrogate Objective PPO replaces the linear importance-weighted term ρA with a clipped version. For a state-action pair (s, a) with advantage A and importance ratio $\rho(\mathbf{w}) = \pi_{\mathbf{w}}(a|s) / \pi_{\mathbf{w}_{\text{old}}}(a|s)$, define the **per-sample clipped objective**:

$$\ell^{\text{CLIP}}(\mathbf{w}; s, a, A) = \min(\rho(\mathbf{w})A, \text{clip}(\rho(\mathbf{w}), 1 - \epsilon, 1 + \epsilon)A) \quad (749)$$

where ϵ is a hyperparameter (typically 0.1 or 0.2) and $\text{clip}(x, a, b) = \max(a, \min(x, b))$ restricts x to the interval $[a, b]$.

The **population-level PPO objective** is then:

$$L^{\text{CLIP}}(\mathbf{w}) = E_{(s,a,A) \sim \xi_{\pi_{\mathbf{w}_{\text{old}}}}} [\ell^{\text{CLIP}}(\mathbf{w}; s, a, A)] \quad (750)$$

where the expectation is taken over the averaged time-marginal distribution (747) induced by $\pi_{\mathbf{w}_{\text{old}}}$.

In practice, we never compute this expectation exactly. Instead, we collect a batch of transitions $\mathcal{D} = \{(s_t^{(i)}, a_t^{(i)}, A_t^{(i)})\}$ by running $\pi_{\mathbf{w}_{\text{old}}}$ and approximate the expectation with an empirical average:

$$\hat{L}^{\text{CLIP}}(\mathbf{w}; \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(s,a,A) \in \mathcal{D}} \ell^{\text{CLIP}}(\mathbf{w}; s, a, A) \quad (751)$$

This is the same plug-in approximation used in [fitted Q-iteration](#): replace the unknown population distribution with the empirical distribution $\hat{P}_{\mathcal{D}}$ induced by the collected batch, then compute the sample average. The empirical surrogate $\hat{L}^{\text{CLIP}}$ is simply an expectation under $\hat{P}_{\mathcal{D}}$. No assumptions about stationarity or discounted visitation are needed. We just average over the transitions we collected.

Intuition for the Clipping Mechanism The min operator in (749) selects the more pessimistic estimate. Consider the two cases:

- **Positive advantage** ($A > 0$): The action is better than average, so we want to increase $\pi_{\mathbf{w}}(a|s)$. The unclipped term ρA increases with ρ . The clipped term stops increasing once $\rho > 1 + \epsilon$. Taking the minimum means we get the benefit of increasing ρ only up to $1 + \epsilon$.
- **Negative advantage** ($A < 0$): The action is worse than average, so we want to decrease $\pi_{\mathbf{w}}(a|s)$. The unclipped term ρA becomes less negative (improves) as ρ decreases. The clipped term stops improving once $\rho < 1 - \epsilon$. Taking the minimum means we get the benefit of decreasing ρ only down to $1 - \epsilon$.

In both cases, the clipping removes the incentive to move the probability ratio beyond the interval $[1 - \epsilon, 1 + \epsilon]$. This keeps the new policy close to the old policy without explicitly computing or constraining the KL divergence.

The algorithm collects a batch of trajectories, then performs K epochs of mini-batch updates on the same data. The empirical surrogate $\hat{L}^{\text{CLIP}}$ approximates the population objective (750) using samples from the averaged time-marginal distribution. The clipped objective ensures that even after multiple updates, the policy does not move too far from the policy that collected the data. The ratio ρ is computed in log-space for numerical stability.

PPO has become one of the most widely used policy gradient algorithms due to its simplicity and robustness. Compared to TRPO, it avoids the computational overhead of constrained optimization while achieving similar sample efficiency. The clip parameter ϵ is the main hyperparameter controlling the trust region size: smaller values keep the policy closer to the behavior policy but may slow learning, while larger values allow faster updates but risk instability.

The Policy Gradient Theorem The algorithms developed so far (REINFORCE, actor-critic, GAE, and PPO) all estimate policy gradients from sampled trajectories. We now establish the theoretical foundation for these estimators by deriving the policy gradient theorem in the discounted infinite-horizon setting.

Sutton et al. [1999] provided the original derivation. Here we present an alternative approach using the Implicit Function Theorem, which frames policy optimization as a bilevel problem:

$$\max_{\mathbf{w}} \alpha^\top \mathbf{v}_\gamma^{\pi_{\mathbf{w}}} \quad (752)$$

subject to:

$$(\mathbf{I} - \gamma \mathbf{P}_{\pi_{\mathbf{w}}}) \mathbf{v}_\gamma^{\pi_{\mathbf{w}}} = \mathbf{r}_{\pi_{\mathbf{w}}} \quad (753)$$

The Implicit Function Theorem states that if there is a solution to the problem $F(\mathbf{v}, \mathbf{w}) = 0$, then we can “reparameterize” our problem as $F(\mathbf{v}(\mathbf{w}), \mathbf{w})$ where $\mathbf{v}(\mathbf{w})$ is an implicit function of $\mathbf{w}$. If the Jacobian $\frac{\partial F}{\partial \mathbf{v}}$ is invertible, then:

$$\frac{d\mathbf{v}(\mathbf{w})}{d\mathbf{w}} = - \left(\frac{\partial F(\mathbf{v}(\mathbf{w}), \mathbf{w})}{\partial \mathbf{v}} \right)^{-1} \frac{\partial F(\mathbf{v}(\mathbf{w}), \mathbf{w})}{\partial \mathbf{w}} \quad (754)$$

Here we made it clear in our notation that the derivative must be evaluated at root $(\mathbf{v}(\mathbf{w}), \mathbf{w})$ of F . For the remaining of this derivation, we will drop this dependence to make notation more compact.

Applying this to our case with $F(\mathbf{v}, \mathbf{w}) = (\mathbf{I} - \gamma \mathbf{P}_{\pi_{\mathbf{w}}}) \mathbf{v} - \mathbf{r}_{\pi_{\mathbf{w}}}$:

$$\frac{\partial \mathbf{v}_\gamma^{\pi_{\mathbf{w}}}}{\partial \mathbf{w}} = (\mathbf{I} - \gamma \mathbf{P}_{\pi_{\mathbf{w}}})^{-1} \left(\frac{\partial \mathbf{r}_{\pi_{\mathbf{w}}}}{\partial \mathbf{w}} + \gamma \frac{\partial \mathbf{P}_{\pi_{\mathbf{w}}}}{\partial \mathbf{w}} \mathbf{v}_\gamma^{\pi_{\mathbf{w}}} \right) \quad (755)$$

Then:

$$\begin{aligned} \nabla_{\mathbf{w}} J(\mathbf{w}) &= \alpha^\top \frac{\partial \mathbf{v}_\gamma^{\pi_{\mathbf{w}}}}{\partial \mathbf{w}} \\ &= \mathbf{x}_\alpha^\top \left(\frac{\partial \mathbf{r}_{\pi_{\mathbf{w}}}}{\partial \mathbf{w}} + \gamma \frac{\partial \mathbf{P}_{\pi_{\mathbf{w}}}}{\partial \mathbf{w}} \mathbf{v}_\gamma^{\pi_{\mathbf{w}}} \right) \end{aligned}$$

where we have defined the discounted state visitation distribution:

$$\mathbf{x}_\alpha^\top \equiv \alpha^\top (\mathbf{I} - \gamma \mathbf{P}_{\pi_{\mathbf{w}}})^{-1}. \quad (756)$$

Recall the vector notation for MDPs from the [dynamic programming chapter](#):

$$\begin{aligned} \mathbf{r}_\pi(s) &\equiv \sum_{a \in \mathcal{A}_s} \pi(a | s) r(s, a), \\ [\mathbf{P}_\pi]_{s, s'} &\equiv \sum_{a \in \mathcal{A}_s} \pi(a | s) p(s' | s, a). \end{aligned}$$

Taking derivatives with respect to $\mathbf{w}$ gives:

$$\begin{aligned}\left[\frac{\partial \mathbf{r}_{\pi_{\mathbf{w}}}}{\partial \mathbf{w}}\right]_s &= \sum_{a \in \mathcal{A}_s} \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) r(s, a), \\ \left[\frac{\partial \mathbf{P}_{\pi_{\mathbf{w}}} \mathbf{v}_{\gamma}^{\pi_{\mathbf{w}}}}{\partial \mathbf{w}}\right]_s &= \sum_{a \in \mathcal{A}_s} \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) \sum_{s'} p(s' | s, a) v_{\gamma}^{\pi_{\mathbf{w}}}(s').\end{aligned}$$

Substituting back:

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= \sum_s x_{\alpha}(s) \left(\sum_a \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) r(s, a) + \gamma \sum_a \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) \sum_{s'} p(s' | s, a) v_{\gamma}^{\pi_{\mathbf{w}}}(s') \right) \\ &= \sum_s x_{\alpha}(s) \sum_a \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) \left(r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_{\gamma}^{\pi_{\mathbf{w}}}(s') \right)\end{aligned}$$

This is the policy gradient theorem, where $x_{\alpha}(s)$ is the discounted state visitation distribution and the term in parentheses is the state-action value function $q^{\pi_{\mathbf{w}}}(s, a)$.

Normalized Discounted State Visitation Distribution The discounted state visitation $x_{\alpha}(s)$ is not normalized. Therefore the expression we obtained above is not an expectation. However, we can transform it into one by normalizing by $1 - \gamma$. Note that for any initial distribution α :

$$\sum_s x_{\alpha}(s) = \alpha^{\top} (\mathbf{I} - \gamma \mathbf{P}_{\pi_{\mathbf{w}}})^{-1} \mathbf{1} = \frac{\alpha^{\top} \mathbf{1}}{1 - \gamma} = \frac{1}{1 - \gamma} \quad (757)$$

Therefore, defining the normalized state distribution $\xi_{\alpha}(s) = (1 - \gamma)x_{\alpha}(s)$, we can write:

$$\begin{aligned}\nabla_{\mathbf{w}} J(\mathbf{w}) &= \frac{1}{1 - \gamma} \sum_s \xi_{\alpha}(s) \sum_a \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) \left(r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_{\gamma}^{\pi_{\mathbf{w}}}(s') \right) \\ &= \frac{1}{1 - \gamma} E_{s \sim \xi_{\alpha}} \left[\sum_a \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(a | s) q^{\pi_{\mathbf{w}}}(s, a) \right]\end{aligned}$$

Now we have expressed the policy gradient theorem in terms of expectations under the normalized discounted state visitation distribution. But what does sampling from ξ_{α} mean? Recall that $\mathbf{x}_{\alpha}^{\top} = \alpha^{\top} (\mathbf{I} - \gamma \mathbf{P}_{\pi_{\mathbf{w}}})^{-1}$. Using the Neumann series expansion (valid when $\|\gamma \mathbf{P}_{\pi_{\mathbf{w}}}\| < 1$, which holds for $\gamma < 1$ since $\mathbf{P}_{\pi_{\mathbf{w}}}$ is a stochastic matrix) we have:

$$\xi_{\alpha}^{\top} = (1 - \gamma) \alpha^{\top} \sum_{k=0}^{\infty} (\gamma \mathbf{P}_{\pi_{\mathbf{w}}})^k \quad (758)$$

We can then factor out the first term from this summation to obtain:

$$\begin{aligned}
\boldsymbol{\xi}_\alpha^\top &= (1 - \gamma)\alpha^\top \sum_{k=0}^{\infty} (\gamma \mathbf{P}_{\pi_w})^k \\
&= (1 - \gamma)\alpha^\top + (1 - \gamma)\alpha^\top \sum_{k=1}^{\infty} (\gamma \mathbf{P}_{\pi_w})^k \\
&= (1 - \gamma)\alpha^\top + (1 - \gamma)\alpha^\top \gamma \mathbf{P}_{\pi_w} \sum_{k=0}^{\infty} (\gamma \mathbf{P}_{\pi_w})^k \\
&= (1 - \gamma)\alpha^\top + \gamma \boldsymbol{\xi}_\alpha^\top \mathbf{P}_{\pi_w}
\end{aligned}$$

The balance equation:

$$\boldsymbol{\xi}_\alpha^\top = (1 - \gamma)\alpha^\top + \gamma \boldsymbol{\xi}_\alpha^\top \mathbf{P}_{\pi_w} \quad (759)$$

shows that $\boldsymbol{\xi}_\alpha$ is a mixture distribution: with probability $1 - \gamma$ you draw a state from the initial distribution α (reset), and with probability γ you follow the policy dynamics $\mathbf{P}_{\pi_w}$ from the current state (continue). This interpretation directly connects to the geometric process: at each step you either terminate and resample from α (with probability $1 - \gamma$) or continue following the policy (with probability γ).

```
import numpy as np

def sample_from_discounted_visitation(
    alpha,
    policy,
    transition_model,
    gamma,
    n_samples=1000
):
    """Sample states from the discounted visitation distribution.

    Args:
        alpha: Initial state distribution (vector of probabilities)
        policy: Function (state -> action probabilities)
        transition_model: Function (state, action -> next state probabilities)
        gamma: Discount factor
        n_samples: Number of states to sample

    Returns:
        Array of sampled states
    """
    samples = []
    n_states = len(alpha)
```

```

rng = np.random.default_rng(2026)

# Initialize state from alpha
current_state = rng.choice(n_states, p=alpha)

for _ in range(n_samples):
    samples.append(current_state)

    # With probability (1 -gamma): reset
    if rng.random() > gamma:
        current_state = rng.choice(n_states, p=alpha)
    # With probability gamma: continue
    else:
        # Sample action from policy
        action_probs = policy(current_state)
        action = rng.choice(len(action_probs), p=action_probs)

        # Sample next state from transition model
        next_state_probs = transition_model(current_state, action)
        current_state = rng.choice(n_states, p=next_state_probs)

return np.array(samples)

# Example usage for a simple 2 -state MDP
alpha = np.array([0.7, 0.3]) # Initial distribution
policy = lambda s: np.array([0.8, 0.2]) # Dummy policy
transition_model = lambda s, a: np.array([0.9, 0.1]) # Dummy transitions
gamma = 0.9

samples = sample_from_discounted_visitation(alpha, policy, transition_model, gamma)

# Check empirical distribution
print("Empirical state distribution:")
print(np.bincount(samples) / len(samples))

```

While the math shows that sampling from the discounted visitation distribution ξ_α would give us unbiased policy gradient estimates, Thomas (2014) demonstrated that this implementation can be detrimental to performance in practice. The issue arises because terminating trajectories early (with probability $1 - \gamma$) reduces the effective amount of data we collect from each trajectory. This early termination weakens the learning signal, as many trajectories don't reach meaningful terminal states or rewards.

Therefore, in practice, we typically sample complete trajectories from the undiscounted process (running the policy until natural termination or a fixed horizon) while still using γ in the advantage estimation. This approach preserves the full learning signal from each trajectory and has been empirically shown to

lead to better performance.

This is one of several cases in RL where the theoretically optimal procedure differs from the best practical implementation.

The Actor-Critic Architecture The policy gradient theorem shows that the gradient depends on the action-value function $q^{\pi_w}(s, a)$. In practice, we do not have access to the true q -function and must estimate it. This leads to the **actor-critic** architecture: the **actor** maintains the policy π_w , while the **critic** maintains an estimate of the value function.

This architecture traces back to Sutton’s 1984 thesis, where he proposed the Adaptive Heuristic Critic. The actor uses the critic’s value estimates to compute advantage estimates for the policy gradient, while the critic learns from the same trajectories generated by the actor. The algorithms we developed earlier (REINFORCE with baseline, GAE, and the one-step actor-critic) are all instances of this architecture.

We are simultaneously learning two functions that depend on each other, which creates a stability challenge. The actor’s gradient uses the critic’s estimates, but the critic is trained on data generated by the actor’s policy. If both change too quickly, the learning process can become unstable.

Konda [2002] analyzed this coupled learning problem and established convergence guarantees under a **two-timescale** condition: the critic must update faster than the actor. Intuitively, the critic needs to “track” the current policy’s value function before the actor uses those estimates to update. If the actor moves too fast, it uses stale or inaccurate value estimates, leading to poor gradient estimates.

In practice, this is implemented by using different learning rates: a larger learning rate α_θ for the critic and a smaller learning rate α_w for the actor, with $\alpha_\theta > \alpha_w$. Alternatively, one can perform multiple critic updates per actor update. The soft actor-critic algorithm discussed earlier in the [amortization chapter](#) follows this same principle, inheriting the actor-critic structure while incorporating entropy regularization and learning Q-functions directly.

The actor-critic architecture also connects to the bilevel optimization perspective of the policy gradient theorem: the outer problem optimizes the policy, while the inner problem solves for the value function given that policy. The two-timescale condition ensures that the inner problem is approximately solved before taking a step on the outer problem.

Reparameterization Methods in Reinforcement Learning When dynamics are known or can be learned, reparameterization provides an alternative to score function methods. By expressing actions and state transitions as deterministic functions of noise, we can backpropagate through trajectories to compute policy gradients with lower variance than score function estimators.

Stochastic Value Gradients The reparameterization trick requires that we can express our random variable as a deterministic function of noise. In

reinforcement learning, this applies naturally when we have a learned model of the dynamics. Consider a stochastic policy $\pi_{\mathbf{w}}(a|s)$ that we can reparameterize as $a = \pi_{\mathbf{w}}(s, \epsilon)$ where $\epsilon \sim p(\epsilon)$, and a dynamics model $s' = f(s, a, \xi)$ where $\xi \sim p(\xi)$ represents environment stochasticity. Both transformations are deterministic given the noise variables.

With these reparameterizations, we can write an n -step return as a differentiable function of the noise:

$$R_n(s_0, \{\epsilon_i\}, \{\xi_i\}) = \sum_{i=0}^{n-1} \gamma^i r(s_i, a_i) \quad (760)$$

where $a_i = \pi_{\mathbf{w}}(s_i, \epsilon_i)$ and $s_{i+1} = f(s_i, a_i, \xi_i)$ for $i = 0, \dots, n-1$. The objective becomes:

$$J(\mathbf{w}) = E_{\{\epsilon_i\}, \{\xi_i\}} [R_n(s_0, \{\epsilon_i\}, \{\xi_i\})] \quad (761)$$

We can now apply the reparameterization gradient estimator:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = E_{\{\epsilon_i\}, \{\xi_i\}} [\nabla_{\mathbf{w}} R_n(s_0, \{\epsilon_i\}, \{\xi_i\})] \quad (762)$$

This gradient can be computed by automatic differentiation through the sequence of policy and model evaluations. The computation requires backpropagating through n steps of model rollouts, which becomes expensive for large n but avoids the high variance of score function estimators.

The Stochastic Value Gradients (SVG) framework [Heess et al., 2015] uses this approach while introducing a hybrid objective that combines model rollouts with value function bootstrapping:

$$J^{\text{SVG}(n)}(\mathbf{w}) = E_{\{\epsilon_i\}, \{\xi_i\}} \left[\sum_{i=0}^{n-1} \gamma^i r(s_i, a_i) + \gamma^n q(s_n, a_n; \theta) \right] \quad (763)$$

The terminal value function $q(s_n, a_n; \theta)$ approximates the value beyond horizon n , allowing shorter rollouts while still capturing long-term value. This creates a spectrum of algorithms parameterized by n .

SVG(0): Model-Free Reparameterization When $n = 0$, the objective collapses to:

$$J^{\text{SVG}(0)}(\mathbf{w}) = E_{s \sim \rho} E_{\epsilon \sim p(\epsilon)} [q(s, \pi_{\mathbf{w}}(s, \epsilon); \theta)] \quad (764)$$

No model is required. We simply differentiate the critic with respect to actions sampled from the reparameterized policy. This is the approach used in DDPG [Lillicrap et al., 2015] (with a deterministic policy where ϵ is absent) and SAC [Haarnoja et al., 2018a] (where ϵ produces the stochastic component). The gradient is:

$$\nabla_{\mathbf{w}} J^{\text{SVG}(0)} = E_{s, \epsilon} \left[\nabla_a q(s, a; \theta) \Big|_{a=\pi_{\mathbf{w}}(s, \epsilon)} \nabla_{\mathbf{w}} \pi_{\mathbf{w}}(s, \epsilon) \right] \quad (765)$$

This requires only that the critic q be differentiable with respect to actions, not a learned dynamics model. All bias comes from errors in the value function approximation.

SVG(1) to SVG(n): Model-Based Rollouts For $n \geq 1$, we unroll a learned dynamics model for n steps before bootstrapping with the critic. Consider SVG(1):

$$J^{\text{SVG}(1)}(\mathbf{w}) = E_{s,\epsilon,\xi} [r(s, \pi_{\mathbf{w}}(s, \epsilon)) + \gamma q(f(s, \pi_{\mathbf{w}}(s, \epsilon), \xi), \pi_{\mathbf{w}}(s', \epsilon'); \theta)] \quad (766)$$

where $s' = f(s, \pi_{\mathbf{w}}(s, \epsilon), \xi)$ is the next state predicted by the model. The gradient now flows through both the reward and the model transition. Increasing n propagates reward information more directly through the model rollout, reducing reliance on the critic. However, model errors compound over the horizon. If the model is inaccurate, longer rollouts can degrade performance.

SVG(∞): Pure Model-Based Optimization As $n \rightarrow \infty$, we eliminate the critic entirely:

$$J^{\text{SVG}(\infty)}(\mathbf{w}) = E_{\{\epsilon_i\}, \{\xi_i\}} \left[\sum_{i=0}^{T-1} \gamma^i r(s_i, \pi_{\mathbf{w}}(s_i, \epsilon_i)) \right] \quad (767)$$

This is pure model-based policy optimization, differentiating through the entire trajectory. Approaches like PILCO [Deisenroth and Rasmussen, 2011] and Dreamer [Hafner et al., 2019] operate in this regime. With an accurate model, this provides the most direct gradient signal. The tradeoff is computational: backpropagating through hundreds of time steps is expensive, and gradient magnitudes can explode or vanish over long horizons.

The choice of n reflects a fundamental bias-variance tradeoff. Small n relies on the critic for long-term value estimation, inheriting its approximation errors. Large n relies on the model, accumulating its prediction errors. In practice, intermediate values like $n = 5$ or $n = 10$ often work well when combined with a reasonably accurate learned model.

Noise Inference for Off-Policy Learning A subtle issue arises when combining reparameterization with experience replay. SVG naturally supports off-policy learning: states s can be sampled from a replay buffer rather than the current policy. However, reparameterization requires the noise variables ϵ that generated each action.

For on-policy data, we can simply store ϵ alongside each transition (s, a, r, s') . For off-policy data collected under a different policy, the noise is unknown. To apply reparameterization gradients to such data, we must *infer* the noise that would have produced the observed action under the current policy.

For invertible policies, this is straightforward. If $a = \pi_{\mathbf{w}}(s, \epsilon)$ with $\epsilon \sim \mathcal{N}(0, I)$, and the policy takes the form $a = \mu_{\mathbf{w}}(s) + \sigma_{\mathbf{w}}(s) \odot \epsilon$ (as in a Gaussian policy), we can recover the noise exactly:

$$\epsilon = \frac{a - \mu_{\mathbf{w}}(s)}{\sigma_{\mathbf{w}}(s)} \quad (768)$$

This recovered ϵ can then be used for gradient computation. However, this introduces a subtle dependence: the inferred ϵ depends on the current policy parameters $\mathbf{w}$, not just the data. As the policy changes during training, the same action a corresponds to different noise values.

For dynamics noise ξ , the situation is more complex. If we have a probabilistic model $s' = f(s, a, \xi)$ and observe the actual next state s' , we could in principle infer ξ . In practice, environment stochasticity is often treated as irreducible: we cannot replay the exact same noise realization. SVG handles this by either: (1) using deterministic models and ignoring environment stochasticity, (2) re-simulating from the model rather than using observed next states, or (3) using importance weighting to correct for the distribution mismatch.

The noise inference perspective connects reparameterization gradients to the broader question of credit assignment in RL. By explicitly tracking which noise realizations led to which outcomes, we can more precisely attribute value to policy parameters rather than to lucky or unlucky samples.

When dynamics are deterministic or can be accurately reparameterized, SVG-style methods offer an efficient alternative to the score function methods developed in the previous section. However, many reinforcement learning problems involve unknown dynamics or dynamics that resist accurate modeling. In those settings, score function methods remain the primary tool since they require only the ability to sample trajectories under the policy.

Summary This chapter developed the mathematical foundations for policy gradient methods. Starting from general derivative estimation techniques in stochastic optimization, we saw two main approaches: the likelihood ratio (score function) method and the reparameterization trick. While the reparameterization trick typically offers lower variance, it requires that the sampling distribution be reparameterizable, making it inapplicable to discrete actions or environments with complex dynamics.

For reinforcement learning, the score function estimator provides a model-free gradient that depends only on the policy parametrization, not the transition dynamics. Through variance reduction techniques (leveraging conditional independence, using control variates, and the Generalized Advantage Estimator), we can make these gradients practical for learning. The likelihood ratio perspective then led to importance-weighted surrogates and PPO’s clipped objective for stable off-policy updates.

We also established the policy gradient theorem, which provides the theoretical foundation for these estimators in the discounted infinite-horizon setting. The actor-critic architecture emerges from approximating the value function

that appears in this theorem, with the two-timescale condition ensuring stable learning.

When dynamics models are available, reparameterization through Stochastic Value Gradients offers lower-variance alternatives. $\text{SVG}(0)$ recovers actor-critic methods like DDPG and SAC, while $\text{SVG}(\infty)$ represents pure model-based optimization through differentiable simulation.

Self-checks

7 Appendix

7.1 Example COCPs

7.1.1 Inverted Pendulum

The inverted pendulum is a classic problem in control theory and robotics that demonstrates the challenge of stabilizing a dynamic system that is inherently unstable. The objective is to keep a pendulum balanced in the upright position by applying a control force, typically at its base. This setup is analogous to balancing a broomstick on your finger: any deviation from the vertical position will cause the system to tip over unless you actively counteract it with appropriate control actions.

We typically assume that the pendulum is mounted on a cart or movable base, which can move horizontally. The system's state is then characterized by four variables:

1. **Cart position:** $x(t)$ — the horizontal position of the base.
2. **Cart velocity:** $\dot{x}(t)$ — the speed of the cart.
3. **Pendulum angle:** $\theta(t)$ — the angle between the pendulum and the vertical upright position.
4. **Angular velocity:** $\dot{\theta}(t)$ — the rate at which the pendulum's angle is changing.

This setup is more complex because the controller must deal with interactions between two different types of motion: linear (the cart) and rotational (the pendulum). This system is said to be “underactuated” because the number of control inputs (one) is less than the number of state variables (four). This makes the problem more challenging and interesting from a control perspective.

We can simplify the problem by assuming that the base of the pendulum is fixed. This is akin to having the bottom of the stick attached to a fixed pivot on a table. You can't move the base anymore; you can only apply small nudges at the pivot point to keep the stick balanced upright. In this case, you're only focusing on adjusting the stick's tilt without worrying about moving the base. This reduces the problem to stabilizing the pendulum's upright orientation using only the rotational dynamics. The system's state can now be described by just two variables:

1. **Pendulum angle:** $\theta(t)$ — the angle of the pendulum from the upright vertical position.
2. **Angular velocity:** $\dot{\theta}(t)$ — the rate at which the pendulum's angle is changing.

The evolution of these two variables is governed by the following ordinary differential equation:

$$\begin{bmatrix} \dot{\theta}(t) \\ \ddot{\theta}(t) \end{bmatrix} = \begin{bmatrix} \dot{\theta}(t) \\ \frac{mgl}{J_t} \sin \theta(t) - \frac{\gamma}{J_t} \dot{\theta}(t) + \frac{l}{J_t} u(t) \cos \theta(t) \end{bmatrix}, \quad y(t) = \theta(t) \quad (769)$$

where:

- m is the mass of the pendulum
- g is the acceleration due to gravity
- l is the length of the pendulum
- γ is the coefficient of rotational friction
- $J_t = J + ml^2$ is the total moment of inertia, with J being the pendulum's moment of inertia about its center of mass
- $u(t)$ is the control force applied at the base
- $y(t) = \theta(t)$ is the measured output (the pendulum's angle)

We expect that when no control is applied to the system, the rod should be falling down when started from the upright position.

7.1.2 Pendulum in the Gym Environment

Let's examine the code and reverse-engineer the original continuous-time problem hidden behind the abstraction layer. Although the pendulum problem may have limited practical relevance as a real-world application, it serves as an excellent example for our analysis. In the current version of [Pendulum](#), we find that the Gym implementation uses a simplified model. Like our implementation, it assumes a fixed base and doesn't model cart movement. The state is also represented by the pendulum angle and angular velocity. However, the equations of motion implemented in the Gym environment are different and correspond to the following ODE:

$$\begin{aligned} \dot{\theta} &= \theta_{dot} \\ \dot{\theta}_{dot} &= \frac{3g}{2l} \sin(\theta) + \frac{3}{ml^2} u \end{aligned}$$

Compared to our simplified model, the Gym implementation makes the following additional assumptions:

1. It omits the term $\frac{\gamma}{J_t} \dot{\theta}(t)$, which represents damping or air resistance. This means that it assumes an idealized pendulum that doesn't naturally slow down over time.

2. It uses ml^2 instead of $J_t = J + ml^2$, which assumes that all mass is concentrated at the pendulum's end (like a point mass on a massless rod), rather than accounting for mass distribution along the pendulum.
3. The control input u is applied directly, without a $\cos\theta(t)$ term, which means that the applied torque has the same effect regardless of the pendulum's position, rather than varying with angle. For example, imagine trying to push a door open. When the door is almost closed (pendulum near vertical), a small push perpendicular to the door (analogous to our control input) can easily start it moving. However, when the door is already wide open (pendulum horizontal), the same push has little effect on the door's angle. In a more detailed model, this would be captured by the $\cos\theta(t)$ term, which is maximum when the pendulum is vertical ($\cos 0 = 1$) and zero when horizontal ($\cos 90 = 0$).

The goal remains to stabilize the rod upright, but the way in which this encoded is through the following instantaneous cost function:

$$c(\theta, \dot{\theta}, u) = (\text{normalize}(\theta))^2 + 0.1\dot{\theta}^2 + 0.001u^2$$

$$\text{normalize}(\theta) = ((\theta + \pi) \bmod 2\pi) - \pi$$

This cost function penalizes deviations from the upright position (first term), discouraging rapid motion (second term), and limiting control effort (third term). The relative weights has been manually chosen to balance the primary goal of upright stabilization with the secondary aims of smooth motion and energy efficiency. The normalization ensures that the angle is always in the range $[-\pi, \pi]$ so that the pendulum positions (e.g., 0 and 2π) are treated identically, which could otherwise confuse learning algorithms.

Studying the code further, we find that it imposes bound constraints on both the control input and the angular velocity through clipping operations:

$$u = \max(\min(u, u_{max}), -u_{max})$$

$$\dot{\theta} = \max(\min(\dot{\theta}, \dot{\theta}_{max}), -\dot{\theta}_{max})$$

Where $u_{max} = 2.0$ and $\dot{\theta}_{max} = 8.0$. Finally, when inspecting the [step](#) function, we find that the dynamics are discretized using forward Euler under a fixed step size of $h = 0.05$. Overall, the discrete-time trajectory optimization

problem implemented in Gym is the following:

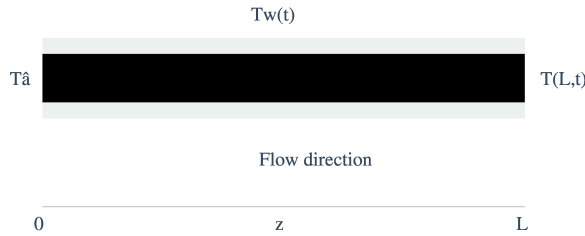
$$\begin{aligned}
\min_{u_k} \quad & J = \sum_{k=0}^{N-1} c(\theta_k, \dot{\theta}_k, u_k) \\
\text{subject to:} \quad & \theta_{k+1} = \theta_k + \dot{\theta}_k \cdot h \\
& \dot{\theta}_{k+1} = \dot{\theta}_k + \left(\frac{3g}{2l} \sin(\theta_k) + \frac{3}{ml^2} u_k \right) \cdot h \\
& -u_{\max} \leq u_k \leq u_{\max} \\
& -\dot{\theta}_{\max} \leq \dot{\theta}_k \leq \dot{\theta}_{\max}, \quad k = 0, 1, \dots, N-1 \\
\text{given:} \quad & \theta_0 = \theta_{\text{initial}}, \quad \dot{\theta}_0 = \dot{\theta}_{\text{initial}}, \quad N = 200
\end{aligned}$$

with $g = 10.0$, $l = 1.0$, $m = 1.0$, $u_{\max} = 2.0$, and $\dot{\theta}_{\max} = 8.0$. This discrete-time problem corresponds to the following continuous-time optimal control problem:

$$\begin{aligned}
\min_{u(t)} \quad & J = \int_0^T c(\theta(t), \dot{\theta}(t), u(t)) dt \\
\text{subject to:} \quad & \dot{\theta}(t) = \dot{\theta}(t) \\
& \ddot{\theta}(t) = \frac{3g}{2l} \sin(\theta(t)) + \frac{3}{ml^2} u(t) \\
& -u_{\max} \leq u(t) \leq u_{\max} \\
& -\dot{\theta}_{\max} \leq \dot{\theta}(t) \leq \dot{\theta}_{\max} \\
\text{given:} \quad & \theta(0) = \theta_0, \quad \dot{\theta}(0) = \dot{\theta}_0, \quad T = 10 \text{ seconds}
\end{aligned}$$

7.1.3 Heat Exchanger

►



We are considering a system where fluid flows through a tube, and the goal is to control the temperature of the fluid by adjusting the temperature of the tube's wall over time. The wall temperature, denoted as $T_w(t)$, can be changed as a function of time, but it remains the same along the length of the tube. On the other hand, the temperature of the fluid inside the tube, $T(z, t)$, depends

both on its position along the tube z and on time t . It evolves according to the following partial differential equation:

$$\frac{\partial T}{\partial t} = -v \frac{\partial T}{\partial z} + \frac{h}{\rho C_p} (T_w(t) - T) \quad (770)$$

where we have:

- v : the average speed of the fluid moving through the tube,
- h : how easily heat transfers from the wall to the fluid,
- ρ and C_p : the fluid's density and heat capacity.

This equation describes how the fluid's temperature changes as it moves along the tube and interacts with the tube's wall temperature. The fluid enters the tube with an initial temperature T_0 at the inlet (where $z = 0$). Our objective is to adjust the wall temperature $T_w(t)$ so that by a specific final time t_f , the fluid's temperature reaches a desired distribution $T_s(z)$ along the length of the tube. The relationship for $T_s(z)$ under steady-state conditions (ie. when changes over time are no longer considered), is given by:

$$\frac{dT_s}{dz} = \frac{h}{v\rho C_p} [\theta - T_s] \quad (771)$$

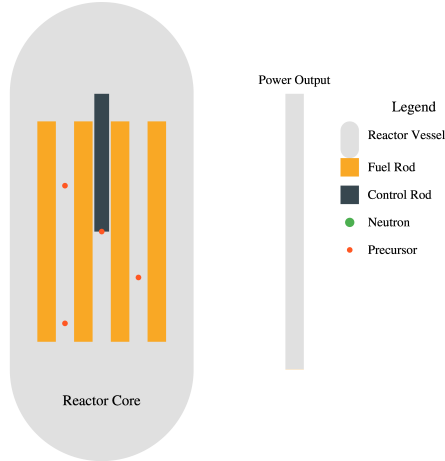
where θ is a constant temperature we want to maintain at the wall. The objective is to control the wall temperature $T_w(t)$ so that by the end of the time interval t_f , the fluid temperature $T(z, t_f)$ is as close as possible to the desired distribution $T_s(z)$. This can be formalized by minimizing the following quantity:

$$I = \int_0^L [T(z, t_f) - T_s(z)]^2 dz \quad (772)$$

where L is the length of the tube. Additionally, we require that the wall temperature cannot exceed a maximum allowable value $T_{\max}$:

$$T_w(t) \leq T_{\max} \quad (773)$$

7.1.4 Nuclear Reactor



In a nuclear reactor, neutrons interact with fissile nuclei, causing nuclear fission. This process produces more neutrons and smaller fissile nuclei called precursors. The precursors subsequently absorb more neutrons, generating “delayed” neutrons. The kinetic energy of these products is converted into thermal energy through collisions with neighboring atoms. The reactor’s power output is determined by the concentration of neutrons available for nuclear fission.

The reaction kinetics can be modeled using a system of ordinary differential equations:

$$\begin{aligned}\dot{x}(t) &= \frac{r(t)x(t) - \alpha x^2(t) - \beta x(t)}{\tau} + \mu y(t), & x(0) &= x_0 \\ \dot{y}(t) &= \frac{\beta x(t)}{\tau} - \mu y(t), & y(0) &= y_0\end{aligned}$$

where:

- $x(t)$: concentration of neutrons at time t
- $y(t)$: concentration of precursors at time t
- t : time
- $r(t) = r[u(t)]$: degree of change in neutron multiplication at time t as a function of control rod displacement $u(t)$
- α : reactivity coefficient
- β : fraction of delayed neutrons
- μ : decay constant for precursors
- τ : average time taken by a neutron to produce a neutron or precursor

The power output can be adjusted based on demand by inserting or retracting a neutron-absorbing control rod. Inserting the control rod absorbs neutrons, reducing the heat flux and power output, while retracting the rod has the opposite effect.

The objective is to change the neutron concentration $x(t)$ from an initial value x_0 to a stable value x_f at time t_f while minimizing the displacement of the control rod. This can be formulated as an optimal control problem, where the goal is to find the control function $u(t)$ that minimizes the objective functional:

$$I = \int_0^{t_f} u^2(t) dt$$

subject to the final conditions:

$$\begin{aligned} x(t_f) &= x_f \\ \dot{x}(t_f) &= 0 \end{aligned}$$

and the constraint $|u(t)| \leq u_{\max}$

7.1.5 Chemotherapy

Chemotherapy uses drugs to kill cancer cells. However, these drugs can also have toxic effects on healthy cells in the body. To optimize the effectiveness of chemotherapy while minimizing its side effects, we can formulate an optimal control problem.

The drug concentration $y_1(t)$ and the number of immune cells $y_2(t)$, healthy cells $y_3(t)$, and cancer cells $y_4(t)$ in an organ at any time t during chemotherapy can be modeled using a system of ordinary differential equations:

$$\begin{aligned} \dot{y}_1(t) &= u(t) - \gamma_6 y_1(t) \\ \dot{y}_2(t) &= \dot{y}_{2,\text{in}} + r_2 \frac{y_2(t)y_4(t)}{\beta_2 + y_4(t)} - \gamma_3 y_2(t)y_4(t) - \gamma_4 y_2(t) - \alpha_2 y_2(t) \left(1 - e^{-y_1(t)\lambda_2}\right) \\ \dot{y}_3(t) &= r_3 y_3(t) (1 - \beta_3 y_3(t)) - \gamma_5 y_3(t)y_4(t) - \alpha_3 y_3(t) \left(1 - e^{-y_1(t)\lambda_3}\right) \\ \dot{y}_4(t) &= r_1 y_4(t) (1 - \beta_1 y_4(t)) - \gamma_1 y_3(t)y_4(t) - \gamma_2 y_2(t)y_4(t) - \alpha_1 y_4(t) \left(1 - e^{-y_1(t)\lambda_1}\right) \end{aligned}$$

where:

- $y_1(t)$: drug concentration in the organ at time t
- $y_2(t)$: number of immune cells in the organ at time t
- $y_3(t)$: number of healthy cells in the organ at time t
- $y_4(t)$: number of cancer cells in the organ at time t

- $\dot{y}_{2,\text{in}}$: constant rate of immune cells entering the organ to fight cancer cells
- $u(t)$: rate of drug injection into the organ at time t
- r_i, β_i : constants in the growth terms
- α_i, λ_i : constants in the decay terms due to the action of the drug
- γ_i : constants in the remaining decay terms

The objective is to minimize the number of cancer cells $y_4(t)$ in a specified time t_f while using the minimum amount of drug to reduce its toxic effects. This can be formulated as an optimal control problem, where the goal is to find the control function $u(t)$ that minimizes the objective functional:

$$I = y_4(t_f) + \int_0^{t_f} u(t) dt$$

subject to the system dynamics, initial conditions, and the constraint $u(t) \geq 0$.

Additional constraints may include:

- Maintaining a minimum number of healthy cells during treatment:

$$y_3(t) \geq y_{3,\text{min}}$$

- Imposing an upper limit on the drug dosage:

$$u(t) \leq u_{\text{max}}$$

7.1.6 Government Corruption

In this model from Feichtinger and Wirl (1994), we aim to understand the incentives for politicians to engage in corrupt activities or to combat corruption. The model considers a politician's popularity as a dynamic process that is influenced by the public's memory of recent and past corruption. The objective is to find conditions under which self-interested politicians would choose to be honest or dishonest.

The model introduces the following notation:

- $C(t)$: accumulated awareness (knowledge) of past corruption at time t
- $u(t)$: extent of corruption (politician's control variable) at time t
- δ : rate of forgetting past corruption
- $P(t)$: politician's popularity at time t
- $g(P)$: growth function of popularity; $g''(P) < 0$

- $f(C)$: function measuring the loss of popularity caused by C ; $f'(C) > 0$, $f''(C) \geq 0$
- $U_1(P)$: benefits associated with being popular; $U_1'(P) > 0$, $U_1''(P) \leq 0$
- $U_2(u)$: benefits resulting from bribery and fraud; $U_2'(u) > 0$, $U_2''(u) < 0$
- r : discount rate

The dynamics of the public's memory of recent and past corruption $C(t)$ are modeled as:

$$\dot{C}(t) = u(t) - \delta C(t), \quad C(0) = C_0$$

The evolution of the politician's popularity $P(t)$ is governed by:

$$\dot{P}(t) = g(P(t)) - f(C(t)), \quad P(0) = P_0$$

The politician's objective is to maximize the following objective:

$$\int_0^\infty e^{-rt} [U_1(P(t)) + U_2(u(t))] dt$$

subject to the dynamics of corruption awareness and popularity.
The optimal control problem can be formulated as follows:

$$\begin{aligned} \max_{u(\cdot)} \quad & \int_0^\infty e^{-rt} [U_1(P(t)) + U_2(u(t))] dt \\ \text{s.t.} \quad & \dot{C}(t) = u(t) - \delta C(t), \quad C(0) = C_0 \\ & \dot{P}(t) = g(P(t)) - f(C(t)), \quad P(0) = P_0 \end{aligned}$$

The state variables are the accumulated awareness of past corruption $C(t)$ and the politician's popularity $P(t)$. The control variable is the extent of corruption $u(t)$. The objective functional represents the discounted stream of benefits coming from being honest (popularity) and from being dishonest (corruption).

7.2 Solving Initial Value Problems

An ODE is an implicit representation of a state-space trajectory: it tells us how the state changes in time but not precisely what the state is at any given time. To find out this information, we need to either solve the ODE analytically (for some special structure) or, as we're going to do, solve them numerically. This numerical procedure is meant to solve what is called an IVP (initial value problem) of the form:

$$\text{Find } x(t) \text{ given } \dot{x}(t) = f(x(t), t) \text{ and } x(t_0) = x_0 \quad (774)$$

7.2.1 Euler's Method

The algorithm to solve this problem is, in its simplest form, a for loop which closely resembles the updates encountered in gradient descent (in fact, gradient descent can be derived from the gradient flow ODE, but that's another discussion). The so-called explicit Euler's method can be implemented as follow:

Consider the following simple dynamical system of a ballistic motion model, neglecting air resistance. The state of the system is described by two variables: $y(t)$: vertical position at time t and $v(t)$, the vertical velocity at time t . The corresponding ODE is:

$$\begin{aligned} \frac{dy}{dt} &= v \\ \frac{dv}{dt} &= -g \end{aligned} \quad (775)$$

where $g \approx 9.81 \text{ m/s}^2$ is the acceleration due to gravity. In our code, we use the initial conditions $y(0) = 0 \text{ m}$ and $v(0) = v_0 \text{ m/s}$ where v_0 is the initial velocity (in this case, $v_0 = 20 \text{ m/s}$). The analytical solution to this system is:

$$\begin{aligned} y(t) &= v_0 t - \frac{1}{2} g t^2 \\ v(t) &= v_0 - g t \end{aligned} \quad (776)$$

This system models the vertical motion of an object launched upward, reaching a maximum height before falling back down due to gravity.

Euler's method can be obtained by taking the first-order Taylor expansion of $x(t)$ at t :

$$x(t+h) \approx x(t) + h \frac{dx}{dt}(t) = x(t) + h f(x(t), t) \quad (777)$$

Each step of the algorithm therefore involves approximating the function with a linear function of slope f over the given interval h .

Another way to understand Euler's method is through the fundamental theorem of calculus:

$$x(t+h) = x(t) + \int_t^{t+h} f(x(\tau), \tau) d\tau \quad (778)$$

We then approximate the integral term with a box of width h and height f , and therefore of area hf .

7.2.2 Implicit Euler's Method

An alternative approach is the Implicit Euler method, also known as the Backward Euler method. Instead of using the derivative at the current point to step forward, it uses the derivative at the end of the interval. This leads to the following update rule:

$$x_{new} = x + hf(x_{new}, t_{new}) \quad (779)$$

Note that x_{new} appears on both sides of the equation, making this an implicit method. The algorithm for the Implicit Euler method can be described as follows:

The main difference in the Implicit Euler method is step 4, where we need to solve a (potentially nonlinear) equation to find x_{new} . This is typically done using iterative methods such as fixed-point iteration or Newton's method.

Stiff ODEs While the Implicit Euler method requires more computation per step, it often allows for larger step sizes and can provide better stability for certain types of problems, especially stiff ODEs.

Stiff ODEs are differential equations for which certain numerical methods for solving the equation are numerically unstable, unless the step size is taken to be extremely small. These ODEs typically involve multiple processes occurring at widely different rates. In a stiff problem, the fastest-changing component of the solution can make the numerical method unstable unless the step size is extremely small. However, such a small step size may lead to an impractical amount of computation to traverse the entire interval of interest.

For example, consider a chemical reaction where some reactions occur very quickly while others occur much more slowly. The fast reactions quickly approach their equilibrium, but small perturbations in the slower reactions can cause rapid changes in the fast reactions.

A classic example of a stiff ODE is the Van der Pol oscillator with a large parameter. The Van der Pol equation is:

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + x = 0 \quad (780)$$

where μ is a scalar parameter. This second-order ODE can be transformed into a system of first-order ODEs by introducing a new variable $y = \frac{dx}{dt}$:

$$\begin{aligned} \frac{dx}{dt} &= y \\ \frac{dy}{dt} &= \mu(1 - x^2)y - x \end{aligned} \quad (781)$$

When μ is large (e.g., $\mu = 1000$), this system becomes stiff. The large μ causes rapid changes in y when x is near ± 1 , but slower changes elsewhere.

This leads to a solution with sharp transitions followed by periods of gradual change.

7.2.3 Trapezoid Method

The trapezoid method, also known as the trapezoidal rule, offers improved accuracy and stability compared to the simple Euler method. The name “trapezoid method” comes from the idea of using a trapezoid to approximate the integral term in the fundamental theorem of calculus. This leads to the following update rule:

$$x_{new} = x + \frac{h}{2}[f(x, t) + f(x_{new}, t_{new})] \quad (782)$$

where $t_{new} = t + h$. Note that this formula involves x_{new} on both sides of the equation, making it an implicit method, similar to the implicit Euler method discussed earlier.

Algorithmically, the trapezoid method can be described as follows:

The trapezoid method can also be derived by averaging the forward Euler and backward Euler methods. Recall that:

1. Forward Euler method:

$$x_{n+1} = x_n + hf(x_n, t_n) \quad (783)$$

2. Backward Euler method:

$$x_{n+1} = x_n + hf(x_{n+1}, t_{n+1}) \quad (784)$$

Taking the average of these two methods yields:

$$\begin{aligned} x_{n+1} &= \frac{1}{2}(x_n + hf(x_n, t_n)) + \frac{1}{2}(x_n + hf(x_{n+1}, t_{n+1})) \\ &= x_n + \frac{h}{2}(f(x_n, t_n) + f(x_{n+1}, t_{n+1})) \end{aligned} \quad (785)$$

This gives us the update rule for the trapezoid method. Recall that the forward Euler method approximates the solution by extrapolating linearly using the slope at the beginning of the interval $[t_n, t_{n+1}]$. In contrast, the backward Euler method extrapolates linearly using the slope at the end of the interval. The trapezoid method, on the other hand, averages these two slopes. This averaging provides better approximation properties than either of the methods alone, offering both stability and accuracy. Note finally that unlike the forward or backward Euler methods, the trapezoid method is also symmetric in time. This means that if you were to reverse time and apply the method backward, you would get the same results (up to numerical precision).

7.2.4 Trapezoidal Predictor-Corrector

The trapezoid method can also be implemented under the so-called predictor-corrector framework. This interpretation reformulates the implicit trapezoid rule into an explicit two-step process:

1. **Predictor Step:**

We make an initial guess for x_{n+1} using the forward Euler method:

$$x_{n+1}^* = x_n + hf(x_n, t_n) \quad (786)$$

This is our “predictor” step, where x_{n+1}^* is the predicted value of x_{n+1} .

2. **Corrector Step:**

We then use this predicted value to estimate $f(x_{n+1}^*, t_{n+1})$ and apply the trapezoid formula:

$$x_{n+1} = x_n + \frac{h}{2} [f(x_n, t_n) + f(x_{n+1}^*, t_{n+1})] \quad (787)$$

This is our “corrector” step, where the initial guess x_{n+1}^* is corrected by taking into account the slope at (x_{n+1}^*, t_{n+1}) .

This two-step process is similar to performing one iteration of Newton’s method to solve the implicit trapezoid equation, starting from the Euler prediction. However, to fully solve the implicit equation, multiple iterations would be necessary until convergence is achieved.

7.2.5 Collocation Methods

The numerical integration methods we discussed earlier are inherently **sequential**: given an initial state, we step forward in time and approximate what happens over a short interval. The accuracy of this procedure depends on the chosen rule (Euler, trapezoid, Runge–Kutta) and on the information available locally. Each new state is obtained by evaluating a formula that approximates the derivative or integral over that small step.

Collocation methods provide an alternative viewpoint. Instead of advancing one step at a time, they approximate the entire trajectory with a finite set of basis functions and require the dynamics to hold at selected points. This replaces the original differential equation with a system of **algebraic equations**: relations among the coefficients of the basis functions that must all be satisfied simultaneously. Solving these equations fixes the whole trajectory in one computation.

Seen from this angle, integration rules, spline interpolation, quadrature, and collocation are all instances of the same principle: an infinite-dimensional problem is reduced to finitely many parameters linked by numerical rules. The difference is mainly in scope. Sequential integration advances the state forward one interval at a time, which makes it simple but prone to error accumulation.

Collocation belongs to the class of **simultaneous methods** already introduced for DOCPs: the entire trajectory is represented at once, the dynamics are imposed everywhere in the discretization, and approximation error is spread across the horizon rather than accumulating step by step.

This global enforcement comes at a computational cost since the resulting algebraic system is larger and denser. However, the benefit is precisely the structural one we saw in simultaneous methods earlier: by exposing the coupling between states, controls, and dynamics explicitly, collocation allows solvers to exploit sparsity and to enforce path constraints directly at the collocation points. This is why collocation is especially effective for challenging continuous-time optimal control problems where robustness and constraint satisfaction are central.

7.2.6 Quick Primer on Polynomials

Collocation methods are based on polynomial approximation theory. Therefore, the first step in developing collocation-based optimal control techniques is to review the fundamentals of polynomial functions.

Polynomials are typically introduced through their standard form:

$$p(t) = a_n t^n + a_{n-1} t^{n-1} + \cdots + a_1 t + a_0 \quad (788)$$

In this expression, the a_i are coefficients which linearly combine the powers of t to represent a function. The set of functions $\{1, t, t^2, t^3, \dots, t^n\}$ used in the standard polynomial representation is called the **monomial basis**.

In linear algebra, a basis is a set of vectors in a vector space such that any vector in the space can be uniquely represented as a linear combination of these basis vectors. In the same way, a **polynomial basis** is such that any function $f(x)$ (within the function space) to be expressed as:

$$f(x) = \sum_{k=0}^{\infty} c_k p_k(x), \quad (789)$$

where the coefficients c_k are generally determined by solving a system of equation.

Just as vectors can be represented in different coordinate systems (bases), functions can also be expressed using various polynomial bases. However, the ability to apply a change of basis does not imply that all types of polynomials are equivalent from a practical standpoint. In practice, our choice of polynomial basis is dictated by considerations of efficiency, accuracy, and stability when approximating a function.

For instance, despite the monomial basis being easy to understand and implement, it often performs poorly in practice due to numerical instability. This instability arises as its coefficients take on large values: an ill-conditioning problem. The following kinds of polynomial often remedy this issues.

Orthogonal Polynomials An **orthogonal polynomial basis** is a set of polynomials that are orthogonal to each other and form a complete basis for a certain space of functions. This means that any function within that space can be represented as a linear combination of these polynomials.

More precisely, let $\{p_0(x), p_1(x), p_2(x), \dots\}$ be a sequence of polynomials where each $p_n(x)$ is a polynomial of degree n . We say that this set forms an orthogonal polynomial basis if any polynomial $q(x)$ of degree n or less can be uniquely expressed as a linear combination of $\{p_0(x), p_1(x), \dots, p_n(x)\}$. Furthermore, the orthogonality property means that for any $i \neq j$:

$$\langle p_i, p_j \rangle = \int_a^b p_i(x) p_j(x) w(x) dx = 0. \quad (790)$$

for some weight function $w(x)$ over a given interval of orthogonality $[a, b]$.

The orthogonality property allows to simplify the computation of the coefficients involved in the polynomial representation of a function. At a high level, what happens is that when taking the inner product of $f(x)$ with each basis polynomial, $p_k(x)$ isolates the corresponding coefficient c_k , which can be found to be:

$$c_k = \frac{\langle f, p_k \rangle}{\langle p_k, p_k \rangle} = \frac{\int_a^b f(x) p_k(x) w(x) dx}{\int_a^b p_k(x)^2 w(x) dx}. \quad (791)$$

Here are some examples of the most common orthogonal polynomials used in practice.

Legendre Polynomials Legendre polynomials $\{P_n(x)\}$ are defined on the interval $[-1, 1]$ and satisfy the orthogonality condition:

$$\int_{-1}^1 P_n(x) P_m(x) dx = \begin{cases} 0 & \text{if } n \neq m, \\ \frac{2}{2n+1} & \text{if } n = m. \end{cases} \quad (792)$$

They can be generated using the recurrence relation:

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \quad (793)$$

with initial conditions:

$$P_0(x) = 1, \quad P_1(x) = x. \quad (794)$$

The first four Legendre polynomials resulting from this recurrence are the following:

Chebyshev Polynomials There are two types of Chebyshev polynomials: **Chebyshev polynomials of the first kind**, $\{T_n(x)\}$, and **Chebyshev polynomials of the second kind**, $\{U_n(x)\}$. We typically focus on the first kind. They are defined on the interval $[-1, 1]$ and satisfy the orthogonality condition:

$$\int_{-1}^1 \frac{T_n(x)T_m(x)}{\sqrt{1-x^2}} dx = \begin{cases} 0 & \text{if } n \neq m, \\ \frac{\pi}{2} & \text{if } n = m \neq 0, \\ \pi & \text{if } n = m = 0. \end{cases} \quad (795)$$

The Chebyshev polynomials of the first kind can be generated using the recurrence relation:

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \quad (796)$$

with initial conditions:

$$T_0(x) = 1, \quad T_1(x) = x. \quad (797)$$

This recurrence relation also admits an explicit formula:

$$T_n(x) = \cos(n \cos^{-1}(x)). \quad (798)$$

Let's now implement it in Python:

Hermite Polynomials Hermite polynomials $\{H_n(x)\}$ are defined on the entire real line and are orthogonal with respect to the weight function $w(x) = e^{-x^2}$. They satisfy the orthogonality condition:

$$\int_{-\infty}^{\infty} H_n(x)H_m(x)e^{-x^2} dx = \begin{cases} 0 & \text{if } n \neq m, \\ 2^n n! \sqrt{\pi} & \text{if } n = m. \end{cases} \quad (799)$$

Hermite polynomials can be generated using the recurrence relation:

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x), \quad (800)$$

with initial conditions:

$$H_0(x) = 1, \quad H_1(x) = 2x. \quad (801)$$

The following code computes the coefficients of the first four Hermite polynomials:

7.2.7 Collocation Conditions

Consider a general ODE of the form:

$$\dot{y}(t) = f(y(t), t), \quad y(t_0) = y_0, \quad (802)$$

where $y(t) \in R^n$ is the state vector, and $f : R^n \times R \rightarrow R^n$ is a known function. The goal is to approximate the solution $y(t)$ over a given interval $[t_0, t_f]$. Collocation methods achieve this by:

1. **Choosing a basis** to approximate $y(t)$ using a finite sum of basis functions $\phi_i(t)$:

$$y(t) \approx \sum_{i=0}^N c_i \phi_i(t), \quad (803)$$

where $\{c_i\}$ are the coefficients to be determined.

2. **Selecting collocation points** $t_1, t_2, \dots, t_N$ within the interval $[t_0, t_f]$. These are typically chosen to be the roots of certain orthogonal polynomials, like Legendre or Chebyshev polynomials, or can be spread equally across the interval.

3. **Enforcing the ODE at the collocation points** for each t_j :

$$\dot{y}(t_j) = f(y(t_j), t_j). \quad (804)$$

To implement this, we differentiate the approximate solution $y(t)$ with respect to time:

$$\dot{y}(t) \approx \sum_{i=0}^N c_i \dot{\phi}_i(t). \quad (805)$$

Substituting this into the ODE at the collocation points gives:

$$\sum_{i=0}^N c_i \dot{\phi}_i(t_j) = f\left(\sum_{i=0}^N c_i \phi_i(t_j), t_j\right), \quad j = 1, \dots, N. \quad (806)$$

The collocation equations are formed by enforcing the ODE at all collocation points, leading to a system of nonlinear equations:

$$\sum_{i=0}^N c_i \dot{\phi}_i(t_j) - f\left(\sum_{i=0}^N c_i \phi_i(t_j), t_j\right) = 0, \quad j = 1, \dots, N. \quad (807)$$

Furthermore when solving an initial value problem (IVP), we also need to incorporate the initial condition $y(t_0) = y_0$ as an additional constraint:

$$\sum_{i=0}^N c_i \phi_i(t_0) = y_0. \quad (808)$$

The collocation conditions and IVP condition are combined together to form a root-finding problem, which we can generically solve numerically using Newton's method.

7.2.8 Common Numerical Integration Techniques as Collocation Methods

Many common numerical integration techniques can be viewed as special cases of collocation methods. While the general collocation method we discussed earlier applies to the entire interval $[t_0, t_f]$, many numerical integration techniques can be viewed as collocation methods applied locally, step by step.

In practical numerical integration, we often divide the full interval $[t_0, t_f]$ into smaller subintervals or steps. In general, this allows us to use simpler basis functions thereby reducing computational complexity, and gives us more flexibility in dynamically adjusting the step size using local error estimates. When we apply collocation locally, we're essentially using the collocation method to "step" from t_n to t_{n+1} . As we did, earlier we still apply the following three steps:

1. We choose a basis function to approximate $y(t)$ over $[t_n, t_{n+1}]$.
2. We select collocation points within this interval.
3. We enforce the ODE at these points to determine the coefficients of our basis function.

We can make this idea clearer by re-deriving some of the numerical integration methods seen before using this perspective.

Explicit Euler Method For the Explicit Euler method, we use a linear basis function for each step:

$$\phi(t) = 1 + c(t - t_n) \quad (809)$$

Note that we use $(t - t_n)$ rather than just t because we're approximating the solution locally, relative to the start of each step. We then choose one collocation point at t_{n+1} where we have:

$$y'(t_{n+1}) = c = f(y_n, t_n) \quad (810)$$

Our local approximation is:

$$y(t) \approx y_n + c(t - t_n) \quad (811)$$

At $t = t_{n+1}$, this gives:

$$y_{n+1} = y_n + c(t_{n+1} - t_n) = y_n + hf(y_n, t_n) \quad (812)$$

where $h = t_{n+1} - t_n$. This is the classic Euler update formula.

Implicit Euler Method The Implicit Euler method uses the same linear basis function:

$$\phi(t) = 1 + c(t - t_n) \quad (813)$$

Again, we choose one collocation point at t_{n+1} . The main difference is that we enforce the ODE using y_{n+1} :

$$y'(t_{n+1}) = c = f(y_{n+1}, t_{n+1}) \quad (814)$$

Our approximation remains:

$$y(t) \approx y_n + c(t - t_n) \quad (815)$$

At $t = t_{n+1}$, this leads to the implicit equation:

$$y_{n+1} = y_n + hf(y_{n+1}, t_{n+1}) \quad (816)$$

Trapezoidal Method The Trapezoidal method uses a quadratic basis function:

$$\phi(t) = 1 + c(t - t_n) + a(t - t_n)^2 \quad (817)$$

We use two collocation points: t_n and t_{n+1} . Enforcing the ODE at these points gives:

- At t_n :

$$y'(t_n) = c = f(y_n, t_n) \quad (818)$$

- At t_{n+1} :

$$y'(t_{n+1}) = c + 2ah = f(y_n + ch + ah^2, t_{n+1}) \quad (819)$$

Our approximation is:

$$y(t) \approx y_n + c(t - t_n) + a(t - t_n)^2 \quad (820)$$

At $t = t_{n+1}$, this gives:

$$y_{n+1} = y_n + ch + ah^2 \quad (821)$$

Solving the system of equations leads to the trapezoidal update:

$$y_{n+1} = y_n + \frac{h}{2}[f(y_n, t_n) + f(y_{n+1}, t_{n+1})] \quad (822)$$

Runge-Kutta Methods Higher-order Runge-Kutta methods can also be interpreted as collocation methods. The RK4 method corresponds to a collocation method using a cubic polynomial basis:

$$\phi(t) = 1 + c_1(t - t_n) + c_2(t - t_n)^2 + c_3(t - t_n)^3 \quad (823)$$

Here, we're using a cubic polynomial to approximate the solution over each step, rather than the linear or quadratic approximations of the other methods above. For RK4, we use four collocation points:

1. t_n (the start of the step)
2. $t_n + h/2$
3. $t_n + h/2$
4. $t_n + h$ (the end of the step)

These points are called the “Gauss-Lobatto” points, scaled to our interval $[t_n, t_n + h]$. The RK4 method enforces the ODE at these collocation points, leading to four stages:

$$\begin{aligned} k_1 &= hf(y_n, t_n) \\ k_2 &= hf(y_n + \frac{1}{2}k_1, t_n + \frac{h}{2}) \\ k_3 &= hf(y_n + \frac{1}{2}k_2, t_n + \frac{h}{2}) \\ k_4 &= hf(y_n + k_3, t_n + h) \end{aligned} \quad (824)$$

The final update formula for RK4 can be derived by solving the system of equations resulting from enforcing the ODE at our collocation points:

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (825)$$

7.2.9 Example: Solving a Simple ODE by Collocation

Consider a simple ODE:

$$\frac{dy}{dt} = -y, \quad y(0) = 1, \quad t \in [0, 2] \quad (826)$$

The analytical solution is $y(t) = e^{-t}$. We apply the collocation method with a monomial basis of order N :

$$\phi_i(t) = t^i, \quad i = 0, 1, \dots, N \quad (827)$$

We select N equally spaced points $\{t_1, \dots, t_N\}$ in $[0, 2]$ as collocation points.

7.3 Nonlinear Programming

Unless specific assumptions are made on the dynamics and cost structure, a DOCP is, in its most general form, a nonlinear mathematical program (commonly referred to as an NLP, not to be confused with Natural Language Processing). An NLP can be formulated as follows:

$$\begin{aligned} & \text{minimize } f(\mathbf{x}) \\ & \text{subject to } \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ & \quad \mathbf{h}(\mathbf{x}) = \mathbf{0} \end{aligned} \tag{828}$$

Where:

- $f : R^n \rightarrow R$ is the objective function
- $\mathbf{g} : R^n \rightarrow R^m$ represents inequality constraints
- $\mathbf{h} : R^n \rightarrow R^\ell$ represents equality constraints

Unlike unconstrained optimization commonly used in deep learning, the optimality of a solution in constrained optimization must consider both the objective value and constraint feasibility. To illustrate this, consider the following problem, which includes both equality and inequality constraints:

$$\begin{aligned} & \text{Minimize } f(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 2.5)^2 \\ & \text{subject to } g(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 1)^2 \leq 1.5, \\ & \quad h(x_1, x_2) = x_2 - (0.5 \sin(2\pi x_1) + 1.5) = 0. \end{aligned}$$

In this example, the objective function $f(x_1, x_2)$ is quadratic, the inequality constraint $g(x_1, x_2)$ defines a circular feasible region centered at (1, 1) with a radius of $\sqrt{1.5}$ and the equality constraint $h(x_1, x_2)$ requires x_2 to lie on a sine wave function. The following code demonstrates the difference between the unconstrained, and constrained solutions to this problem.

Karush-Kuhn-Tucker (KKT) conditions While this example is simple enough to convince ourselves visually of the solution to this particular problem, it falls short of providing us with actionable characterization of what constitutes an optimal solution in general. The Karush-Kuhn-Tucker (KKT) conditions provide us with an answer to this problem by generalizing the first-order optimality conditions in unconstrained optimization to problems involving both equality and inequality constraints. This result relies on the construction of an auxiliary function called the Lagrangian, defined as:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\mu}^\top \mathbf{g}(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x}) \tag{829}$$

where $\boldsymbol{\mu} \in R^m$ and $\boldsymbol{\lambda} \in R^\ell$ are known as Lagrange multipliers. The first-order optimality conditions then state that if $\mathbf{x}^*$, then there must exist corresponding Lagrange multipliers $\boldsymbol{\mu}^*$ and $\boldsymbol{\lambda}^*$ such that:

Definition 7.1. 1. The gradient of the Lagrangian with respect to $\mathbf{x}$ must be zero at the optimal point (**stationarity**):

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) = \nabla f(\mathbf{x}^*) + \sum_{i=1}^m \mu_i^* \nabla g_i(\mathbf{x}^*) + \sum_{j=1}^{\ell} \lambda_j^* \nabla h_j(\mathbf{x}^*) = \mathbf{0} \quad (830)$$

In the case where we only have equality constraints, this means that the gradient of the objective and that of constraint are parallel to each other at the optimum but point in opposite directions.

2. A valid solution of a NLP is one which satisfies all the constraints (**primal feasibility**)

$$\mathbf{g}(\mathbf{x}^*) \leq \mathbf{0}, \text{ and } \mathbf{h}(\mathbf{x}^*) = \mathbf{0} \quad (831)$$

3. Furthermore, the Lagrange multipliers for **inequality** constraints must be non-negative (**dual feasibility**)

$$\boldsymbol{\mu}^* \geq \mathbf{0} \quad (832)$$

This condition stems from the fact that the inequality constraints can only push the solution in one direction.

4. Finally, for each inequality constraint, either the constraint is active (equality holds) or its corresponding Lagrange multiplier is zero at an optimal solution (**complementary slackness**)

$$\mu_i^* g_i(\mathbf{x}^*) = 0, \quad \forall i = 1, \dots, m \quad (833)$$

Let's now solve our example problem above, this time using [Ipopt](#) via the [Pyomo](#) interface so that we can access the Lagrange multipliers found by the solver.

```
# label: appendix_nlp -cell -02
# caption: Rendered output from the preceding code cell.

from pyomo.environ import *
from pyomo.opt import SolverFactory
import math

# Define the Pyomo model
model = ConcreteModel()

# Define the variables
model.x1 = Var(initialize=1.25)
```

```

model.x2 = Var(initialize=1.5)

# Define the objective function
def objective_rule(model):
    return (model.x1 - 1)**2 + (model.x2 - 2.5)**2
model.obj = Objective(rule=objective_rule, sense=minimize)

# Define the inequality constraint (circle)
def inequality_constraint_rule(model):
    return (model.x1 - 1)**2 + (model.x2 - 1)**2 <= 1.5
model.ineq_constraint = Constraint(rule=inequality_constraint_rule)

# Define the equality constraint (sine wave) using Pyomo's math functions
def equality_constraint_rule(model):
    return model.x2 == 0.5 * sin(2 * math.pi * model.x1) + 1.5
model.eq_constraint = Constraint(rule=equality_constraint_rule)

# Create a suffix component to capture dual values
model.dual = Suffix(direction=Suffix.IMPORT)

# Create a solver
solver=SolverFactory('ipopt')

# Solve the problem
results = solver.solve(model, tee=False)

# Check if the solver found an optimal solution
if (results.solver.status == SolverStatus.ok and
    results.solver.termination_condition == TerminationCondition.optimal):

    # Print the results
    print(f"x1: {value(model.x1)}")
    print(f"x2: {value(model.x2)}")

    # Print the objective value
    print(f"Objective value: {value(model.obj)}")

    # Print the Lagrange multipliers (dual values)
    print("\nLagrange multipliers:")
    ineq_lambda = None
    eq_lambda = None
    for c in model.component_objects(Constraint, active=True):
        for index in c:
            dual_val = model.dual[c[index]]
            print(f"{c.name}[{index}]: {dual_val}")
            if c.name == "ineq_constraint":

```

```

        ineq_lambda = dual_val
    elif c.name == "eq_constraint":
        eq_lambda = dual_val
else:
    print("Solver did not find an optimal solution.")
    print(f"Solver Status: {results.solver.status}")
    print(f"Termination Condition: {results.solver.termination_condition}")

```

After running the code above, we can observe the Lagrange multipliers. The Lagrange multiplier associated with the inequality constraint is very small (close to zero), suggesting that the inequality constraint is not active at the optimal solution—meaning that the solution point lies inside the circle defined by this constraint. This can be verified visually in the figure above. As for the equality constraint, its corresponding Lagrange multiplier is non-zero, indicating that this constraint is active at the optimal solution. In general, when we find a Lagrange multiplier close to zero (like the one for the inequality constraint), it means that constraint is not “binding”—the optimal solution does not lie on the boundary defined by this constraint. In contrast, a non-zero Lagrange multiplier, such as the one for the equality constraint, indicates that the constraint is active and that any relaxation would directly affect the objective function’s value, as required by the stationarity condition.

Lagrange Multiplier Theorem The KKT conditions introduced above characterize the solution structure of constrained optimization problems with equality constraints. In this particular context, these conditions are referred to as the first-order optimality conditions, as part of the Lagrange multiplier theorem. Let’s just re-state them in that simpler setting:

Definition 7.2 (Lagrange Multiplier Theorem). *Consider the constrained optimization problem:*

$$\begin{aligned}
 & \min_{\mathbf{x}} f(\mathbf{x}) \\
 & \text{subject to } h_i(\mathbf{x}) = 0, \quad i = 1, \dots, m
 \end{aligned} \tag{834}$$

where $\mathbf{x} \in R^n$, $f : R^n \rightarrow R$, and $h_i : R^n \rightarrow R$ for $i = 1, \dots, m$. Assume that:

1. f and h_i are continuously differentiable functions.
2. The gradients $\nabla h_i(\mathbf{x}^*)$ are linearly independent at the optimal point $\mathbf{x}^*$.

Then, there exist unique Lagrange multipliers $\lambda_i^* \in R$, $i = 1, \dots, m$, such that the following first-order optimality conditions hold:

1. *Stationarity:* $\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla h_i(\mathbf{x}^*) = \mathbf{0}$
2. *Primal feasibility:* $h_i(\mathbf{x}^*) = 0$, for $i = 1, \dots, m$

Note that both the stationarity and primal feasibility statements are simply saying that the derivative of the Lagrangian in either the primal or dual variables must be zero at an optimal constrained solution. In other words:

$$\nabla_{\mathbf{x}, \boldsymbol{\lambda}} L(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0} \quad (835)$$

Letting $\mathbf{F}(\mathbf{x}, \boldsymbol{\lambda})$ stand for $\nabla_{\mathbf{x}, \boldsymbol{\lambda}} L(\mathbf{x}, \boldsymbol{\lambda})$, the Lagrange multipliers theorem tells us that an optimal primal-dual pair is actually a zero of that function $\mathbf{F}$: the derivative of the Lagrangian. Therefore, we can use this observation to craft a solution method for solving equality constrained optimization using Newton's method, which is a numerical procedure for finding zeros of a nonlinear function.

Newton's Method Newton's method is a numerical procedure for solving root-finding problems. These are nonlinear systems of equations of the form:

Find $\mathbf{z}^* \in R^n$ such that $\mathbf{F}(\mathbf{z}^*) = \mathbf{0}$

where $\mathbf{F} : R^n \rightarrow R^n$ is a continuously differentiable function. Newton's method then consists in applying the following sequence of iterates:

$$\mathbf{z}^{k+1} = \mathbf{z}^k - [\nabla \mathbf{F}(\mathbf{z}^k)]^{-1} \mathbf{F}(\mathbf{z}^k) \quad (836)$$

where $\mathbf{z}^k$ is the k-th iterate, and $\nabla \mathbf{F}(\mathbf{z}^k)$ is the Jacobian matrix of $\mathbf{F}$ evaluated at $\mathbf{z}^k$.

Newton's method exhibits local quadratic convergence: if the initial guess $\mathbf{z}^0$ is sufficiently close to the true solution $\mathbf{z}^*$, and $\nabla \mathbf{F}(\mathbf{z}^*)$ is nonsingular, the method converges quadratically to $\mathbf{z}^*$ [Ortega and Rheinboldt \[1970\]](#). However, the method is sensitive to the initial guess; if it's too far from the desired solution, Newton's method might fail to converge or converge to a different root. To mitigate this problem, a set of techniques known as numerical continuation methods [Allgower and Georg \[1990\]](#) have been developed. These methods effectively enlarge the basin of attraction of Newton's method by solving a sequence of related problems, progressing from an easy one to the target problem. This approach is reminiscent of several concepts in machine learning and statistical inference: curriculum learning in machine learning, where models are trained on increasingly complex data; tempering in Markov Chain Monte Carlo (MCMC) samplers, which gradually adjusts the target distribution to improve mixing; and modern diffusion models, which use a similar concept of gradually transforming noise into structured data.

Efficient Implementation of Newton's Method Note that each step of Newton's method involves computing the inverse of a Jacobian matrix. However, a cardinal rule in numerical linear algebra is to avoid computing matrix inverses explicitly: rarely, if ever, should there be a `np.linalg.inv` in your code. Instead, the numerically stable and computationally efficient approach is to solve a linear system of equations at each step. Given the Newton's method iterate:

$$\mathbf{z}^{k+1} = \mathbf{z}^k - [\nabla \mathbf{F}(\mathbf{z}^k)]^{-1} \mathbf{F}(\mathbf{z}^k) \quad (837)$$

We can reformulate this as a two-step procedure:

1. Solve the linear system: $\underbrace{[\nabla \mathbf{F}(\mathbf{z}^k)]}_{\mathbf{A}} \Delta \mathbf{z}^k = -\mathbf{F}(\mathbf{z}^k)$
2. Update: $\mathbf{z}^{k+1} = \mathbf{z}^k + \Delta \mathbf{z}^k$

The structure of the linear system in step 1 often allows for specialized solution methods. In the context of automatic differentiation, matrix-free linear solvers are particularly useful. These solvers can find a solution without explicitly forming the matrix $\mathbf{A}$, requiring only the ability to evaluate matrix-vector or vector-matrix products. Typical examples of such methods include classical matrix-splitting methods (e.g., Richardson iteration) or conjugate gradient methods through [sparse.linalg.cg](#) for example. Another useful method is the Generalized Minimal Residual method (GMRES) implemented in SciPy via [sparse.linalg.gmres](#), which is useful when facing non-symmetric and indefinite systems.

By inspecting the structure of matrix $\mathbf{A}$ in the specific application where the function $\mathbf{F}$ is the derivative of the Lagrangian, we will also uncover an important structure known as the KKT matrix. This structure will then allow us to derive a Quadratic Programming (QP) sub-problem as part of a larger iterative procedure for solving equality and inequality constrained problems via Sequential Quadratic Programming (SQP).

Solving Equality Constrained Programs with Newton's Method To solve equality-constrained optimization problems using Newton's method, we begin by recognizing that the problem reduces to finding a zero of the function $\mathbf{F}(\mathbf{z}) = \nabla_{\mathbf{x}, \boldsymbol{\lambda}} L(\mathbf{x}, \boldsymbol{\lambda})$. Here, $\mathbf{F}$ represents the derivative of the Lagrangian function, and $\mathbf{z} = (\mathbf{x}, \boldsymbol{\lambda})$ combines both the primal variables $\mathbf{x}$ and the dual variables (Lagrange multipliers) $\boldsymbol{\lambda}$. Explicitly, we have:

$$\mathbf{F}(\mathbf{z}) = \begin{bmatrix} \nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\lambda}) \\ \mathbf{h}(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \nabla f(\mathbf{x}) + \sum_{i=1}^m \lambda_i \nabla h_i(\mathbf{x}) \\ \mathbf{h}(\mathbf{x}) \end{bmatrix}. \quad (838)$$

Newton's method involves linearizing $\mathbf{F}(\mathbf{z})$ around the current iterate $\mathbf{z}^k = (\mathbf{x}^k, \boldsymbol{\lambda}^k)$ and then solving the resulting linear system. At each iteration k , Newton's method updates the current estimate by solving the linear system:

$$\mathbf{z}^{k+1} = \mathbf{z}^k - [\nabla \mathbf{F}(\mathbf{z}^k)]^{-1} \mathbf{F}(\mathbf{z}^k). \quad (839)$$

However, instead of explicitly inverting the Jacobian matrix $\nabla \mathbf{F}(\mathbf{z}^k)$, we solve the linear system:

$$\underbrace{\nabla \mathbf{F}(\mathbf{z}^k)}_{\mathbf{A}} \Delta \mathbf{z}^k = -\mathbf{F}(\mathbf{z}^k), \quad (840)$$

where $\Delta \mathbf{z}^k = (\Delta \mathbf{x}^k, \Delta \boldsymbol{\lambda}^k)$ represents the Newton step for the primal and dual variables. Substituting the expression for $\mathbf{F}(\mathbf{z})$ and its Jacobian, the system becomes:

$$\begin{bmatrix} \nabla_{\mathbf{x}\mathbf{x}}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) & \nabla \mathbf{h}(\mathbf{x}^k)^T \\ \nabla \mathbf{h}(\mathbf{x}^k) & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x}^k \\ \Delta \boldsymbol{\lambda}^k \end{bmatrix} = - \begin{bmatrix} \nabla f(\mathbf{x}^k) + \nabla \mathbf{h}(\mathbf{x}^k)^T \boldsymbol{\lambda}^k \\ \mathbf{h}(\mathbf{x}^k) \end{bmatrix}. \quad (841)$$

The matrix on the left-hand side is known as the KKT matrix, as it stems from the Karush-Kuhn-Tucker conditions for this optimization problem. The solution of this system provides the updates $\Delta \mathbf{x}^k$ and $\Delta \boldsymbol{\lambda}^k$, which are then used to update the primal and dual variables:

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta \mathbf{x}^k, \quad \boldsymbol{\lambda}^{k+1} = \boldsymbol{\lambda}^k + \Delta \boldsymbol{\lambda}^k. \quad (842)$$

Demonstration The following code demonstrates how we can implement this idea in Jax. In this demonstration, we are minimizing a quadratic objective function subject to a single equality constraint, a problem formally stated as follows:

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = (x_1 - 2)^2 + (x_2 - 1)^2 \\ \text{subject to} \quad & h(x) = x_1^2 + x_2^2 - 1 = 0 \end{aligned} \quad (843)$$

Geometrically speaking, the constraint $h(x)$ describes a unit circle centered at the origin. To solve this problem using the method of Lagrange multipliers, we form the Lagrangian:

$$L(x, \lambda) = f(x) + \lambda h(x) = (x_1 - 2)^2 + (x_2 - 1)^2 + \lambda(x_1^2 + x_2^2 - 1) \quad (844)$$

For this particular problem, it happens so that we can also find an analytical solution without even having to use Newton's method. From the first-order optimality conditions, we obtain the following linear system of equations:

$$\begin{aligned} 2(x_1 - 2) + 2\lambda x_1 &= 0 \\ 2(x_2 - 1) + 2\lambda x_2 &= 0 \\ x_1^2 + x_2^2 - 1 &= 0 \end{aligned}$$

From the first two equations, we then get:

$$x_1 = \frac{2}{1 + \lambda}, \quad x_2 = \frac{1}{1 + \lambda} \quad (845)$$

which we can substitute these into the 3rd constraint equation to obtain:

$$\left(\frac{2}{1 + \lambda}\right)^2 + \left(\frac{1}{1 + \lambda}\right)^2 = 1 \Leftrightarrow \lambda = \sqrt{5} - 1 \quad (846)$$

This value of the Lagrange multiplier can then be backsubstituted into the above equations to obtain $x_1 = \frac{2}{\sqrt{5}}$ and $x_2 = \frac{1}{\sqrt{5}}$. We can verify numerically (and visually on the following graph) that the point $(2/\sqrt{5}, 1/\sqrt{5})$ is indeed the point on the unit circle closest to $(2, 1)$.

7.3.1 The SQP Approach: Taylor Expansion and Quadratic Approximation

Sequential Quadratic Programming (SQP) tackles the problem of solving constrained programs by iteratively solving a sequence of simpler subproblems. Specifically, these subproblems are quadratic programs (QPs) that approximate the original problem around the current iterate by using a quadratic model of the objective function and a linear model of the constraints. Suppose we have the following optimization problem with equality constraints:

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{h}(\mathbf{x}) = \mathbf{0}. \end{aligned} \tag{847}$$

At each iteration k , we approximate the objective function $f(\mathbf{x})$ using a second-order Taylor expansion around the current iterate $\mathbf{x}^k$. The standard Taylor expansion for f would be:

$$f(\mathbf{x}) \approx f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^T (\mathbf{x} - \mathbf{x}^k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^k)^T \nabla^2 f(\mathbf{x}^k) (\mathbf{x} - \mathbf{x}^k).$$

This expansion uses the **Hessian of the objective function** $\nabla^2 f(\mathbf{x}^k)$ to capture the curvature of f . However, in the context of constrained optimization, we also need to account for the effect of the constraints on the local behavior of the solution. If we were to use only $\nabla^2 f(\mathbf{x}^k)$, we would not capture the influence of the constraints on the curvature of the feasible region. The resulting subproblem might then lead to steps that violate the constraints or are less effective in achieving convergence. The choice that we make instead is to use the Hessian of the Lagrangian, $\nabla_{\mathbf{xx}}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k)$, leading to the following quadratic model:

$$f(\mathbf{x}) \approx f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^T (\mathbf{x} - \mathbf{x}^k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^k)^T \nabla_{\mathbf{xx}}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) (\mathbf{x} - \mathbf{x}^k). \tag{848}$$

Similarly, the equality constraints $\mathbf{h}(\mathbf{x})$ are linearized around $\mathbf{x}^k$:

$$\mathbf{h}(\mathbf{x}) \approx \mathbf{h}(\mathbf{x}^k) + \nabla \mathbf{h}(\mathbf{x}^k) (\mathbf{x} - \mathbf{x}^k). \tag{849}$$

Combining these approximations, we obtain a Quadratic Programming (QP) subproblem, which approximates our original problem locally at $\mathbf{x}^k$ but is easier to solve:

$$\begin{aligned}
& \text{Minimize} && \nabla f(\mathbf{x}^k)^T \Delta \mathbf{x} + \frac{1}{2} \Delta \mathbf{x}^T \nabla_{\mathbf{x}\mathbf{x}}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) \Delta \mathbf{x} \\
& \text{subject to} && \nabla \mathbf{h}(\mathbf{x}^k) \Delta \mathbf{x} + \mathbf{h}(\mathbf{x}^k) = \mathbf{0},
\end{aligned} \tag{850}$$

where $\Delta \mathbf{x} = \mathbf{x} - \mathbf{x}^k$. The QP subproblem solved at each iteration focuses on finding the optimal step direction $\Delta \mathbf{x}$ for the primal variables. While solving this QP, we obtain not only the step $\Delta \mathbf{x}$ but also the associated Lagrange multipliers for the QP subproblem, which correspond to an updated dual variable vector $\boldsymbol{\lambda}^{k+1}$. More specifically, after solving the QP, we use $\Delta \mathbf{x}^k$ to update the primal variables:

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta \mathbf{x}^k.$$

Simultaneously, the Lagrange multipliers from the QP provide the updated dual variables $\boldsymbol{\lambda}^{k+1}$. We summarize the SQP algorithm in the following pseudo-code:

Connection to Newton's Method in the Equality-Constrained Case

The QP subproblem in SQP is directly related to applying Newton's method for equality-constrained optimization. To see this, note that the KKT matrix of the QP subproblem is:

$$\begin{bmatrix} \nabla_{\mathbf{x}\mathbf{x}}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k) & \nabla \mathbf{h}(\mathbf{x}^k)^T \\ \nabla \mathbf{h}(\mathbf{x}^k) & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x}^k \\ \Delta \boldsymbol{\lambda}^k \end{bmatrix} = - \begin{bmatrix} \nabla f(\mathbf{x}^k) + \nabla \mathbf{h}(\mathbf{x}^k)^T \boldsymbol{\lambda}^k \\ \mathbf{h}(\mathbf{x}^k) \end{bmatrix}$$

This is exactly the same linear system that have to solve when applying Newton's method to the KKT conditions of the original program! Thus, solving the QP subproblem at each iteration of SQP is equivalent to taking a Newton step on the KKT conditions of the original nonlinear problem.

7.3.2 SQP for Inequality-Constrained Optimization

So far, we've applied the ideas behind Sequential Quadratic Programming (SQP) to problems with only equality constraints. Now, let's extend this framework to handle optimization problems that also include inequality constraints. Consider a general nonlinear optimization problem that includes both equality and inequality constraints:

$$\begin{aligned}
& \min_{\mathbf{x}} && f(\mathbf{x}) \\
& \text{subject to} && \mathbf{g}(\mathbf{x}) \leq \mathbf{0}, \\
& && \mathbf{h}(\mathbf{x}) = \mathbf{0}.
\end{aligned}$$

As we did earlier, we approximate this problem by constructing a quadratic approximation to the objective and a linearization of the constraints. QP subproblem at each iteration is then formulated as:

$$\begin{aligned} \text{Minimize} \quad & \nabla f(\mathbf{x}^k)^T \Delta \mathbf{x} + \frac{1}{2} \Delta \mathbf{x}^T \nabla_{\mathbf{xx}}^2 L(\mathbf{x}^k, \boldsymbol{\lambda}^k, \boldsymbol{\nu}^k) \Delta \mathbf{x} \\ \text{subject to} \quad & \nabla \mathbf{g}(\mathbf{x}^k) \Delta \mathbf{x} + \mathbf{g}(\mathbf{x}^k) \leq \mathbf{0}, \\ & \nabla \mathbf{h}(\mathbf{x}^k) \Delta \mathbf{x} + \mathbf{h}(\mathbf{x}^k) = \mathbf{0}, \end{aligned}$$

where $\Delta \mathbf{x} = \mathbf{x} - \mathbf{x}^k$ represents the step direction for the primal variables. The following pseudocode outlines the steps involved in applying SQP to a problem with both equality and inequality constraints:

Demonstration with JAX and CVXPY Consider the following equality and inequality-constrained problem:

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = (x_1 - 2)^2 + (x_2 - 1)^2 \\ \text{subject to} \quad & g(x) = x_1^2 - x_2 \leq 0 \\ & h(x) = x_1^2 + x_2^2 - 1 = 0 \end{aligned}$$

This example builds on our previous one but adds a parabola-shaped inequality constraint. We require our solution to lie not only on the circle defining our equality constraint but also below the parabola. To solve the QP subproblem, we will be using the [CVXPY](#) package. While the Lagrangian and derivatives could be computed easily by hand, we use [JAX](#) for generality:

7.3.3 The Arrow-Hurwicz-Uzawa algorithm

While the SQP method addresses constrained optimization problems by sequentially solving quadratic subproblems, an alternative approach follows from viewing constrained optimization as a min-max problem. This perspective leads to a simpler algorithm, originally introduced by the Arrow-Hurwicz-Uzawa [Arrow et al. \[1958\]](#). Consider the following general constrained optimization problem encompassing both equality and inequality constraints:

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \\ & \mathbf{h}(\mathbf{x}) = \mathbf{0} \end{aligned} \tag{851}$$

Using the Lagrangian function $L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathbf{x}) + \boldsymbol{\mu}^T \mathbf{g}(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{h}(\mathbf{x})$, we can reformulate this problem as the following min-max problem:

$$\min_{\mathbf{x}} \max_{\boldsymbol{\lambda}, \boldsymbol{\mu} \geq 0} L(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \tag{852}$$

The role of each component in this min-max structure can be understood as follows:

1. The outer minimization over $\mathbf{x}$ finds the feasible point that minimizes the objective function $f(\mathbf{x})$.
2. The maximization over $\boldsymbol{\mu} \geq \mathbf{0}$ ensures that inequality constraints $\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$ are satisfied. If any inequality constraint is violated, the corresponding term in $\boldsymbol{\mu}^T \mathbf{g}(\mathbf{x})$ can be made arbitrarily large by choosing a large enough μ_i .
3. The maximization over $\boldsymbol{\lambda}$ ensures that equality constraints $\mathbf{h}(\mathbf{x}) = \mathbf{0}$ are satisfied.

Using this observation, we can devise an algorithm which, like SQP, will update both the primal and dual variables at every step. But rather than using second-order optimization, we will simply use a first-order gradient update step: a descent step in the primal variable, and an ascent step in the dual one. The corresponding procedure, when implemented by gradient descent, is called Gradient Ascent Descent in the learning and optimization communities. In the case of equality constraints only, the algorithm looks like the following:

Now to account for the fact that the Lagrange multiplier needs to be non-negative for inequality constraints, we can use our previous idea from projected gradient descent for bound constraints and consider a projection, or clipping step to ensure that this condition is satisfied throughout. In this case, the algorithm looks like the following:

Here, $[\cdot]_+$ denotes the projection onto the non-negative orthant, ensuring that $\boldsymbol{\mu}$ remains non-negative.

However, as it is widely known from the lessons of GAN (Generative Adversarial Network) training [Goodfellow et al. \[2014\]](#), Gradient Descent Ascent (GDA) can fail to converge or suffer from instability. The Arrow-Hurwicz-Uzawa algorithm, also known as the first-order Lagrangian method, is known to converge only locally, in the vicinity of an optimal primal-dual pair.

7.3.4 Projected Gradient Descent

The Arrow-Hurwicz-Uzawa algorithm provided a way to handle constraints through dual variables and a primal-dual update scheme. Another commonly used approach for constrained optimization is **Projected Gradient Descent (PGD)**. The idea is simple: take a gradient descent step as if the problem were unconstrained, then project the result back onto the feasible set. Formally:

$$\mathbf{x}_{k+1} = \mathcal{P}_C(\mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)), \quad (853)$$

where $\mathcal{P}_C$ is the projection onto the feasible set C , α is the step size, and $f(\mathbf{x})$ is the objective function.

PGD is particularly effective when the projection is computationally cheap. A common example is **box constraints** (or bound constraints), where the feasible set is a hyperrectangle:

$$C = \{\mathbf{x} \mid \mathbf{x}_{\text{lb}} \leq \mathbf{x} \leq \mathbf{x}_{\text{ub}}\}. \quad (854)$$

In this case, the projection reduces to an element-wise clipping operation:

$$[\mathcal{P}_C(\mathbf{x})]_i = \max(\min([\mathbf{x}]_i, [\mathbf{x}_{\text{ub}}]_i), [\mathbf{x}_{\text{lb}}]_i). \quad (855)$$

For bound-constrained problems, PGD is almost as easy to implement as standard gradient descent because the projection step is just a clipping operation. For more general constraints, however, the projection may require solving a separate optimization problem, which can be as hard as the original task. Here is the algorithm for a problem of the form:

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{x}_{\text{lb}} \leq \mathbf{x} \leq \mathbf{x}_{\text{ub}}. \end{aligned} \quad (856)$$

The clipping function is defined as:

$$\text{clip}(x, x_{\text{lb}}, x_{\text{ub}}) = \max(\min(x, x_{\text{ub}}), x_{\text{lb}}). \quad (857)$$

Under mild conditions such as Lipschitz continuity of the gradient, PGD converges to a stationary point of the constrained problem. Its simplicity and low cost make it a common choice whenever the projection can be computed efficiently.

References

- C. P. Adams and V. V. Brantner. Spending on new drug development1. *Health Economics*, 19(2):130–141, 2 2009. ISSN 1099-1050. doi: 10.1002/hec.1454. URL <http://dx.doi.org/10.1002/hec.1454>.
- E. L. Allgower and K. Georg. *Numerical Continuation Methods: An Introduction*, volume 13 of *Springer Series in Computational Mathematics*. Springer-Verlag, Berlin, Heidelberg, 1990.
- K. J. Arrow, L. Hurwicz, and H. Uzawa. *Studies in linear and non-linear programming*. Stanford University Press, 1958.
- K. E. Atkinson and F. A. Potra. Projection and Iterated Projection Methods for Nonlinear Integral Equations. *SIAM Journal on Numerical Analysis*, 24(6):1352–1373, 1987. doi: 10.1137/0724087.
- L. Baird. Residual algorithms: Reinforcement learning with function approximation. *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37, 1995.
- V. S. Borkar and S. P. Meyn. Ode Methods for Temporal Difference Learning. *SIAM Journal on Control and Optimization*, 38(2):447–465, 2000.
- C3S. Era5 hourly data on single levels from 1940 to present, 2018. URL <https://cds.climate.copernicus.eu/doi/10.24381/cds.adbb2d47>.
- S. Chakraverty, N. R. Mahato, P. Karunakar, and T. D. Rao. *Weighted Residual Methods*, chapter 3, pages 25–44. John Wiley & Sons, Inc., Hoboken, NJ, 2019. ISBN 9781119423461. doi: 10.1002/9781119423461.ch3.
- M. Chang. *Monte Carlo Simulation for the Pharmaceutical Industry: Concepts, Algorithms, and Case Studies*. CRC Press, 9 2010. ISBN 9780429152382. doi: 10.1201/ebk1439835920. URL <http://dx.doi.org/10.1201/EBK1439835920>.
- W. J. Cole, K. M. Powell, E. T. Hale, and T. F. Edgar. Reduced-order residential home modeling for model predictive control. *Energy and Buildings*, 74:69–77, 2014. ISSN 0378-7788. doi: <https://doi.org/10.1016/j.enbuild.2014.01.033>. URL <https://www.sciencedirect.com/science/article/pii/S0378778814000711>.
- M. J. Conroy and J. T. Peterson. *Decision Making in Natural Resource Management: A Structured, Adaptive Approach: A Structured, Adaptive Approach*. Wiley, 1 2013. ISBN 9781118506196. doi: 10.1002/9781118506196. URL <http://dx.doi.org/10.1002/9781118506196>.
- M. P. Deisenroth and C. E. Rasmussen. Pilco: A Model-Based and Data-Efficient Approach to Policy Search. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pages 465–472. Omnipress, 2011.

- C. D'Eramo, A. Nuara, M. Restelli, and A. Nowe. Estimating the Maximum Expected Value through Gaussian Approximation. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1032–1040. PMLR, 2016.
- D. Ernst, P. Geurts, and L. Wehenkel. Tree-Based Batch Mode Reinforcement Learning. *Journal of Machine Learning Research*, 6:503–556, 2005.
- J. Farebrother, M. C. Machado, and M. Bowling. Stop Regressing: Training Value Functions via Classification for Scalable Deep RL. *arXiv preprint arXiv:2403.03950*, 2024. URL <https://arxiv.org/abs/2403.03950>.
- S. Fujimoto, H. Hoof, and D. Meger. Addressing Function Approximation Error in Actor-Critic Methods. In *International Conference on Machine Learning (ICML)*, pages 1587–1596, 2018.
- D. Garg, J. Tang, G. Kahn, S. Levine, and C. Finn. Extreme Q-Learning: Max-ent RL without Entropy. *International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=SJ0Lde3tRL>.
- M. Gargiani, A. Zanelli, D. Liao-McPherson, T. H. Summers, and J. Lygeros. Dynamic Programming Through the Lens of Semismooth Newton-Type Methods. *IEEE Control Systems Letters*, 6:2996–3001, 2022. doi: 10.1109/LCSYS.2022.3181213.
- M. Geist, B. Scherrer, and O. Pietquin. A Theory of Regularized Markov Decision Processes. In K. Chaudhuri and R. Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2160–2169. PMLR, jun 9 2019. URL <https://proceedings.mlr.press/v97/geist19a.html>.
- I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, volume 27, 2014.
- G. J. Gordon. Stable function approximation in dynamic programming. In *Proceedings of the Twelfth International Conference on International Conference on Machine Learning, ICML'95*, pages 261–268, Tahoe City, California, USA, 1995. Morgan Kaufmann Publishers Inc. ISBN 1558603778.
- G. J. Gordon. Approximate Solutions to Markov Decision Problems, 1999. Technical Report CMU-CS-99-111.
- A. I. Grancharova and T. A. Johansen. *Explicit nonlinear model predictive control*. Lecture notes in control and information sciences. Springer, Berlin, Germany, 2012 edition, 3 2012.
- J. Gravdahl and O. Egeland. Compressor surge control using a close-coupled valve and backstepping. In *Proceedings of the 1997 American Control Conference (Cat. No.97CH36041)*, pages 982–986 vol.2. IEEE, 1997. doi: 10.1109/acc.1997.609673. URL <http://dx.doi.org/10.1109/ACC.1997.609673>.

- T. Haarnoja, H. Tang, P. Abbeel, and S. Levine. Reinforcement Learning with Deep Energy-Based Policies. *Proceedings of the 34th International Conference on Machine Learning*, 70:1352–1361, 2017.
- T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018a.
- T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018b.
- D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- R. Hafner and M. Riedmiller. Reinforcement learning in feedback control: Challenges and benchmarks from technical process control. *Machine Learning*, 84(1-2):137–169, 2 2011. ISSN 1573-0565. doi: 10.1007/s10994-011-5235-x. URL <http://dx.doi.org/10.1007/s10994-011-5235-x>.
- N. Heess, G. Wayne, D. Silver, T. Lillicrap, T. Erez, and Y. Tassa. Learning Continuous Control Policies by Stochastic Value Gradients. In *Advances in Neural Information Processing Systems*, volume 28, pages 2944–2952. Curran Associates, Inc., 2015.
- T. Jaakkola, M. I. Jordan, and S. P. Singh. Convergence of Stochastic Iterative Dynamic Programming Algorithms. *Neural Computation*, 6(6):1185–1201, 1994.
- K. L. Judd. Projection methods for solving aggregate growth models. *Journal of Economic Theory*, 58(2):410–452, 1992.
- K. L. Judd. *Approximation, perturbation, and projection methods in economic analysis*, volume 1, pages 509–585. Elsevier, 1996.
- K. L. Judd. *Numerical Methods in Economics*. MIT Press, Cambridge, MA, 1998. ISBN 9780262100717.
- M. P. Keane and K. I. Wolpin. The Solution and Estimation of Discrete Choice Dynamic Programming Models by Simulation and Interpolation: Monte Carlo Evidence. *The Review of Economics and Statistics*, 76(4):648, 11 1994. ISSN 0034-6535. doi: 10.2307/2109768. URL <http://dx.doi.org/10.2307/2109768>.
- V. R. Konda. Actor-Critic Algorithms, 2002.
- H. J. Kushner and G. G. Yin. *Stochastic Approximation and Recursive Algorithms and Applications*, volume 35 of *Applications of Mathematics*. Springer, 2nd edition, 2003.

- D. Lee and W. B. Powell. Bias-Corrected Q-Learning With Multistate Extension. *IEEE Transactions on Automatic Control*, 64(10):4011–4023, 10 2019. ISSN 2334-3303. doi: 10.1109/tac.2019.2912443. URL <http://dx.doi.org/10.1109/TAC.2019.2912443>.
- D. Lee, B. Defourny, and W. B. Powell. Bias-corrected Q-learning to control max-operator bias in Q-learning. In *2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, pages 93–99. IEEE, 4 2013. doi: 10.1109/adprl.2013.6614994. URL <http://dx.doi.org/10.1109/ADPRL.2013.6614994>.
- M. Legrand and S. Junca. Weighted Residual Solution Methods. HAL Preprint, 2025.
- S. Levine, A. Kumar, G. Tucker, and J. Fu. Reinforcement Learning as a Framework for Control: A Survey. *arXiv preprint arXiv:1806.04222*, 2018.
- T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous Control with Deep Reinforcement Learning. *arXiv preprint arXiv:1509.02971*, 2015.
- E. R. McGrattan. Application of Weighted Residual Methods to Dynamic Economic Models. Mimeo, Federal Reserve Bank of Minneapolis, 1997.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, and others. Playing Atari with Deep Reinforcement Learning. In *NIPS Deep Learning Workshop*, 2013.
- O. Nachum, M. Norouzi, K. Xu, and D. Schuurmans. Bridging the Gap Between Value and Policy Based Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 30, pages 2775–2785. Curran Associates, Inc., 2017.
- J. M. Ortega and W. C. Rheinboldt. *Iterative Solution of Nonlinear Equations in Several Variables*. Computer Science and Applied Mathematics. Academic Press, New York, 1970.
- M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New York, 1994. ISBN 978-0-471-61977-3. First published in 1994.
- M. Riedmiller. Neural Fitted Q Iteration – First Experiences with a Data Efficient Neural Reinforcement Learning Method. In *Proceedings of the 16th European Conference on Machine Learning (ECML)*, pages 317–328, Berlin, Heidelberg, 2005. Springer.
- J. Rust. Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher. *Econometrica*, 55(5):999–1033, 1987.

- J. Rust. *Chapter 14 Numerical dynamic programming in economics*, pages 619–729. Elsevier, 1996. doi: 10.1016/S1574-0021(96)01016-7. URL [http://dx.doi.org/10.1016/S1574-0021\(96\)01016-7](http://dx.doi.org/10.1016/S1574-0021(96)01016-7).
- M. S. Santos and J. Vigo-Aguiar. Analysis of a numerical dynamic programming algorithm applied to economic models. *Econometrica*, 66(2):409–426, 1998.
- C. Savorgnan, C. Romani, A. Kozma, and M. Diehl. Multiple shooting for distributed systems with applications in hydro electricity production. *Journal of Process Control*, 21(5):738–745, 6 2011. ISSN 0959-1524. doi: 10.1016/j.jprocont.2011.01.011. URL <http://dx.doi.org/10.1016/j.jprocont.2011.01.011>.
- Y. Sawaguchi, E. Furutani, G. Shirakami, M. Araki, and K. Fukuda. A Model-Predictive Hypnosis Control System Under Total Intravenous Anesthesia. *IEEE Transactions on Biomedical Engineering*, 55(3):874–887, 3 2008. ISSN 0018-9294. doi: 10.1109/tbme.2008.915670. URL <http://dx.doi.org/10.1109/tbme.2008.915670>.
- J. Stachurski. *Economic Dynamics: Theory and Computation*. MIT Press, Cambridge, MA, 2009. ISBN 9780262012775.
- J. Sun. Openap.top: Open Flight Trajectory Optimization for Air Transport and Sustainability Research. *Aerospace*, 9(7):383, 7 2022. ISSN 2226-4310. doi: 10.3390/aerospace9070383. URL <http://dx.doi.org/10.3390/aerospace9070383>.
- R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In *Advances in Neural Information Processing Systems*, volume 12, pages 1057–1063. MIT Press, 1999.
- J. N. Tsitsiklis. Asynchronous Stochastic Approximation and Q-Learning. *Machine Learning*, 16(3):185–202, 1994.
- H. Van Hasselt, A. Guez, and D. Silver. Deep Reinforcement Learning with Double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2016.
- C. J. C. H. Watkins. *Learning from Delayed Rewards*, 1989.
- C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3-4): 279–292, 1992.
- G. Williams, A. Aldrich, and E. A. Theodorou. Model Predictive Path Integral Control: From Theory to Parallel Computation. *Journal of Guidance, Control, and Dynamics*, 40(2):344–357, 2017. doi: 10.2514/1.G001921.
- R. J. Williams. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3):229–256, 1992. doi: 10.1007/BF00992696.

- Y. Zang, G. Bao, X. Ye, and H. Zhou. Weak adversarial networks for high-dimensional partial differential equations. *Journal of Computational Physics*, 411:109409, 2020. doi: 10.1016/j.jcp.2020.109409.
- B. D. Ziebart, A. L. Maas, J. A. Bagnell, and A. K. Dey. Maximum Entropy Inverse Reinforcement Learning. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, pages 1433–1438, 2008.